



Generación de imágenes de ecocardiogramas mediante estrategias de aprendizaje generativo

Salomón Hernández Velandia

Facultad de Ingeniería
Ingeniería en Multimedia
Bogotá
2025

Generación de imágenes de ecocardiogramas mediante estrategias de aprendizaje generativo

Salomón Hernández Velandia

Tesis presentada como requisito parcial para optar por el título de:
Ingeniero en Multimedia

Director(a):

Dr. Ing. Alexander Cerón Correa
Profesor Asociado
Facultad de Ingeniería
Universidad Militar Nueva Granada

Línea de investigación:

Multimedia Analítica

Grupo de investigación:

ACCEDER

Universidad Militar Nueva Granada

Facultad de Ingeniería
Ingeniería en Multimedia

2025

Declaración

Me permito afirmar que he realizado ésta tesis de manera autónoma y con la única ayuda de los medios permitidos y no diferentes a los mencionados el presente texto. Todos los pasajes que se han tomado de manera textual o figurativa de textos publicados y no publicados, los he reconocido en el presente trabajo. Ninguna parte del presente trabajo se ha empleado en ningún otro tipo de tesis.

Bogotá., 08/12/2025

A handwritten signature in black ink, appearing to read 'Salomón', with several long, sweeping strokes extending from the letters.

Salomón Hernández Velandia

Agradecimientos

Quiero expresar mi más profundo agradecimiento a mi maestra, la Dra. Hilda Marcela Iregui Guerrero, quien me abrió las puertas de su grupo de investigación ACCEDER y dedicó una parte invaluable de su tiempo a enseñarme los fundamentos algebraicos y estadísticos del machine learning. En los momentos en los que aún no tenía claridad sobre el rumbo profesional que deseaba seguir, ella despertó en mí una verdadera pasión por este campo, llenándome de ilusión y nuevas expectativas, su guía siempre fue cercana, paciente, amable, inspiradora, motivante y profundamente formativa. Aun en los días difíciles de su vida por culpa de su enfermedad, me regaló su apoyo, su conocimiento y un acompañamiento casi personalizado, hoy, cada avance que he logrado lleva implícita su huella, sus enseñanzas, la motivación y las profundas bases que me dejó, me permitieron continuar aprendiendo incluso en su ausencia.

Gracias por ser una muy buena profesional y una gran calidad de persona, que inspiró mi carrera profesional, siempre me quedarán guardadas sus enseñanzas y su vocación en mi corazón profesional.

Agradezco también a la Universidad Militar Nueva Granada y, especialmente, a la Universidad Nacional de Colombia, por brindarme el apoyo institucional y económico necesario para desarrollar este proyecto, gracias a sus espacios académicos pude relacionarme con profesionales de maestría y doctorado cuyas amplias experiencias y conocimientos enriquecieron mi proceso, ampliaron mi panorama en este campo y me ofrecieron orientación adicional en ausencia de mi maestra Marcela.

A su vez, extiendo mi agradecimiento a mi profesor, el Dr. Alexander Cerón Correa, quien asumió el acompañamiento de este trabajo, aunque su área principal no tenía relación directa en este campo académico, supo guiarme con claridad cuando me desviaba del objetivo esencial, su orientación me permitió enfocar adecuadamente el problema, fortalecer mi criterio académico y adquirir experiencia a través de su insistencia en que participara en diversos simposios como el 19CCC y el SIPAIM, encuentros que ampliaron mi perspectiva en el ámbito de la ingeniería aplicada a la medicina, también le agradezco su comprensión y paciencia cuando circunstancias familiares me obligaron a pausar temporalmente el proyecto.

Finalmente, expreso mi gratitud a las personas que han sido el fundamento de mi formación, gracias a sus sacrificios, su apoyo, su bendición, me dieron la oportunidad de cursar esta carrera, a mi abuelo Luis Adolfo Velandia Rojas, a mi abuela Cecilia Amado de Velandia, y, sobre todo, a mi madre Alcira Velandia Amado, cuyo esfuerzo, dedicación y amor han hecho posible que hoy llegue hasta este punto de mi vida académica y profesional y a Dios por su luz divina que iluminó mis días de incertidumbre, la paz que calmó mis dudas y me dio la

Generación de imágenes de ecocardiogramas mediante estrategias de aprendizaje generativo

oportunidad de finalizar este paso de mi vida aun con tantas adversidades.

Listado de símbolos y abreviaturas

Abreviaturas

Abreviatura	Término
<i>A2C</i>	Apical 2-cámaras
<i>A3C</i>	Apical 3-cámaras
<i>A4C</i>	Apical 4-cámaras
<i>AD</i>	Aurícula derecha
<i>ADA</i>	Adaptive Discriminator Augmentation
<i>Adam</i>	Adaptive Moment Estimation
<i>AEs</i>	Autoencoders
<i>AI</i>	Aurícula izquierda
<i>AM – Score</i>	Accuracy and Diversity Score
<i>ANN</i>	Artificial Neural Networks
<i>ASD</i>	Atrial Septal Defect
<i>AsaIN</i>	Adaptative Instance Normalization
<i>AV</i>	Atrioventricular Valves
<i>BUS</i>	Breast Ultrasound
<i>C2ST</i>	Classifier Two-Sample Test
<i>cGANs</i>	Conditional Generative Adversarial Networks
<i>CNN</i>	Convolutional Neural Networks
<i>CT</i>	Computed Tomography
<i>DCGAN</i>	Deep Convolutional Generative Adversarial Network
<i>DCNN</i>	Deep Convolutional Neural Network
<i>DKL</i>	Kullback-Leibler Divergence
<i>DL</i>	Deep Learning
<i>DNNs</i>	Deep Neural Networks
<i>EDV</i>	End-Diastolic Volume (Volumen telediastólico)
<i>EE</i>	Expected Entropy
<i>EF</i>	Ejection Fraction (Fracción de eyección)
<i>EHRs</i>	Electronic Health Record
<i>EMD</i>	Earth-Mover Distance
<i>ENN</i>	Expected Nearest-Neighbour Distant
<i>ERM</i>	Empirical Risk Optimization

Generación de imágenes de ecocardiogramas mediante estrategias de aprendizaje generativo

Abreviatura	Término
<i>ESV</i>	End-Systolic Volume (Volumen telesistólico)
<i>FC</i>	Fully Connected
<i>FEVI</i>	Fracción de Eyección del Ventrículo Izquierdo
<i>FFNNs</i>	Feed-Forward Neural Networks
<i>FID</i>	Fréchet Inception Distance
<i>GANs</i>	Generative Adversarial Networks
<i>GCN</i>	Graph Convolutional Networks
<i>$\mathcal{G}$</i>	Generator
<i>$\mathcal{G}_{\text{loss}}$</i>	Generator Loss
<i>GRU</i>	Gated Recurrent Units
<i>IA</i>	Inteligencia Artificial
<i>IS</i>	Inception Score
<i>JSD</i>	Jensen–Shannon Divergence
<i>KID</i>	Kernel Inception Distance
<i>LR</i>	Learning Rate
<i>LPIPS</i>	Learned Perceptual Image Patch Similarity
<i>LReLU</i>	Leaky Rectified Linear Unit
<i>LS</i>	LeNet Score
<i>LSGAN</i>	Least Squares Generative Adversarial Network
<i>LSTM</i>	Long Short-Term Memory
<i>$m - IS$</i>	Modality-specific Inception Score
<i>ML</i>	Machine Learning
<i>MLP</i>	Multi-Layer Perceptron
<i>MMD</i>	Maximum Mean Discrepancy
<i>MRI</i>	Magnetic Resonance Imaging
<i>MS</i>	Modality Score
<i>MSE</i>	Mean Squared Error
<i>NDB</i>	Nearest Neighbor Distance
<i>NLM</i>	Non-Local Means
<i>PCA</i>	Principal Analisis Components
<i>PAC</i>	Probably Approximately Correct
<i>PET</i>	Positron Emission Tomography
<i>PLAX</i>	Parasternal eje largo
<i>POCUS</i>	Point-Of-Care Ultrasound
<i>PSAX</i>	Parasternal eje corto
<i>PSNR</i>	Peak Signal-to-Noise Ratio
<i>RBM</i>	Restricted Boltzmann Machine
<i>ReLU</i>	Rectified Linear Unit
<i>RNNs</i>	Recurrent Neural Networks
<i>$\mathcal{D}$</i>	Discriminator
<i>$\mathcal{D}_{\text{loss}}$</i>	Discriminator Loss
<i>SGD</i>	Stochastic Gradient Descent
<i>SL</i>	Válvulas semilunares

Abreviatura	Término
<i>SVM</i>	Support Vector Machines
<i>Spade – GAN</i>	Spatially-Adaptive DE-normalization Generative Adversarial Network
<i>SRM</i>	Structural Risk Minimization
<i>SSIM</i>	Structural Similarity Index Measure
<i>StyleGAN</i>	Style-Based Generative Adversarial Network
s_i	Vector de Estilos
$t – SNE$	t-distributed Stochastic Neighbor Embedding
<i>TTE</i>	Transthoracic Echocardiogram
<i>TVDs</i>	Total Variation Distances
<i>US</i>	Ultra Sound
<i>VAE</i>	Variational Autoencoder
<i>VD</i>	Ventrículo derecho
<i>VI</i>	Ventrículo izquierdo
<i>VM</i>	Válvula Mitral
<i>WGAN</i>	Wasserstein Generative Adversarial Network
$\mathcal{W}$	Espacio de Estilos
$\mathcal{Z}$	Espacio Latente

Resumen

Generación de imágenes de ecocardiogramas mediante estrategias de aprendizaje generativo

La generación realista de ecocardiogramas con redes generativas adversarias representa un avance importante en la simulación médica, ya que es una de las técnicas más usada en el ámbito clínico pero de las menos representadas en *datasets* abiertos, estableciendo una necesidad de disponer de mas data que facilite el entrenamiento de modelos de inteligencia artificial, capaces de identificar patologías cardíacas, predecir sintomatologías, reducir costos de adquisición y mantener la privacidad de los pacientes, ante la limitación de aplicar técnicas tradicionales de data augmentation en imágenes cardíacas — por el riesgo de distorsionar la anatomía y generar patrones irreales —, las redes generativas adversariales emergen como una alternativa sólida para ampliar conjuntos de datos con coherencia estructural.

Este trabajo evaluó comparativamente cuatro arquitecturas generativas — StyleGAN2-ADA, MedGAN, WGAN y VQ-GANs — empleando el conjunto de datos EchoNet-Dynamic, tras un proceso exhaustivo de limpieza y selección de fases cardíacas relevantes, se entrenaron los modelos bajo 8 configuraciones de hiperparámetros diferentes, para determinar el impacto de diferentes combinaciones en la eficiencia de cada arquitectura, estabilidad y capacidad de generación fiel de imágenes ecocardiográficas, se utilizaron métricas de distribución, métricas de similitud perceptual y el comportamiento de las funciones de pérdida.

Los resultados demostraron que StyleGAN2-ADA fue la arquitectura con el desempeño más robusto, generando las imágenes más estables y morfológicamente fieles en una corta cantidad de tiempo, MedGAN presentó un rendimiento cercano y consistente, aunque con menor diversidad que StyleGAN, en contraste, WGAN evidenció un desempeño bajo, con problemas de estabilidad y baja calidad perceptual.

Palabras clave: GANs, Aprendizaje Automático, Aprendizaje Profundo, Ecocardiograma, Aumento de Datos, Distancia de Inception de Fréchet, Distancia de Inception del Kernel, Divergencia de Jensen-Shannon

Abstract

Echocardiogram image generation using generative learning strategies

The realistic generation of echocardiographic images using generative adversarial networks represents a significant advancement in medical simulation, particularly given that echocardiography is one of the most frequently used clinical imaging modalities yet remains underrepresented in publicly available datasets. This limitation highlights the need for synthetic data capable of supporting the training of artificial intelligence models for cardiac pathology detection, symptom prediction, cost reduction, and patient privacy preservation. Because traditional data-augmentation methods may distort cardiac anatomy or introduce unrealistic patterns, generative models emerge as a suitable alternative for expanding datasets while preserving structural coherence.

This study presents a comparative evaluation of four generative architectures—StyleGAN2-ADA, MedGAN, WGAN, and VQ-GAN with Transformer—trained on the EchoNet-Dynamic dataset following an extensive preprocessing and cardiac-phase selection pipeline. Each model was trained under eight different hyperparameter configurations to assess the influence of these variations on training efficiency, stability, and fidelity of the generated echocardiographic images. Performance was measured using distribution-based metrics, perceptual similarity metrics, and the behavior of the loss functions.

The results show that StyleGAN2-ADA achieved the most robust performance, producing the most stable and morphologically accurate images with comparatively short training times. MedGAN showed competitive and consistent results, albeit with lower diversity, in contrast, WGAN demonstrated weak performance, with stability issues and poor perceptual quality.

Keywords: GANs, Machine Learning, Deep Learning, Echocardiogram, Data Augmentation, Fréchet Inception Distance, Kernel Inception Distance, Jensen–Shannon Divergence

Lista de figuras

3-1	Diagrama ANNs.	31
3-2	Diagrama FullyConnected	35
3-3	Diagrama GAN	39
4-1	Diagrama simplificado StyleGAN2-ADA	51
4-2	Diagrama detallado StyleGAN2-ADA.	58
4-3	Diagrama inicial MedGAN	61
4-4	Diagrama definitivo MedGAN	63
4-5	Diagrama básico VQGAN.	67
5-1	Ecocardiograma con tinte azulado pasada a escala de grises.	71
5-2	Combinación de mascara triangular con mascara de semi elipse.	72
5-3	Frames binarizados del video 0X102C51641C321436 en los momentos en que se detecto diástole y sístole.	73
6-1	FID-S-LP.	117
6-2	FID-S-Violin	118
6-3	KID-S-LP	119
6-4	KID-S-Violin	120
6-5	JSD-S-LP	121
6-6	JSD-S-Violin	122
6-7	SSIM-S-LP	123
6-8	SSIM-S-Violin.	124
6-9	PSNR-S-LP.	125
6-10	PSNR-S-Violin	126
6-11	LPIPS-S-LP	127
6-12	LPIPS-S-Violin.	128
6-13	FID-M-LP.	133
6-14	FID-M-Violin	134
6-15	KID-M-LP	135

**Generación de imágenes de ecocardiogramas mediante estrategias de aprendizaje
generativo**

6-16	KID-M-Violin	136
6-17	JSD-M-LP.	137
6-18	JSD-M-Violin	137
6-19	SSIM-M-LP	139
6-20	SSIM-M-Violin	139
6-21	PSNR-M-LS	140
6-22	PSNR-M-Violin	140
6-23	LPIPS-M-LS.	141
6-24	ˆLPIPS-M-Violin	142
6-25	FID-W-LP.	146
6-26	FID-W-Violin	147
6-27	KID-W-LP	148
6-28	KID-W-Violin	149
6-29	JSD-W-LP	150
6-30	JSD-W-Violin	151
6-31	SSIM-W-LP	152
6-32	SSIM-W-Violin.	152
6-33	PSNR-W-LS.	153
6-34	PSNR-W-Violin	154
6-35	LPIPS-W-LS.	155
6-36	LPIPS-W-Violin	156
6-37	FID-VQ-LP	158
6-38	FID-VQ-Violin	159
6-39	SSIM-VQ-LP	160
6-40	SSIM-VQ-Violin	160
6-41	PSNR-VQ-LP	161
6-42	PSNR-VQ-Violin	162
6-43	LPIPS-VQ-LP	163
6-44	LPIPS-VQ-Violin.	163
6-45	Muestras sintéticas generadas por StyleGAN2-ADA (Configuración E4) . . .	172
6-46	Matrices de confusión de cada evaluador	176
6-47	Accuracy	178
6-48	Fooling	180

Lista de tablas

4-2	Comparison Models	69
5-1	StyleGAN-hiper	76
5-2	MedGAN-hiper	79
5-3	WGAN-hiper	81
5-4	VQGAN-hiper.	84
5-5	VQGAN-Transformer-hiper	86
5-6	VQGAN-Transformer-hiper	86
5-7	Umbrales orientativos del (FID) para la evaluación de modelos GAN en imágenes médicas, los rangos son aproximados y deben interpretarse en función del dominio, el extractor de características y el número de muestras utilizadas en el cálculo.. . . .	107
5-8	Umbrales orientativos adoptados para KID en evaluación de modelos GAN (heurísticos, no universales, estos valores son heurísticos y deben usarse solo para comparar modelos dentro del mismo experimento con las mismas condiciones, mismo extractor, mismo número de muestras, mismo <i>kernel</i> / espacio).	108
5-9	Umbrales orientativos adoptados para JSD en evaluación de modelos GAN (heurísticos, no universales).	109
5-10	Umbrales orientativos adoptados para PSNR en evaluación de modelos GAN, adaptación y valor que fue ajustado para GANs sin correspondencia de imagen directa.110	
5-11	Umbrales orientativos adoptados para SSIM en evaluación de modelos GAN, adaptación y valor que fue ajustado para GANs sin correspondencia de imagen directa.110	
5-12	Umbrales orientativos adoptados para LPIPS en evaluación de modelos GAN, adaptación y valor que fue ajustado para GANs sin correspondencia de imagen directa.110	
6-2	StyleGAN-losses-exp	115
6-3	Mínimos, máximos y promedios de FID registrados para los diferentes experimentos de StyleGAN2-ADA, de mejor a peor.. . . .	118
6-4	Mínimos, máximos y promedios de KID registrados para los diferentes experimentos de StyleGAN2-ADA, de mejor a peor.. . . .	120
6-5	Mínimos, máximos y promedios de JSD registrados para los diferentes experimentos de StyleGAN2-ADA, de mejor a peor.. . . .	122

6-6	Mínimos, máximos y promedios de SSIM registrados para los diferentes experimentos de StyleGAN2-ADA, de mejor a peor..	124
6-7	Mínimos, máximos y promedios de PSNR registrados para los diferentes experimentos de StyleGAN2-ADA, de mejor a peor..	126
6-8	Mínimos, máximos y promedios de LPIPS registrados para los diferentes experimentos de StyleGAN2-ADA, de mejor a peor..	128
6-10	MedGAN-losses-exp.	130
6-11	Mínimos, máximos y promedios de FID registrados para los diferentes experimentos de MedGAN, de mejor a peor.	134
6-12	Mínimos, máximos y promedios de KID registrados para los diferentes experimentos de MedGAN, de mejor a peor.	136
6-13	Mínimos, máximos y promedios de JSD registrados para los diferentes experimentos de MedGAN, de mejor a peor.	138
6-14	Mínimos, máximos y promedios de SSIM registrados para los diferentes experimentos de MedGAN, de mejor a peor.	138
6-15	Mínimos, máximos y promedios de PSNR registrados para los diferentes experimentos de MedGAN, de mejor a peor.	141
6-16	Mínimos, máximos y promedios de LPIPS registrados para los diferentes experimentos de MedGAN, de mejor a peor.	142
6-18	WGAN-losses-exp	143
6-19	Mínimos, máximos y promedios de FID registrados para los diferentes experimentos de WGAN, de mejor a peor..	147
6-20	Mínimos, máximos y promedios de KID registrados para los diferentes experimentos de WGAN, de mejor a peor..	149
6-21	Mínimos, máximos y promedios de JSD registrados para los diferentes experimentos de WGAN, de mejor a peor..	151
6-22	Mínimos, máximos y promedios de SSIM registrados para los diferentes experimentos de WGAN, de mejor a peor..	153
6-23	Mínimos, máximos y promedios de PSNR registrados para los diferentes experimentos de WGAN, de mejor a peor..	154
6-24	Mínimos, máximos y promedios de LPIPS registrados para los diferentes experimentos de WGAN, de mejor a peor.	156
6-26	VQGAN-losses-exp	157
6-27	Mínimos, máximos y promedios de FID registrados para los diferentes experimentos de VQ-GAN, de mejor a peor.	159
6-28	Mínimos, máximos y promedios de SSIM registrados para los diferentes experimentos de VQ-GAN, de mejor a peor.	161
6-29	Mínimos, máximos y promedios de PSNR registrados para los diferentes experimentos de VQ-GAN, de mejor a peor.	162

Generación de imágenes de ecocardiogramas mediante estrategias de aprendizaje generativo

6-30	Mínimos, máximos y promedios de LPIPS registrados para los diferentes experimentos de VQ-GAN, de mejor a peor.	162
6-31	Diagramas del mejor valor obtenido por cada configuración en cada métrica para las 4 diferentes arquitecturas.	164
6-32	Top mejores configuraciones para cada modelo de GANs.. . . .	165
6-33	Kings para configuración E3 y E4 donde tuvieron sus mejores resultados según cada métrica.	166
6-34	Resultados de imágenes generadas en diferentes kings a medida que avanza el entrenamiento del StyleGAN2-ADA	167
6-35	Épocas para configuración M2 y M3 donde tuvieron sus mejores resultados según cada métrica.	168
6-36	Épocas para configuración W8 y W6 donde tuvieron sus mejores resultados según cada métrica.	168
6-37	Resultados de imágenes generadas en diferentes épocas a medida que avanza el entrenamiento de MedGAN	169
6-38	Resultados de imágenes generadas en diferentes épocas a medida que avanza el entrenamiento de WGAN	170
6-39	Épocas para configuración VQ6 y VQ1 donde tuvieron sus mejores resultados según cada métrica.	171
6-40	Resultados de imágenes generadas en diferentes épocas a medida que avanza el entrenamiento de WGAN	171
6-42	Nivel 1 – Evaluación binaria, pares de imágenes compuestos por una imagen real y una sintética, la columna “Real” indica la posición correcta.	174
6-43	Nivel 2 – Conjuntos de cuatro imágenes con dos reales y dos sintéticas, se indica la distribución correcta (R = real, F = sintética).	175
6-44	Nivel 3 – Conjuntos de cuatro imágenes con una real y tres sintéticas, se indica la posición correcta (R = real, F = sintética)..	175

Contenido

Agradecimientos	II
Listado de símbolos y abreviaturas	IV
Resumen	VII
Abstract	VIII
Lista de figuras	IX
Lista de tablas	XI
Contenido	XIV
1 Introducción	1
1.1 Planteamiento del problema	6
1.1.1 Identificación	6
1.2 Objetivos	6
1.2.1 Objetivo General	6
1.2.2 Objetivos Específicos	6
2 Estado del Arte	8
2.1 Limitaciones de los <i>datasets</i>	9
2.2 Aplicaciones de GANs en ecocardiografías	10
2.2.1 StyleGAN en el dominio medico	10
2.2.2 Pix2Pix en denoising y mejora de calidad	11
2.2.3 MedGAN y sus variantes	11
2.2.4 VQ-GAN en generación de datos diversos y estables	13
2.2.5 CGAN para traducción de dominios y Outpainting	14
2.2.6 WGAN en la eficiencia de la estabilidad	14
2.3 Métricas y protocolos de evaluación	14

Generación de imágenes de ecocardiogramas mediante estrategias de aprendizaje generativo

2.3.1	FID reportados en la literatura	15
2.3.2	KID reportados en la literatura	16
2.3.3	JSD reportados en la literatura	16
2.3.4	SSIM y PSNR reportados en la literatura	17
2.4	Riesgos y consideraciones	18
3	Marco teórico	20
3.1	Ecocardiografía	20
3.1.1	Ciclo cardíaco sístole y diástole	20
3.1.2	Ecocardiografía Transtorácica (TTE) y fundamentos	20
3.1.3	Anatomía cardíaca relevante	21
3.1.4	Dataset EchoNet-Dynamic (Stanford)	23
3.1.5	Patologías detectables en ecocardiografía con procesamiento de imagen y <i>deep learning</i>	24
3.2	Fundamentos del Machine Learning	26
3.2.1	Paradigmas principales de ML	27
3.3	<i>Deep Learning</i>	30
3.3.1	Principales Arquitecturas de Deep Learning	34
3.4	Generative Adversarial Networks (GANs)	39
3.4.1	Problemas Fundamentales en GANs	42
3.5	Métricas de evaluación de imágenes sintéticas	44
3.5.1	Métricas basadas en distancias estadísticas de distribuciones	45
3.5.2	Métricas basadas en similitud imagen a imagen	47
4	Arquitecturas GANs	51
4.1	StyleGAN2-ADA	51
4.1.1	StyleGAN (v1)	52
4.1.2	StyleGAN2	53
4.1.3	StyleGAN2-ADA	56
4.2	MedGAN	60
4.3	Wasserstein Generative Adversarial Network (WGAN)	64
4.4	Vector-Quantized GAN (VQ-GAN)	66
4.5	Comparación entre diferentes modelos GANs	69
5	Metodología	70
5.1	Preprocesamiento	70
5.2	Configuraciones experimentales	74
5.2.1	Configuraciones base del setup	74

Generación de imágenes de ecocardiogramas mediante estrategias de aprendizaje generativo	
5.2.2	Hiperparámetros StyleGAN2-ADA 75
5.2.3	Hiperparámetros MedGAN 77
5.2.4	Hiperparámetros WGAN 80
5.2.5	Hiperparámetros VQ-GAN + <i>Transformer</i> 82
5.3	Construcción y adaptación de las arquitecturas 87
5.3.1	Implementación StyleGAN2-ADA 87
5.3.2	Implementación MedGAN 90
5.3.3	Implementación WGAN 96
5.3.4	Implementación VQ-GAN + <i>Transformer</i> 100
5.4	Construcción de evaluación de métricas 107
5.4.1	Evaluación perceptual por especialistas médicos 112
6	Resultados y Análisis 114
6.1	Resultados preliminares de las 8 configuraciones 115
6.1.1	Resultados en configuraciones de StyleGAN2-ADA 115
6.1.2	Resultados en configuraciones de MedGAN 130
6.1.3	Resultados en configuraciones de WGAN 143
6.1.4	Resultados en configuraciones de VQGAN 157
6.2	Mejores configuraciones de cada arquitectura 164
6.3	Muestras sintéticas generadas por el mejor modelo 172
6.4	Resultados de la evaluación perceptual por especialistas 174
6.4.1	Análisis de matrices de confusión por especialista 176
6.4.2	Análisis de la exactitud por especialista 178
6.4.3	Análisis del <i>Fooling Rate</i> 180
6.4.4	Análisis integrador de resultados y consideraciones clínicas de los evaluadores 181
7	Conclusiones 183
	Referencias Bibliográficas 187

1 Introducción

La generación de imágenes médicas sintéticas de alta calidad ha tenido un crecimiento considerable en la integración en múltiples campos de aplicaciones biomédicas, especialmente en contextos donde la recopilación de datos reales resulta compleja, limitada o éticamente delicada, en este marco, la integración de técnicas de aprendizaje automático (*Machine Learning, ML*) y aprendizaje profundo (*Deep Learning, DL*) para detección de características especiales, anomalías o generación de nuevos datos, ha permitido avances notables en múltiples áreas medicas.

En *endoscopia*, los métodos basados en DL se han utilizado en gran medida para la extracción de características, el diagnóstico de enfermedades y la mejora de la calidad de las imágenes [1]; en el diagnóstico de *neumonía* a partir de radiografías de tórax, se han implementado modelos como VGG16, ResNet-50 e Inception V3, así como redes residuales preentrenadas de 50 capas, para clasificar y detectar la presencia de esta patología [2]; en el *análisis de retina* la extracción de puntos característicos y su posterior clasificación mediante máquinas de vectores de soporte (*Support Vector Machines, SVM*), k-Nearest Neighbors y k-Means ha permitido diferenciar retinas sanas de aquellas afectadas por enfermedades [3]; en la clasificación de *tumores cerebrales* y la detección de Alzheimer, a partir de imágenes de resonancia magnética, se han comparado arquitecturas convolucionales basadas en *transfer learning* como VGG-19, ResNet-50, DenseNet-201 e InceptionV3, demostrando que la incorporación de redes generativas adversariales (*Generative Adversarial Networks, GANs*) posibilita un aumento de datos eficaz y una mejora sustancial de las métricas [4]; en la identificación de diferentes tipos de tejido, donde tres de estos son de tipo cancerosos en pulmón y colón, se ha aplicado aprendizaje supervisado basado en redes convolucionales multicanal tras la concatenación de características [5]; también se ha implementado técnicas IA para la detección de *cáncer de mama* mediante la combinación de CNN con la arquitectura BreastNet [6] y para la detección de tumores pulmonares utilizando el modelo LTD-CNN [7].

De particular relevancia para este trabajo, el análisis de imágenes ecocardiográficas con registros dinámicos del corazón obtenidos mediante técnicas de ultrasonidos, constituyen una herramienta diagnóstica de primer nivel en la cardiología moderna, gracias a su carácter no invasivo, su capacidad para revelar estructuras cardíacas en tiempo real y su utilidad para monitorear diversas patologías, lo que complementa la labor médica, aumenta la precisión diagnóstica y reduce los tiempos de intervención [8], estas y otras aplicaciones médicas se han

visto beneficiadas por la incorporación de diferentes técnicas avanzadas de procesamiento de imágenes apoyadas en diferentes arquitecturas de redes neuronales con propósitos variados, dichos enfoques permiten clasificar patrones latentes presentes en las imágenes, comprender las características que las componen de manera individual y conjunta, así como identificar la relación que se da entre todas sus características, todo esto con el propósito de detectar patologías, predecir la presencia de condiciones malignas o incluso mejorar la calidad de imágenes existentes mediante la implementación conjunta de dos redes neuronales, lo que amplía la posibilidad y hasta necesidad de generación de nuevos conjuntos de datos que preservan la estructura morfológica apropiada para su uso en necesidades posteriores.

En la investigación biomédica, las redes neuronales profundas han ganado gran relevancia debido a su capacidad para analizar imágenes medicas con un alto nivel de rendimiento, según Drukker et al. [9], estas técnicas han mostrado un notable impacto en numerosas aplicaciones médicas, desde la segmentación y clasificación de imágenes hasta la predicción de resultados clínicos, a diferencia de los métodos tradicionales que requieren un análisis e intervención de conocimiento experto, el DL permite detectar representaciones complejas directamente a partir de los datos, como lo destacan Balaji et al. [10], este avance ha facilitado el reconocimiento, clasificación y análisis de patrones en imágenes médicas, contribuyendo significativamente al progreso en la medicina personalizada.

El desafío presente en el DL se encuentra en la limitada disponibilidad de datos médicos de alta calidad que dificulta enormemente el entrenamiento de modelos robustos. Un gran conjunto de datos permite un apropiado entrenamiento reduciendo el desequilibrio de clases, los sesgos, mejorando la generalización y evitando el sobreajuste, pero la dificultad de adquisición de este tipo de datos a nivel de tiempo, dinero, recursos humanos junto a diferentes normas éticas, dificultan enormemente la adquisición o creación de *dataset* robustos para entrenamientos en IA y se convierte en un gran obstáculo para la aplicación de redes neuronales profundas, este inconveniente ha llevado a la búsqueda del desarrollo de técnicas de aumento de datos (*Data Augmentation*) para incrementar la cantidad y diversidad de datos [11].

Técnicas típicas de aumento de datos no pueden ser implementadas en imágenes médicas, técnicas tales como transformaciones afines, transformaciones de borrado, transformaciones elásticas o transformaciones a nivel de pixel, ya que es necesario mantener la integridad de la morfología del corazón, pequeños cambios que afecten la estructura crearían imágenes erróneas con características inexistentes o patrones inconsistentes que darían lugar a conjeturas erróneas, cambios espaciales tales como traslaciones, rotaciones, escalamientos, deformaciones, etc, afectarían la lógica original y real de la toma de datos realizadas en un dispositivo determinado, por ende las técnicas típicas básicas de aumento de datos no son adecuadas.

En este contexto, las GANs han surgido como una alternativa sólida para la expansión sintética de conjuntos de datos médicos, ofreciendo el potencial de generar imágenes indistinguibles de las reales: Introducidas por Goodfellow et al. en 2014 [12], las GANs consisten en una arquitectura dual compuesta por dos redes neuronales, un generador (*Generator*, $\mathcal{G}$) y un discriminador (*Discriminator*, $\mathcal{D}$) que compiten constantemente, el generador busca producir datos sintéticos convincentes, mientras que el discriminador intenta diferenciar

1. Introducción

entre datos reales y generados, a través de este entrenamiento adversarial, se alcanza un equilibrio en el que el generador puede sintetizar ejemplos altamente realistas.

Pese a su rápida y amplia acogida, la implementación de GANs en imágenes médicas plantea desafíos complejos, las imágenes clínicas no solo deben verse convincentes, capaces de engañar al ojo humano, sino que deben preservar patrones estructurales y morfológicos clave, como se mencionó antes, cualquier distorsión o hasta diferencia de ruido podría inducir errores en sistemas diagnósticos automatizados o sesgos en modelos predictivos, por ello, el diseño de arquitectura y la selección de funciones de pérdida empleadas en GANs médicas requieren una evaluación cuidadosa, incluyendo el uso de métricas perceptuales, regularización morfológica, y codificadores latentes clínicamente interpretables [13].

En el caso particular del corazón, el análisis ecocardiográfico presenta una complejidad singular que lo diferencia de otros órganos la cual es su resolución temporal, dado que no basta con limitarse a una visión estática, sino que puede ampliarse su comprensión por medio del análisis de sus distintas fases y estados funcionales producto del movimiento cardíaco, como la diástole y la sístole [14], estas variaciones ecocardiográficas muestran una relación significativa con aspectos demográficos y clínicos como la edad, el sexo, la presión arterial, diversidad, estadística de la población, tal como lo evidencian estudios recientes [15].

Por ejemplo, Carvalho et al. [16] reportaron que la relación E/A — que refleja el equilibrio entre el llenado temprano (onda E) y el llenado tardío por contracción auricular (onda A) — disminuye con la edad y que la relación E/e' , un indicador de las presiones de llenado del ventrículo izquierdo, aumenta con la edad siendo más alta en mujeres que en hombres; de manera similar, el Rotterdam Study identificó que los parámetros estructurales cardíacos son mayores en hombres y están asociados con la edad, el índice de masa corporal y la presión arterial, mientras que la relación E/A era consistentemente menor en mujeres y variaba según edad y presión arterial diastólica [17], además, un estudio sobre población saludable mostró diferencias en parámetros diastólicos — como la velocidad de llenado temprano (E), la relación E/A y la velocidad de relajación miocárdica e' — entre hombres y mujeres menores de 50 años, que desaparecen en grupos de edad mayores [18].

Todas estas variabilidades según grupos demográficos puede cambiar teniendo correlaciones diferentes, sin embargo, en gran parte del mundo la disponibilidad de datos ecocardiográficos sigue siendo muy limitada, lo que representa un desafío importante para el entrenamiento de modelos de IA y aún más enfocadas para grupos poblaciones específicos, frente a este panorama, el presente trabajo propone estrategias de aumento de datos por medio de diferentes GANs que permitan generar nuevas imágenes cardíacas sintéticas a partir de la información disponible, manteniendo la coherencia estructural, la distribución del espacio donde han sido tomados los datos, el ruido existe y los aspectos morfológicos relevantes.

Para ello, se plantea una evaluación rigurosa que inicia con el preprocesamiento exhaustivo del conjunto de datos EchoNet-Dynamic de la Universidad de Stanford, donde los datos son tratados y limpiados para el correcto funcionamiento de los GANs, en ML si la data usada para el entrenamiento es basura, por más eficiente u óptimo que sea el algoritmo no será

capaz de generar data apropiada, la limpieza de los datos comprende la uniformidad, donde todos los datos son convertidos a escala de grises, evitando distorsión en la distribución de entrenamiento que pueda ocasionar colapsos e dispersión de la distribución, a su vez son reescalados a un tamaño de 128x128 pixeles de manera que tengan una resolución en potencia de dos, ideales para el eficiente funcionamiento de los GANs, estos a su vez son normalizados de $[-1,1]$ garantizando la estabilización de los modelos, es usada una máscara de segmentación para la eliminación de ruido, son implementadas diferentes técnicas de procesamiento de imágenes para la eliminación de ruido producto del reescalamiento y por último se hace un proceso de binarización y conteo de pixeles para seleccionar los momentos o fases más relevantes del corazón enriqueciendo el *dataset* implementado en el entrenamiento.

Posteriormente, estas representaciones finales se emplean en GANs configuradas con distintos hiperparámetros, con el propósito de identificar aquellas configuraciones que permitan no solo una reproducción morfológica de alta fidelidad, sino también un análisis más detallado de las características cardíacas, contribuyendo así a la generación de nuevos conjuntos de datos ecocardiográficos aplicables y ampliables en investigación clínica, este paso es fundamental ya que los GANs son muy sensibles a la configuración inicial de hiperparámetros, aunque el modelo sea bueno, si la configuración no es apropiado puede estallar los gradientes, es por eso que en el presente trabajo se realiza una observación exhaustiva para determinar qué configuraciones son ideales y determinar la razón por la que se da la máxima eficiencia o el colapso de modos. Es así como se pretende que en trabajos futuros al fusionar datos sintéticos con datos de imágenes reales para entrenamientos variados de DL aumente la heterogeneidad de ese nuevo *dataset*, lo que mejora el rendimiento creando modelos entrenados más eficientes por tener una mayor exposición de variedad de imágenes congruentes.

En este estudio 4 arquitecturas de GANs fueron seleccionadas de acuerdo a su diversidad conceptual, amplio uso en la literatura o características posiblemente innovadoras, para evaluar su desempeño en un escenario controlado con el mismo *dataset*, mismas métricas de rendimiento y evaluación, pero variación de parámetros e hiperparámetros, entre las arquitecturas a evaluar esta StyleGAN2-ADA desarrollado por Karras et al. [19] destaca por su realismo y diversidad sin necesidad de imágenes pareadas — imágenes con una correspondiente directa —, es un modelo no supervisado que introduce un control explícito del estilo en diferentes niveles de la imagen, tiende a ser uno de los modelos más estables gracias al uso de técnicas de aumento del discriminador adaptativo (*Adaptive Discriminator Augmentation, ADA*) que mitigan el sobreajuste, aun que genera imágenes altamente realistas, su control estructural es menos preciso, lo que puede introducir variabilidad morfológica indeseada, en cambio VQ-GAN destaca ampliamente donde StyleGAN2-ADA es más débil ya que ofrece un balance entre fidelidad visual y estructura anatómica, presentado por Esser et al. [20] introduce un enfoque basado en codificación latente discreta mediante vector quantization, esta arquitectura combina la capacidad de un *autoencoder* para mapear imágenes a un espacio latente comprimido con la potencia de una GAN para refinar la calidad perceptual de las reconstrucciones, sin embargo, es una arquitectura altamente exigente ya que va acompañada de un *Transformer*, aunque tiene capacidad para generar datos variados y realistas tiene una exigencia de entre 10 a 15 veces mayor a el resto de arquitecturas, requiriendo una capacidad computacional excesiva y más tiempo de entrenamiento.

1. Introducción

MedGAN tiene una orientación medica mayor donde la coherencia estructural es prioritaria, introducido inicialmente por Choi et al. [21], tiene un enfoque que combina un *autoencoder* con una GAN adversarial, lo que permite capturar representaciones latentes compactas y generar imágenes sintéticas a partir de ellas, el término MedGAN no obedece a un solo autor ni una sola arquitectura y la primera mención realizada funciona para datos tabulares, la versión usada en este proyecto está más vinculado al modelo propuesto por Guo et al. [22] con una arquitectura apropiada para trabajar en imágenes, su ventaja es su capacidad de preservar la distribución estadística de los datos médicos originales, logrando un balance entre variabilidad y fidelidad anatómica debido al uso de representaciones latentes médicamente informadas, aunque puede ser menos potente en la generación de detalles de alta resolución.

La implementación de las arquitecturas seleccionadas serán evaluadas mediante métricas cuantitativas, estas se desglosan entre métricas que calculan la distribución como, la distancia de inception de Fréchet (*Fréchet Inception Distance, FID*), la distancia de inception del kernel (*Kernel Inception Distance, KID*), y la divergencia de Jensen-Shannon (*Jensen-Shannon Divergence, JSD*); además de las métricas basadas en similitud de imagen a imagen tales como, la relación señal-ruido máxima (*Peak Signal-to-Noise Ratio, PSNR*), la medida de índice de similitud estructural (*Structural Similarity Index Measure, SSIM*) y la similitud de parches de imágenes perceptuales aprendidas (*Learned Perceptual Image Patch Similarity, LPIPS*); todas estas métricas con el fin de estimar el nivel de fidelidad morfológica y perceptual de las imágenes sintéticas generadas.

Paralelamente, se analiza la evolución de la función de pérdida del $\mathcal{G}$ y del $\mathcal{D}$ como indicador de la estabilidad del entrenamiento, la capacidad de generalización del modelo y la detección de posibles casos de memorización, a partir de estos análisis será posible identificar qué arquitectura resulta más efectiva para la generación de imágenes ecocardiográficas sintéticas de alta calidad, así como determinar la configuración de parámetros e hiperparámetros más adecuada, o bien reconocer los puntos fuertes y limitaciones particulares de cada enfoque comparado.

Finalmente, parte de este trabajo fue presentada en el SIPAIM 2025 en la ponencia titulada “*Comparative Study of Methods for Generating Echocardiographic Images*”, esta contribución hace parte del proyecto “Estimación del trabajo cardíaco como índice de función cardiovascular en videos ecocardiográficos”, financiado por la convocatoria SUE Distrito Capital 2023 (código 60946) y por la Universidad Militar Nueva Granada (código EXT_ING 4073), y se desarrolla de forma más completa y ampliada en la presente tesis.

1.1 Planteamiento del problema

1.1.1 Identificación

La principal problemática radica en la limitada disponibilidad de ecocardiogramas que presenten patologías cardíacas visibles, lo que dificulta el desarrollo de modelos de inteligencia artificial (IA) robustos y efectivos para el análisis y diagnóstico, esta carencia de datos afecta tanto la capacidad de estudiar a profundidad estas imágenes como la posibilidad de entrenar modelos capaces de identificar de manera precisa las enfermedades cardíacas más relevantes. La solución a este problema no solo permitiría analizar las sintomatologías, sino también predecir posibles riesgos de enfermedades presentes en el corazón en el futuro, al contar con un conjunto de datos más amplio y variado, sería posible desarrollar modelos de IA que apoyen a los profesionales en la identificación temprana y en la predicción del riesgo de enfermedades cardíacas, contribuyendo así a mejorar la prevención, diagnóstico y tratamiento.

Se pueden identificar los siguientes desafíos en la implementación de GANs para el análisis Ecocardiográfico, como la escasez de imágenes ECO para el entrenamiento de algoritmos IA, donde el numero de imágenes disponibles en base de datos existentes de imágenes y como ampliarlas, ademas de las patologías presentes en los diferentes videos Ecos disponibles; otro desafío a abarcar son las dificultades en la precisión del diagnóstico automatizado, como la tasa de error en algoritmos de IA para diagnósticos, los problemas de sobreajuste que se puedan presentar y determinar cuales son las métricas de rendimiento mas eficientes para determinar las mejores imágenes generadas.

1.2 Objetivos

1.2.1 Objetivo General

Determinar el modelo GANs más efectivo para la generación de imágenes ecocardiográficas sintéticas, con el fin de contribuir a la ampliación y diversificación de *datasets* en el análisis cardíaco.

1.2.2 Objetivos Específicos

- Explorar un *dataset* amplio, representativo y confiable de imágenes ecocardiográficas, estudiar su composición, morfología y características clave para replicar en la generación

de nuevos datos sintéticos.

- Analizar el desempeño de diferentes arquitecturas GANs en la generación de imágenes ecocardiográficas, utilizando imágenes reales como base para el entrenamiento.
- Evaluar diferentes configuraciones de parámetros e hiperparámetros en cada arquitectura GAN, para así determinar la combinación mas adecuada en la generación de nuevas imágenes sintéticas morfológicamente excelentes.
- Generar un conjunto de datos ECO a partir de los métodos generativos que tuvieron el mejor desempeño y hacer una evaluación con un medico profesional experto.

2 Estado del Arte

La aplicación de GANs a la imagenología cardiovascular ha crecido rápidamente en los últimos 8 años como respuesta directa a la escasez de datos anotados y a la gran variabilidad técnica y operativa de las ecocardiografías, las limitaciones más citadas —cantidad limitada de pacientes y vistas, diferencias entre equipos y operadores, y presencia de ruido tipo *speckle*— hacen que los conjuntos de datos reales a menudo no sean suficientes ni representativos para entrenar modelos robustos de clasificación y segmentación, por ello, la literatura propone el uso de GANs tanto para generar imágenes sintéticas realistas que amplíen y balanceen *datasets*.

Tal como lo proponen Skandarani et al. [23] aunque no se hace una comparación cuantitativa robusta, se hace de manera superficial una comparación de resultados de 6 diferentes arquitecturas GANs, seleccionadas según: la adopción e implementación en generación de imágenes, según la calidad generada en estas arquitecturas, de manera que no excedan la capacidad de VRAM de 12 gigabytes y la evolución de estas arquitecturas desde 2016; entre las arquitecturas concretas escogidas se encuentran: *StyleGAN*, considerada de las arquitecturas mas avanzadas permitiendo imágenes de alta calidad y diversidad; *DCGAN*, siendo de las primeras GANs en implementar capas convolucionales, muy utilizada como GAN de referencia; *WGAN* que utiliza Wasserstein como función de perdidas estabilizando el entrenamiento y reduciendo el colapso; *LSGAN* que reemplaza la función de perdida de entropía cruzada por la del error cuadrático medio, reduciendo los problemas de gradientes que desaparecen; *HingeGAN* que utiliza como función de perdida la maximización de margen estabilizando el proceso de optimización; y *Spade-GAN* usado para la traducción de imagen a imagen mejorando el modelo de *Pix2Pix*.

En la investigación de Skandarani et al. [23] se realiza el ajuste de los hiperparámetros para todos los modelos con diferentes configuraciones no mencionadas o detalladas y el uso de GPU equiparable a 200 días de procesamiento, destacando que la mayoría de configuraciones divergieron o produjeron datos muy degradados, para la evaluación de los modelos se implementaron métricas tales como IS, FID, y KID que se acercan a una evaluación humana, mas aun así no se correlaciona con los evaluadores expertos.

2.1 Limitaciones de los *datasets*

La calidad de las anotaciones es un cuello de botella presente en la ecocardiografía, las etiquetas útiles (contornos de cavidades, segmentaciones del ventrículo izquierdo, marcas temporales, mediciones hemodinámicas) requieren experiencia clínica para su creación y son laboriosas de obtener; además, existen convenciones de segmentación y definiciones de contorno que varían entre laboratorios y estudios, lo que introduce heterogeneidad en los *ground truths* y dificulta la comparación directa entre trabajos [24].

A lo anterior se añade la variabilidad técnica entre dispositivos (*inter-vendor*) y operadores (*inter-operator*): diferencias en la electrónica de los transductores, en los *presets* de adquisición, en las configuraciones de ganancia/enfocamiento y en la técnica de exploración producen imágenes con contrastes, texturas y escalas espaciales distintas, estudios que comparan medidas entre distintos proveedores de ecógrafos muestran desviaciones sistemáticas en parámetros funcionales y en la apariencia de las imágenes, lo cual reduce la capacidad de los modelos entrenados en un dominio para generalizar a otro sin adaptación de dominio, esta heterogeneidad operacional es citada repetidamente como una causa de falla en la validación externa de algoritmos de segmentación y clasificación [25].

En el dominio de la señal de ultrasonido, un problema intrínseco y persistente es la formación de moteado y otro tipo de ruidos, en específico el ruido *speckle* —un ruido multiplicativo de naturaleza coherente que da a las imágenes ecocardiográficas una apariencia granulada— y otros artefactos de reconstrucción (sombra acústica, reverberación, *aliasing Doppler*, desenfoque por movimiento), el *speckle* degrada la resolución aparente y el contraste, puede ocultar bordes finos y alterar texturas relevantes para la detección automática, y por ello constituye una dificultad tanto para clínicos como para algoritmos, se han buscado múltiples técnicas de filtrado de medidas no locales (*Non-Local Means*, *NLM*), tales como reducción de ruido de imágenes, la eliminación de ruido en imágenes multiespectrales y el desenfoque de imágenes, pero solo eliminan el ruido y no mejoran la calidad de las imágenes siendo ineficaces; la bibliografía técnica y trabajos de procesamiento de ultrasonido documentan cómo las técnicas de *despeckling* clásicas y modernas intentan mitigar estos efectos, pero muchas de ellas implican una pérdida de resolución o requieren pares imagen–imagen difíciles de obtener en condiciones reales [26].

Otro aspecto frecuente es el sesgo de clases y de población: algunas patologías, vistas o fenotipos son raros, lo que produce *datasets* desbalanceados donde las clases patológicas (e.g., cardiopatías poco prevalentes) están poco representadas, este desbalance incentiva a que los modelos aprendan a privilegiar la clase mayoritaria (imágenes normales) y falla en detectar o segmentar correctamente casos clínicamente relevantes pero menos frecuentes [27].

Frente a estos problemas, las GANs y otros métodos generativos se proponen como solución para generar imágenes sintéticas que amplíen la variabilidad de los conjuntos de entrenamiento y, en algunos casos, produzcan pares imagen–máscara cuando estas no existen o son

costosas de anotar, trabajos concretos han usado *pipelines* basados en GANs para sintetizar ecocardiogramas 2D y 3D emparejados con etiquetas anatómicas, demostrando que los datos sintéticos pueden aumentar el rendimiento de segmentadores y mejorar la generalización cuando se combinan con imágenes reales en el entrenamiento [28].

Sin embargo, la augmentación sintética no es una “solución milagrosa” y plantea desafíos propios: las imágenes generadas pueden introducir anatomías implausibles o artefactos sutiles que enmascaren sesgos o produzcan falsas correlaciones; por ello, la literatura recomienda evaluar los sintéticos no solo con métricas visuales (FID, SSIM) sino con pruebas de utilidad clínica —e.g., medir el impacto en tareas *downstream* (segmentación, clasificación) y someter las imágenes a valoración por expertos—, además, se sugiere cuidar la trazabilidad (qué ejemplos son sintéticos), realizar validación por paciente y por dispositivo, y combinar enfoques como generación incondicional para diversidad + GANs condicionados (*Conditional Generative Adversarial Networks*, *cGANs*) para maximizar la utilidad y minimizar riesgos [28].

2.2 Aplicaciones de GANs en ecocardiografías

La generación de ecocardiogramas sintéticos con GANs se ha propuesto como solución directa a la escasez de datos anotados y al desbalance de clases en problemas cardiológicos, trabajos dedicados han demostrado que arquitecturas condicionadas y no condicionadas (e.g., modelos GAN con discriminadores por parches o variantes adaptadas) pueden producir vistas apicales y parasternal realistas que sirven para aumentar la variabilidad de los conjuntos de entrenamiento y para equilibrar clases poco representadas, un método propuesto para resolver esto es implementando la generación de ecocardiogramas a partir de mascarar con segmentaciones específicas de los puntos mas relevantes del corazón, adoptando una ruta determinista sin aleatoriedad en el entrenamiento ni en la inferencia, hace que el generador aprende una correspondencia determinista entre las distribuciones de las imágenes pareadas [29], es importante destacar que en trabajos como este se recomienda evaluar el beneficio real mediante tareas *downstream* más que con métricas visuales aisladas.

2.2.1 StyleGAN en el dominio medico

Entre las arquitecturas concretas aplicadas al dominio médico, tales como tomografías computarizadas (*Computed Tomography*, *CT*) o radiografías de tórax o imagen de resonancia magnética (*Magnetic Resonance Imaging*, *MRI*) o tumores cerebrales, destacan varias familias con roles diferenciados, StyleGAN2-ADA y variantes estilo StyleGAN se han usado cuando el objetivo es producir imágenes estáticas de alta fidelidad y variedad de estilos, mostrando cómo mecanismos de adaptive augmentation ayudan a entrenar con menos imágenes reales reduciendo el sobreajuste en dominios médicos de pequeña muestra, se destaca la métrica

FID como un estándar para la evaluación de imágenes naturales, debido a su capacidad para distinguir entre muestras reales y generadas, concordancia con los juicios perceptuales humanos, la sensibilidad a las distorsiones y su capacidad computacional, sin embargo múltiples publicaciones han mencionado que FID no es apropiado para evaluación en imágenes medicas debido a que no existen en ImageNet, pero se denota que es ampliamente correlacionable a la percepción humana, si FID decae para una imagen de CT, la percepción humana lo nota como reales, determinando una efectividad en su implementación, teniendo en cuenta que FID generalmente solo es útil en imágenes medicas si se utiliza el mismo modelo de codificación y el como el diseñador de la métrica formula su propia métrica de evaluación lo que inducirá a un sesgo no cuantificado; las GANs son sumamente costosas y difíciles de entrenar, son muy sensibles a la inestabilidad y al cambio de hiperparámetros, mostrándose por medio de un estudio independiente que al realizar 1500 configuraciones de hiperparámetros diferentes en 3 conjuntos de datos de imágenes, muy pocos modelos logaban generar imágenes de calidad [30].

2.2.2 Pix2Pix en denoising y mejora de calidad

Los GANs han sido utilizados en la reducción del ruido *speckle* y mejora de calidad (preservación anatómica), una aplicación muy estudiada de cGANs (especialmente pix2pix y variantes condicionadas) en ultrasonido es la reducción de *speckle* y la mejora del contraste sin perder contornos anatómicos relevantes, estudios aplican pix2pix entrenado con pares (imagen ruidosa $\rightarrow$ imagen filtrada/limpia) y reportan mejoras visuales y métricas de similitud estructural frente a filtros tradicionales; además, las GAN tienden a preservar bordes y detalles finos mejor que métodos de *despeckling* clásicos que sobre-suavizan, en ecocardiografía esto se ha explorado tanto en imágenes estáticas como en protocolos que extienden el método a secuencias cine (series dinámicas que capturan el ciclo cardíaco), con el objetivo de mejorar la entrada a *pipelines* automáticos de segmentación o medición funcional [31].

2.2.3 MedGAN y sus variantes

MedGAN y trabajos relacionados se centran en la traducción médica end-to-end (corrección de artefactos, mapeos entre modalidades o dominio $\rightarrow$ dominio), proveen arquitecturas adaptadas a las necesidades clínicas y han servido como linea base en múltiples estudios de imagen médica [32], sin embargo, MedGAN no es una arquitectura única universal, el término MedGAN ha sido utilizado en diferentes contextos más allá de su concepción original, generando múltiples variantes que, aunque comparten la idea de aplicar GANs en el ámbito biomédico, se enfocan en problemas distintos y con arquitecturas adaptadas.

Una de las primeras adaptaciones fue dada por Armanious et al. [33] con un MedGAN para traducción de imágenes médicas, tomó inspiración de modelos de *image-to-image*

translation como *pix2pix*, aplicándolos a la conversión entre modalidades médicas, por ejemplo, resonancias magnéticas $T1 \leftrightarrow T2$ o en $CT \leftrightarrow$ tomografías por emisión de positrones (*Positron Emission Tomography, PET*), la traducción se da por el mapeo de un tensor de entrada de alta dimensión a un tensor de salida que conserva la misma estructura pero tiene una apariencia diferente de estructura, la arquitectura se basa en un generador tipo U-Net, encargado de mapear imágenes de un dominio a otro, y un discriminador tipo PatchGAN, utilizada como extractor de características entrenables que castiga las imágenes traducidas y las modalidades buscadas, evalúa la verosimilitud de las imágenes de salida a nivel local, y utiliza pérdidas por transferencia de estilos para asemejarse a texturas y a las estructuras finas, el objetivo principal de este enfoque fue reducir la dependencia de múltiples adquisiciones médicas costosas, permitiendo traducir una imagen obtenida en una modalidad hacia otra que aporte información complementaria para el diagnóstico.

Posteriormente, Macedo et al. [34] uso el nombre MedGAN aplicandolo al problema del diseño de nuevas moléculas, dando lugar a MedGAN con redes convolucionales de grafos (*Graph Convolutional Networks, GCN*), en este caso, las moléculas se representan como grafos, donde los nodos corresponden a átomos y las aristas a enlaces químicos, la arquitectura combinó GANs capaces de generar grafos válidos con redes convolucionales en grafos GCN, que modelan propiedades estructurales y químicas de las moléculas, el objetivo fue generar compuestos novedosos con propiedades químicas plausibles y potencial terapéutico, contribuyendo al proceso de descubrimiento de fármacos y acelerando la identificación de candidatos en etapas tempranas de investigación.

Otra propuesta relevante fue dada por Armanious et al. [35] llamada Cycle-MedGAN, orientada a la traducción no supervisada de imágenes médicas, inspirada en CycleGAN, esta variante se diseñó para superar la limitación de contar con *datasets* pareados en el ámbito médico, que suelen ser escasos, su arquitectura emplea dos generadores ($A \rightarrow B$ y $B \rightarrow A$) y dos discriminadores, junto con la pérdida de consistencia cíclica (cycle-consistency loss), que garantiza que si una imagen se transforma de un dominio A a otro dominio B y luego de vuelta a A, se recupere la información esencial de la original, y así logre traducir imágenes entre modalidades sin necesidad de pares alineados, lo que constituye una ventaja significativa en contextos clínicos donde la recolección de datos es limitada o costosa.

Finalmente, de Guo et al. [22] surge un MedGAN enfocado en la síntesis de imágenes médicas realistas, este trabajo propone un mecanismo de entrenamiento adaptativo para el discriminador, ajustando dinámicamente su capacidad para evitar problemas de sobreajuste que suelen aparecer en GANs entrenados con conjuntos de datos pequeños, la arquitectura incluye un generador que produce imágenes sintéticas y un discriminador con capacidad adaptativa que evalúa su realismo, el enfoque de esa investigación es alternar el entrenamiento del generador y discriminador apoyándose en las pérdidas de Wasserstein como métrica de convergencia, ese enfoque adaptativo permite que el descenso de gradiente alterno funcione bien logrando mejorar la diversidad y calidad de las imágenes generadas, esta propuesta busca generar *datasets* médicos sintéticos que permitan fortalecer modelos de aprendizaje profundo en tareas de diagnóstico, segmentación o clasificación, especialmente en escenarios donde los datos reales son limitados por razones éticas, legales o de disponibilidad.

2.2.4 VQ-GAN en generación de datos diversos y estables

Como lo explica Khader et al. [36], en los últimos años se ha documentado el uso de arquitecturas VQ-GAN/VQ-VAE en tareas médicas variadas: síntesis y aumento de imágenes (para ampliar *datasets*), compresión y tokenización latente (útil para acelerar generación por modelos posteriores), reconstrucción de alta resolución y generación condicionada a mapas semánticos, la gran mayoría de las arquitecturas GANs generan imágenes en bidimensionales (2D); sin embargo, la mayoría de implementaciones y diagnósticos médicos dan imágenes tridimensionales (3D) haciendo que múltiples estudios de la implementación de la IA en entornos médicos sea ignorado perdiéndose datos potencialmente útiles.

Frente a esas limitaciones trabajos recientes que exploran generación de volúmenes 3D o trayectorias temporales han adoptado variantes 3D/4D de VQ-GAN para modelar estructuras anatómicas volumétricas con buena calidad y coherencia temporal, mostrando que los códigos discretos (índices en un diccionario aprendido) facilitan tanto la síntesis como el análisis posterior de biomarcadores, además, combinaciones VQ-GAN + difusión latente han emergido como un patrón exitoso: primero tokenizar con VQ-GAN y luego modelar la distribución de esos tokens con un modelo de difusión para obtener muestras realistas y eficientes [36].

Un dato importante mencionado sobre la dimensión a usarse del espacio latente es que, al comprimir cada dimensión espacial por un factor de 8, causa que se pierdan características anatómicas fundamentales, hay una gran pérdida de detalle, un factor de compresión máximo de 4 conserva detalles anatómicos aun precisos, además, la implementación de VQ-GAN en el aprendizaje de un diccionario de *tokens* junto con un modelo de difusión permiten diversidad de datos, ya que por lo general las imágenes sintéticas creadas suelen ser idénticas y tener muy poca variabilidad, esto se comprueba al usar un SSIM como métrica de evaluación y obtener un valor alto pero no de forma excesiva. Según Zhao et al. [37], varios trabajos de síntesis 3D y 4D en medicina usan exactamente este patrón —e.g., 4D-VQ-GAN o *pipelines* 3D VQ-GAN + Latent Semantic Diffusion— para tareas como simulación de progresión de enfermedad o generación de volúmenes sintéticos etiquetados.

Aunque el uso de VQ-GAN en ecocardiografía es más reciente que en dominios como CT o MRI, ya hay trabajos y preprints específicos que aplican tokenización latente y modelos basados en VQ-VAE/VQ-GAN, por ejemplo, proyectos recientes presentan *pipelines* que usan VQ-VAE/VQ-GAN para tokenizar vídeos de ecocardiograma y luego modelar esos *tokens* con técnicas de *masked token modeling* o difusión condicionada (trabajos denominados “ECHOPulse”, “4D-VQ-GAN” o propuestas de tokenización para generación de vídeos ECHO condicionados a señales ECG).

Estos enfoques muestran que la tokenización latente permite generar vídeos ecocardiográficos coherentes en el tiempo, condicionar la generación a señales fisiológicas (ECG) y acelerar la decodificación frente a modelos píxel-a-píxel, las primeras evaluaciones sugieren utilidad para generar secuencias sintéticas coherentes con la fisiología cardíaca, aunque aún se enfatiza la necesidad de validación clínica y pruebas de privacidad [38].

2.2.5 CGAN para traducción de dominios y Outpainting

Diseñados para traducción imagen→máscara, máscara→imagen y soporte de segmentación son muy valiosos cuando se requieren pares anotados costosos de generar manualmente, estudios recientes han utilizado versiones 3D para sintetizar volúmenes ecocardiográficos emparejados con etiquetas anatómicas (ventrículos, aurículas, miocardio), y luego han demostrado que entrenar segmentadores con reales + sintéticos mejora la generalización, especialmente cuando las anotaciones reales son escasas, existen trabajos que integran esquemas adversariales dentro del proceso de segmentación para forzar el acercamiento anatómico de las máscaras generadas [28], estas aproximaciones reducen la necesidad de grandes grupos anotados y facilitan la creación de benchmarks con más variabilidad anatómica.

Últimos trabajos han adaptado arquitecturas cGAN para realizar *outpainting* en ecocardiografía —es decir, generar regiones adyacentes útiles fuera del campo de visión original— con el propósito de ofrecer vistas más completas para suplementar *frames* incompletos, modelos específicos proponen discriminadores y pérdidas conscientes del dominio cardíaco para reconstruir tejido anatómico creíble en áreas no capturadas originalmente, lo que puede reducir la curva de aprendizaje en exploraciones y mejorar la información disponible para algoritmos que requieren vistas más completas, aunque prometedoras requieren validación clínica para asegurar que el contenido extrapolado no induzca errores diagnósticos [39].

2.2.6 WGAN en la eficiencia de la estabilidad

Para estabilizar el entrenamiento y mejorar la fidelidad, variantes como WGAN / WGAN-GP han sido utilizadas en contextos médicos (retina, radiología, etc.) porque su función de coste basada en Wasserstein mejora la convergencia y reduce problemas de colapso del generador; en pruebas, WGAN-GP suele ofrecer imágenes con texturas más conservadas respecto a GANs estándar cuando la cantidad de datos es limitada [40].

2.3 Métricas y protocolos de evaluación

La evaluación de generadores en ecocardiografía debe combinar métricas de calidad de imagen, utilidad y evaluación por expertos humanos, en la literatura sobre imágenes médicas generativas se usan habitualmente tres tipos de métricas: medidas de similitud distributiva (e.g., FID) que comparan la distribución de características entre reales y sintéticas; métricas de similitud de par referencia-a-referencia (e.g., SSIM, PSNR, MSE) cuando existen pares *ground truth*; y métricas orientadas a la tarea (e.g., IoU, *accuracy* en segmentación / clasificación) que miden el impacto real de los sintéticos al entrenar modelos *downstream*.

2.3.1 FID reportados en la literatura

La FID se ha convertido en la métrica de facto importante para cuantificar la similitud entre la distribución de características de imágenes reales y sintetizadas por un generador, en la literatura de imagen médica, aunque FID no es la mas recomendada, se usa extensamente porque resume en un solo número la “pertinencia” estadística del conjunto sintético con respecto al real y suele correlacionar con juicios humanos de calidad visual.

Varios trabajos que aplican StyleGAN2-ADA a dominios médicos presentan FID como la métrica principal para cuantificar fidelidad: por ejemplo, Woodland et al. [30] reportaron $FID = 5.22 (\pm 0.17)$ para un *dataset* de cortes de CT hepáticos y FID record de ~ 10.78 , ~ 3.52 y ~ 21.17 en otros conjuntos médicos (ACDC, SLIVER07, ChestX-ray14), asociados a resultados que los autores consideran de “muy alta” calidad perceptual. Esto muestra que, en dominios relativamente homogéneos y bien tratados, FID por debajo de ≈ 5 – 10 suele asociarse a imágenes de excelente fidelidad en la literatura médica.

En el subcampo de ultrasonido y ecocardiografía Gazda et al. [39] muestran ejemplos concretos que usan FID y que ilustran cómo interpretar sus valores en imágenes ruidosas y heterogéneas, un caso reciente es echoGAN (*outpainting*, es decir extrapolación de imagen en eco-cardiograma transtorácico (*Transthoracic Echocardiogram, TTE*)), que reportó $FID \approx 122.08$ para un escenario de ampliación relativamente moderada (30°) y FID crecientes (≈ 144 – 150) para cortes de *outpainting* mayores; los autores interpretan estos números en el contexto específico del problema (*outpainting* complejo y extrapolación de información faltante) y muestran que, pese a FID altos comparados con dominios “limpios”, las imágenes pintadas mantienen coherencia anatómica comprobada por medidas clínicas, demostrando que, en ecocardiografía FID tiende a ser mayor que en CT/MRI porque las imágenes son más ruidosas y heterogéneas, además que valores absolutos altos no implican necesariamente inutilidad clínica —es imprescindible complementar con evaluación funcional y juicio experto.

Otros estudios de ultrasonido han reportado valores intermedios de FID en augmentación y síntesis, por ejemplo, trabajos en imágenes de mamario/biopsia y ecografías oftálmicas presentado por Alruily et al. [41] han usado FID para comparar las arquitecturas, IGAN, DCGAN, Speckle GAN y UltraGAN en mejoras de pérdida/regularización: con FID en torno a ~ 40 para modelos de augmentación en conjuntos de ultrasonido, mientras que valores más cercanos a 10 o menos aparecen en dominios menos ruidosos o con *datasets* mayores.

En conjunto, la experiencia recogida en la literatura indica rangos orientativos muy aproximados, donde $FID \approx 10$ – 50 puede ser considerado bueno/aceptable en tareas de imagen médica donde la heterogeneidad o el ruido aumentan la dificultad; y $FID > 50$ – 100 suele indicar que la distribución generada difiere notablemente de la real —aunque, como en echoGAN, valores altos pueden todavía ser clínicamente útiles si la métrica no captura aspectos relevantes del objetivo. Como limitaciones, FID puede correlacionar con percepción humana global, pero es insensible a fallos anatómicos puntuales (e.g., generación de válvulas mal posicionadas) y depende fuertemente del extractor (Inception fue entrenada en imágenes naturales).

2.3.2 KID reportados en la literatura

Como explica Pan et al. [42] su estimador es no sesgado, lo que significa que las estimaciones convergen al valor verdadero sin el sesgo apreciable que puede presentar FID en muestras pequeñas, razón por la que es especialmente recomendable en dominios médicos; KID tiene una escala cuyo ideal es 0, en la práctica médica los valores de KID que se consideran “excelentes” o “aceptables” dependen mucho del dominio, del tamaño del conjunto de evaluación y de la normalización del cálculo: en trabajos de imágenes médicas bien curadas se han reportado KID del orden de 0.01–0.05 para modelos muy afinados, mientras que en ultrasonido y tareas que implican extrapolación difícil (*outpainting*, síntesis de texturas *speckle*).

Los KID suelen ser mayores y pueden situarse en valores superiores a 0.1 o más, sin que ello automáticamente implique incapacidad clínica —la magnitud refleja en parte la complejidad del dominio y la sensibilidad de KID a diferencias de textura y varianza, en trabajos de generación de tumores en ultrasonido mamario (*Breast Ultrasound, BUS*) o en *pipelines* clínico-guiados-previamente se reportan KID para evaluar el rendimiento de la generación en torno a 0.019–0.03 para modelos bien ajustados tras adaptación (con mejoras relativas entre métodos), mientras que tareas de ultrasonido más difíciles muestran KID sensiblemente mayores [42].

Trabajos como 2S-BUSGAN o *frameworks* para síntesis de lesiones usan KID como métrica principal junto a FID y SSIM para comparar variantes; trabajos de generación clínica guiada muestran tablas con FID y KID juntos, reportando KID entre 0.76–0.33 como indicador de buen ajuste, en ecocardiografía, artículos de simulación y de *dehazing/denoising* han empezado a incorporar KID (y ocasionalmente a usarlo como término de pérdida) para mejorar la fidelidad perceptual de imágenes sintéticas en dominios con pocos datos [43].

2.3.3 JSD reportados en la literatura

Es una medida simétrica y acotada de discrepancia entre dos distribuciones de probabilidad (ideal = 0), se emplea de dos maneras principales: como herramienta de evaluación cuantitativa comparando histogramas de intensidad entre reales y sintéticas y en la teoría del entrenamiento de GAN, donde el objetivo original del GAN clásico se puede interpretar como la minimización empírica de la JSD entre la distribución real y la generada, esto se desarrollo añadiendo ruido a las entradas del discriminador, donde las densidades del real y generador se reemplazan por estimaciones de densidad del *kernel*, estableciendo una convergencia casi segura evitando el sesgo —lo que explica por qué el uso directo de esta divergencia puede llevar a problemas de convergencia que motivaron variantes como WGAN; se plantean el uso de diversas métricas cuantitativas tales como (*Expected Entropy, EE*), (*Expected nearest-neighbour distance, ENN*), (*LeNet score, LS*), (*Maximum Mean Discrepancy, MMD*), donde la métrica mas relevante es JSD que da valor aproximado de -1.384 [44].

Respecto a qué valores de JSD se consideran “buenos” o “malos”, como indican Tom et al. [45] hay que enfatizar que la escala y el significado absoluto dependen fuertemente de cómo se calcula (discretización/histogramas, normalización, número de bits, o si se estima en un espacio latente), en trabajos concretos (como de ultrasonido/IVUS) se propuso un pipeline de dos etapas (*Stage I* / *Stage II* GAN) para refinar simulaciones físicas y evaluó la “*patho-realism*” midiendo la JSD entre las distribuciones de intensidad de las regiones *lumen* / *media* / *externa* en imágenes reales vs. sintetizadas, en este paper los autores muestran una reducción notable de la JSD tras la etapa de refinamiento: *Stage II* GAN obtuvo $JSD = 0.0458$ (*lumen*), 0.1159 (*media*), 0.3343 (*externa*), frente a valores mucho mayores en la etapa inicial (*Stage 0*: 0.0957, 0.5245, 0.6685, respectivamente), los autores interpretan valores bajos (e.g., ≈ 0.0458 en *lumen*) como indicativos de muy buena coincidencia estadística local del *speckle*, mientras que valores más altos en regiones complejas reflejan mayor dificultad de modelado, estos umbrales derivan de comparaciones empíricas en estudios que informan la JSD entre regiones anatómicas y versiones de generadores que deben tomarse como orientativos.

2.3.4 SSIM y PSNR reportados en la literatura

En el ámbito del ultrasonido, los GANs condicionales (e.g., pix2pix / cGAN) y arquitecturas híbridas UNet+GAN se emplean frecuentemente para tareas de *despeckling*/*denoising* y traducción imagen→imagen; estos trabajos reportan SSIM como métrica principal junto a PSNR, en ciertos papers se aplicó pix2pix para despeckling en ultrasonido y reportó un $SSIM \approx 0.789$ (con $PSNR \sim 21.08$ dB) frente a valores mucho menores con filtros clásicos, señalando una mejora estructural apreciable tras el tratamiento con GAN [31].

De forma análoga, Cammarasana et al. [46] compararon un pix2pix frente a una red neuronal convolucional (*Convolutional Neural Network, CNN*) tradicional, donde las imágenes tienen una mediana de SSIM de 0.964 y de PSNR de 36.13 y con imágenes musculo-esqueléticas se tiene una mediana de SSIM de 0.881 y de PSNR de 26.58, donde se especifica que si se aumenta el conjunto de entrenamiento de 500 a 3,500 o hasta 50,000 imágenes hay una mejora cuantitativa en las métricas de rendimiento, además se obtuvieron SSIM promedio ≈ 0.878 para pix2pix (vs. $SSIM \approx 0.651$ para la CNN), mostrando que los enfoques adversariales preservan mejor las estructuras relevantes en (*Ultra Sound, US*)).

Estos ejemplos ilustran que en ultrasonido clínico la SSIM suele situarse en rangos intermedios (≈ 0.7 – 0.9) cuando el modelo mejora visiblemente la calidad y preserva morfología, la revisión empírica permite determinar que SSIM es más informativa cuando existen pares (referencia real), en escenarios *unpaired* o generación incondicional su uso exige estrategias para crear una referencia válida, un SSIM muy alto exclusivamente entre imágenes sintéticas (sin contraste con reales) puede indicar *mode collapse*, por ello la literatura recomienda medir SSIM entre reales y sintéticos y otros indicadores de diversidad (FID/KID), con todo esto se permite tener una interpretación de que específicamente en imágenes con un par-a-par, un $SSIM \geq 0.90$ es excelente, indicando una alta conservación estructural entre referencia

y resultado, un SSIM entre 0.75 - 0.90 es bueno y clínicamente aceptable, la estructura principal se preserva, pequeños suavizados o diferencias textuales pueden existir pero no impiden el uso diagnóstico y un $SSIM \leq 0.75$ es especialmente preocupante y pobre, con una pérdida estructural y suavizado notable.

2.4 Riesgos y consideraciones

La aplicación de GANs a ecocardiografía ofrece beneficios claros (aumento de datos, denoising, generación de pares anotados), pero conlleva limitaciones técnicas importantes que deben reconocerse explícitamente, entre las limitaciones recurrentes reportadas en la literatura están la necesidad de grandes cantidades de datos representativos para capturar la variabilidad anatómica y de adquisición.

Tal como indican Woodland et al. [30] la elevada exigencia computacional en todo tipo de GANs, tal como StyleGAN2 donde para su creación tomo al rededor de 51 años de GPU y la dificultad para estabilizar el entrenamiento (*mode collapse*, oscilaciones y sensibilidad a hiperparámetros); aunque variantes modernas como StyleGAN2-ADA alivian parte de la necesidad de datos mediante ADA, estudios encuentran que la calidad, la diversidad y la robustez de las muestras sintéticas siguen condicionadas por la similitud del *dataset* y por la configuración experimental (*transfer learning*, aumentaciones, tamaño de entrenamiento), estas limitaciones técnicas afectan directamente la aplicabilidad clínica de los datos sintéticos en ecocardiografía que exigen experimentos cuidadosos y validación externa.

Las imprecisiones anatómicas y artefactos sutiles son otro vector de riesgo clínico: un generador puede producir texturas o estructuras que parecen plausibles pero que contienen errores morfológicos como, contornos de ventrículo mal posicionados, tamaño de cavidades inconsistentes o artefactos que podrían confundir algoritmos de segmentación, las métricas visuales (FID, SSIM) no garantizan la ausencia de errores anatómicos [47].

Aunque FID mejore continuamente a medida que avanza el entrenamiento de una GAN, no puede evaluarse de forma posterior como en segmentación de las imágenes generadas ya que el FID no produce un buen rendimiento; por eso la literatura recomienda que además de métricas numéricas se realicen evaluaciones por cardiólogos y pruebas de utilidad en tareas *downstream* (e.g., impacto en segmentadores dados) para detectar fallos clínicamente relevantes [47], esto es especialmente importante en ecocardiografía, donde estructuras finas (válvulas, paredes delgadas) y el ruido *speckle* pueden ocultar o falsear hallazgos.

El sesgo es una preocupación ética y técnica recurrente, como indica Jain et al. [48], en el contexto de GANs, por “sesgo” se entiende cualquier discrepancia sistemática entre la representación en los datos de entrenamiento y la población o casos reales que el modelo encontrará en producción, los sesgos pueden ser de clase (desbalance de patologías), demográficos (edad, sexo, etnia), técnicos (un fabricante de ecógrafo sobre-representado) o de

adquisición (vistas y *presets*), los GANs no corrigen automáticamente estos sesgos —de hecho, pueden amplificarlos—, si el conjunto contiene mayoritariamente casos normales o imágenes de un solo proveedor, $\mathcal{G}$ tenderá a producir más ejemplos similares y a apartar las minorías (fenómenos de *mode collapse* o *bias amplification*), un $\mathcal{G}$ si es un vector de baja dimensión de ruido aleatorio, colapsa, porque la entrada de ruido z corresponde a algún punto minoritario de la distribución causando una probabilidad baja o nula, por lo que $\mathcal{G}(z)$ atrae poco o ningún gradiente del $\mathcal{D}$, teniendo una total incapacidad de atraer gradientes de $\mathcal{D}$ capaces de crear imágenes de esos puntos minoritarios; en ecocardiografía esto puede traducirse en peor representación de patologías raras o en imágenes sintéticas de peor calidad para ciertos subgrupos de pacientes, con el riesgo de empeorar la equidad del sistema.

Evitar el sesgo permite tener un enfoque completo en características primordiales, mas que nada en entornos médicos, donde pequeñas características no comunes pueden pasar desapercibidas impidiendo detecciones tempranas de patologías importantes, existen varios posibles enfoques que pueden impedir la retropropagación de estos en un *dataset* generado tales como, la curación y balanceo del *dataset*, condicionamiento controlado usando etiquetas o atributos clínicos que permiten generar ejemplos de subgrupos escasos, auditorias de memorización, privacidad diferencial y arquitecturas privadas, evaluación humana y pruebas *downstream*; es importante conocer como en ciertas arquitecturas el sesgo puede manifestarse:

- *StyleGAN / StyleGAN2-ADA*: al ser modelos no condicionales (o condicionales mediante etiquetas externas), su salida depende fuertemente de la representación en el *dataset* y del muestreo en el espacio latente, si la distribución de entrenamiento es sesgada, el *mapping* de estilos favorecerá la generación de modos bien representados; la augmentación de datos adaptativa ayuda, pero no elimina la amplificación de sesgos si las minorías son muy escasas, además, el diseño de la pérdida y el proceso de truncamiento en el espacio latente pueden reducir diversidad y priorizar la “media” del *dataset* [30].
- *pix2pix / cGANs (image→image)*: aquí el sesgo puede originarse en los pares de entrenamiento, si los pares (imagen, máscara/objetivo) se crearon mayoritariamente en pacientes o condiciones concretas, el modelo aprenderá ese mapeo sesgado (e.g., estilos de despeckling que funcionan mejor para un tipo de ecógrafo), al ser condicional, pix2pix reproducirá las correlaciones aprendidas entre entrada y salida —incluyendo correlaciones espurias o dependencias del equipo— lo que implica que la corrección/transformación aprendida podría no generalizar a datos de distinto origen [31].
- *VQ-GAN / modelos latentes discretos*: estos modelos aprenden un diccionario de códigos discretos que representan patrones locales, si el diccionario está dominado por patrones asociados a la mayoría de datos (e.g., imágenes normales o de un proveedor), la reconstrucción y síntesis tenderán a omitir o reconstruir mal patrones minoritarios, sin embargo, la estructura latente discreta también puede facilitar control y condicionamiento (e.g, forzar códigos que representen patologías), lo que, bien usado, permite mitigar sesgos mediante diseño de *dataset* y condicionamiento explícito [49].

3 Marco teórico

3.1 Ecocardiografía

3.1.1 Ciclo cardíaco sístole y diástole

La *diástole* comprende el llenado ventricular, en la *relajación isovolumétrica*, el ventrículo se relaja pero las válvulas auriculoventriculares (*Atrioventricular Valves, AV*) aún permanecen cerradas, al bajar la presión ventricular por debajo de la auricular, la válvula mitral se abre y ocurre el *llenado rápido*, luego viene la fase de *diástasis* (llenado lento), y finalmente la *contracción auricular* que impulsa más sangre al ventrículo. Por su parte, la *sístole* ventricular consiste en una fase de contracción isovolumétrica (cuando todas las válvulas están cerradas) seguida por la eyección de sangre al sobrepasar la presión arterial y la presión ventricular [50].

3.1.2 Ecocardiografía Transtorácica (TTE) y fundamentos

La TTE es la técnica de imagen cardíaca no invasiva más utilizada para valorar anatomía y función ventricular, valvulopatías y hemodinámica en tiempo real, emplea transductores de matriz en fase *phased-array* típicamente de 1-5 MHz en adultos, con modos 2D, *M-mode*, Doppler pulsado/continuo y Doppler color, la combinación de alta resolución temporal y portabilidad la hace ideal para evaluar dinámica ciclo a ciclo, a diferencia de modalidades estáticas, para el montaje y la adquisición se usan transductores *phase-array presets* cardíacos con compuestos de imagen y *frame rates* altos para preservar cinética valvular y de paredes [51, 52], algunos planos ecocardiográficos (TTE estándar) son:

- Parasternal eje largo (PLAX) y eje corto (PSAX) a múltiples niveles
- Apical 4-cámaras (A4C), 2-cámaras (A2C) y 3-cámaras (A3C).
- Subcostal y suprasternal según indicación.

3.1.3 Anatomía cardíaca relevante

El corazón humano es un órgano muscular cuya función principal es garantizar la circulación de la sangre a través de todo el organismo, asegurando el transporte de oxígeno y nutrientes a los tejidos, así como la eliminación de productos de desecho, su estructura anatómica está organizada en cuatro cámaras: dos aurículas (derecha e izquierda) y dos ventrículos (derecho e izquierdo), las cuales trabajan de forma coordinada mediante contracciones secuenciales que forman el ciclo cardíaco [53], de manera detallada para entender la importancia de cada cámara o cavidad, se tiene:

■ Aurículas

- **Aurícula derecha (AD):** se sitúa detrás del ventrículo derecho en dirección derecha, consta de un componente venoso, una orejuela y un vestíbulo, recibe la sangre venosa sistémica desoxigenada a través de la vena cava superior, la vena cava inferior y el seno coronario, su función es servir como cámara de llenado y regulación antes de que la sangre pase al ventrículo derecho (VD), en ecocardiografía, la AD suele tener menor relevancia cuantitativa, pero un aumento de su tamaño puede evidenciar hipertensión pulmonar o insuficiencia tricuspídea [54].
- **Aurícula izquierda (AI):** recibe la sangre oxigenada proveniente de los pulmones a través de las cuatro venas pulmonares, la AI juega un papel crucial como regulador del llenado del ventrículo izquierdo (VI), actuando como reservorio durante la sístole del VI y almacena energía en forma de presión y actúa como conducto durante la diástole que transfiere sangre pasivamente desde las venas pulmonares al VI, un agrandamiento de la AI es un biomarcador de eventos cardíacos adversos como infartos en el miocardio, insuficiencia cardíaca, accidentes cerebro vasculares, insuficiencia mitral degenerativa y fibrilación auricular [55].

■ Ventrículos

- **Ventrículo derecho (VD):** Tiene una forma creciente única que genera un grado de complejidad en calcular su tamaño y función, sus dimensiones se estiman mejor con una vista apical de cuatro cámaras enfocadas en el VD, se presenta poca uniformidad del corazón en el lado derecho debido a la falta de familiaridad con diversas técnicas y a la gran atención prestada a cuantificar rasgos del corazón en su lado izquierdo [53], bombea sangre hacia los pulmones a través de la arteria pulmonar, anatómicamente es más delgado y trabeculado que el VI, no debe tener mas de dos tercios del tamaño de este, lo que puede dificultar su segmentación automática en ecocardiografía, la cuantificación de la función del VD es clínicamente relevante en hipertensión pulmonar, insuficiencia cardíaca derecha y cardiopatías congénitas [56].

- **Ventrículo izquierdo (VI):** es la cámara principal encargada de impulsar sangre oxigenada hacia la aorta y el resto del organismo, es la estructura más analizada en ecocardiografía y en bases de datos como EchoNet-Dynamic, donde se enfatiza la medición de volúmenes al final de la sístole (ESV) y la diástole (EDV), así como el cálculo de la fracción de eyección (EF) [57], por su importancia clínica y su relativa facilidad para delimitarse en ecocardiogramas (gracias a la diferenciación entre endocardio y miocardio), el VI es el objetivo principal de la mayoría de algoritmos de segmentación en DL [58].
- **Valvulas cardiacas** Las válvulas cardíacas son estructuras esenciales para garantizar el flujo unidireccional de la sangre dentro del corazón, abriéndose y cerrándose en fases precisas del ciclo cardíaco, las cuatro válvulas (tricúspide, pulmonar, mitral y aórtica) actúan coordinadamente para separar atrios de ventrículos y ventrículos de los grandes vasos, evitando el reflujo y permitiendo una eficiente circulación sanguínea [59]. Desde el punto de vista estructural, las válvulas cardíacas se pueden clasificar en dos grupos:
 - **Válvulas auriculoventriculares (AV):** Separan las aurículas de los ventrículos, estas tienen valvas, la válvula tricúspide y la válvula mitral, conectan un atrio con su correspondiente ventrículo, y están apoyadas sobre un complejo aparato subvalvular (cuerdas tendinosas, músculos papilares) [59].
 - **Válvulas semilunares (SL):** Estas válvulas tienen tricúspides, la válvula pulmonar y la válvula aórtica están situadas entre los ventrículos y los grandes vasos (arteria pulmonar y aorta), su estructura de soporte difiere de la de las AV, pues no tienen cuerdas tendinosas, sino que sus cúspides se apoyan en la pared de la raíz arterial [59].

También es importante el esqueleto fibroso del corazón (o “*fibrous annulus / cardiac skeleton*”), que forma anillos de soporte que rodean las válvulas, contribuyendo a su fijación y al aislamiento eléctrico entre atrios y ventrículos. A continuación se describirá las válvulas mas importantes con mas detalle:

- **Válvula tricúspide:** Localizada entre la AD y el VD, es la mas grande de las 4 válvulas cardíacas, compuesta normalmente de 3 valvas(cuspides), durante la diástole se abre para permitir que la sangre procedente de la AD fluya al VD, en la sístole, la válvula se cierra para impedir el reflujo hacia la AD [60].
- **Válvula pulmonar:** Ubicada entre el VD y la arteria pulmonar, formada por tres cúspides semilunares, durante la sístole derecha, la válvula pulmonar se abre para permitir que la sangre pobre en oxígeno del VD llegue a los pulmones para su oxigenación, durante la diástole, se cierra para impedir el reflujo de sangre de la arteria pulmonar al VD [59].
- **Válvula mitral (VM):** conecta la AI con el VI, a diferencia de las demás, es una válvula bicúspide (dos valvas: anterior y posterior), durante la diástole ventricular izquierda, la VM se abre para permitir que la sangre oxigenada desde la AI pase al VI, en la sístole, la válvula debe cerrarse herméticamente para prevenir el reflujo

al atrio izquierdo, ejerciendo la coaptación adecuada de sus valvas gracias al soporte del aparato valvular. es una de las más evaluadas por ecocardiografía, ya que sus disfunciones (estenosis o insuficiencia) son de alta prevalencia clínica [61].

- **Válvula aórtica:** Conecta el VI con la aorta, posee tres cúspides semilunares, durante la sístole del VI, la válvula aórtica se abre para permitir el paso de sangre oxigenada hacia la aorta y así hacia la circulación sistémica, en la diástole, la válvula cierra para prevenir el reflujo de sangre desde la aorta de vuelta al ventrículo, de esta forma, mantiene la presión diastólica de perfusión y protege al ventrículo, su degeneración, como ocurre en la estenosis aórtica, puede identificarse mediante alteraciones en los flujos y morfología valvular [62].

3.1.4 Dataset EchoNet-Dynamic (Stanford)

El EchoNet-Dynamic es uno de los *datasets* más influyentes en el área del DL aplicado a imágenes cardíacas, fue desarrollado por el equipo de medicina de Stanford, este recurso contiene aproximadamente 10,030 estudios de ecocardiogramas en la vista apical de cuatro cámaras (A4C), cada uno en formato de video, lo que permite analizar la dinámica cardíaca a lo largo del ciclo completo para diferentes pacientes [57], cada estudio incluye:

- **Etiquetas clínicas:** En este *dataset* para cada paciente de estudio se extrajeron los valores de *Volumen telediastólico* (*End-Diastolic Volume, EDV*), que es el volumen máximo del VI al final de la diástole y el *Volumen telesistólico* (*End-Systolic Volume, ESV*), que es el volumen mínimo del VI al final de la sístole, ambos se expresan en mililitros y son esenciales para cuantificar la función de bombeo del corazón, usando estos valores se puede obtener la Fracción de Eyección del Ventrículo Izquierdo (FEVI), esta es de las métricas clínicas más utilizadas para evaluar la función sistólica global del corazón, se calcula a partir de los volúmenes EDV y ESV:

$$FEVI(\%) = \frac{EDV - ESV}{EDV} \times 100 \quad (3-1)$$

Los valores normales rondan del 55–70 % y los valores reducidos son <50 %, asociados a disfunción sistólica, insuficiencia cardíaca y riesgo de eventos adversos.

- **Máscaras de segmentación manuales:** Para cada estudio se incluyen máscaras binarizadas del VI correspondientes a los cuadros representativos de la sístole y la diástole, anotadas manualmente por cardiólogos expertos utilizando herramientas de trazado médico, estas máscaras delimitan con alta precisión el contorno endocárdico, sirviendo como referencia para el entrenamiento y validación de modelos de segmentación automática, gracias a estas anotaciones, el EchoNet-Dynamic permite evaluar métricas de segmentación como el coeficiente de IoU, fundamentales para cuantificar la concordancia entre predicciones del modelo y segmentaciones reales, cada cuadro de referencia se encuentra alineado temporalmente con el ciclo cardíaco, facilitando el análisis comparativo entre la morfología del VI y sus variaciones dinámicas [57].

- Una división estandarizada en entrenamiento, validación y prueba, el *dataset* cuenta con una partición predefinida que garantiza la independencia entre los conjuntos y evita fugas de información entre pacientes, en la división propuesta por los autores, aproximadamente el 70 % de los estudios se destinan al conjunto de entrenamiento, 10 % al de validación y 20 % restante al de prueba, cada subconjunto incluye tanto los videos de ecocardiogramas como sus etiquetas clínicas y máscaras correspondientes [57].

El estudio original demostró que modelos de video, como redes convolucionales 3D y arquitecturas híbridas con memoria larga de corto plazo (*Long Short-Term Memory, LSTM*), podían alcanzar una precisión comparable a la de expertos en la estimación de la FEVI directamente a partir de videos ecocardiográficos, sin necesidad de intervención manual [57], este *dataset* ha sido crucial para estandarizar la evaluación de métodos de DL en cardiología, explorar técnicas de análisis de beat-to-beat, donde la FEVI puede calcularse en cada latido en lugar de una única estimación promedio y facilitar la investigación en segmentación automática del VI, un paso clave para la cuantificación de volúmenes (EDV/ESV).

3.1.5 Patologías detectables en ecocardiografía con procesamiento de imagen y *deep learning*

La ecocardiografía constituye una de las herramientas más versátiles en la evaluación cardiovascular, permitiendo la caracterización estructural y funcional del corazón, tal como se menciono antes gracias a su naturaleza no invasiva y su bajo costo relativo frente a otras técnicas de imagen, ha sido considerada el método de primera línea en el diagnóstico de múltiples enfermedades cardíacas, sin embargo, la interpretación de imágenes ecocardiográficas presenta importantes retos de variabilidad interobservador y dependencia del operador, en este orden de ideas, el uso de procesamiento de imagen y técnicas de DL permite automatizar, estandarizar y mejorar la detección y cuantificación de patologías cardíacas, especialmente aquellas con marcadas alteraciones morfológicas y funcionales [53, 63].

- **Cardiomiopatía dilatada/ Fracción de eyección reducida:** se caracteriza por la dilatación progresiva del VI acompañada de un disminuido acortamiento sistólico global, lo que conlleva a una reducción significativa de la FEVI [63, 64]. Desde el punto de vista ecocardiográfico, puede observarse un aumento de los diámetros y volúmenes ventriculares (EDV, ESV) y una contractilidad reducida en múltiples segmentos, estas características hacen que la cardiomiopatía dilatada sea particularmente adecuada para el análisis automatizado, ya que la dilatación geométrica y la disminución de la motilidad se reflejan de forma clara en imágenes y secuencias de video, modelos de DL han demostrado capacidad para predecir la FEVI y estimar dimensiones ventriculares con precisión comparable a la de expertos, lo que es útil tanto en cribado poblacional como en el seguimiento longitudinal de pacientes [63].

- **Cardiomiopatía hipertrófica** es la enfermedad genética cardíaca más frecuente y se caracteriza por una hipertrofia ventricular izquierda no explicada por sobrecarga de presión, la forma más común es la hipertrofia septal asimétrica, en la cual el tabique interventricular se engrosa de forma desproporcionada respecto a la pared libre [63, 65], en ecocardiografía, la cardiomiopatía hipertrofica se manifiesta como un engrosamiento focal o difuso, que puede acompañarse de obstrucción dinámica al tracto de salida del VI y alteraciones en el llenado diastólico, estos patrones son detectables en múltiples planos (A4C, PLAX, PSAX), la capacidad de DL para reconocer patrones sutiles de engrosamiento miocárdico y desorganización estructural permite clasificar estudios TTE de manera automatizada, incluso en casos tempranos, facilitando el cribado en poblaciones de riesgo [65].
- **Estenosis aórtica** es una valvulopatía degenerativa progresiva caracterizada por la calcificación y estrechamiento del orificio valvular aórtico, lo que limita el flujo sanguíneo del VI hacia la aorta [66], tradicionalmente, su diagnóstico y graduación requieren estudios Doppler espectral, que permiten calcular el gradiente de presión y el área valvular, sin embargo, estudios recientes han mostrado que modelos de DL aplicados sobre videos 2D de ecocardiografía (particularmente en vistas PLAX y PSAX) son capaces de detectar la estenosis aórtica y estimar su severidad sin necesidad de datos Doppler explícitos, aprovechando patrones sutiles de movimiento valvular y flujo [67], esto es especialmente relevante para aplicaciones en ecografía en el punto de atención (*Point-Of-Care Ultrasound, POCUS*)), donde no siempre se dispone de mediciones Doppler completas.
- **Insuficiencia mitral** es la regurgitación retrógrada de sangre desde el VI hacia la aurícula izquierda debido a un cierre incompleto de la VM, puede ser primaria (degenerativa) o secundaria (funcional, asociada a dilatación ventricular), en ecocardiografía, se evalúa tradicionalmente mediante color-Doppler, cuantificando la extensión del chorro regurgitante y parámetros asociados, recientemente, CNN y modelos de DL han demostrado capacidad para identificar automáticamente videos con regurgitación mitral y estimar su severidad en categorías ordinales (leve, moderada, severa) a partir de ecocardiogramas TTE completos [68], esta capacidad de clasificación automatizada puede facilitar la estratificación rápida en estudios de tamizaje y seguimiento clínico.
- **Amiloidosis cardíaca y otras entidades** es una enfermedad infiltrativa en la que proteínas amiloides se depositan en el miocardio, produciendo un patrón ecocardiográfico característico: engrosamiento difuso de las paredes ventriculares, aspecto “granular” del miocardio y restricción diastólica, si bien el diagnóstico definitivo requiere técnicas avanzadas, los sistemas de DL han demostrado capacidad para detectar patrones ecocardiográficos sutiles sugestivos de amiloidosis [69, 70], asimismo, *pipelines* de DL han mostrado un desempeño comparable al de expertos en la clasificación de múltiples diagnósticos ecocardiográficos, incluyendo defecto septal auricular (*Atrial Septal Defect, ASD*)), cardiomiopatía dilatada, cardiomiopatía hipertrófica e infarto agudo de miocardio previo a partir de videos TTE [63], esto sugiere que las técnicas de DL pueden servir como herramientas de apoyo al diagnóstico diferencial en tiempo real.

3.2 Fundamentos del Machine Learning

El aprendizaje automático (*ML*) es un subcampo de la Inteligencia Artificial, que se enfoca en el diseño de algoritmos capaces de aprender patrones a partir de datos y mejorar su desempeño en una tarea específica, desde una definición mas clásica, primitiva y citada es la de Mitchell(1997) [71]:

“Un programa de computadora se dice que aprende de la experiencia E , con respecto a una clase de tareas T y una medida de desempeño P , si su desempeño en las tareas T , medido por P , mejora con la experiencia E .”

Dado un conjunto de datos de entrenamiento, $D = \{(x_i, y_i)\}_{i=1}^n$, $x_i \in X$, $y_i \in Y$, se asume que los pares (x_i, y_i) provienen de una distribución conjunta desconocida $D(x, y)$.

Como indica Shwartz et al. [72] la meta es aprender una función $f : X \rightarrow Y$ que minimice el riesgo esperado, la función f debe aproximar la relación subyacente entre la entrada y salida de los datos, básicamente tomar una nueva entrada y hacer una predicción, de manera que pueda generalizar a instancias nuevas no vistas minimizando el *riesgo esperado* o error de generalización; el modelo no debe memorizar *overfitting*, si no generalizar a datos nuevos, para evaluarlo los datos se dividen en, Entrenamiento, Prueba y *cross validation*; en el riesgo esperado ℓ es una función de pérdida y D la distribución desconocida de datos, como D no se observa, es desconocido, se minimiza el riesgo empírico sobre las muestras:

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \hat{R}_n(f) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(x_i)) \quad (3-2)$$

este principio es conocido como minimización del riesgo empírico (*Empirical Risk Minimization*, *ERM*), de esta manera para controlar sobreajuste se usa regularización o se restringe la complejidad del modelo [73], se puede descomponer el riesgo en dos términos:

$$R(\hat{f}) - R^* = \underbrace{[R(\hat{f}) - \inf_{f' \in \mathcal{F}} R(f')]}_{\text{error de estimación}} + \underbrace{[\inf_{f' \in \mathcal{F}} R(f') - R^*]}_{\text{error de aproximación}} \quad (3-3)$$

Un concepto interesante que permite definir el aprendizaje es la teoría (*Probably Approximately Correct*, *PAC*) definida de la siguiente manera, se toma un conjunto fijo de conceptos posibles H llamado conjunto de hipótesis, recibe una muestra $S = (x_1, \dots, x_m)$, así como las etiquetas $c(x_1), \dots, c(x_m)$ que se basan en un concepto objetivo para aprender, el objetivo es usar la muestra etiquetada S para seleccionar una hipótesis $h_S \in H$ que tenga un pequeño error de generalización con respecto a c , el error de generalización de una hipótesis también es denominado error verdadero o error justo de h , entonces desde un punto de vista mas matemático, una clase de conceptos C es PAC-aprendible si existe un algoritmo A y una

función polinómica de manera que cualquier $\epsilon > 0$ y $\delta > 0$, para todas las distribuciones D sobre X [74].

El desempeño de un modelo ML puede analizarse a través de la descomposición sesgo-varianza, la esencia del dilema radica en que modelos incorrectos generan sesgo elevado, tareas de inferencia complejas tardan en converger requiriendo grandes muestras de entrenamiento para lograr un rendimiento aceptable, el efecto de alta varianza es consecuencia del gran número de parámetros, arquitecturas paralelas y hardware rápido no es suficiente en estos casos, la única manera de controlar la varianza es usar estimación basada en modelos, pero en consecuencia esto hace que sea propensa a sesgos; el problema de regresión consiste en construir una función $f(x)$ basada en un conjunto de entrenamiento con el fin de aproximar en observaciones futuras de $\mathbf{x}$, esto suele conocerse como generalización, el error cuadrático medio esperado en un punto $\mathbf{x}$ se descompone como [75]:

$$\mathbb{E}_{\mathcal{D}}[(f(x; \mathcal{D}) - \mathbb{E}[y | x])^2] = \underbrace{(\mathbb{E}_{\mathcal{D}}[f(x; \mathcal{D})] - \mathbb{E}[y | x])^2}_{\text{sesgo}} + \underbrace{\mathbb{E}_{\mathcal{V}}[(f(x; \mathcal{V}) - \mathbb{E}_{\mathcal{V}}[f(x; \mathcal{V})])^2]}_{\text{varianza}} \quad (3-4)$$

el análisis sesgo-varianza ilustra el dilema fundamental en ML: reducir sesgo, incrementar varianza y viceversa.

3.2.1 Paradigmas principales de ML

- **Aprendizaje Supervisado:**

El aprendizaje se refiere a alguna forma del algoritmo de reducir el error en un conjunto de datos de entrenamiento, para el aprendizaje supervisado los datos se les proporciona una etiqueta de categoría, que es lo mismo a un valor objetivo, o un costo para cada patrón en un conjunto de entrenamiento, con la intención de reducir la suma de costos de estos patrones, en resumen algoritmos que aprenden de X a Y [76], de manera similar se puede describir como el escenario donde la experiencia adquirida por medio del entrenamiento contiene información significativa que falta en los ejemplos de prueba a los cuales se les aplicara la experiencia adquirida, de manera que se pueda predecir la información faltante en los datos de prueba [72].

- *Regresión lineal:* Según Shwartz et al. [72], es una herramienta estadística para modelar la relación entre algunas variables de entrada y su resultado correspondiente al valor real, donde será necesario definir una función de pérdida, en clasificación, la función de pérdida es directa, ya que $\ell(h, (x, y))$ indica si $h(x)$ predice correctamente y o no, por lo tanto para la predicción se debe definir una

penalización entre la discrepancia entre el valor predicho $h(x)$ y el valor objetivo y , una forma de hacerlo es con la función de perdida cuadrática, la cual es:

$$\ell(h, (\mathbf{x}, y)) = (h(\mathbf{x}) - y)^2 \quad (3-5)$$

Para esta función de perdida, la función de riesgo empírico se denomina, error cuadrático medio (*Mean Squared Error, MSE*), el cual es:

$$L_S(h) = \frac{1}{m} \sum_{i=1}^m (h(x_i) - y_i)^2 \quad (3-6)$$

- *Regresión Logística*: Según Shwartz et al. [72], se utiliza para tareas de clasificación, se puede interpretar $h(x)$ como la probabilidad de que la etiqueta $\mathbf{x}$ sea 1 o 0, la clase de hipótesis asociada es la composición de una función sigmoidea

$$\phi_{\text{sig}} : \mathbb{R} \rightarrow [0, 1]$$

sobre la clase de las funciones lineales L_d , donde la formula de uso es:

$$\phi_{\text{sig}}(z) = \frac{1}{1 + e^{-z}} \quad (3-7)$$

- *Máquinas de Vectores de Soporte (SVM)*: Según Shwartz et al. [72], se usa para el aprendizaje de predictores lineales en espacios de características de alta dimensionalidad, sea $S = (x_1, y_1), \dots, (x_m, y_m)$ un conjunto de entrenamiento de ejemplos, se dice que este conjunto es linealmente separable si existe un semiplano (w, b) tal que $y_i = \text{sign}(\langle w, x_i \rangle + b)$ para todo i , todos los semiplanos que satisfacen esta condición son hipótesis ERM (su error 0-1 es cero, que es el error mínimo posible)
- **Aprendizaje No Supervisado**: En este no hay un objetivo explicito, y el sistema forma grupos o agrupaciones naturales de los patrones de entrada "Naturales", se define de forma explicita o implícita en su propia agrupación, y dado sus patrones o funciones de costo, diferentes algoritmos de agrupación conducen a diferentes grupos [76], en este aprendizaje no hay distinción entre datos de entrenamiento y datos de prueba, el aprendizaje procesa los datos de entrada con el objetivo de obtener un resumen o una versión comprimida agrupando ese conjunto de datos [72], el aprendizaje no supervisado evita la necesidad de aprender a dividir el mundo en categorías arbitrarias, obliga al modelo a explicar las entradas de alta dimensión en lugar de solo salidas de baja dimensión, permitiendo aprender modelos mas ricos de como funciona el mundo [77].
- *K-means*: Según Shwartz et al. [72], la agrupación (*clustering*) es una de las técnicas mas utilizadas en la exploración de datos, que consiste en agrupar un grupo de objetos de manera que los objetos similares queden en el mismo grupo, y los disimiles se separen en otros, k-means es de los mas populares en el agrupamiento,

ya que mide la distancia al cuadrado entre cada punto de $\mathbf{x}$ y el centroide de su *cluster*, el centroide de C_i se define como:

$$\mu(C_i) = \arg \min_{\mu \in X} \sum_{x \in C_i} d(x, \mu)^2 \quad (3-8)$$

donde la función objetivo k-means seria:

$$G_{k\text{-means}}((X, d), \{C_1, \dots, C_k\}) = \min_{\{\mu_1, \dots, \mu_k \in X\}} \sum_{i=1}^k \sum_{x \in C_i} d(x, \mu_i)^2 \quad (3-9)$$

- *PCA (Análisis de Componentes Principales)*: Según Shwartz et al. [72], la reducción de dimensionalidad es el proceso de transformar datos de alta dimensión a un nuevo espacio de menor dimensión, el concepto se relaciona a la compresión por perdida, hay varias razones para reducir la dimensión, tales como desafíos computacionales, baja capacidad de generalización y mejorar la interpretabilidad de los datos, uno de los métodos para esto es PCA, sea $\mathbf{x}_1, \dots, \mathbf{x}_m$ vectores m en $\mathbb{R}^d$, se desea reducir la dimensionalidad de estos vectores mediante una transformación lineal, una matriz $W \in \mathbb{R}^{n \times d}$, donde $n < d$, induce una función $\mathbf{x} \mapsto W\mathbf{x}$ donde $W\mathbf{x} \in \mathbb{R}^n$ es la representación de menor dimensionalidad de $\mathbf{x}$, la matriz de compresión $\mathbf{W}$ y la de recuperación $\mathbf{U}$ se encuentran de manera que la distancia cuadrática entre los 2 vectores originales y recuperados sea mínima, por medio de:

$$\arg \min_{W \in \mathbb{R}^{n,d}, U \in \mathbb{R}^{d,n}} \sum_{i=1}^m \|\mathbf{x}_i - UW\mathbf{x}_i\|^2 \quad (3-10)$$

Una forma mas clara de entenderlo es usando la interpretación de Murphy [77] con el uso de la formula aquí expuesta, este menciona que el objetivo básico consiste en encontrar una proyección lineal y ortogonal de los datos de alta dimensión a un subespacio de baja dimensión, si se proyecta o codifica $\mathbf{x}$ para obtener $\mathbf{z} = W^T \mathbf{x}$ y luego se des proyecta o decodifica $\mathbf{z}$ para obtener $\mathbf{z}'$, entonces se debe estar cerca de $\mathbf{x}$ a una distancia ℓ_2 , de esta manera se define el siguiente error de distorsión o reconstrucción:

$$\mathcal{L}(W) \triangleq \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - \text{decode}(\text{encode}(\mathbf{x}_n; W); W)\|_2^2 \quad (3-11)$$

Se puede minimizar esta función objetivo definiendo $\hat{W} = U_L$, donde $\mathbf{U}_L$ contiene los L autovectores con los mayores autovalores de la matriz de covarianza empírica:

$$\hat{\Sigma} = \frac{1}{N} \sum_{n=1}^N (x_n - \bar{x})(x_n - \bar{x})^T = \frac{1}{N} X_c^T X_c \quad (3-12)$$

donde $\mathbf{X}_c$ es una versión centrada de la matriz $N \times D$.

- *t-Distributed Stochastic Neighbor Embedding (t-SNE)*: Según Murphy [77], la idea básica de la incrustación estocástica de vecinos con distribución t, es convertir distancias euclidianas de alta dimensión en probabilidades condicionales que representan similitudes, en la siguiente formula se ve como la varianza se fija a una constante, cambiarla solo re escalaría el mapa aprendido, sin modificar su topología, si la representación vectorial es buena, entonces q_{ij} debería coincidir con p_{ij} :

$$\mathcal{L} = \sum_i \text{KL}(P_i \| Q_i) = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}} \quad (3-13)$$

donde P_i es la distribución condicional sobre todos los demás puntos de datos dados de x_i , Q_i es la distribución condicional sobre todos los demás puntos latentes dado z_i y $\text{KL}(P_i \| Q_i)$ es la divergencia de *Kullback-Leibler (KL)* entre las distribuciones.

3.3 Deep Learning

Para comenzar a abordar que es el DL es fundamental comenzar con aquello que constituye la base fundamental del DL que son las redes neuronales artificiales (*Artificial Neural Networks, ANNs*), inspirándose en la forma en que las neuronas biológicas procesan información, hay 3 elementos fundamentales que componen del modelo neuronal tal y como se ve en la Figura **3-1**: la sinapsis o enlace de conexión, que tienen sus propios *pesos sinápticos* ($\mathbf{w}_i$), una señal $\mathbf{x}_j$ en la entrada de la sinapsis j conectada a la neurona k , se multiplica por el peso sináptico (que puede contener valores tanto negativos como positivos) [78, 79].

También consta de un sumador que suma las señales de entrada, estas operaciones componen un combinador lineal, y por ultimo, esta una función de activación para limitar la amplitud de la salida de una neurona, también se conoce como función de compresión ya que limita el rango de amplitud a un valor finito, este también contiene un sesgo denotado por $\mathbf{b}_k$ que tiene el efecto de aumentar o disminuir la entrada de la función de activación, de forma matemática todo esto puede ser descrito de la siguiente manera:

$$y_k = \phi \left(\sum_{i=1}^n w_{ki} x_i + b_k \right) \quad (3-14)$$

donde $\mathbf{x}_i$ representa las entradas, $\mathbf{w}_i$ los pesos sinápticos de cada neurona k , y b_k el sesgo, $\phi(\cdot)$ es la función de activación y $\mathbf{y}_k$ es la salida [78, 79].

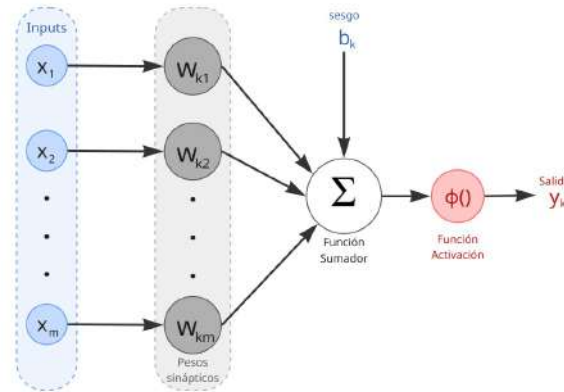


Figura 3-1: Modelo de una red neuronal con entradas k .

El perceptrón, propuesto por Rosenblatt en 1958, fue el primer modelo de red neuronal descrito algorítmicamente, utilizada en la clasificación de patrones linealmente separables (patrones en lados opuestos del hiperplano), consiste en una sola neurona con pesos sinápticos y sesgo ajustables, demostró que si los vectores usados provienen de dos clases linealmente separables, entonces el algoritmo del perceptrón converge, el objetivo del perceptrón es clasificar correctamente el conjunto de estímulos aplicados externamente $x_1, x_2, \dots, x_m$ en una de dos clases, c_1 o c_2 ; el algoritmo de convergencia del perceptrón es no paramétrico debido a que no presupone la forma de las distribuciones subyacentes, su funcionamiento se basa en errores en las zonas de solapamiento de las distribuciones, por lo tanto puede funcionar bien en distribuciones no gaussianas, además el algoritmo de convergencia es adaptativo, de fácil simplificación, su almacenamiento se limita a los pesos sinápticos [78].

La elección de la *función de activación* es crítica para el desempeño de las redes, para calcular la de cada neurona se requiere conocer la derivada de la función de activación $\phi(\cdot)$, para que exista esa derivada la función $\phi(\cdot)$ debe ser continua, dicho de otra manera la diferenciabilidad es el único requisito, entre las funciones de activación más utilizadas se encuentra la *sigmoide* que comprime valores en el rango $(0, 1)$, la *función logística* es una forma de no linealidad sigmoideal, una característica del aprendizaje por propagación que ayuda a la estabilidad del algoritmo, es deducir para el sigmoide como los pesos sinápticos experimentan el mayor cambio en aquellas neuronas donde las señales de la función se encuentran en su rango medio; otra forma común de no linealidad es la función hiperbólica $\tanh$ $\phi(x) = \tanh(x)$, con salida en $(-1, 1)$, definida como $\varphi_i(v_i(n)) = a \tanh(bv_i(n))$ donde a y b son constantes positivas, la función *tangente hiperbólica* simplemente es la función logística re escalada y sesgada [78].

El descenso de gradiente estocástico (*Stochastic Gradient Descent, SGD*) sus variantes son

de vital importancia, ya que marca la manera en que se acerca al mínimo local o global, cuando una función es diferenciable respecto a sus parámetros, el descenso de gradiente tiende a ser un método eficiente, debido a que en el calculo de las derivadas de primer orden tiene la misma complejidad computacional que la simple evaluación de la función, cabe resaltar cierto métodos de optimizaciones típicos en GANs tales como Adam y RMSProp [80].

Estimación adaptativa de momentos (*Adaptive Moment Estimation, Adam*) es un método de optimización estocástica que solo requiere gradientes de primer orden y poca memoria, Adam también mantiene un promedio de gradientes pasados m_t que decae exponencialmente., similar a SGD con impulso:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (3-15)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (3-16)$$

donde m_t y v_t son estimadores de media y varianza, β_1, β_2 parámetros de decaimiento y g_t^2 los gradientes en pasos de tiempos sucesivos [80].

RMSProp es un método de optimización muy relacionado con Adam, se ha utilizado versiones con *momentum*, esta genera sus actualizaciones utilizando un momentum sobre el gradiente re escalado, carece de un termino de corrección de sesgo, esto es relevante cuando el valor de β_2 es cercano a 1 para gradientes dispersos, si no se corrige dicho sesgo puede provocar pasos muy largos al punto de llegar a divergir [80], de esta manera la regla de actualización para cada parametro puede ser escrita como:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \cdot g_t \quad (3-17)$$

Para hablar de la tasa de aprendizaje (*Learning Rate, LR*), es necesario mencionar el (*Backpropagation*) o algoritmo de retropropagación, que define la trayectoria en el espacio de pesos mediante el descenso de gradiente, entre mas pequeño sea la LR, menores serán los cambios de los pesos sinápticos de una iteración a la siguiente, se podría decir que mas cortos serán sus pasos, y de esta manera sera mas suave la trayectoria en el espacio de pesos, pero a su vez mas tiempo tardara en converger, teniendo una menor velocidad de aprendizaje, si por el contrario, se aumenta demasiado la LR los cambios en los pesos sinápticos serán mayores pero mas inestables pudiendo producir oscilaciones o rebotes, una manera sencilla de aumentar la velocidad evitando la inestabilidad consiste en modificar la regla delta incorporando un termino de momento de manera que se obtenga esta ecuación:

$$\Delta w_{ji}(n) = \alpha \Delta w_{ji}(n-1) + \eta \delta_j(n) y_i(n) \quad (3-18)$$

donde tiende a ser un numero positivo llamado constante de momento, realizando diversos cambios al obtener la siguiente ecuación:

$$\Delta w_{ji}(n) = -\eta \sum_{t=0}^n \alpha^{n-t} \frac{\partial \mathcal{E}(t)}{\partial w_{ji}(t)} \quad (3-19)$$

se puede inferir y observar que: el ajuste de $\Delta w_{ji}(n)$ representa la suma de una serie temporal ponderada exponencialmente, para que converja se debe restringir al rango $0 \leq |\alpha| \leq 1$ cuando es cero; cuando la derivada parcial de $\frac{\partial \mathcal{E}(t)}{\partial w_{ji}(t)}$ tiene el mismo signo en iteraciones consecutivas; la suma ponderada exponencialmente $\Delta w_{ji}(n)$ aumenta en magnitud y , haciendo que el peso se ajuste considerablemente, esto tiende a acelerar el descenso en direcciones descendentes constantes, de forma contraria, si la misma derivada parcial tiene signos opuestos en iteraciones consecutivas, la suma ponderada exponencialmente se ajusta ligeramente, esto tiene un efecto estabilizador en direcciones con signos oscilantes [78].

Las redes neuronales profundas (*Deep Neural Networks, DNNs*), tienen como característica distintiva la capacidad de aprender representaciones jerárquicas de los datos, donde las capas iniciales extraen características simples y las capas más profundas capturan patrones más abstractos y de alto nivel, estos están conformados por perceptrones multicapa (*Multi-layer Perceptron, MLP*), se puede pensar en 3 funciones $f^{(1)}$, $f^{(2)}$, y $f^{(3)}$ conectados de forma lineal, para este caso $f^{(1)}$ se denomina primera capa de la red o capa de entrada, $f^{(2)}$ sería la segunda capa o capa oculta, la última capa de la red neuronal de propagación hacia adelante se denomina capa de salida, cada capa oculta de la red suele ser vectorial, la dimensionalidad de estas determina el ancho del modelo, se puede entender como un conjunto de unidades que actúan en paralelo, cada una representando una función vectorial-escalar, cada una se asemeja a una neurona en que recibe información de muchas otras unidades y calcula su propio valor de activación [79].

Una implementación recurrente en el entrenamiento de ANNs profundas es el uso de Lotes o minilotes (conocido como *Batch* o *Minibatch*), la mayoría de los algoritmos utilizan mas de un ejemplo de entrenamiento, denominados métodos de minilotes, o métodos estocásticos, la implementación clásica para utilizarlos es el SGD, entre mas grande sea el lote, se puede hacer una estimación mas precisa del gradiente, pero con una eficiencia sublineal, si todos los ejemplos del lote se procesan en paralelo, la cantidad de memoria utilizada aumenta proporcionalmente al tamaño del lote, en términos de hardware ese es el limitante del lote, algunos hardware logran un mejor tiempo de ejecución con tamaños de matrices específicos, en GPUs tamaños en potencias de 2 ofrecen un mejor tiempo de ejecución, los lotes pequeños pueden ofrecer un efecto de regularización, posiblemente al ruido que ofrece el aprendizaje [79].

Se puede obtener un estimador insesgado del gradiente utilizando un minibatch de ejemplos $x^{(1)}, \dots, x^{(m)}$ con objetivos correspondientes y^i calculando el gradiente de la función de pérdida con respecto a los parámetros del minibatch:

$$\hat{g} = \frac{1}{m} \nabla_{\theta} \sum_i \mathcal{L}(f(x^{(i)}; \theta), y^{(i)}) \quad (3-20)$$

actualizando en la dirección de g realiza el SGD sobre el error de generalización [79].

El aumento de la profundidad en las redes trae consigo desafíos importantes, uno de ellos

es el problema del gradiente explosivo, donde los gradientes que se propagan hacia capas iniciales se reducen o amplifican exponencialmente, ralentizando el proceso y dificultando el aprendizaje, para mitigar este fenómeno, se han propuesto técnicas como la normalización por lotes (*Batch Normalization*), esta permite utilizar tasas de aprendizaje mucho mas altas, y es menos cuidadoso con la inicialización, la normalización de las activaciones que depende del minilote permite un entrenamiento eficiente, se quiere que la salida dependa unicamente de la entrada de forma determinista [81], para ello una vez entrenada la red se utiliza la siguiente normalización :

$$\hat{x} = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}} \quad (3-21)$$

3.3.1 Principales Arquitecturas de Deep Learning

Las arquitecturas de DL han evolucionado para resolver distintos tipos de problemas y datos, desde imágenes y secuencias hasta texto y señales biomédicas, cada arquitectura se construye sobre la base de las DNNs, pero introduce adaptaciones estructurales que las hacen más eficaces en contextos específicos, a continuación, se presentan las más relevantes en el ámbito de visión por computador y generación de imágenes, que son la base conceptual de las GANs.

Recurrent Neural Networks (RNNs)

Según Pascanu et al. [82], las redes neuronales recurrentes se diseñaron para modelado de series temporales, procesar datos secuenciales, introduciendo conexiones recurrentes que permiten mantener un estado interno o “memoria”, la estructura es similar a la de un MLP, con la diferencia de que se permiten conexiones en las capas ocultas asociado a un retardo temporal, permitiendo descubrir correlaciones temporales entre eventos que posiblemente esten muy alejados entre si, pero es sumamente difícil de entrenar ya que sufre de problemas de gradiente desapareciente/explosivo.

Para mitigar esto se propusieron variantes como las LSTM y las unidades recurrentes cerradas (*Gated Recurrent Units, GRU*), que utilizan compuertas para regular el flujo de información y conservar dependencias de largo plazo [83].

Feed-Forward Neural Networks (FFNNs)

Las redes neuronales de propagación directa son el modelo mas simple y general de una red profunda, se distingue por la presencia de una o mas capas ocultas, “oculto” por la parte de la red que no es visible ni desde la entrada o salida de la red, al agregar mas capas ocultas la red puede extraer estadísticas de orden superior de su entrada, adquiriendo mas perspectiva global [78], se puede observar en la Figura 3-2 un diagrama de como se compone una red multicapa totalmente conectada, en este caso con una capa de entrada con 5 neuronas, 2 capas ocultas con diferentes cantidad de neuronas y una capa de salida con 2 neuronas, por lo general en la capa de salida se dispone de una sola neurona, pero no es una regla establecida, en este caso la red seria de 5-3-7-2 por la cantidad de neuronas que tiene por capa,

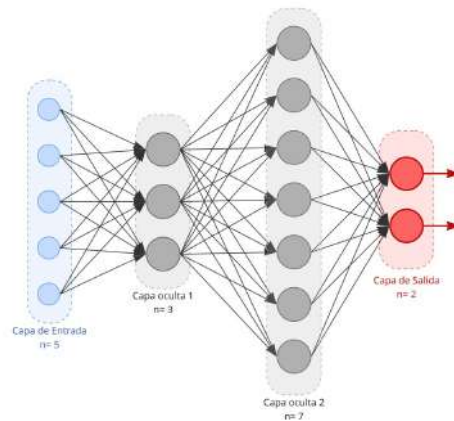


Figura 3-2: Red neuronal totalmente conectada, de 4 capas con diferente cantidad de neuronas o perceptrones.

Convolutional Neural Networks (CNNs)

Como explica Goodfellow et al. [79] el nombre de redes neuronales convolucionales hace referencia a que utilizan una operación matemática típica en el procesamiento de señales llamada convolución, su implementación conjunta con redes neuronales permite procesar datos estructurados, con una topología conocida en forma de cuadrícula o rejilla, tal como datos de series temporales, y datos de imágenes, estas redes se diferencian sencillamente en que utilizan la convolución en lugar de la multiplicación de matrices, aunque sea en una de sus capas, la convolución es una operación entre 2 funciones que se puede denotar de la siguiente manera:

$$s(t) = (w * x)(t) \quad (3-22)$$

Esta se suele denotar con un asterisco, para este contexto la función se denomina entrada, suele ser un vector multidimensional de datos y el *kernel*, la función w vendría siendo el núcleo, la salida tiende a denominarse mapa de características, tanto la entrada como el *kernel* deben almacenarse por separado, se asume que este conjunto es cero en todas partes excepto en el conjunto finito donde se almacenan los valores, haciendo que en la practica se pueda implementar una sumatoria infinita así:

$$s(t) = (w * x)(t) = \sum_{a=-\infty}^{\infty} x(a) w(t - a) \quad (3-23)$$

Al utilizar una convolución de mas de un eje seria:

$$S(i, j) = (w * x)(i, j) = \sum_m \sum_n w(i - m, j - n) w(m, n) \quad (3-24)$$

las CNNs emplean el principio de conectividad local y compartición de pesos, lo que permite reducir significativamente el número de parámetros y, al mismo tiempo, explotar la naturaleza espacial de las imágenes.

Además de la convolución, las CNNs emplean una operación de *pooling* para reducir la dimensionalidad de los mapas de características, una CNN tradicional consta de 3 etapas, en la primera etapa la capa realiza varias convoluciones en paralelo para generar un conjunto de activaciones lineales, en la segunda etapa, cada activación lineal se procesa mediante una función de activación no lineal, esta etapa se denomina etapa detectora, y la tercera etapa es el pooling, una función de agrupación que reemplaza la salida de la red con un resumen estadístico de las salidas cercanas, esto aporta cierta invarianza a traslaciones locales, la operación de *max pooling* selecciona el valor máximo dentro de una región definida, mientras que el *average pooling* calcula el valor promedio:

$$S(i, j) = \max_{(m,n) \in R(i,j)} I(m, n) \quad (\text{Max Pooling}) \quad (3-25)$$

$$S(i, j) = \frac{1}{|R(i, j)|} \sum_{(m,n) \in R(i,j)} I(m, n) \quad (\text{Average Pooling}) \quad (3-26)$$

donde $R(i, j)$ representa la región local de la entrada que se está submuestreando [78, 84].

Autoencoders (AEs)

Los AEs son un tipo de red neuronal especial no supervisada, a diferencia de la típica retropropagación hacia adelante, los AEs pueden entrenarse mediante recirculación, fueron propuestos para tareas de reducción de dimensionalidad (similar al PCA), o aprendizaje de características, esto con el objetivo de encontrar la inicialización de los pesos que permitiera

aprender códigos de baja dimensionalidad que funcionaran mejor en el PCA, es por esto que los AEs están diseñados para aprender a copiar, a aprender una representación comprimida de los datos, denominas espacio latente, sin embargo están restringidos a copiar de forma aproximada, entradas que se asemejen a los datos de entrenamiento, es por esto que son eficientes aprendiendo y captando datos útiles y relevantes que permitan reconstruir lo mas fielmente posible la entrada en la salida [79, 85], constan de 2 componentes:

- **Encoder(Codificador):** Es un componente de una red neuronal que tiene como función mapear una entrada de alta dimensión a una representación latente de menor dimensión, capturando las características más relevantes de los datos, el encoder reduce dimensionalidad, pero a diferencia de métodos lineales como PCA, permite capturar relaciones no lineales complejas mediante funciones de activación como ReLU, Sigmoid o Tanh, en resumen, obtiene la representación comprimida que luego será reconstruida, si se les otorga demasiada capacidad el *autoencoder* aprendera a hacer la tarea de copia sin extraer mucha información relevante [79], formalmente, un encoder puede expresarse como una función parametrizada:

$$z = f_{\theta}(x) \quad (3-27)$$

Aquí f_{θ} es una composición de capas lineales y funciones de activación no lineales, donde cada σ_i es una función de activación que introduce no linealidad [79]:

$$f_{\theta}(x) = \sigma_L \left(W_L (\sigma_{L-1} (W_{L-1} (\dots \sigma_1 (W_1 x + b_1) \dots) + b_{L-1})) + b_L \right), \quad (3-28)$$

- **Decoder(Decodificador):** Es un modelo de red neuronal no supervisado que utiliza un encoder y un decoder para aprender representaciones compactas de los datos, su objetivo principal es la reconstrucción de la entrada a partir de una versión comprimida, toma la representación latente z en una representación latente $\hat{\mathbf{x}}$, este aprende a mapear índices enteros de los valores de entrenamiento [79]:

$$\hat{x} = g_{\phi}(z) \quad (3-29)$$

A lo largo de los años se han propuesto variantes de *Autoencoders* con distintos objetivos tales como: *Undercomplete AE*: el espacio latente es de menor dimensión, restringiendo h a una dimensión menor que $\mathbf{x}$, forzando al modelo a aprender representaciones compactas y relevantes; *Denoising AE*: aprende información útil modificando el termino de error de reconstrucción de dicha función, entrenado para reconstruir $\mathbf{x}$ a partir de una versión ruidosa $\tilde{x}$, queriendo minimizar $L = g_{\phi}(f_{\theta}(\tilde{x}))$, lo que mejora la robustez; *Sparse AE*: introduce una penalización de regularización por dispersion en la capa de codificación h , ademas del error de reconstrucción $L = \|x - g_{\phi}(f_{\theta}(x))\|^2 + \Omega(z)$ donde $g(h)$ es la salida del decodificador y se tiene $h = f(x)$ como la salida del codificador, se utilizan para fomentar representaciones dispersas en z , y aprender características para otra tarea, aspectos estadísticos únicos en

lugar de actuar simplemente como una función de identidad; *Convolutional AE (CAE)*: emplea capas convolucionales, lo cual es especialmente útil en imágenes; *Variational AE (VAE)*: incorpora un enfoque probabilístico en el espacio latente, permitiendo la generación de nuevos datos (fundamental en VQ-GAN) [79].

Variational Autoencoder (VAE)

Constituyen una de las arquitecturas generativas más influyentes en el campo del DL, propuestos por Kingma y Welling en 2013 [86], demostrando que con una reparametrización de la cota inferior variacional se logra producir un estimador in sesgado, diferenciable, y simple, los VAEs extienden el concepto clásico de los AEs al marco probabilístico, permitiendo que el estimador se pueda diferenciar y optimizar fácilmente por medio de métodos estándar de gradiente estocástico, lo que les permite no solo aprender representaciones latentes compactas, sino también generar nuevas muestras de datos de forma coherente y realista.

A diferencia de los AEs tradicionales, cuyo espacio latente carece de estructura, los VAEs imponen una distribución generalmente gaussiana sobre el espacio latente, lo que permite un muestreo directo para la generación de datos sintéticos, esta característica los convierte en un puente entre la estadística bayesiana y el DL, siendo fundamentales en áreas como síntesis de imágenes, compresión, procesamiento de lenguaje natural y análisis de imágenes médicas [79].

El objetivo central de un VAE es que se pueda entrenar maximizando la cota inferior variacional asociada al punto de datos $\mathbf{x}$:

$$\mathcal{L}(q) = \mathbb{E}_{\mathbf{z} \sim q(\mathbf{z}|\mathbf{x})} [\log p_{\text{model}}(\mathbf{z}, \mathbf{x})] + \mathcal{H}(q(\mathbf{z} | \mathbf{x})) \quad (3-30)$$

$$= \mathbb{E}_{\mathbf{z} \sim q(\mathbf{z}|\mathbf{x})} [\log p_{\text{model}}(\mathbf{x} | \mathbf{z})] - D_{\text{KL}}(q(\mathbf{z} | \mathbf{x}) \parallel p_{\text{model}}(\mathbf{z})) \quad (3-31)$$

$$\leq \log p_{\text{model}}(\mathbf{x}). \quad (3-32)$$

En las ecuaciones Goodfellow et al. [79] explica que se componen de, el primer termino que es la *log-verosimilitud* conjunta de las variables visibles y ocultas bajo la distribución posterior aproximada sobre las variables latentes, el segundo termino es la entropía de la distribución posterior aproximada, q se elige como una distribución gaussiana, con ruido añadido a un valor predicho. Maximizar ese termino de la entropía fomenta el aumento de la desviación estándar en ese ruido, este segundo termino busca que la distribución posterior aproximada y la distribución a priori del modelo converjan.

3.4 Generative Adversarial Networks (GANs)

Las redes generativas adversarias fueron introducidas en 2014 por Goodfellow et al. [12] como un nuevo paradigma dentro de los modelos generativos en el campo del DL, los modelos discriminatorios son capaces de mapear una entrada sensorial rica y de alta dimensión a una etiqueta de clase, viéndose beneficiados algoritmos de retropropagación y de abandono; la motivación principal detrás de este enfoque fue superar las limitaciones de modelos generativos previos, tales como las máquinas de Boltzmann restringidas (*Restricted Boltzmann Machines*, *RBM*) y los VAEs, los cuales, si bien eran capaces de aproximar distribuciones de datos complejos, presentaban dificultades en la generación de muestras con un alto grado de realismo perceptual [87].

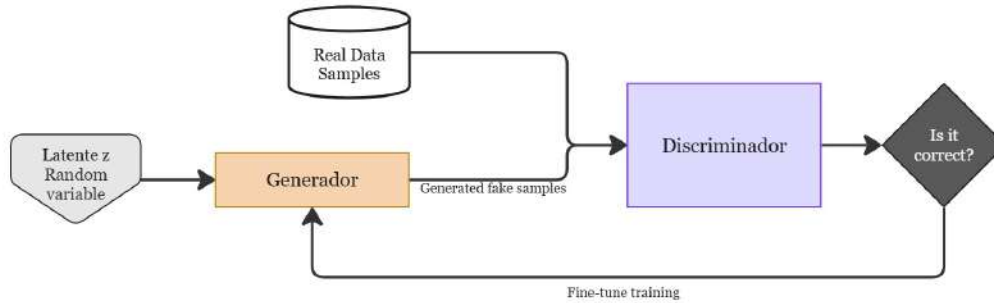


Figura 3-3: Flujo básico de un modelo GAN con el proceso adversarial del Generador $\mathcal{G}$ y el Discriminador $\mathcal{D}$

La idea central de las GANs de Goodfellow et al. [12] es establecer un juego de suma cero entre dos DNNs, idealmente cuando ambos modelos son MLP: el *Generador* ($\mathcal{G}$) y el *Discriminador* ($\mathcal{D}$), como se observa en la Figura 3-3, $\mathcal{G}$ tiene como objetivo aprender una función que mapee un vector de ruido aleatorio $z \sim p_z(z)$, proveniente de una distribución simple (e.g., gaussiana o uniforme) sobre los datos $\mathbf{x}$, hacia el espacio de datos $x \in \mathbb{R}^n$.

En este esquema, $\mathcal{G}$ es una función diferenciable representada por un MLP con parámetros θ_g , de manera que las muestras generadas $\mathcal{G}(\mathbf{z})$ sean indistinguibles de las muestras reales, por su parte, $\mathcal{D}$ es el segundo MLP $\mathcal{D}(x; \theta_d)$ que busca aprender una función de clasificación binaria $\mathcal{D}(\mathbf{x})$, que estime la probabilidad de que una muestra provenga de la distribución real de los datos $p_{\text{data}}(x)$ en lugar de la distribución generada por $\mathcal{G}$, maximizando la probabilidad de asignar la etiqueta correcta tanto a los ejemplos de entrenamientos como a los datos o muestras generados, es decir, el problema se formula como un *minimax game* entre ambos

modelos, donde $\mathcal{G}$ intenta minimizar y $\mathcal{D}$ maximizar la siguiente función de valor:

$$\min_{\mathcal{G}} \max_{\mathcal{D}} V(\mathcal{D}, \mathcal{G}) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log \mathcal{D}(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - \mathcal{D}(\mathcal{G}(z)))]. \quad (3-33)$$

El **Generador** $\mathcal{G}$ es una red neuronal que aprende a mapear un vector de ruido aleatorio, muestreado de una distribución a prior simple $p_z(z)$ (uniforme o gaussiana), hacia una muestra sintética, el objetivo de $\mathcal{G}$ es aprender una función que produzca datos cuya distribución se aproxime lo más posible a la distribución real $p_{\text{data}}(x)$, es decir, el $\mathcal{G}$ busca minimizar la distancia entre la distribución de datos generados $p_g(x)$ y la distribución real $p_{\text{data}}(x)$, al principio del entrenamiento $\mathcal{G}$ es deficiente, $\mathcal{D}$ lo rechaza fácilmente ya que son sumamente diferentes de las muestras originales, haciendo que $\log(1 - \mathcal{D}(\mathcal{G}(z)))$ se sature, en lugar de entrenar $\mathcal{G}$ para minimizar $\log(1 - \mathcal{D}(\mathcal{G}(z)))$ se entrena $\mathcal{G}$ para maximizar $\log \mathcal{D}(\mathcal{G}(z))$ [12], en términos prácticos, $\mathcal{G}$ suele estar implementado como una red neuronal profunda convolucional (*Deep Convolutional Neural Network*, *DCNN*), especialmente en el caso de datos visuales [88], una arquitectura típica incluye o puede incluir:

- *Capa de entrada:* $\mathcal{G}$ parte de un vector latente $z \in \mathbb{R}^d$, muestreado de una distribución a prior simple $p_z(z)$, la idea es que z codifica puntos en un espacio latente en el que distintas direcciones o regiones corresponden a variaciones semánticas de la salida (e.g., tamaño, orientación, presencia/ausencia de rasgos) [12].
- *Capas densas iniciales:* El primer bloque suele ser una (o varias) capas densas totalmente conectadas que elevan la dimensionalidad del vector z hasta un tensor con forma espacial (canales $\times$ altura $\times$ anchura), Radford et al. [88] resumen estas decisiones arquitectónicas en DCGAN y muestran que reemplazar las estructuras totalmente conectadas por estructuras convolucionales y normalización sistemática mejora mucho la estabilidad.
- *Capas convolucionales transpuestas (o deconvoluciones):* permiten la expansión progresiva de la representación latente hasta alcanzar la resolución de la imagen de salida, las transposed-conv pueden producir patrones de tipo ajedrez (checkerboard) cuando el solapamiento de los “pasos” del *kernel* no es uniforme; esto genera artefactos visuales indeseados en imágenes generadas [89].
- *Funciones de activación:* Las activaciones introducen no linealidad y condicionan la dinámica de gradiente:
 - *Tanh (Tangente Hiperbólica):* Representa la no linealidad aplicada a todas las unidades ocultas de la DNN, una sola activación de la unidad oculta viene dada por:

$$h^{(i)} = \sigma(w^{(i)T} x) \quad (3-34)$$

donde sigma es la función tangente hiperbólica, esta función de activación es asimétrica respecto a 0 y tiene un gradiente mas gradual que una sigmoide,

permitiendo una optimización mas robusta en el entrenamiento, si las imágenes de entrenamiento han sido normalizadas de esa manera, permite que la salida esté en la misma escala que los datos de referencia [90].

- *ReLU (Rectified Linear Unit)*: Explicado por Maas et al. [90], ReLU ofrece una alternativa a las no linealidades sigmoideas, aporta simplicidad, gradiente no saturante en la parte positiva y buen comportamiento empírico, matemáticamente se puede expresar como:

$$h^{(i)} = \max(w^{(i)T}x, 0) = \begin{cases} w^{(i)T}x, & w^{(i)T}x > 0, \\ 0, & \text{else.} \end{cases} \quad (3-35)$$

Cuando una unidad ReLU se activa por encima de 0, su derivada parcial es 1, haciendo que no existan gradientes desvanecientes a lo largo de las trayectorias de las unidades ocultas, ReLU se saturan exactamente en 0, útil en redes ocultas.

- *LeakyReLU*: Además Maas et al. [90], menciona que ReLU se encuentra en una desventaja potencial, como el gradiente siempre es 0, la unidad no esta activa, dando lugar a que el algoritmo no ajuste los pesos, y el entrenamiento sea lento, para evitar eso aparece LReLU, el rectificador con fugas permite un gradiente pequeño, no cero, cuando la unidad esta saturada, se puede expresar como:

$$h^{(i)} = \max(w^{(i)T}x, 0) = \begin{cases} w^{(i)T}x, & w^{(i)T}x > 0, \\ 0, 0.1w^{(i)T}x, & \text{en caso contrario.} \end{cases} \quad (3-36)$$

de esta manera se evitan “neuronas muertas” cuando los gradientes se anulan.

$\mathcal{G}$ por tanto, no recibe retroalimentación directa sobre su rendimiento, sino que aprende a partir de los gradientes transmitidos por $\mathcal{D}$.

El **Discriminador** $\mathcal{D}$ siguiendo la teoría y formulación de Goodfellow et al. [12], se puede considerar como la contraparte adversaria del generador en una GAN, es una red neuronal que actúa como un clasificador binario, encargado de distinguir entre ejemplos reales y ejemplos sintéticos producidos por el generador, el descenso del gradiente estocástico en minilotes. Para cada k pasos se extrae el minilote de las muestras de ruido z del ruido previo $p_g(z)$ y el minilote de las muestras reales m para la distribución generada p_{data} , de manera que la actualización del ascenso del gradiente estocástico se puede ver como:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m [\log D(x^{(i)}) + \log(1 - D(G(z^{(i)})))] . \quad (3-37)$$

Goodfellow et al. [12] en adición, menciona que se busca obtener la optimalidad global para $p_g = p_{\text{data}}$, donde para un $\mathcal{G}$ fijo, el discriminador optimo $\mathcal{D}$ es:

$$\mathcal{D}_{\mathcal{G}}^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}. \quad (3-38)$$

donde en el entrenamiento de $\mathcal{D}$, dado cualquier $\mathcal{G}$, busca maximizar la cantidad $V(\mathcal{G}, \mathcal{D})$ asi:

$$\begin{aligned} V(G, D) &= \int_x p_{\text{data}}(x) \log(D(x)) dx + \int_z p_z(z) \log(1 - D(G(z))) dz \\ &= \int_x p_{\text{data}}(x) \log(D(x)) + p_g(x) \log(1 - D(x)) dx. \end{aligned} \quad (3-39)$$

de esta manera $\mathcal{D}$ es entrenado para maximizar la probabilidad de asignar valores cercanos a 1 para datos reales ($x \sim p_{\text{data}}$) y valores cercanos a 0 para datos falsos ($x \sim p_g$).

Algunos componentes fundamentales que suele tener un Discriminador son:

- *Funciones de activación LeakyReLU*: El discriminador emplea comúnmente LeakyReLU en lugar de ReLU estándar para mitigar el problema de “neuronas muertas” (cuando las salidas se fijan en cero y no reciben gradiente), aunque ReLU es el generador para todas las capas generalmente (excepto para la salida que usa Tanh), la activación LeakyReLU se usa en todas las capas del discriminador [88, 90].
- *Dropout y normalización espectral*: Dropout se refiere a la eliminación de unidades ocultas o visibles de una red neuronal junto con todas sus conexiones de entrada y salida, consiste en anular aleatoriamente activaciones de una capa con probabilidad p durante el entrenamiento, de manera que evite el sobreajuste y permita combinar de forma eficiente un numero exponencial de arquitecturas de redes neuronales diferentes, la operación de alimentación directa se puede describir como:

$$\begin{aligned} r_j^{(l)} &\sim \text{Bernoulli}(p), \\ \tilde{y}^{(l)} &= r^{(l)} * y^{(l)}, \\ y_i^{(l+1)} &= f\left(z_i^{(l+1)}\right). \end{aligned} \quad (3-40)$$

donde $y^{(l)}$ denota el vector de entrada de la capa l , p la probabilidad y f como cualquier funcion de activación [91]

El discriminador proporciona el gradiente adversarial que guía a $\mathcal{G}$ durante el entrenamiento, si bien su objetivo es maximizar la clasificación correcta, en el equilibrio ideal termina actuando como un crítico imparcial que entrega información valiosa al generador.

3.4.1 Problemas Fundamentales en GANs

A pesar de los avances introducidos por Goodfellow et al. [12], el entrenamiento de las GANs continúa siendo uno de los mayores desafíos en el campo del DL, la dinámica de

entrenamiento entre $\mathcal{G}$ y el $\mathcal{D}$ es altamente inestable debido a la naturaleza competitiva del juego minimax, lo que genera múltiples dificultades prácticas y teóricas que afectan la convergencia, la diversidad de las muestras y la reproducibilidad de los resultados, a continuación, se describen los problemas fundamentales más relevantes en las GANs.

Inestabilidad del Entrenamiento

Radford et al. [88] menciona que el principal problema de las GANs es la inestabilidad en el proceso de optimización, lo que causa que los datos sintéticos producidos por $\mathcal{G}$ carezcan de sentido, a diferencia de los métodos convencionales de aprendizaje supervisado, que optimiza una única función de pérdida, las GANs se optimiza un juego de dos jugadores, lo que introduce comportamientos no triviales en el espacio de parámetros, el entrenamiento puede derivar en oscilaciones, divergencia o colapso prematuro sin alcanzar el equilibrio deseado, al adoptar un enfoque de la normalización de la capa de entrada para que tenga una media y una varianza de cero, se logra una estabilización del aprendizaje, reduciendo problemas producto de la inicialización deficiente facilitando el flujo de gradientes, sin embargo, esta aplicación produce oscilación de las muestras e inestabilidad, al no aplicar la normalización por lotes ni a la capa de salida de $\mathcal{G}$ ni la capa de entrada de $\mathcal{D}$ se evitan estos problemas.

Complementando con lo mencionado por Salimans et al. [92], el entrenamiento busca resolver que $\mathcal{D}$ y $\mathcal{G}$ actualicen sus parámetros en direcciones opuestas, sin embargo, este problema no siempre presenta un mínimo global alcanzable mediante SDG, en muchos casos, la dinámica del entrenamiento se asemeja a un ciclo sin convergencia estable, en lugar de converger a un punto de equilibrio de Nash que es muy difícil, donde en determinados casos las funciones no son convexas los parámetros son continuos y su espacio es extremadamente dimensional.

Salimans et al. [92] indica que algunas formas para atacar el problema de inestabilidad y hacer que el algoritmo converja han sido: coincidencia de características, donde en lugar de maximizar la salida de $\mathcal{D}$ se busca que $\mathcal{G}$ de datos que coincidan con las características estadísticas de los datos reales, para así escoger cuales valen la pena comparar, también se puede implementar la discriminación del minibatch, promedio histórico, suavizado de etiquetas de una cara y normalización de lotes virtuales.

Colapso de Modos (Mode Collapse)

Otro problema ampliamente documentado es el colapso de modos, fenómeno en el cual $\mathcal{G}$ aprende a producir un conjunto limitado de muestras, reduciendo drásticamente la diversidad

de las salidas, mas detalladamente, se suelen presentar colapsos de grandes volúmenes de masa de probabilidad en unos pocos modos, aunque $\mathcal{G}$ produce muestras significativas, estas a menudo provienen de unas pocas regiones donde se concentra una alta probabilidad de distribución de datos, muchos modos de la distribución de datos no están representados en muestras generadas, produciendo una distribución de entropía mucho menor, una forma de solucionarlo es implementando una regularización como proponen Che et al. [93].

Asimismo la tecnica de discriminación del minibatch ayuda a atacar este problema, en lugar de aplicar los múltiples ejemplos de combinación del discriminador de forma aislada, analiza conjuntamente múltiples ejemplos dentro del minibatch [92]

3.5 Métricas de evaluación de imágenes sintéticas

La evaluación de GANs constituye un desafío importante en el campo del ML, a diferencia de técnicas de evaluación clásicas del ML tales como, métricas de rendimiento o el área bajo la curva ROC, la calidad de los modelos generativos es más difícil de cuantificar, ya que involucra tanto la fidelidad visual de las muestras individuales como la capacidad del modelo para capturar la diversidad estadística de los datos reales, un buen modelo generativo no solo debe producir imágenes convincentes, sino también representar de manera adecuada la distribución completa del conjunto de datos. Aunque hay formas de evaluar el comportamiento del modelo por medio de las pérdidas en $\mathcal{G}$ y $\mathcal{D}$, estas no constituyen una medida confiable de calidad, durante el entrenamiento, las oscilaciones en las pérdidas no reflejan necesariamente el realismo ni la diversidad de las imágenes producidas [92].

Esta desconexión ha motivado el desarrollo de métricas específicas para evaluar el rendimiento de los modelos generativos, abarcando tanto criterios objetivos (basados en estadísticas matemáticas o perceptuales) como subjetivos (basados en evaluaciones humanas), no obstante, las evaluaciones humanas resultan costosas, poco escalables y sujetas a sesgos, por lo que en la práctica predominan métricas cuantitativas automáticas [94].

En la literatura se distinguen dos grandes categorías de métricas para la evaluación de GANs, la primera se basa en la comparación de distribuciones, la segunda corresponde a las métricas de similitud imagen a imagen; a pesar de los avances, ninguna métrica es universal ni está exenta de limitaciones, esto ha llevado a la comunidad científica a recomendar el uso combinado de múltiples métricas, complementando las ventajas de cada una, especialmente en aplicaciones como la generación de imágenes biomédicas, donde se requiere tanto realismo visual como precisión estructural para garantizar interpretaciones clínicas confiables [32].

3.5.1 Métricas basadas en distancias estadísticas de distribuciones

Las métricas basadas en distribución constituyen una de las aproximaciones más relevantes y extendidas para evaluar GANs, estas buscan comparar las distribuciones de probabilidad de los datos reales y la distribución de datos generados, con el objetivo de determinar en qué medida el modelo generativo logra aproximarse al espacio estadístico de las muestras auténticas, las principales métricas de distribución son IS, m-IS, MS, AM Score, FID, MMD, The Wassertein Critic, C2ST, NDB, KID, JSD, Earth Mover's Distance, TVDs [94] (solo 3 de estos serán abordados e implementados). Estas métricas operan a nivel agregado, midiendo propiedades estadísticas globales de los conjuntos de datos reales y sintéticos, el fundamento teórico de estas métricas radica en la definición de una distancia o divergencia entre distribuciones de probabilidad, como resulta impracticable trabajar directamente con distribuciones en el espacio de píxeles, estas métricas suelen calcularse sobre un espacio latente de características extraído mediante una red preentrenada (e.g., Inception-V3) [95].

Fréchet Inception Distance (FID)

Se ha consolidado como la métrica más utilizada para evaluar la calidad de imágenes generadas por GANs, debido a su capacidad de medir tanto el realismo visual como la diversidad estadística de las muestras [94], FID fue introducido por Heusel et al. (2017) [96] como una mejora sobre el Inception Score (IS), superando sus limitaciones al considerar explícitamente la distribución de los datos reales y no solo la entropía de las predicciones de una red de clasificación, la gaussiana es la distribución de máxima entropía para una media y covarianza dadas, de manera que la codificación sigue una gaussiana multidimensional, la diferencia entre 2 gaussianas (sintética y real); se mide mediante la distancia de Frechet (también conocido como distancia de Wasserstein-2), su formula puede describirse como:

$$\text{FID}(r, g) = \|\mu_r - \mu_g\|_2^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}), \quad (3-41)$$

donde (μ_r, Σ_r) y (μ_g, Σ_g) son la media y la covarianza de las distribuciones de datos reales, un FID mas bajo cercano a cero indica distancias menores entre las distribuciones de los datos sintéticos y los reales [94]. FID es consistente con el juicio humano, es mas robusto al ruido que IS, FID es capaz de detectar el modo intraclase, pero empeorara a medida que se agregan varios tipos de artefactos, las puntuaciones IS y AM miden la diversidad y calidad, mientras que FID la distancia entre las distribuciones [94],

Kernel Inception Distance (KID)

Una de las métricas más empleadas para evaluar la calidad de las imágenes generadas por modelos generativos adversarios es KID, propuesta por Bińkowski et al. [97] como una alternativa al FID, usa un *kernel* polinomial $k(x, y) = \left(\frac{1}{d}x^\top y + 1\right)^3$ donde d es la representación de la dimensión, el objetivo de KID es medir qué tan cercanas están las distribuciones de las imágenes reales y sintéticas en un espacio de características de alto nivel, usualmente extraídas mediante la red Inception-V3 preentrenada en ImageNet, al igual que en el caso del FID, la principal diferencia entre KID y FID radica en que, mientras FID asume que las distribuciones de características pueden aproximarse mediante distribuciones gaussianas multivariadas, KID evita esta suposición y en su lugar utiliza la *Maximum Mean Discrepancy (MMD)* con un *kernel* polinomial para comparar directamente las muestras, esto hace que KID sea una métrica no paramétrica para la distribución de activación, muy apropiado en activaciones tipo ReLU, haciendo que nunca sean negativas, esto es menos sensible a la validez de la aproximación gaussiana que limita al FID.

Formalmente, KID se define como el cuadrado de la MMD entre dos conjuntos de características $KID = MMD(X, Y)^2$, de manera que KID puede representarse como:

$$KID = \frac{1}{m(m-1)} \sum_{\substack{i,j=1 \\ i \neq j}}^m k(x_i, x_j) + \frac{1}{n(n-1)} \sum_{\substack{i,j=1 \\ i \neq j}}^n k(y_i, y_j) - \frac{2}{mn} \sum_{i=1}^m \sum_{j=1}^n k(x_i, y_j) \quad (3-42)$$

Jensen–Shannon Divergence (JSD)

A diferencia de FID o KID, que dependen de representaciones intermedias obtenidas de redes neuronales como Inception-V3, la JSD se formula directamente sobre las distribuciones de probabilidad de los datos reales y los datos generados, Lin et al. [98], da una definición de la divergencia de Jensen-Shannon de forma detallada, esta puede generalizarse la medida global de 2 distribuciones para cualquier número finito de distribuciones, útil en problemas de decisión con mas de dos clases, sean $p_1, p_2, \dots, p_n$ distribuciones de probabilidad con sus respectivos pesos $\pi_1, \pi_2, \dots, \pi_n$, la JSD generalizada puede definirse como:

$$JS_\pi(p_1, p_2, \dots, p_n) = H\left(\sum_{i=1}^n \pi_i p_i\right) - \sum_{i=1}^n \pi_i H(p_i). \quad (3-43)$$

La JSD es siempre finita y está acotada en el intervalo $[0, \log 2]$, donde valores cercanos a 0 indican que las distribuciones son muy similares, mientras que valores cercanos a $\log 2$ indican una fuerte disimilitud.

Es posible estimar muestreadores neuronales mediante la minimización aproximada de la JSD, usando:

$$D_{JS}(P \parallel Q) = \frac{1}{2}D_{KL}(P \parallel \frac{1}{2}(P + Q)) + \frac{1}{2}D_{KL}(Q \parallel \frac{1}{2}(P + Q)) \quad (3-44)$$

donde D_{KL} denota la divergencia de KL, donde se introduce una segunda red neuronal discriminadora que se optimiza simultáneamente [99].

3.5.2 Métricas basadas en similitud imagen a imagen

Las métricas basadas en similitud imagen a imagen constituyen un enfoque diferente al de las métricas distribucionales, en lugar de comparar distribuciones globales de características entre conjuntos de imágenes reales y generadas, estas métricas evalúan el grado de parecido entre dos imágenes individuales, una considerada como referencia (generalmente una imagen real) y otra como candidata (generalmente una imagen generada por un modelo GAN), esto las hace especialmente relevantes en contextos donde existe un “ground truth” claro, como en tareas de image-to-image translation (ejemplo: pix2pix o CycleGAN) [32, 100].

Estas métricas resultan complementarias a las distribucionales: mientras que métricas como FID y KID miden calidad y diversidad global, métricas de similitud imagen a imagen como, MAE, PSNR, SSIM y LPIPS permiten evaluar de forma cuantitativa la fidelidad local o estructural, sin embargo, estas métricas no se corresponden con la calidad visual de la imagen, en aplicaciones médicas, esta combinación es particularmente valiosa, ya que no basta con que las imágenes generadas pertenezcan a la misma distribución estadística que las reales; es esencial también que reproduzcan con precisión las estructuras anatómicas específicas que tienen relevancia clínica [32].

Peak Signal-to-Noise Ratio (PSNR)

Para poder profundizar sobre PSNR es necesario primero mencionar la métrica MSE, como indica Wang et al. [101], MSE es una medida de fidelidad de la señal, donde el objetivo es comparar 2 señales y determinar su grado de similitud, o el error de distorsión entre estas, el MSE entre señales se define como:

$$MSE(x, y) = \frac{1}{N} \sum_{i=1}^N (x_i - y_i)^2, \quad (3-45)$$

donde $(x_i - y_i)$ es la diferencia entre las señales originales y distorsionadas, cualquier tipo de procesamiento aplicado a una imagen causa pérdida importante de información o calidad,

dejando esto claro se puede entender que MSE se convierte en una medida de relación señal-ruido máxima (PSNR) que se puede describir con la formula:

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{L^2}{\text{MSE}} \right) \quad (3-46)$$

donde L es el rango dinámico de intensidades de pixeles permitidos en la imagen (e.g., 255 en imágenes de 8 bits), un PSNR más alto indica mayor similitud con la referencia, siendo valores mayores a 30 dB generalmente considerados como buena calidad perceptual, el valor de PSNR se acerca al infinito a medida que el MSE se acerca a cero, demostrando que un PSNR mas alto indica una mayor calidad de la imagen y un valor pequeño indica altas diferencias numéricas entre imágenes, se han comprobado que tiene un alto rendimiento en imágenes ruidosas, es decir es sensible al ruido gaussiano aditivo [101, 102].

Cabe aclarar que su aplicación presenta limitaciones importantes, al ser una métrica pixel a pixel, el PSNR no correlaciona bien con la percepción visual humana ni con criterios clínicos, ya que pequeñas variaciones estructurales pueden producir grandes caídas en el valor de PSNR aunque la imagen siga siendo válida, además, en tareas de GANs no pareados donde no existe una correspondencia directa entre la imagen generada y la real, el PSNR deja de ser adecuado porque no hay un “ground truth” con el cual comparar [103], a pesar de estas limitaciones el PSNR sigue siendo relevante como métrica complementaria, especialmente en escenarios de superresolución, reducción de ruido o reconstrucción de modalidades diagnósticas, donde la fidelidad cuantitativa respecto a una imagen original es esencial [32, 103].

Structural Similarity Index Measure (SSIM)

Es una métrica perceptual propuesta por Wang et al. [104], como alternativa al MSE y al PSNR, con el objetivo de medir la calidad de una imagen en función de la similitud estructural entre una referencia y una imagen distorsionada, el SSIM busca capturar la forma en que el sistema visual humano percibe diferencias de luminancia ($l(\mathbf{x}, \mathbf{y})$), contraste ($c(\mathbf{x}, \mathbf{y})$) y estructura ($s(\mathbf{x}, \mathbf{y})$), esas medidas de comparación se pueden observar de la siguiente manera:

$$l(\mathbf{x}, \mathbf{y}) = \frac{2\mu_x\mu_y - C_1}{\mu_x^2 + \mu_y^2 + C_1} \quad (3-47)$$

$$c(\mathbf{x}, \mathbf{y}) = \frac{2\sigma_x\sigma_y + C_2}{\sigma_x^2 + \sigma_y^2 + C_2} \quad (3-48)$$

$$s(\mathbf{x}, \mathbf{y}) = \frac{\sigma_{xy} + C_3}{\sigma_x\sigma_y + C_3} \quad (3-49)$$

donde C_1 esta dada por $C_1 = (K_1, L)^2$ siendo una constante para evitar inestabilidad cuando $\mu_x^2 + \mu_y^2$ es muy cercano a cero, L es el rango de intensidad de los pixeles, C_2 esta dada por

$C_2 = (K_2, L)^2$ y por ultimo C_3 dada por $C_3 = C_2/2$, al combinar las ecuaciones anteriores como una medida de similitud resultante se obtiene el SSIM entre las señales $\mathbf{x}$ y $\mathbf{y}$:

$$\text{SSIM}(\mathbf{x}, \mathbf{y}) = [l(\mathbf{x}, \mathbf{y})]^\alpha \cdot [c(\mathbf{x}, \mathbf{y})]^\beta \cdot [s(\mathbf{x}, \mathbf{y})]^\gamma \quad (3-50)$$

donde α , β y γ son mayores a cero y son parámetros utilizados para ajustar la importancia relativa de los tres componentes, con esto da como forma específica el índice SSIM:

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \quad (3-51)$$

así, los valores μ_x, μ_y son medias locales de las ventanas en $\mathbf{x}$ e $\mathbf{y}$, σ_x^2, σ_y^2 son varianzas locales y σ_{xy} es la covarianza entre $\mathbf{x}$ e $\mathbf{y}$; el índice resultante toma valores en el rango $[-1, 1]$, donde 1 indica similitud estructural perfecta y valores cercanos a 0 o negativos indican alta disimilitud, usualmente, en la práctica, se reportan valores entre 0 y 1, siendo $> 0,95$ considerado excelente en aplicaciones visuales estándar [102, 104].

Learned Perceptual Image Patch Similarity (LPIPS)

Fue introducido por Zhang et al. [105] como una métrica perceptual entrenada que busca cuantificar la similitud entre imágenes de una manera que se correlacione mejor con la percepción humana en comparación a las métricas anteriormente mencionadas, a diferencia de métricas basadas en diferencias estadísticas o estructurales, LPIPS emplea CNNs preentrenadas para modelar representaciones de alto nivel de las imágenes y calcular la similitud en dicho espacio latente, formalmente, dado un par de imágenes $\mathbf{x}$ y $\mathbf{y}$, ambas se procesan a través de una red preentrenada ϕ (e.g., AlexNet, VGG o SqueezeNet), extrayendo activaciones intermedias en distintas capas, luego, la distancia entre las representaciones normalizadas de los parches correspondientes se calcula mediante:

$$d(x, x_0) = \sum_l \frac{1}{H_l W_l} \sum_{h,w} \|w_l \odot (\hat{y}_{hw}^l - \hat{y}_{0hw}^l)\|_2^2 \quad (3-52)$$

donde, l indexa las capas de la red, H_l, W_l son las dimensiones espaciales del mapa de características en la capa l y $\hat{\phi}^l(x)$ es la activación normalizada del mapa de características para la imagen $\mathbf{x}$.

Posteriormente, los autores de LPIPS entrenaron un modelo lineal sobre estas distancias para ajustarlas a evaluaciones perceptuales humanas, resultando en una métrica con fuerte correlación con juicios de calidad subjetivos, capacidad de capturar diferencias de alto nivel (textura, estilo, semántica) que PSNR y SSIM no detectan, adecuado para tareas no deterministas [105], donde no existe una correspondencia estricta imagen a imagen, sin embargo, presenta también limitaciones como: dependencia del extractor preentrenado: el rendimiento varía según la arquitectura usada (AlexNet, VGG, etc.) [106].

Puede ser menos interpretable en dominios especializados (como imágenes médicas) debido a que los extractores suelen entrenarse en *datasets* naturales como ImageNet, no mide directamente la fidelidad anatómica por ejemplo, lo que obliga a combinarla con métricas estructurales (e.g., SSIM) o basadas en distribución (e.g., FID, KID) [32].

4 Arquitecturas GANs

4.1 StyleGAN2-ADA

Esta sección explica la teoría de la arquitectura y funcionamiento con foco en StyleGAN2-ADA donde se analiza su evolución histórica desde su primera propuesta hasta sus avances actuales, la extensión matemática respecto a un GAN clásico, las características relevantes y diferenciadoras que tiene respecto a otros modelos GANs relevantes y el análisis de evaluación necesario para construir a este modelo GAN específico.

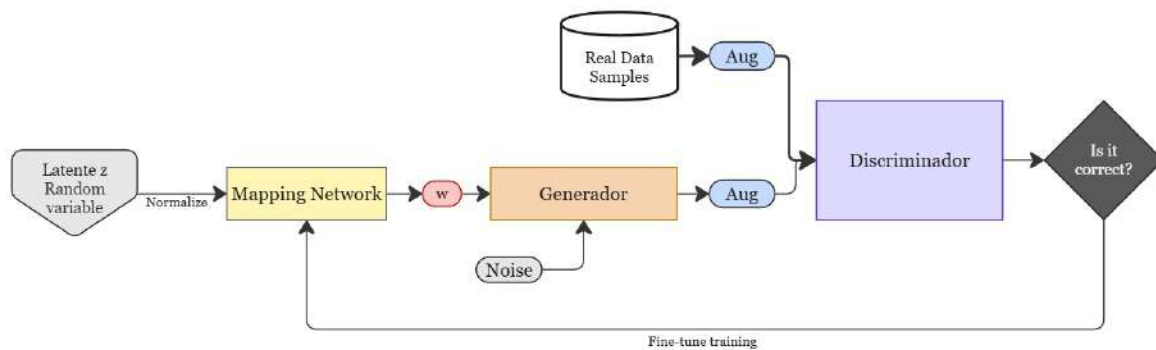


Figura 4-1: Flujo simplificado del modelo StyleGAN2-ADA donde ya se incluye el cambio del vector latente a un vector de estilos por medio del mapping network y funciones aumentativas después del Generator y después del dataset de imágenes originales.

4.1.1 StyleGAN (v1)

La idea clave de *StyleGAN (2018/2019)* propuesto por Karras et al. [107] es re diseñar la arquitectura de $\mathcal{G}$, desde la invención de GANs la literatura ha propuesto diversas técnicas para mejorar el $\mathcal{D}$ usando diferentes variaciones de este o varios en un modelo GAN con la idea de que al mejorar la eficiencia de la red para juzgar las imágenes se mejoraría en mayor medida la capacidad de $\mathcal{G}$ de ser más eficiente, por el lado de $\mathcal{G}$ se han probado diferentes distribuciones espacio latente o en generar variaciones de este, el modelo de StyleGAN se centra en mejorar las capacidades de $\mathcal{G}$ de manera que esté basado en la captura de estilos por medio de su red de mapeo y transformaciones afines teniendo en cuenta la implementación por separado de un *espacio latente* tradicional y un *espacio de estilos* intermedio.

Un espacio latente (z) es básicamente un vector de ruido inicial $z \sim \mathcal{N}(0, I)$ que captura representaciones compactas, es un espacio matemático de representación [12], en GANs clásicos, este vector z se introducía directamente a $\mathcal{G}$ en su primera capa de entrada; generalmente este espacio latente es de dimensión mucho menor que la del espacio de píxeles de las imágenes reales, del espacio latente al espacio de datos se puede expresar de manera más formal como $\mathcal{G} : \mathcal{G}(\mathbf{z}) \rightarrow \mathbb{R}^{|\mathbf{x}|}$, donde cada vector $\mathbf{z} \in \mathbb{R}^{|\mathbf{z}|}$ corresponde a un código latente que captura atributos de alto nivel de las imágenes que $\mathcal{G}$ puede sintetizar [108].

StyleGAN propone transformar este vector a través de una *mapping network* $f : \mathcal{Z} \rightarrow \mathcal{W}$, que redefine estas representaciones es un espacio de estilos más controlable, esta red de mapeo consiste en un *MLP* de 8 capas totalmente conectadas (*Fully Connected, FC*) [107], que son capas densas en las que cada neurona está conectada con todas las de la capa anterior, formalmente, una capa FC aplica una transformación lineal seguida de una no linealidad.

El espacio resultante $\mathcal{W}$ —denominado *espacio de estilos*— tal como mencionan Karras et al. [107] permite una representación más desentrelazada y controlable, es decir, coordenadas que controlan un solo factor generativo interpretable (rotación, brillo, tamaño, etc), a partir de $w \in \mathcal{W}$, se obtienen vectores de estilo s_i para cada nivel de la red de síntesis, así controlando de las características de la imagen en diferentes escalas, junto con ruido inyectado conduce a una separación no supervisada de atributos de alto nivel a partir de la variación estocástica y mediante una transformación afín que son una forma de extraer muestras para cada estilo de una distribución aprendida, estos estilos se inyectan en las capas convolucionales mediante el mecanismo de normalización de instancias adaptativas (*Adaptive Instance Normalization, AdaIN*), AdaIN normaliza primero la activación x de una capa a media cero y varianza unitaria, y luego la re escala y desplaza según los parámetros de estilo (γ, β) :

$$\text{AdaIN}(x_i, \mathbf{y}) = \mathbf{y}_{\mathbf{s},i} \cdot \frac{x_i - \mu(x_i)}{\sigma(x_i)} + \mathbf{y}_{\mathbf{b},i} \quad (4-1)$$

donde $\mu(x)$ y $\sigma(x)$ son la media y la desviación estándar de las activaciones por canal, aquí,

$\mathbf{y}_{s,i}$ y $\mathbf{y}_{b,i}$ corresponden respectivamente a la escala y al sesgo controlados por los estilos derivados de w .

$\mathcal{G}$ incorpora en cada una de sus capas convolucionales un ruido gaussiano no correlacionado, inyectado en cada bloque para producir detalles estocásticos, este mecanismo permite que cada capa controle distintos atributos de la imagen en función de la escala de representación, en el caso de imágenes ecocardiográficas, dichos detalles estocásticos pueden asociarse con fenómenos propios del ultrasonido, como el ruido ecocardiográfico, sombras, reverberaciones, zonas oscuras, variaciones en el ángulo de la sonda, texturas miocárdicas o bordes de las válvulas, estos elementos pueden variar sin afectar la morfología o percepción general de la imagen, siempre que se mantenga la distribución estadística apropiada. En particular, las capas cercanas a la entrada de $\mathcal{G}$ controlan atributos de baja frecuencia o de alto nivel (e.g., estructura global, forma y disposición general de un ventrículo), mientras que la inyección de ruido estocástico añade realismo a los detalles locales, a diferencia de los estilos, el ruido no modifica la estructura global, sino únicamente la microvariación de píxeles, lo cual evita que las imágenes sintéticas se perciban excesivamente “perfectas” y artificiales [107].

Asimismo, un aspecto central de este modelo es la priorización del espacio latente $\mathcal{W}$, el cual busca desenredar los factores de variación y hacer que estas variaciones se expresen de manera más lineal, de este modo, es posible identificar vectores de dirección que correspondan a factores individuales de variación [107], en contraste, cuando se trabaja en el espacio $\mathcal{Z}$, las características ausentes entre dos puntos de una interpolación lineal suelen aparecer de forma abrupta en medio del recorrido, lo que evidencia que $\mathcal{Z}$ permanece enredado [109], esto impide obtener transiciones suaves entre las características de dos imágenes, dificultando el control semántico sobre la generación.

4.1.2 StyleGAN2

La arquitectura *StyleGAN2* (2020) propuesta por Karras et al. [110] surge como una evolución directa de *StyleGAN* (2019), con el objetivo de resolver limitaciones existentes en la calidad visual y en la estabilidad del entrenamiento.

Una de sus mejoras es la implementación de *Weight Demodulation*, en StyleGAN(v1), la inyección de estilos se realizaba con AdaIN, lo que consistía en normalizar las activaciones intermedias y posteriormente reescalarlas y sesgarlas de acuerdo con el vector de estilo, para ser más específicos, realiza la transferencia de estilos en el espacio de características mediante la transferencia de estadísticas de características, siendo la media y varianza por canal, permitiendo así la transferencia de estilo arbitrario en tiempo real [107, 111], este procedimiento, aunque efectivo, introducía artefactos estructurados (patrones repetitivos

y alineados con las coordenadas de píxel) en las imágenes generadas [110], para resolverlo, StyleGAN2 propone la técnica de weight modulation/demodulation.

- *Weight modulation* usado en StyleGAN2 como indican Karras et al. [110], los pesos convolucionales son modulados por el estilo correspondiente en función del estilo entrante, para un estilo s , escalando los pesos de la convolución como:

$$w'_{i,j,k} = s_i \cdot w_{i,j,k}, \quad (4-2)$$

donde $w_{i,j,k}$ representa el peso del kernel convolucional entre el canal de entrada i , el canal de salida j y la posición espacial k , s_i es el factor de estilo aplicado al canal de entrada i , y $\tilde{w}_{i,j,k}$ es el peso resultante tras la modulación.

- *Weight demodulation* Posteriormente se corrige la norma de estos pesos para estabilizar la salida, de manera que se elimine los efectos de las estadísticas s de los mapas de características de salida de la convolución, dividiendo por la raíz cuadrada de la suma de cuadrados, convirtiendo todo el bloque de estilo en una sola capa:

$$w''_{i,j,k} = \frac{w'_{i,j,k}}{\sqrt{\sum_{i,k} w'^2_{i,j,k} + \epsilon}}. \quad (4-3)$$

De este modo, se elimina la dependencia artificial de la normalización a nivel de activación y se evita que surjan artefactos dependientes de las coordenadas, el resultado son imágenes más coherentes, libres de patrones espurios y con mejor fidelidad estructural [110].

En el contexto de GANs, los *artefactos* se refieren a patrones indeseados que aparecen en las imágenes generadas pero que no tienen correlación en los datos reales, estos pueden ser líneas repetitivas, manchas, estructuras cuadrículadas o patrones dependientes de la posición en la malla de píxeles, en StyleGAN (v1) dichos artefactos se estaban presentando debido a la normalización de instancias a partir de la resolución 64x64 con artefactos en forma de gotas que no son del todo visibles pero al observar las activaciones dentro de $\mathcal{G}$ el problema se nota, esto está ligado al uso de AdaIN que normaliza la media y la varianza de cada mapa de características de forma independiente destruyendo información en las magnitudes de características relativas entre si y también ligado al progressive growing [107, 110].

StyleGAN2 mitiga estos artefactos principalmente mediante weight demodulation y un re-diseño de la arquitectura de síntesis, la eliminación de AdaIN se realiza partiéndolo en sus 2 partes constituyentes por separado, las cuales son Normalización y Modulación, originalmente el ruido era aplicado dentro del bloque de estilo, pero esto provocaba efectos inversamente proporcionales a las magnitudes de estilo, al aplicar el ruido fuera de este bloque se mantienen resultados mas predecibles al operar sobre datos normalizados [110].

En *Progressive GAN* y *StyleGAN (v1)*, la red se entrenaba aumentando gradualmente la resolución ($4 \times 4 \rightarrow 8 \times 8 \rightarrow \dots \rightarrow 1024 \times 1024$), lo que estabilizaba el entrenamiento pero producía transiciones bruscas y artefactos específicos notorios, denotando una preferencia de dejar ciertos detalles como dientes u ojos en lugares específicos de la imagen, estos deberían desplazarse pero quedan estancados en un punto específico de la imagen debido a que cada resolución actúa en determinados momentos como resolución de salida lo que genera detalles de frecuencia máxima [107, 112].

En StyleGAN2, se eliminó el progressive growing y se adoptó una arquitectura fija, donde la imagen es generada desde el inicio a la resolución completa mediante un proceso jerárquico de upsampling, este cambio redujo los artefactos introducidos por los cambios de resolución y mejoró la estabilidad del entrenamiento a escalas altas, se incorporaron correcciones en los pesos (weight demodulation) para mitigar los artefactos conocidos como texture sticking, permitiendo un control más preciso sobre las estadísticas de activación, estas mejoras combinadas permitieron a StyleGAN2 producir imágenes sintéticas con mayor fidelidad:

- *Upsample* operación de interpolación (típicamente bilineal) seguida de convoluciones, que permite aumentar progresivamente la resolución de los mapas de características mientras se mantienen detalles consistentes [110].
- *Skip-toRGB* en cada nivel de resolución, se aplica una convolución 1×1 que convierte las características a una imagen RGB parcial, estas salidas intermedias se combinan con las imágenes RGB de resoluciones inferiores (vía skip connections) para mejorar la coherencia multiescala [110].

En adición Karras et al. [110] indican que se hicieron cambios en la *regularización*, esta es una técnica que agrega un término adicional a la función de pérdida con el objetivo de mejorar la generalización del modelo y estabilizar el entrenamiento, en StyleGAN2 se introduce el path length regularization, su objetivo es regularizar la sensibilidad de la imagen generada respecto al espacio latente $\mathcal{W}$, intuitivamente, busca que pequeños cambios en $\mathcal{W}$ produzcan cambios proporcionales y suaves en la imagen. Formalmente, se penaliza la desviación de la norma del gradiente de $\mathcal{G}$ con respecto a un valor objetivo μ :

$$L_{pl} = \mathbb{E}_{w,y} \left[\left(\|J_w^T \mathbf{y}\|^2 - \mu \right)^2 \right], \quad (4-4)$$

donde $J_w y$ es el jacobiano de la imagen respecto al vector latente. Este regularizador mejora la suavidad del espacio latente, facilita la interpolación entre imágenes y aumenta la disentrelazabilidad de atributos, aunque añade cierto costo computacional durante el entrenamiento.

La *inicialización de pesos* es crucial para la estabilidad de las redes profundas, StyleGAN2 introduce mejoras en la normalización e inicialización de los pesos convolucionales para

evitar explosiones o desapariciones del gradiente en capas profundas [110], además, se ajustó el pipeline de síntesis ($\mathcal{G}$) para mejorar la calidad de detalle fino, entre estos ajustes se encuentra, la normalización de convoluciones moduladas para estabilizar la varianza, mejor manejo del aliasing en el upsampling y uso de LeakyReLU con parámetros ajustados en capas críticas, estas mejoras reducen el ruido estructural y mejoran la nitidez de los detalles sin comprometer la coherencia global.

4.1.3 StyleGAN2-ADA

Karras et al. [19] proponen StyleGAN2-ADA (Adaptive Discriminator Augmentation) como una extensión directa de StyleGAN2 orientada a un problema clave: el entrenamiento con *datasets* pequeños, algunos problemas con StyleGAN2 normal se hallaban en que para obtener imágenes de alta calidad, el modelo requería *datasets* de decenas o cientos de miles de imágenes, cuando el número de ejemplos era reducido (como ocurre en ecocardiografía), el discriminador se sobreajustaba rápidamente, lo que llevaba al colapso del entrenamiento ($\mathcal{G}$ dejaba de mejorar), con métricas como FID es observable que entre menos imágenes existan más rápido llega el punto de sobreajuste y en que los resultados empiezan a decaer y ser más erróneos.

Como explican Karras et al. [19] la solución propuesta por ADA es introducir un esquema de aumento de datos adaptativo aplicado dinámicamente a las imágenes reales y generadas, pero controlado de manera que no provoque sesgo de distribución, la mejor manera de esto es controlando las operaciones de aumento llamadas "Sin Fugas", aunque existan imágenes corruptas el entrenamiento deshace dichas corrupciones mientras encuentra la distribución mas apropiada, esto es posible mientras se represente con transformaciones invertibles, para las aumentaciones se consideraron una secuencia de 18 transformaciones agrupadas en 6 diferentes categorías las cuales son: pixel blitting (x-flips, rotaciones de 90°, traslación de enteros) transformaciones afines generales, transformaciones de color, filtrado del espacio de la imagen, ruido adaptativo, y recorte.

Algorithm 1 Mapping Network StyleGAN2-ADA

Require: Latente $z \in \mathbb{R}^{z_dim}$, etiqueta $c \in \mathbb{R}^{c_dim}$ (opcional), parámetros $w_dim, L, num_ws, \beta, \psi, truncation_cutoff$

Ensure: Vectores intermedios $w \in \mathbb{R}^{num_ws \times w_dim}$

```

1: Inicialización:
2: Crea un MLP de  $L$  capas totalmente conectadas ▷ Equalized LR + LeakyReLU
3: if  $c\_dim > 0$  then
4:   Crear capa de embedding  $E(c)$ 
5: end if

6:  $x \leftarrow normalize(z)$  ▷ si  $z\_dim > 0$ 
7: if  $c\_dim > 0$  then
8:    $y \leftarrow normalize(E(c))$ 
9:    $x \leftarrow concat(x, y)$ 
10: end if
11: for  $i = 0 \rightarrow L - 1$  do
12:    $x \leftarrow fc_i(x)$ 
13: end for
14: if modo entrenamiento y  $\beta \neq None$  then
15:    $w\_avg \leftarrow \beta \cdot w\_avg + (1 - \beta) \cdot mean(x)$ 
16: end if
17: if  $num\_ws \neq None$  then
18:    $x \leftarrow repeat(x, num\_ws)$ 
19: end if
20: if  $\psi \neq 1$  then
21:   if  $truncation\_cutoff = None$  then
22:      $x \leftarrow (1 - \psi) \cdot w\_avg + \psi \cdot x$ 
23:   else
24:     Truncar solo las primeras  $truncation\_cutoff$  capas
25:   end if
26: end if
27: return  $x$ 

```

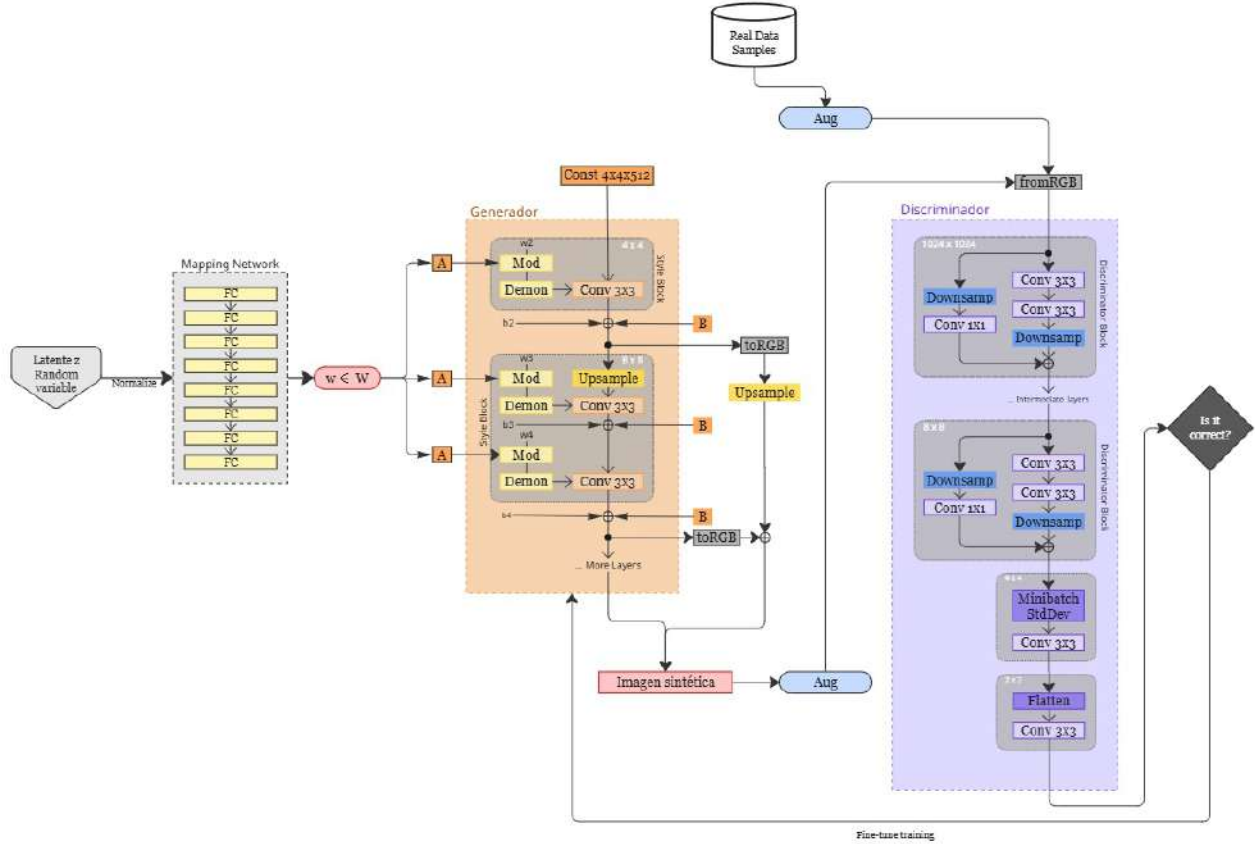


Figura 4-2: Flujo detallado del modelo StyleGAN2-ADA donde es apreciable de mejor manera los (*Full Connection*) en la red de mapeo (*mapping network*) antes de convertir el vector latente a un vector de estilos, la manera en que están contruidos los bloques de estilos para $\mathcal{G}$, las partes donde son aplicadas las funciones adaptativas y la manera en que se distribuyen y componen los bloques del discriminador

La probabilidad de aplicar estas augmentaciones se ajusta automáticamente en cada iteración en función de la señal de entrenamiento del discriminador, formalmente, si p es la probabilidad de aplicar augmentación, ADA ajusta p para mantener constante una métrica llamada signo de la predicción de $\mathcal{D}$, para cada transformación se aplica una probabilidad p o se omite con una probabilidad $1-p$ utilizando siempre el mismo valor de p para todas las transformaciones, así, se evita tanto el overfitting como el underfitting, la aleatorización se realiza por separado en grupos de mini-batch, ya que al ser tantos aumentos impide que $\mathcal{D}$ obtenga una imagen limpia, todo esto hace que StyleGAN2-ADA pueda entrenarse con *datasets* tan pequeños como unas pocas miles de imágenes, manteniendo calidad comparable a *datasets* grandes [19].

La capacidad de StyleGAN2(-ADA) para capturar detalles locales (como bordes, texturas o “*speckle*” en ecocardiografía) puede deberse a dos factores clave:

1. Weight Demodulation + convoluciones moduladas: permiten aplicar los pesos asociados a cada capa de convolución, haciendo que cada estilo afecte selectivamente cómo se aplican los filtros en distintas escalas, logrando un control fino en la generación de detalles sin introducir artefactos de coordenadas, en comparación con la normalización de instancias la demodulación es más débil ya que se basa en suposiciones estadísticas sobre la señal en lugar del contenido real de los mapas de características.
2. Control multiescala de estilos, algunos de estos diferentes aspectos se pueden detallar por Karras et al. [110] y por Sauer et al. [113]:
 - Los estilos w extraídos del (*mapping network*) se inyectan en distintas capas de $\mathcal{G}$, se usan para modular los núcleos de convolución de $\mathcal{G}$ para controlar el proceso de síntesis.
 - Las capas de baja resolución (4×4 , 8×8 , 16×16) controlan la estructura global (e.g., la forma general del ventrículo, orientación de la cámara ecocardiográfica).
 - Las capas de alta resolución (128×128 , 256×256 , 512×512) modulan detalles locales (e.g., podrían con los bordes de válvulas, texturas finas, ruido *speckle* característico de ultrasonido).
 - Esta descomposición jerárquica permite que StyleGAN2-ADA produzca imágenes que conservan tanto coherencia anatómica global como realismo visual en detalles locales.

El concepto de multiscale style control proviene desde StyleGAN (2019) [107] y se mantiene en StyleGAN2-ADA, el vector latente $\mathcal{Z}$ pasa por un (*mapping network*) para transformarse en $\mathcal{W}$, que luego se distribuye a diferentes capas de $\mathcal{G}$, cada capa recibe un “estilo” distinto (a través de weight modulation), lo que otorga control independiente sobre la imagen en múltiples niveles de abstracción, esta propiedad puede ser particularmente útil en ecocardiografía porque permite preservar la forma cardíaca global mientras se genera realismo en detalles locales clínicamente relevantes.

La gran innovación de StyleGAN2-ADA es la estrategia ADA [110, 113], donde la motivación yace en que si se aplican demasiadas aumentaciones, la distribución de datos se distorsiona $\rightarrow$ el $\mathcal{G}$ aprende a imitar aumentaciones en lugar de datos reales, para ello la solución ADA está en ajustar automáticamente el grado de aumentación, el discriminador mide el overfitting a través de la métrica “discriminator overfitting heuristic”, si detecta sobreajuste, se incrementa la probabilidad p de aplicar aumentaciones, si no hay señales de sobreajuste, p se reduce, esto hace que el modelo se auto-regule durante el entrenamiento, sin necesidad de que el investigador ajuste manualmente el nivel de aumentación.

4.2 MedGAN

El término MedGAN fue introducido por primera vez en 2017 con el trabajo de Choi et al. [21], en el cual se propuso un modelo generativo capaz de sintetizar registros médicos electrónicos (*Electronic Health Records, EHRs*), estos registros presentan una naturaleza discreta y multietiquetada —e.g., diagnósticos, procedimientos médicos o resultados de laboratorio—, lo que dificultaba el uso de GANs tradicionales basadas en distribuciones continuas, la motivación principal era generar datos sintéticos realistas que pudieran emplearse en investigación clínica y en el desarrollo de modelos predictivos, reduciendo al mismo tiempo los riesgos de privacidad asociados con el uso de datos de pacientes reales.

La arquitectura original de MedGAN se basó en la combinación de un AE y un GAN, el AE, compuesto por un encoder y un decoder como se observa en la Figura 4-3, se utilizó para proyectar los vectores discretos de los EHRs en un espacio latente continuo, facilitando el aprendizaje en el marco de los GANs, esta estrategia fue fundamental, ya que los GANs presentan dificultades al trabajar directamente con datos categóricos o binarios, mientras que en un espacio latente continuo es posible garantizar gradientes más estables. [21].

$\mathcal{G}$ recibió como entrada un vector de ruido aleatorio, comúnmente denotado como z , para producir representaciones latentes sintéticas, posteriormente, el decoder del *autoencoder* transformaba estas representaciones en registros médicos discretos y multi-etiqueta, finalmente, $\mathcal{D}$ evaluaba si un registro correspondía a datos reales del conjunto original o a una muestra sintética producida por $\mathcal{G}$, en términos generales, MedGAN puede considerarse como una hibridación entre AE y GAN diseñada específicamente para trabajar con datos clínicos discretos, diferenciándose de los modelos típicos centrados en imágenes [21].

En términos de logros como explican Choi et al. [21], MedGAN demostró la capacidad de generar datos sintéticos que preservaban las distribuciones estadísticas y las correlaciones multietiqueta presentes en los registros clínicos reales, los experimentos incluyeron la evaluación de utilidad, donde modelos predictivos entrenados con datos sintéticos lograban desempeños comparables a aquellos entrenados con datos reales, adicionalmente, los autores exploraron el aspecto de privacidad, argumentando que los registros generados por MedGAN podían reducir el riesgo de exposición de información sensible de pacientes.

Es importante resaltar que, en su concepción original, MedGAN no fue diseñado para imágenes médicas, sino para datos tabulares y categóricos, su éxito radicó en ofrecer una solución novedosa al problema de la generación de variables discretas, algo que los GANs convencionales no manejaban bien; su denominación inspiró una serie de trabajos posteriores en los que el nombre “MedGAN” comenzó a aplicarse a distintas variantes de GANs enfocadas en imágenes médicas, estas adaptaciones posteriores incluyeron arquitecturas como la

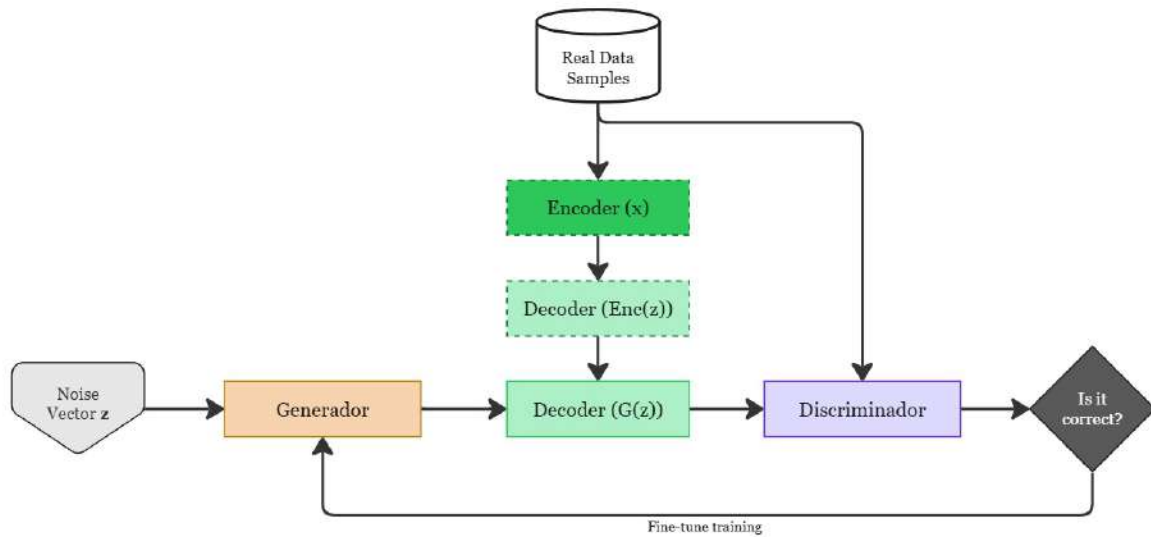


Figura 4-3: Flujo simplificado del primer modelo MedGAN con la implementación de un AE para datos tabulares.

propuesta por Armanious et al. [35] de un Cycle-MedGAN para traducción de modalidades médicas, y la propuesta por Guo et al. [22] donde se implementa un Adaptive MedGAN para mejorar la estabilidad y calidad de imágenes clínicas generadas, en este sentido, aunque el MedGAN original no servía directamente para la generación de imágenes, su concepto sentó un precedente y abrió la puerta a investigaciones posteriores.

Tras la aparición del MedGAN en registros tabulares, el término fue adoptado en investigaciones variadas para referirse a variantes de GANs diseñadas para el procesamiento y generación de imágenes médicas, en este nuevo contexto, los retos eran diferentes: mientras el modelo original debía manejar datos discretos y categóricos, en imágenes médicas los desafíos principales se centran en la preservación de detalles anatómicos, la calidad estructural, la diversidad de las muestras generadas y la reducción de artefactos, de esta forma, varias arquitecturas denominadas “MedGAN” aparecieron con el propósito de resolver problemas específicos de síntesis y traducción de dominios como MRI, CT, ultrasonido y PET.

Una adaptación destacada es con la combinación de Cycle-MedGAN, desarrollada por Armanious et al. [35] para abordar el problema de la traducción no supervisada entre modalidades médicas, inspirada en CycleGAN, esta arquitectura se diseñó para transformar imágenes de una modalidad a otra (e.g., de PET a CT o de MRI con movimiento a MRI corregida) sin necesidad de pares de imágenes perfectamente alineadas, a diferencia de enfoques tradicionales que podían producir imágenes borrosas o con pérdida de textura, Cycle-MedGAN introdujo pérdidas perceptuales y texturales además de la pérdida adversarial

y de ciclo-consistencia, esto permitió preservar mejor los detalles finos y las estructuras anatómicas, generando imágenes útiles tanto para inspección visual como para tareas clínicas posteriores, como segmentación o cuantificación de tejidos, estas variantes demuestran que, en el campo de la imagen médica, el término MedGAN se ha convertido en un “refugio” conceptual bajo el cual se agrupan distintas arquitecturas adaptadas a problemas específicos, mientras que Adaptive MedGAN se centra en la mejora de estabilidad y calidad en la generación directa desde ruido, Cycle-MedGAN amplía el alcance hacia la traducción de modalidades y la mejora de calidad de imágenes médicas no supervisadas, en conjunto, estos desarrollos resaltan el potencial de las GANs en aplicaciones clínicas: desde la aumentación de datos en escenarios con bases limitadas, hasta la reconstrucción y traducción multimodal en entornos diagnósticos.

Otra contribución particular es la de Macedo et al. [34] la cual presenta una versión de MedGAN orientada al diseño de moléculas (específicamente, generación de moléculas con scaffold quinolínico) empleando una combinación de WGAN y GCN para modelar moléculas como grafos (átomos = nodos, enlaces = aristas) y así generar estructuras químicas válidas, novedosas y sintetizables, el objetivo práctico es explorar y optimizar hiperparámetros (latent size, optimizadores, activaciones, unidades en $\mathcal{G}/\mathcal{D}$, etc.) para maximizar métricas relevantes en descubrimiento de fármacos: validity, connected validity, novelty, uniqueness y propiedades tipo drug-likeness (Lipinski), synthetic accessibility y predicciones de toxicidad, este trabajo documenta además un pipeline completo (SMILES $\rightarrow$ Mol $\rightarrow$ grafos $\rightarrow$ generación $\rightarrow$ deserialización) y los pasos de optimización y fine-tuning del modelo, este MedGAN muestra cómo el mismo nombre puede cubrir aplicaciones muy distintas.

La principal adaptación usada para esta tesis es la propuesta por Guo et al. [22] que introduce una arquitectura novedosa dentro de la familia de GANs, diseñada para la síntesis de imágenes médicas realistas, a diferencia del MedGAN original de Choi, enfocado en registros médicos discretos, esta implementación se centra específicamente en el dominio de la imagen médica, con el objetivo de mejorar la calidad visual y estructural de los datos sintéticos, el modelo se denomina “MedGAN” porque mantiene la orientación hacia aplicaciones médicas y conserva el principio generativo adversarial GAN, adaptando la arquitectura a los retos particulares del procesamiento de imágenes clínicas, desde el punto de vista arquitectónico.

Esta variante de Guo et al. [22] se fundamenta en un marco adaptativo de GAN que introduce mejoras tanto en $\mathcal{G}$ como en $\mathcal{D}$ para enfrentar problemas comunes en imágenes médicas, como el colapso de modos y la pérdida de detalles anatómicos, $\mathcal{G}$ está inspirado en estructuras de tipo encoder-decoder con capas convolucionales profundas, tal como se puede observar en la Figura 4-4, lo que le permite mapear vectores latentes a imágenes de alta resolución mientras conserva la coherencia espacial, $\mathcal{D}$ por su parte, se implementa como una red convolucional que no solo clasifica entre imágenes reales y sintéticas, sino que también se entrena con un esquema adaptativo que ajusta dinámicamente su retroalimentación a $\mathcal{G}$, esta adaptación consiste en modificar los coeficientes de la función de pérdida para balancear la calidad visual y la diversidad estadística de las imágenes generadas.

El modelo utiliza una función de pérdida híbrida, en la que se combina la pérdida adversarial tradicional con términos adicionales de regularización, la pérdida adversarial se basa en el clásico esquema de minimax entre $\mathcal{G}$ y $\mathcal{D}$: Adaptive MedGAN, propuesta para mejorar la calidad de generación en *datasets* de imágenes médicas heterogénea; es así como la innovación del modelo consiste en añadir factores adaptativos que modifican dinámicamente los pesos de cada término de la función de costo, con el fin de reducir inestabilidades y mejorar la fidelidad estructural.

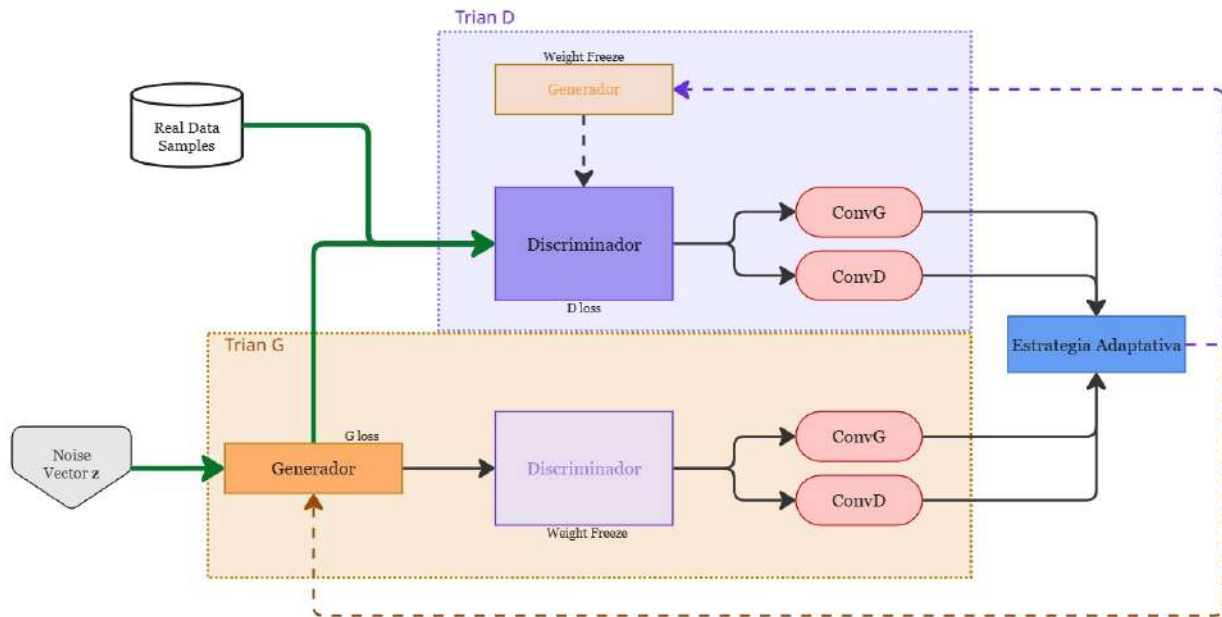


Figura 4-4: Flujo simplificado del modelo MedGAN usado, con la implementación adaptativa para evitar colapso de alguna de las redes neuronales.

En cuanto al entorno médico, Guo et al. [22] prueba su eficacia en imágenes médicas bidimensionales de distintas modalidades (e.g., MRI y CT), donde la preservación de detalles anatómicos y la reducción de artefactos son fundamentales, este enfoque permite no solo aumentar la disponibilidad de datos en entornos clínicos donde los conjuntos de imágenes suelen ser limitados, sino también mejorar la capacidad de modelos posteriores de clasificación y segmentación cuando se entrenan con imágenes sintéticas, respecto a las métricas de evaluación, el trabajo utiliza tanto indicadores cuantitativos como cualitativos, entre los cuantitativos destacan: SSIM, PSNR, FID, además de estas métricas, el artículo enfatiza la importancia de la evaluación clínica cualitativa, es decir, la inspección de radiólogos o especialistas médicos para verificar si las imágenes generadas conservan información diagnóstica útil.

4.3 Wasserstein Generative Adversarial Network (WGAN)

La distancia Wasserstein como punto de partida se puede comprender por Rubner et al. [114] por medio de la distancia de movimiento de tierra (*Earth-Mover Distance, EMD*), es una distancia entre distribuciones de probabilidad que cuantifica el coste mínimo necesario para transformar una distribución en otra trasladando “masa” a lo largo de un espacio métrico, por medio del transporte a partir de la optimización lineal para la recuperación de imágenes basada en el contenido, permite coincidencias parciales y se puede aplicar a representaciones de distribución de longitud de variable, esta formulación conecta el problema con la teoría del transporte óptimo iniciada por Monge y formalizada por Kantorovich.

El problema clásico de transporte óptimo se explica a profundidad por Tupitsa et al. [115], donde dado un “montón” de masa con distribución μ y un “montón” objetivo con distribución ν (ambas medidas de probabilidad), lograr reubicar la masa de μ para obtener ν con el costo total mínimo, siendo el costo de mover una unidad de masa desde x hasta y igual a $c(x, y)$, esto dio lugar a formulaciones clásicas como la de Monge (mapas de transporte) y la de Kantorovich (acoples o planes de transporte).

De manera superficial, Tupitsa et al. [115] expone que la formulación primal (Kantorovich) de la distancia W_p (Wasserstein de orden p) entre dos medidas de probabilidad μ y ν sobre un espacio métrico (X, d) viene dada por:

$$W_p(\mu, \nu) = \left(\inf_{\gamma \in \Gamma(\mu, \nu)} \int_{X \times X} d(x, y)^p d\gamma(x, y) \right)^{1/p}, \quad (4-5)$$

donde $\Gamma(\mu, \nu)$ es el conjunto de planes de transporte con márgenes μ y ν , es decir, se busca el acople γ que minimiza el coste total esperado de mover masa desde x a y , esta expresión formaliza la idea del “mínimo trabajo” para transformar μ en ν .

Teniendo en cuenta todo esto, se puede ahondar en el modelo WGAN propuesto por Arjovsky et al. [116] como una alternativa a las GANs clásicas con el fin de mejorar la estabilidad del entrenamiento, superar problemas relacionados a divergencias usuales como la JSD y curvas útiles para depuración y búsqueda de hiperparámetros; su idea central radica en reemplazar la función de pérdida tradicional por una basada en la distancia de Wasserstein (EMD). En su artículo fundacional, los autores proponen reemplazar $\mathcal{D}$ tradicional por un “critic”, este critic no tiene el propósito de clasificar entre real o falso, sino de estimar la distancia de Wasserstein ---específicamente la distancia W_1 --- entre la distribución real P_r y la distribución generada P_θ , este cambio aporta mejora en la estabilidad del aprendizaje, y provee curvas de pérdida significativas, es decir, la pérdida deja de ser una señal poco interpretable y pasa a correlacionarse mejor con la calidad de las muestras generadas, facilitando la depuración y la selección de hiperparámetros.

Como detallan Arjovsky et al. [116], $W(P_r, P_\theta)$ puede tener mejores propiedades cuando se optimiza $JSD(P_r, P_\theta)$, la dualidad Kantorovich-Rubinstein dice que:

$$W(P_r, P_\theta) = \sup_{\|f\|_L \leq 1} (\mathbb{E}_{x \sim P_r}[f(x)] - \mathbb{E}_{x \sim P_\theta}[f(x)]) \quad (4-6)$$

donde que la estimación por parte del critic sea válida como una aproximación de la distancia W_1 , es necesario que el critic pertenezca al conjunto de funciones 1-Lipschitz, se puede notar que al reemplazar $\|f\|_L \leq 1$ por $\|f\|_L \leq K$ se obtiene $K \cdot W(P_r, P_\theta)$, por ende si se tiene una familia parametrizada de funciones $\{f_w\}_{w \in \mathcal{W}}$ donde todas las K -Lipschitz para una constante K se podría resolver el problema:

$$\max_{w \in \mathcal{W}} \mathbb{E}_{x \sim P_r}[f_w(x)] - \mathbb{E}_{z \sim p(z)}[f_w(g_\theta(z))] \quad (4-7)$$

este proceso daría un calculo de $W(P_r, P_g)$ hasta una constante multiplicativa.

Todo esto permite que los autores Arjovsky et al. [116] establezcan un teorema el cual dice:

“Sea P_r cualquier distribución. Sea P_θ la distribución de $g_\theta(Z)$, donde Z es una variable aleatoria con densidad p y g_θ es una función que satisface el supuesto 1. Entonces, existe una solución $f : X \rightarrow \mathbb{R}$ para el problema.

$$\max_{\|f\|_L \leq 1} (\mathbb{E}_{x \sim P_r}[f(x)] - \mathbb{E}_{x \sim P_\theta}[f(x)]) \quad (4-8)$$

y tenemos

$$\nabla_\theta W(P_r, P_\theta) = -\mathbb{E}_{z \sim p(z)}[\nabla_\theta f(g_\theta(z))] \quad (4-9)$$

cuando ambos términos están definidos.”

Este teorema asegura varios controles durante el entrenamiento tales como, que la distancia de Wasserstein sea continua en los parámetros θ , sea diferenciable en casi todas sus partes, no se sature incluso cuando las distribuciones no se tocan permitiendo estabilidad en el entrenamiento, evitando que $\mathcal{D}$ aprenda demasiado rápido, y produzca gradientes utilizables para $\mathcal{G}$ incluso si $\mathcal{G}$ esta produciendo basura, curvas de perdidas interpretables y permite aproximar Wasserstein en ANN.

Por ultimo los autores Arjovsky et al. [116] implementan la restricción Lipschitz mediante recorte (“clipping”) de los pesos del critic: después de cada actualización de sus parámetros, se restringe cada peso a un rango compacto $[-c, +c]$ para alguna constante c pequeña, forzando que la red no crezca demasiado y actúe como una función con derivadas acotadas, acercando su comportamiento al de una función 1-Lipschitz, sin embargo puede generar efectos adversos dependiendo del valor de c , si c es muy grande, los pesos pueden tardar mucho en saturarse y el critic puede tener demasiada libertad, lo que dificulta su convergencia.

4.4 Vector-Quantized GAN (VQ-GAN)

En la generación de imágenes mediante GANs se han realizado varias propuestas con diferentes modelos que han tenido avances en estabilidad, fidelidad y eficiencia, sin embargo, la mayoría de estos enfoques trabajan con *latentes continuos* dependiendo de arquitecturas generativas puras, sin incorporar procesos de *comprensión perceptual*, en este contexto surge VQ-GAN, que integra un *autoencoder con latentes discretos*, cuantización vectorial y pérdidas perceptuales, logrando una reconstrucción realista de imágenes de alta resolución, es una técnica clásica en procesamiento de señales que permite representar datos continuos en un espacio discreto de símbolos, seleccionados de un conjunto finito denominado código o diccionario. Su aplicación en el contexto de modelos generativos fue introducida de manera innovadora en el (*Vector Quantized Variational Autoencoder, VQ-VAE*) propuesto por Van den Oord et al. (2017) [117], la motivación principal radica en superar las limitaciones de los VAE, que, al modelar distribuciones continuas gaussianas, suelen producir imágenes borrosas debido al suavizado probabilístico en el espacio latente, este modelo no solo permite compresión y reconstrucción perceptual, si no que también facilita la tokenización de imágenes, habilitando el uso de *Transformers* para modelar datos visuales como secuencias discretas [20].

Un *autoencoder* busca aprender una representación comprimida de los datos, consiste en un *encoder* E y un *decoder* G , el encoder transforma una imagen de entrada x en una representación latente continua z_e , luego, en lugar de transmitir directamente este vector al *decoder*, se proyecta al vector más cercano de un diccionario discreto de *códigos embeddings* $e \in \mathbb{R}^K$, donde K es el número de entradas en el diccionario, la operación de cuantización vectorial se define como:

$$z_q(x) = e_k \quad \text{donde} \quad k = \arg \min_j \|z_e(x) - e_j\|_2, \quad (4-10)$$

donde e_j representa el j -ésimo vector del diccionario de *embeddings*. De esta forma, cada representación continua del *encoder* se sustituye por un índice discreto en el diccionario.

Este proceso convierte el espacio latente continuo que es simplemente una matriz de valores reales (como un feature map extraído de una CNN) en una representación discreta, el *codebook* puede entenderse como una tabla de “símbolos visuales”, el *decoder* tiene como función reconstruir la imagen $\hat{x} = G(z_q)$ a partir de la latente o versión cuantizada z_q , cada vector de este mapa se cuantiza, es decir, se fuerza a que tome el valor mas cercano de un conjunto limitado de representaciones, es decir, el codebook, su rol es crítico: debe expandir el mapa comprimido y reinyectar detalles visuales que aseguren que la reconstrucción sea perceptualmente realista [20], en otras palabras, el decoder convierte una secuencia de “*tokens* visuales” en una imagen completa y realista, en aplicaciones médicas, este paso puede ser crucial para preservar estructuras anatómicas relevantes y evitar artefactos irreales.

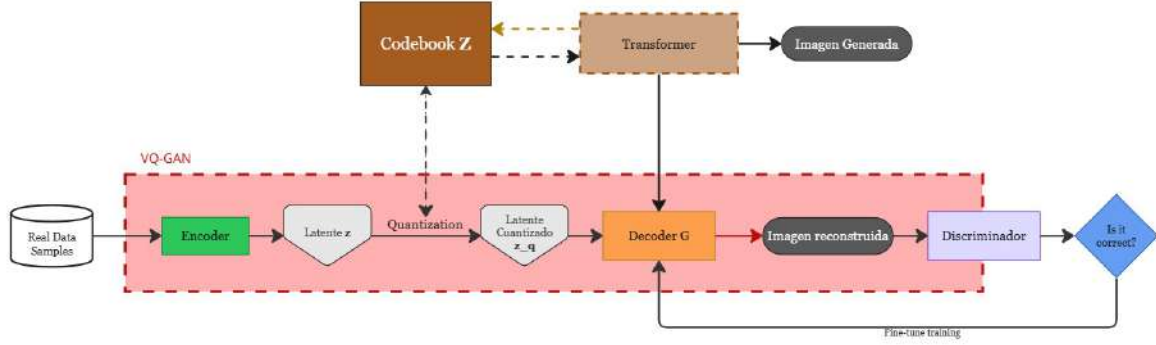


Figura 4-5: Flujo simplificado del modelo VQ-GAN, donde se aprecia la implementación de los *autoencoders*, el transformador y el codebook.

El entrenamiento de un VQ-VAE requiere una función de pérdida que equilibre la reconstrucción de la imagen con la actualización adecuada del diccionario. Para ello se define la siguiente pérdida total:

$$\mathcal{L} = \|x - \hat{x}\|_2^2 + \|\text{sg}[z_e(x)] - e\|_2^2 + \beta \|z_e(x) - \text{sg}[e]\|_2^2 \quad (4-11)$$

donde el primer término corresponde al error de reconstrucción (x es la imagen original y $\hat{x}$ la reconstruida), el segundo término (*codebook loss*) actualiza el diccionario para acercar los *embeddings* al encoder, el tercero (*commitment loss*) penaliza al encoder para que permanezca próximo al *embedding* asignado, evitando que el encoder se desplace arbitrariamente, la operación $\text{sg}[\cdot]$ (*stop-gradient*) asegura que ciertos gradientes no se propaguen durante el entrenamiento. El parámetro β controla el compromiso entre el encoder y el diccionario; valores inadecuados pueden producir *colapso del diccionario* (cuando solo unos pocos *embeddings* son usados), reduciendo la diversidad representacional [117].

Este mecanismo de cuantización representa un cambio conceptual importante respecto a los VAE y GAN clásicos: mientras los primeros imponen una regularización probabilística (KL-divergence) y los segundos operan en espacios continuos de ruido, los VQ-VAE utilizan un espacio discreto latente, lo que permite un modelado más cercano a la naturaleza simbólica de muchos datos (por ejemplo, el lenguaje, o bien patrones discretos en imágenes). Posteriormente, el VQ-GAN de Esser et al. (2021) [20] integró este mismo principio de cuantización con una pérdida adversarial, logrando generar imágenes de alta resolución con gran calidad perceptual y manteniendo un espacio latente eficiente y discreto.

Para dar mas contexto, la *cuantización* es el proceso de aproximar un valor continuo a uno de un conjunto discreto predefinido, en VQ-GAN, cada vector del mapa latente z_e es reemplazado por el vector más cercano de un diccionario (codebook) $E = \{e_1, e_2, \dots, e_K\}$,

donde K es el número de entradas, matemáticamente:

$$k^* = \arg \min_k \|z_e - e_k\|^2, \quad z_q = e_{k^*}. \quad (4-12)$$

El codebook actúa como una “tabla de símbolos visuales”, donde cada entrada representa un patrón característico (ejemplo: bordes, texturas, formas recurrentes), esta discretización evita que el modelo se apoye únicamente en representaciones continuas muy ricas, lo cual puede llevar a problemas como el colapso posterior en VAEs [86], en ecocardiografía, el codebook puede aprender patrones recurrentes como el *speckle* ultrasónico, bordes valvulares o el movimiento del ventrículo.

Para entrenar el *autoencoder*, VQ-GAN utiliza una combinación de pérdidas perceptuales y adversariales, en lugar de entrenar solo con pérdidas como el error cuadrático medio (MSE) o el error absoluto medio (MAE), las pérdidas perceptuales comparan imágenes en un espacio de features extraídos por redes convolucionales pre-entrenadas (ej. VGG), en lugar de píxel a píxel.

1. *Pérdida de reconstrucción perceptual (LPIPS):*

$$L_{\text{perc}}(x, \hat{x}) = \sum_l \frac{1}{H_l W_l} \|w_l \odot (\phi_l(x) - \phi_l(\hat{x}))\|_2^2 \quad (4-13)$$

donde ϕ_l son features de una red preentrenada, esta pérdida se correlaciona mejor con la percepción humana que métricas tradicionales como PSNR o SSIM [105], esta pérdida es útil porque, el ojo humano es más sensible a texturas y estructuras locales que a diferencias globales de intensidad.

2. *Pérdida adversarial con PatchGAN:* en lugar de un discriminador global, se utiliza un discriminador local por parches, propuesto originalmente en pix2pix decide si la imagen completa es real o falsa, un PatchGAN analiza parches locales de tamaño $N \times N$, que obliga a que cada región de la imagen sea realista [100]:

$$L_D = \mathbb{E}_x [\max(0, 1 - D(x))] + \mathbb{E}_{\hat{x}} [\max(0, 1 + D(\hat{x}))] \quad (4-14)$$

$$L_G = -\mathbb{E}_{\hat{x}} [D(\hat{x})] \quad (4-15)$$

Tiene varias ventajas como: centrarse en texturas locales, lo cual es crucial para mantener detalles finos, es más ligero computacionalmente que un discriminador global, promueve que cada parche de la imagen sea plausible, en lugar de solo un aspecto global, lo cual explica por que VQ-GAN logra reconstrucciones con alto detalle perceptual, esto es especialmente útil en ecocardiografía, ya que permite preservar la textura granular del eco (*speckle*) y la coherencia anatómica local.

3. *Pérdida de codebook:* asegura que los vectores del diccionario se actualicen adecuadamente y que el encoder se comprometa con los códigos seleccionados, evitando dead codes [117].

4.5 Comparación entre diferentes modelos GANs

Modelo	Tipo de GAN / Supervisión	Núcleo arquitectónico	Truco/ pérdida clave	Datos requeridos	Pros	Contras
ProgGAN	No condicional	Crecimiento progresivo (res)	Curriculum por resolución	Muchos	Estabilidad a ultra-alta res	Artefactos de transición; pipeline complejo [112].
StyleGAN2	No condicional	Modulated conv + weight demodulation, skip-toRGB	Path-length reg.	Muchos (<BigGAN)	Control multi-escala, texturas finas, menos artefactos	Aún exige datos; sin ADA puede sobreajustar [110].
StyleGAN2-ADA	No condicional	Igual a StyleGAN2	ADA (p adaptativo)	Pocos-miles	Mantiene calidad con <i>data-sets</i> pequeños; gran opción clínica	Ajuste de ADA añade complejidad; sigue siendo no condicional [19].
StyleGAN3	No condicional	Filtros/upsampling alias-free	Band-limiting, equivariancia	Muchos	Coherencia geométrica (rot/trasl)	Entrenamiento algo más delicado; coste similar [118].
DCGAN	No condicional, no pareado	Conv/Deconv simples	BCE adversarial	Miles-decenas de miles	Sencillo, estable “básico”	Control limitado; calidad moderada [88].
BigGAN	Condicional por clase	ResNets muy grandes	Truncation trick, ortho-reg	Muchísimos + cómputo	SOTA en fidelidad/diversidad	Costoso; menos práctico en clínica [119].
WGAN / WGAN-GP	— (pérdida alternativa)	Critic Lipschitz	Wasserstein + GP	Variable	Más estabilidad y métricas	Aporta pérdida, no arquitectura [116].
Pix2Pix	Condicional pareado ($A \rightarrow B$)	U-Net + Patch-GAN	cGAN + L1	Pares	Traducciones precisas	Requiere pares, menos diversidad [100].
CycleGAN	No pareado ($X \leftrightarrow Y$)	Dos G + dos D	Ciclo-consistencia	No pares	Útil sin pares	Riesgo de mapeos espurios / sesgos [120].
VQ-GAN	<i>Autoencoder</i> + GAN	VQ codebook discreto	GAN + perceptual	Variable	Texturas nítidas, alta res, compresión semántica	Complejidad (codebook), puede “bloquear” modos si mal entrenado [20].
VAE-GAN	Híbrido (likelihood + adversarial)	Encoder/Decoder + D	Reconstr. feature-wise	Menos que GAN puro	Latentes útiles + fidelidad >VAE	Aún menos nítido que StyleGAN2-like en alta res [121].
MediGAN	— (ecosistema)	N/A	Catálogo, evaluación	N/A	Acceso rápido a modelos médicos	Dependes de modelos existentes/dominios compatibles [122].

Tabla 4-2: Comparación de diferentes GANs en cuanto a arquitectura, funcionamiento y adecuación

5 Metodología

El presente trabajo tiene como objetivo la generación de imágenes ecocardiográficas sintéticas a partir de arquitecturas basadas en GANs, para ello se implementaron y evaluaron diferentes modelos, con el fin de evaluar el desempeño en la creación de imágenes medicas ecocardiograficas realistas, se consideraron cuatro arquitecturas principales, *StyleGAN2-ADA*, *MedGAN*, *WGAN* y *VQ_GAN*, seleccionadas debido a sus aportes representativos en la generación de imágenes de alta calidad, aparición en trabajos de investigación o mención de implementación en técnicas de dominio medico, cada arquitectura fue entrenada bajo múltiples configuraciones de hiperparámetros con el fin de evaluar la influencia de dichas variaciones en la calidad, realismo y la sensibilidad de diversos cambios en cada arquitectura.

5.1 Preprocesamiento

El conjunto de datos corresponde al repositorio público EchoNet-Dynamic correspondiente a la Facultad de Medicina de la Universidad de Stanford, la base original incluye 10,030 videos en formato .avi correspondientes a distintos pacientes, cada uno con una resolución de 112×112 píxeles por *frame*, cada video consta de 30 a 140 *frames* aproximadamente, dada la naturaleza de las GANs, que requieren imágenes preprocesadas y homogéneas para un entrenamiento estable, fue necesario diseñar un *pipeline* específico de preparación de los datos que permitió obtener un total de 40,120 imágenes estáticas a partir de los videos.

El preprocesamiento se realizo en varias etapas:

- *Conversión a escala de grises:* Cada *frame* fue transformado de RGB a escala de grises, ya que había cierta cantidad de videos de la adquisición original (menor al 5 %) con un tinte azulado, tal como se puede observar en la Figura 5-1, esto aseguró homogeneidad cromática evitando que la red generativa aprendiera variaciones irrelevantes de color.

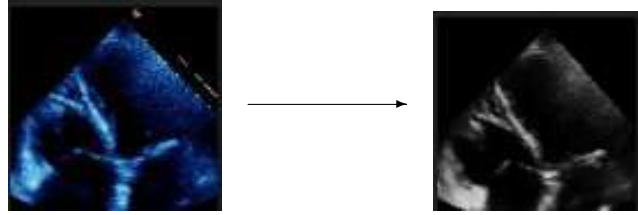


Figura 5-1: Ecocardiograma con tinte azulado pasada a escala de grises.

- *Aplicación de máscara anatómica:* Se implementó una máscara combinada que integró una región o mascara triangular y una semi-elipse combinada que se puede observar en la Figura 5-2, diseñada para restringir el área de interés a las 4 cámaras del corazón, reduciendo ruido proveniente de regiones externas (texto superpuesto, bordes negros o estructuras extracardiácas), fue necesario hacer la mascara con valores exactos de manera que se lograra imitar el campo de interés existente en las imágenes, estos valores se pueden observar en el pseudocódigo del Algoritmo 2, esta segmentación previa permitió que las redes se enfocaran en las dinámicas cardíacas relevantes, facilitando un entrenamiento más eficiente y reduciendo artefactos.

Algorithm 2 Construcción de máscara combinada para ecocardiograma

```

1: function CREARMÁSCARA
2:    $shape \leftarrow (112, 112)$                                 ▷ Tamaño de imagen en escala de grises
3:    $top \leftarrow (60, 5)$                                     ▷ Vértice superior del triángulo
4:    $left \leftarrow (-15, 80)$                                 ▷ Vértice izquierdo del triángulo
5:    $right \leftarrow (127, 80)$                                ▷ Vértice derecho del triángulo
6:    $center \leftarrow (55, 78)$                                 ▷ Centro de la elipse
7:    $axes \leftarrow (70, 40)$                                 ▷ Ejes mayor y menor de la elipse
8:    $angle \leftarrow 0, startAngle \leftarrow 0, endAngle \leftarrow 180$   ▷ Semi-elipse inferior
9:    $maskT \leftarrow$  matriz de ceros de tamaño  $shape$ 
10:   $points \leftarrow$  arreglo con  $top, left, right$ 
11:   $mask\_triangular \leftarrow$  RELLENARPOLÍGONOCONVEXO( $maskT, points, 255$ )
12:   $maskC \leftarrow$  matriz de ceros de tamaño  $shape$ 
13:  DIBUJARÉLIPSE( $maskC, center, axes, angle, startAngle, endAngle, 255$ )
14:   $combined\_mask \leftarrow$  COMBINARMÁSCARAS( $mask\_triangular, maskC$ )
15:  return  $combined\_mask$ 
16: end function
17: function APLICARMÁSCARA( $imagen, mscara$ )
18:   return AND_BITWISE( $imagen, mscara$ )
19: end function

```



Figura 5-2: Combinación de mascara triangular con mascara de semi elipse

- *Redimensión y normalización*: Tras aplicar la máscara, los *frames* fueron redimensionados desde 112×112 a 128×128 píxeles --dado que es el valor en potencia de 2 mas cercano evitando un redimensionamiento brusco donde se pierda ruido *speakle* relevante-- utilizando interpolación Lanczos4, técnica adecuada para preservar detalles finos en estructuras anatómicas, posteriormente, se aplicó normalización de intensidades para asegurar que los valores de entrada estuvieran en un rango estándar, al estar con resolución en potencia de 2 se garantiza que el entrenamiento en los GANs sea totalmente eficiente y estable.
- *Detección de fases cardíacas (sístole y diástole)*: Se realizó un proceso de binarización por umbral adaptado al área enmascarada, seguido de un conteo de píxeles blancos para cada *frame*, las cavidades del corazón presentan menor área blanca en la fase diastólica ya que el corazón al expandirse ensancha las cavidades y las paredes del corazón (en blanco por la binarización) presenta una menor cantidad de pixeles y de forma contraria es menor en la fase sistólica tal y como se ve en la Figura 5-3, este procedimiento permitió identificar de forma sencilla los *frames* correspondientes a ambas fases.

Algorithm 3 Construcción de máscara combinada para ecocardiograma

```

1: function DETECTARSÍSTOLEDIÁSTOLE(frames_gray, mscara, umbral  $\leftarrow$  30)
2:   counts  $\leftarrow$  lista vacía
3:   for all frame en frames_gray do
4:     masked  $\leftarrow$  APLICARMÁSCARA(frame, mscara)
5:     binary  $\leftarrow$  UMBRALIZAR(masked, umbral)
6:     AGREGAR(counts, número de píxeles blancos en binary)
7:   end for
8:   sstole  $\leftarrow$  índice de máximo en counts
9:   distole  $\leftarrow$  índice de mínimo en counts
10:  return sstole, distole, counts
11: end function

```

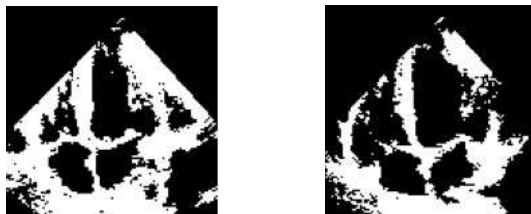


Figura 5-3: Frames binarizados del video 0X102C51641C321436 en los momentos en que se detecto diástole y sístole.

- *Selección de frames representativos:* De cada video se extrajeron cuatro imágenes clave: el primer *frame*, el *frame* correspondiente a sístole, el *frame* correspondiente a diástole y el último *frame*, con ello se garantizó una representación balanceada de las dinámicas cardíacas, preservando tanto información temporal como estructural.
- *Suavizado y homogeneización:* Como etapa final, las imágenes fueron sometidas a un filtro bilateral que redujo ruido de alta frecuencia preservando bordes y contornos anatómicos, evitando inclusión de ruido erróneo al ruido *speckle* y evitando artefactos indeseados productos del redimensionamiento realizado anteriormente y así optimizando la calidad de entrada al modelo.

El resultado de este *pipeline* fue un *dataset* balanceado de 40,120 imágenes ecocardiográficas estáticas en escala de grises y resolución 128×128 píxeles, adecuadamente preprocesadas para su uso en arquitecturas GANs, dicho preprocesamiento garantizó la eliminación de sesgos de color, la reducción de artefactos y la preservación de información anatómica esencial, factores fundamentales para un entrenamiento estable y la generación de imágenes sintéticas de alta fidelidad.

Con el propósito de optimizar la manipulación del conjunto de datos y facilitar su carga en Google Drive, la carpeta que contenía las 40,120 imágenes ecocardiográficas fue comprimida en un archivo ZIP mediante un *script* en Python, esta estrategia permitió empaquetar de forma estructurada todos los archivos sin necesidad de transferirlos manualmente uno a uno, lo cual habría sido altamente ineficiente y propenso a errores, posteriormente, en el entorno de Google Colab, el archivo comprimido fue descomprimido automáticamente con otro *script*, que además se encargó de omitir directorios vacíos y reconstruir únicamente los archivos de imagen en el destino especificado, de este modo, se garantizó una carga rápida, ordenada y reproducible del *dataset*, reduciendo significativamente el tiempo requerido para la preparación de datos y preservando la integridad de las imágenes para su posterior uso en el entrenamiento de los modelos.

5.2 Configuraciones experimentales

En esta sección se describen las configuraciones experimentales utilizadas durante el entrenamiento de los diferentes modelos, antes de eso es necesario destacar que cada modelo está compuesto por dos tipos de variables: los *parámetros* y los *hiperparámetros*. Los *parámetros* son aquellos valores internos que la red ajusta automáticamente durante el proceso de optimización, se actualizan en cada iteración del entrenamiento con el objetivo de minimizar la función de pérdida, algunos ejemplos de parámetros son: los pesos y sesgos de las capas, los estilos latentes en el espacio intermedio o los mapas de ruido que se incorporan a las imágenes durante la síntesis, estos valores no son definidos manualmente, sino que emergen del propio aprendizaje del modelo.

Por otro lado, los *hiperparámetros* corresponden a valores que deben fijarse del entrenamiento y que condicionan directamente la dinámica de aprendizaje, determinan aspectos críticos como la capacidad de generalización, la estabilidad del entrenamiento y la calidad final de las imágenes generadas, entre los hiperparámetros más relevantes se incluyen la tasa de aprendizaje, el tamaño de lote (*batch size*), la intensidad de la regularización aplicada a $\mathcal{D}$, la duración del entrenamiento expresada en miles de imágenes procesadas, y la frecuencia con la que se almacenan los modelos intermedios o *snapshots*, según el modelo implementado puede tener mas o menos hiperparámetros que requieren variación, su importancia radica en que pequeños cambios en sus valores pueden tener efectos significativos en el modelo.

Por ejemplo, un *batch size* más grande suele aportar mayor estabilidad al entrenamiento, pero incrementa considerablemente el consumo de memoria; un LR elevado puede acelerar el proceso de convergencia inicial, aunque con el riesgo de que la red colapse o genere resultados inconsistentes, de manera similar, aplicar aumentos de datos demasiado agresivos ayuda a prevenir el sobreajuste en conjuntos de datos pequeños, pero puede dificultar que $\mathcal{G}$ aprenda a reproducir detalles finos en las imágenes.

5.2.1 Configuraciones base del setup

- **-data:** Ruta al conjunto de datos. Puede ser un archivo .zip con imágenes o una carpeta, es un hiperparametro obligatorio.
- **-outdir:** Carpeta de salida donde se guardan resultados (modelos .pkl, imágenes de muestra, logs), necesario para guardar checkpoints.
- **-resume:** Ruta a un modelo .pkl existente para reanudar entrenamiento o usar como preentrenado, mejora la convergencia usando transferencia.

5.2.2 Hiperparámetros StyleGAN2-ADA

En el caso particular de StyleGAN2-ADA, el uso de ADA permite incrementar artificialmente la diversidad de los ejemplos de entrenamiento de manera controlada --esto adquiere una especial relevancia en datos médicos que tienden a ser limitados--, ajustando dinámicamente la intensidad de las transformaciones, esto lo convierte en una herramienta clave para evitar que el modelo memorice imágenes y, en cambio, logre aprender representaciones más generalizables, algunos de sus hiperparámetros configurables son los siguientes:

- Configuraciones principales.
 - **-gpus**: Número de GPUs a usar, aunque en un entorno como colab se usa 1 solo.
 - **-batch**: Tamaño de *batch* global o por GPU, modifica la estabilidad y el consumo de memoria, los valores típicos son entre 16 y 32, entre mayor sea, el entrenamiento es mas estable, pero requiere mucha VRAM.
 - **-mirror**: Reflejo horizontal aleatorio de imágenes, 1 significa que esta activado, 0 es desactivado, por defecto esta activado si el *dataset* no contiene caras con orientación fija, aunque en ecocardiografías no es adecuado cambiar el eje anatómico, eso es sumamente relevante.
 - **-snap**: Cada cuantos *ticks* guardar *snapshot* y métricas por defecto es 50, bajo mas *checkpoints* y mas espacio, un valor alto da a menos guardados.
- Regulación, *augmentation*.
 - **-gamma**: Coeficiente de R1 para el discriminador, controla sobreajuste y estabilidad: Al aumentar, menos sobreajuste pero entrenamiento mas lento, bajarlo puede sobreajustar si *dataset* es pequeño, para *dataset* pequeños conviene que sea mayor a 10.
 - **-aug**: Estrategia de *data augmentation*, los tipos de aumentos posibles son: *noaug*, *ADA* (Adaptativa), *fixed* (aumentos fijos), por defecto es ADA, recomendado en *dataset* pequeños.
 - **-target**: Tasa objetivo de activación para ADA, modifica el nivel de aumentos que el sistema intentará mantener, por defecto el valor es 0.6, y el rango recomendado es 0.5-0.8.
 - **-p**: (solo para *fixed*): 0-1, probabilidad de aplicar aumentos.
- Optimizacion fina.
 - **-kimg**: Duración del entrenamiento en miles de imágenes procesadas, mas kimg mas tiempo para aprender, útil para test cortos (e.g., 500 es equivalente a 500,000 imágenes recorridas en el entrenamiento).

- **-lrate:** Tasa de aprendizaje (LR), entre mayor sea el aprendizaje es más rápido pero inestable; menor es más estable pero le toma tiempo converger.
- **-metrics:** StyleGAN por defecto tiene una métricas que permite calcular (fid50kfull, prc10k3, ppl2_wend...) útil para análisis.
- **-workers:** Numero de hilos de lectura de datos, mayor carga mas rápida, en colab se usa 2-4.

Para evaluar el impacto de los hiperparámetros, se diseñaron 8 configuraciones con los hiperparámetros que mayor relevancia pudieran tener en los resultados del entrenamiento, cada experimento, identificado como E1, E2, etc., representa una combinación particular de valores, de esta manera, de forma organizada y etiquetada fue posible analizar cómo las distintas configuraciones afectan tanto la evolución de las funciones de pérdida de $\mathcal{G}$ y $\mathcal{D}$, como las métricas de calidad utilizadas, en la Tabla 5-1 se puede observar todas las configuraciones junto con los hiperparámetros mas importantes tomados para la variación junto con las ventajas y desventajas que podría ofrecer esa configuración.

Tabla 5-1: Combinaciones implementadas para cada configuración y evaluación de pros y contras de los diferentes valores de hiperparámetros en StyleGAN2-ADA

Experimento	Hiper	Pros	Contras
E1 - Original Básico	batch=32, gamma=32, lr=0.002, aug=ada, target=0.7	Estabilidad y resistencia al sobreajuste, el batch grande aporta estabilidad al gradiente, imágenes mas consistentes, el gamma alto refuerza la regularización, menos sobreajuste en <i>dataset</i> pequeños o medianos, el LR estándar ofrece equilibrio entre velocidad y estabilidad, con un ADA suficiente para enriquecer el <i>dataset</i> sin saturar.	Sacrifica detalles pequeños, gamma tan alto también provoca que el discriminador aprenda muy lento, posible perdida en detalle fino.
E2 - Learning rate mas alto	batch=32, gamma=32, lr=0.004, aug=ada, target=0.6	Exploración rápida de resultados, doblar el LR acelera la convergencia, útil para un entrenamiento rápido, mantiene regularidad de E2, menos aumentos por ende menos deformación artificial.	Riesgo de que la calidad no sea consistente, puede generar inestabilidad, oscilaciones fuertes de perdidas, imágenes borrosas o colapso, y un gamma de 32 puede ralentizar aun mas la capacidad de seguir el ritmo de un LR tan alto.
E3 - Lote pequeño mas exploración	batch=16, gamma=32, lr=0.002, aug=ada, target=0.6	Flexibilidad (resolución mayor posible), útil en un Colab limitado, batch mas pequeño consume menos memoria, permite entrenar en resoluciones mayores, hay equilibrio en LR, y el gamma es lo suficiente para ayudar con posibles sobreajustes.	Sacrifica estabilidad en perdidas, <i>batch</i> pequeño por ende gradientes mas ruidosos, entrenamiento menos estable, métricas pueden fluctuar, menos muestras por iteración por ende convergencia mas lenta.
E4 - Regularización baja	batch=32, gamma=8, lr=0.002, aug=ada, target=0.6	Mejor detalle visual a corto plazo, el gamma permite que el discriminador aprenda rápido, puede captar mas detalles finos, el <i>batch</i> aporta estabilidad al gradiente, LR estándar mantiene balance.	Peor generalización y riesgo de colapso, gamma bajo disminuye regularización, riesgo de sobreajuste fuerte en <i>dataset</i> pequeños, puede generar imágenes que parecen realistas en entrenamiento pero no generalizan bien (FID alto).

Experimento	Hiper	Pros	Contras
E6 - Learning rate mas bajo	batch=32, gamma=32, lrate=0.001, aug=ada, target=0.6	Estabilidad a largo plazo, ideal si se tiene paciencia para entrenar mas tiempo, LR bajo permite entrenamiento muy estable, menos riesgo de colapso, el <i>batch</i> y <i>gamma</i> mantiene regularización y estabilidad.	Convergencia lenta, puede necesitar muchos mas king para alcanzar calidad, puede atascarse en un "punto seguro" sin mejorar detalles.
E7 - <i>Augmentation</i> más agresivo	batch=32, gamma=32, lrate=0.002, aug=ada, target=0.85	Generalización fuerte, mas aumentos es útil para evitar sobreajuste en <i>dataset</i> pequeños, mantiene regularización y estabilidad base estable.	Sacrifica fidelidad a la distribución real del <i>dataset</i> , muchos aumentos puede distorsionar demasiado las imágenes y el generador aprende ruido o patrones irreales.
E8 - <i>Augmentation</i> más suave	batch=32, gamma=32, lrate=0.002, aug=ada, target=0.5	Mejor fidelidad en corto plazo, menos aumentos mas fidelidad a los datos originales, batch y gamma estables.	Riesgo de sobreajuste, FID puede empeorar al evaluar con imágenes nuevas, en <i>dataset</i> pequeño menos aumentos aumenta la probabilidad de sobreajuste, el modelo puede memorizar patrones de entrenamiento en lugar de generalizar.
E9 - Sin <i>augmentation</i>	batch=32, gamma=32, lrate=0.002, aug=noaug, target=0.6	Muy buen resultado visual de entrenamiento, el generador aprende directamente la distribución sin transformaciones, fidelidad máxima al <i>dataset</i> entrenado.	No generaliza, útil solo como experimento de control, con <i>dataset</i> pequeño y sin aumentos sobreajuste seguro, se disparan métricas como FID al evaluar muestras no vistas, generalización pobre.

En síntesis, el análisis comparativo de los parámetros e hiperparámetros no solo permite identificar la configuración óptima para el dominio ecocardiográfico, sino también comprender las interacciones entre los distintos factores que influyen en el desempeño del modelo, esta comprensión es esencial para el diseño de futuras investigaciones que busquen aplicar arquitecturas generativas a imágenes médicas, ya que proporciona un marco de referencia sobre las ventajas y limitaciones de cada decisión experimental.

5.2.3 Hiperparámetros MedGAN

En el caso de MedGAN (implementación basada en AdaGAN con $\mathcal{G}$ y $\mathcal{D}$ alternativos), los hiperparámetros controlan tanto la dinámica del entrenamiento adversarial como la capacidad de generalización de las arquitecturas evaluadas (MLP, DCGAN y ResNet), a diferencia de StyleGAN2-ADA, este modelo no emplea mecanismos avanzados de regularización adaptativa como ADA, sino que depende de la alternancia y ajuste dinámico entre las actualizaciones de $\mathcal{G}$ y $\mathcal{D}$, siguiendo umbrales de control (T_1 , T_2) que determinan cuándo priorizar uno sobre el otro, los hiperparámetros configurables más relevantes son los siguientes:

- Configuración del modelo:

- **-batch-size**: Número de imágenes procesadas en paralelo, valores típicos entre 32 y 128.
 - **-model**: Arquitectura a emplear para $\mathcal{G}$ y $\mathcal{D}$ (`mlp`, `drgan`, `resnet`).
 - **-nc**: número de canales de entrada de la imagen.
 - **-nz**: Dimensión del vector latente de entrada a $\mathcal{G}$, controla la capacidad de variabilidad de las muestras generadas, si es mas grande es mas expresivo y diverso pero mas difícil de entrenar, si es mas pequeño es lo contrario y puede sufrir colapso de modo.
 - **-ngf / -ndf**: Número de filtros iniciales en $\mathcal{G}$ y $\mathcal{D}$, escala nivel de filtros en convoluciones, actúan como ancho de la red, mas grande mas potentes y pesadas, as pequeño menos capacidad.
 - **-n_extra_layers**: Añade capas de convolución adicionales, útil si el *dataset* es muy complejo y requiere mas profundidad, aporta mas poder de representación, pero aumenta riesgo de sobreajuste.
- Optimización:
- **-niter**: Número total de épocas objetivo.
 - **-lrG / -lrD**: Tasa de aprendizaje inicial de $\mathcal{G}$ y $\mathcal{D}$, comúnmente 2×10^{-5} .
 - **-adam**: Algoritmo de optimización (`adam`, `sgd`), puede usar Adam o RMSprop.
 - **-beta1, -beta2**: Parámetros de momento de Adam, valores típicos 0.5 y 0.999.
- Regulación y dinámica adversarial:
- **-T1, -T2**: Umbrales de control que definen si se entrena más a $\mathcal{G}$ o a $\mathcal{D}$, permitiendo un equilibrio dinámico en el proceso., con esto se evita colapso, se mantiene el juego equilibrado, si la perdida de $\mathcal{D}$ es demasiado baja se entrena mas a $\mathcal{G}$ y viceversa.
 - **-warmup-epochs**: Número de épocas iniciales donde se prioriza el entrenamiento de $\mathcal{D}$.
 - **-ditors**: número de veces que se entrena $\mathcal{D}$ por cada paso de $\mathcal{G}$, sirve para estabilizar, si $\mathcal{G}$ empieza muy flojo, $\mathcal{D}$ se entrena mas veces para mantener un equilibrio.
 - **-mlp_G / -mlp_D**: Define capas ocultas en arquitectura MLP, cuando se usa en vez de DCGAN, se define la cantidad de capas densas de 512 a 128 neuronas antes de la capa de salida.

Para explorar el impacto de estos hiperparámetros, se diseñaron diferentes configuraciones experimentales, variando principalmente la tasa de aprendizaje, el tamaño de lote, el vector latente y los umbrales de control adversarial (T_1 , T_2), cada configuración se identificó como M1, M2, etc., lo que permitió comparar cómo las distintas variantes afectan la estabilidad

del entrenamiento y la calidad final de las imágenes generadas, en la Tabla 5-2 se resumen las configuraciones más representativas junto con sus ventajas y desventajas, cabe aclarar que estas ventajas y desventajas son una idea de lo que se pretendía conseguir, mas no es una verdad absoluta.

Tabla 5-2: Combinaciones implementadas para cada configuración y evaluación de pros y contras de los diferentes valores de hiperparámetros en (MedGAN / AdaGAN).

Experimento	Hiperparámetros	Pros	Contras
M1 - No <i>Batch-Norm</i> (Entrenamiento WGAN-like)	batchSize=20, nz=100, ngf=64, ndf=64, lrD=5e-5, lrG=5e-5, Diters=5, optimizer=RMSprop, T1=0.3, T2=0.5	Favorece estabilidad cuando los <i>batches</i> son pequeños (evita la inestabilidad de las estadísticas de <i>BatchNorm</i>), RMSprop + Diters=5 emulan el estilo WGAN/critic, estabilizando $\mathcal{D}$ y proporcionando gradientes más regulares, <i>Batch</i> pequeño (20) permite mayores resoluciones en GPUs con memoria limitada, adecuado para <i>datasets</i> médicos pequeños donde la variabilidad por <i>batch</i> es alta.	Sin <i>BatchNorm</i> la convergencia puede ser más lenta y $\mathcal{G}$ necesitar mejores inicializaciones (peso/normalización alternativa), gradientes ruidosos por <i>batch</i> pequeño $\rightarrow$ necesita más épocas para converger, riesgo de que la red quede con menor diversidad si no se compensa con aumentos o regularización alternativa (e.g., <i>spectral norm</i> / <i>instance norm</i>).
M2 - DCGAN clásico (Adam, <i>batch</i> grande)	batchSize=64, nz=100, ngf=64, ndf=64, lrD=2e-5, lrG=2e-5, Diters=1, optimizer=Adam (beta1=0.5), T1=0.3, T2=0.5	Estándar reproducible: Adam con $\beta_1 = 0.5$ y <i>batch</i> 64 es la receta DCGAN clásica que suele converger bien en imágenes naturales, <i>Batch</i> grande entrega gradientes más estables y estadísticas de <i>batchnorm</i> más fiables (si se usa BN), buena eficiencia en GPU (menos pasos de D por iteración).	Las tasas de aprendizaje muy bajas (e.g., 2e-5) harán la convergencia muy lenta; si se mantienen, requieren muchas más épocas, dependencia de <i>Batch-Norm</i> (si se usa) puede ser problemática en <i>datasets</i> pequeños o cuando la variabilidad del <i>dataset</i> es alta, si $\mathcal{D}$ es demasiado fuerte o débil respecto a $\mathcal{G}$ se observan oscilaciones o colapso — requiere ajuste fino de LR y arquitectura.
M3 - Alta capacidad para alta resolución (detalles finos)	batchSize=16, nz=200, ngf=128, ndf=128, lrD=5e-5, lrG=5e-5, Diters=5, optimizer=RMSprop, T1=0.4, T2=0.6	Mayor capacidad de modelo (ngf/ndf=128) y latente amplio (nz=200) para capturar variaciones y detalles finos en imágenes de mayor resolución, ideal cuando el objetivo es mejorar textura y estructura fina (e.g., bordes, pequeñas anomalías en ecocardiogramas), el <i>batch small</i> (16) permite ejecutar modelos grandes en memoria limitada.	Coste computacional y memoria significativamente mayores (menos <i>batching</i> , más pasos por época), ruido mayor en gradientes al reducir <i>batch</i> $\rightarrow$ requiere más épocas y/o técnica de estabilización (<i>spectral norm</i> , <i>weight decay</i> , <i>augmentation</i>), mayor riesgo de sobreajuste si el <i>dataset</i> es pequeño y no se añade aumentos potentes o regularización.
M4 - Generador potenciado ($\mathcal{G} > \mathcal{D}$)	batchSize=32, nz=150, ngf=128, ndf=128, lrD=5e-5, lrG=1e-5, Diters=3, optimizer=Adam, T1=0.2, T2=0.5	Arquitectura con generador más amplio (ngf>ndf) favorece la síntesis de detalles y la expresividad de $\mathcal{G}$, lrG reducida (1e-5) protege a $\mathcal{G}$ grande de actualizaciones demasiado agresivas; Diters=3 da tiempo a $\mathcal{D}$ para reaccionar, útil cuando la prioridad es maximizar la capacidad de generación (texturas complejas o estructura fina).	Desbalance de capacidad ($\mathcal{G}$ mucho más potente que $\mathcal{D}$) puede llevar a que $\mathcal{G}$ produzca artefactos o “ <i>hallucinations</i> ” si $\mathcal{D}$ no ofrece señal de corrección suficiente, lrG demasiado baja puede ralentizar el aprendizaje efectivo del $\mathcal{G}$; puede necesitarse ajustar lrD/diters para restablecer equilibrio, más sensibilidad a la inicialización y al esquema de regularización.
M5 - Discriminador potenciado ($\mathcal{D}$ fuerte)	batchSize=32, nz=100, ngf=64, ndf=128, lrD=1e-5, lrG=5e-5, Diters=7, optimizer=RMSprop, T1=0.3, T2=0.6	$\mathcal{D}$ con más parámetros y Diters elevado detecta sutiles artefactos; obliga a $\mathcal{G}$ a mejorar características finas, beneficio cuando la prioridad es reducir falsos positivos y mejorar realismo en detalles clínicos, RMSprop y muchas iteraciones de D son robustos para esquemas tipo WGAN/critic.	Riesgo de que $\mathcal{D}$ se vuelva demasiado dominante $\rightarrow$ gradientes vanishing para $\mathcal{G}$ y estancamiento del aprendizaje de $\mathcal{G}$, requiere compensación (e.g., aumentar lrG, reducir Diters o simplificar $\mathcal{D}$) si $\mathcal{G}$ deja de mejorar, mayor coste computacional por entrenar D repetidamente (<i>Diters</i> =7).

Experimento	Hiperparámetros	Pros	Contras
M6 - <i>Baseline</i> (Configuración recomendada por defecto)	batchSize=32, nz=100, ngf=64, ndf=64, lrD=5e-5, lrG=5e-5, Diters=5, noBN=True, optimizer=RMSprop, T1=0.3, T2=0.5	Configuración equilibrada y robusta para <i>datasets</i> médicos pequeños: capacidad moderada, <i>stability-enforcing</i> (RMSprop) y sin <i>BatchNorm</i> para evitar inestabilidad en batches pequeños, buena primera línea para comparar variantes (<i>baseline</i> reproducible).	No optimiza ni para velocidad (lr intermedia) ni para máxima calidad de detalle —es un punto medio, puede requerir ajustes finos (lr, <i>Diters</i> , <i>augmentation</i>) para alcanzar calidad clínica exigente.
M7 - Exploración con MLP (<i>baseline</i> alternativa)	batchSize=40, nz=120, ngf=96, ndf=96, lrD=1e-5, lrG=5e-5, Diters=2, optimizer=RMSprop, T1=0.25, T2=0.55	Útil como experimento de contraste / ablation: MLPs son rápidos de entrenar y consumen menos memoria, permite comprobar cuánto aporta la inductiva convolucional comparado con una arquitectura sin sesgo espacial.	Para imágenes (especialmente $>32 \times 32$) la calidad visual suele ser pobre: pérdida de estructura espacial y peores métricas (FID/SSIM), no recomendable para generación de imágenes médicas con detalles finos; sirve solo como control metodológico.
M8 - Entrenamiento agresivo (rápida experimentación)	batchSize=64, nz=100, ngf=128, ndf=128, lrD=2e-5, lrG=2e-5, Diters=1, optimizer=Adam, T1=0.1, T2=0.3	Configuración pensada para obtener resultados visuales rápidos: batch grande + arquitecturas potentes acelera la aparición de imágenes plausibles, menor número de pasos de $\mathcal{D}$ por $\mathcal{G}$ reduce coste por iteración y permite más épocas por tiempo real.	Muy susceptible a inestabilidad y colapso si no se supervisa; métricas pueden ser engañosas (buen aspecto inicial, mala generalización), requiere semilla y réplicas múltiples para validar resultados (gran varianza entre corridas).

5.2.4 Hiperparámetros WGAN

Para la implementación de los hiperparámetros de WGAN se implementaron los mismos de MedGAN exceptuando -T1 y -T2 ya que WGAN carece de criterios de control mediante umbrales adaptativos, de manera que entrena un bucle fijo sin ajuste dinámico que priorice $\mathcal{D}$ o $\mathcal{G}$ cuando alguno adquiere mayor fuerza; WGAN solo realiza un *warmup* corto, (25 iteraciones) manteniéndose constante en todo el entrenamiento manteniendo un régimen fijo en la actualización de cada red, diferenciándose de MedGAN que implementa una lógica de estados para alternar diatónicamente la prioridad.

Las diferentes configuraciones para este modelo se identificaron como W1, W2, etc., donde al igual que con los anteriores modelos establece un índice claro de diferenciación para identificar las variadas configuraciones, estas se pueden observar en la Tabla **5-3** con las diferentes variaciones, su característica distintiva y sus pros y contra, cada una de estas configuraciones busca analizar características específicas del comportamiento de WGAN, tales como estabilidad, suavidad en la convergencia, sensibilidad al *clipping* en los pesos y capacidad de generar distribuciones complejas bajo arquitecturas con distinta expresividad.

Tabla 5-3: Combinaciones implementadas para cada configuración y evaluación de pros y contras de los diferentes valores de hiperparámetros en WGAN.

Experimento	Hiperparámetros	Pros	Contras
W1 - Configuración Base Estándar (<i>Baseline</i> WGAN)	batchSize=64, nz=100, ngf=64, ndf=64, lrD=5e-5, lrG=5e-5, Diters=5, clamp=0.01, optimizer=RMSprop, BatchNorm=True, mlpG/mlpD=False	Forma más estable de entrenar un WGAN, el uso de ngf=64 y ndf=64 ofrece una arquitectura equilibrada, suficientemente expresiva sin riesgo elevado de sobreajuste, RMSprop es el sugerido originalmente en el <i>paper</i> de WGAN debido a su compatibilidad con el recorte de pesos (clamp=0.01), garantizando la restricción Lipschitz se mantenga sin fluctuaciones bruscas y el <i>BatchNorm</i> ayuda a estabilizar el flujo de gradientes, disminuye la varianza y facilita que $\mathcal{G}$ converja suavemente.	A pesar de su estabilidad, esta configuración puede resultar limitada en tareas complejas o <i>datasets</i> de alta variabilidad, donde la capacidad representacional es insuficiente, el clamp=0.01 puede restringir excesivamente $\mathcal{D}$, reduciendo su habilidad para modelar la <i>critic function</i> con precisión, aunque <i>BatchNorm</i> estabiliza, también puede introducir correlaciones no deseadas entre muestras y afectar negativamente la validez estricta de la condición 1-Lipschitz.
W2 - Arquitectura Profunda y de Alta Capacidad	batchSize=128, nz=128, ngf=128, ndf=128, lrD=5e-5, lrG=5e-5, Diters=5, clamp=0.01, optimizer=RMSprop, BatchNorm=True, mlpG/mlpD=False	Expande la capacidad del modelo mediante mayores anchos de capa (ngf=128 , ndf=128), permitiendo capturar distribuciones más complejas y de mayor dimensión, batch mayor (128) reduce la varianza del estimador del gradiente y contribuye a una convergencia más suave, la combinación de profundidad incrementada y RMSprop conserva la estabilidad típica de WGAN, pero con un mayor potencial de fidelidad en las muestras generadas.	El incremento en el tamaño del modelo hace que el entrenamiento sea más costoso y susceptible a sobreajuste cuando los datos son escasos, mantener el mismo clamp=0.01 , las capas profundas pueden verse excesivamente restringidas, generando gradientes débiles y un <i>critic</i> poco expresivo pese a su tamaño y <i>BatchNorm</i> + redes más grandes puede incrementar oscilaciones si el <i>dataset</i> es reducido.
W3 - Eliminación de <i>BatchNorm</i> (WGAN Sin Normalización Interna)	batchSize=64, nz=100, ngf=64, ndf=64, lrD=5e-5, lrG=5e-5, Diters=5, clamp=0.01, optimizer=RMSprop, BatchNorm=False, mlpG/mlpD=False	Deshabilitar <i>BatchNorm</i> elimina las dependencias entre muestras dentro del <i>batch</i> y respeta mejor la condición Lipschitz, beneficioso en contextos donde incluso pequeñas violaciones afectan la <i>critic loss</i> , puede producir gradientes más consistentes y comparables entre iteraciones, haciendo al modelo más fiel al diseño teórico del WGAN original, sin normalización interna, el comportamiento del <i>critic</i> es más directo y controlado por el <i>clipping</i> .	La estabilidad del entrenamiento disminuye notablemente, ya que <i>BatchNorm</i> actúa como regularizador, su ausencia puede hacer que tanto $\mathcal{D}$ como $\mathcal{G}$ sufran de <i>vanishing</i> o <i>exploding gradients</i> , especialmente cuando los pesos están sujetos al recorte.
W4 - Arquitectura MLP (WGAN Totalmente Conectado)	batchSize=64, nz=256, ngf=64, ndf=64, lrD=1e-4, lrG=1e-4, Diters=5, clamp=0.02, optimizer=Adam, BatchNorm=False, mlpG/mlpD=True	Usar mlpG/mlpD=True elimina las <i>inductive biases</i> propios de las convoluciones, permitiendo evaluar si la distribución puede modelarse sin estructuras espaciales, útil en dominios tabulares o <i>embeddings</i> , junto con Adam y un clamp=0.02 , la <i>critic function</i> recibe gradientes más suaves y flexibles, y el aumento del tamaño latente (nz=256) permite explorar regiones más amplias del espacio de generación.	Los MLP tienden a ser menos estables que las CNN bajo el régimen de <i>clipping</i> , la ausencia de convoluciones dificulta la captación de patrones locales y puede degradar la calidad si los datos tienen estructura espacial, Adam combinado con <i>weight clipping</i> es más propenso a provocar oscilaciones y a romper la estabilidad teórica del <i>critic</i> , esta configuración puede tener la peor estabilidad global de todas.
W5 - Critic Fortalecido (Discriminador Dominante)	batchSize=64, nz=100, ngf=64, ndf=128, lrD=1e-4, lrG=5e-5, Diters=10, clamp=0.05, optimizer=RMSprop, BatchNorm=True, mlpG/mlpD=False	Incrementar ndf=128 y usar más iteraciones de <i>critic</i> por paso (Diters=10) fortalece significativamente a $\mathcal{D}$, lo cual puede resultar en estimaciones Wasserstein más precisas y mejor separación entre distribuciones real/falsa, un <i>critic</i> más fuerte es ideal para <i>datasets</i> complejos donde $\mathcal{D}$ débil no puede capturar relaciones estructurales profundas.	Un <i>critic</i> demasiado dominante puede causar que $\mathcal{G}$ reciba gradientes extremadamente pequeños, lo que lleva al estancamiento del entrenamiento. esta configuración puede incrementar el tiempo de entrenamiento debido al mayor número de pasos de <i>critic</i> por iteración, y puede inducir sobreajuste rápido del <i>critic</i> , reduciendo el poder generalizador.

Experimento	Hiperparámetros	Pros	Contras
W6 - Generador Agresivo	batchSize=64, nz=128, ngf=128, ndf=64, lrD=5e-5, lrG=1e-4, Diters=3, clamp=0.01, noBN=True, optimizer=Adam, BatchNorm=True, mlpG/mlpD=False	El aumento en la capacidad de $\mathcal{G}$ (ngf=128, nz=128) y una tasa de aprendizaje mayor (lrG=1e-4) permiten que $\mathcal{G}$ actualice más rápido que el <i>critic</i> , interesante para analizar escenarios donde $\mathcal{G}$ trata de escapar activamente al <i>critic</i> y produce diversidad alta en las primeras fases del entrenamiento.	$\mathcal{G}$ demasiado agresivo puede “abrumar” temporalmente al <i>critic</i> , produciendo pérdida Wasserstein errática, la reducción de Diters=3 disminuye aún más la capacidad del <i>critic</i> para seguir el ritmo de $\mathcal{G}$, puede culminar en problemas como <i>overshooting</i> , colapso de modo o gradientes poco útiles.
W7 - Recorte amplio con Adam	batchSize=64, nz=100, ngf=64, ndf=64, lrD=1e-4, lrG=1e-4, Diters=5, clamp=0.1, optimizer=Adam, BatchNorm=True, mlpG/mlpD=False	Aumentar el recorte a clamp=0.1 otorga al <i>critic</i> una libertad mayor para ajustar pesos, lo que en ocasiones permite capturar relaciones complejas que quedarían comprimidas por un <i>clipping</i> estricto.	Esta es una de las configuraciones más inestables: Adam + <i>clipping</i> amplio rompe casi completamente la garantía Lipschitz y tiende a generar gradientes ruidosos o incluso divergentes, <i>critic</i> puede desarrollar pesos grandes que inducen explosiones en la <i>critic loss</i> , $\mathcal{G}$ puede colapsar debido a gradientes esporádicos y no estacionarios.
W8 - Modelo Ligero con Batch Pequeño (WGAN de Baja Capacidad)	batchSize=16, nz=100, ngf=32, ndf=32, lrD=5e-5, lrG=5e-5, Diters=5, clamp=0.01, optimizer=RMSprop, BatchNorm=True, mlpG/mlpD=False	Reducir ngf y ndf a 32 produce redes muy ligeras, útiles para análisis rápidos, recursos limitados o conjuntos de datos simples, <i>batch</i> pequeño (16) aumenta la frecuencia de actualización de parámetros por época y permite estudiar la robustez del <i>critic</i> ante gradientes ruidosos.	<i>Batch</i> pequeño introduce varianza elevada en los gradientes, lo cual dificulta la convergencia del <i>critic</i> en WGAN, la baja capacidad del modelo puede resultar insuficiente para distribuciones complejas, generando muestras borrosas o con artefactos.

5.2.5 Hiperparámetros VQ-GAN + *Transformer*

El VQGAN tiene 2 entrenamientos independientes, uno es el de VQGAN para enseñarle al modelo una apropiada representación del vector latente por medio de una reconstrucción de las imágenes del *dataset*, y luego el entrenamiento del *Transformer* que sus resultados son dependientes del entrenamiento previo para el VQGAN, estos dos momentos de entrenamiento tienen sus propios hiperparámetros, por eso para esta arquitectura es requerido realizar mas configuraciones de las trabajadas en los otros modelos, para el VQGAN puro combina un AE vectorial con un discriminador adversarial, sus hiperparámetros gobiernan tanto la representación latente como la dinámica del entrenamiento.

- De la arquitectura:
 - -latent-dim: Define el número de características en el espacio comprimido que produce el *encoder* antes de la cuantización. valores pequeños como 64-128 produ-

cen latentes mas compactos pero pueden perder detalle, mas grandes como 256-512 captura mayor información pero aumenta memoria y riesgo de sobreajuste.

- **-num-codebook-vectors**: Son el tamaño del diccionario de *embeddings* discretos, cada vector representa una “palabra visual” en el espacio latente, un diccionario pequeño como de 256-512 fuerza el modelo a aprender representaciones mas genéricas, y msa grande como de 1024-2048 permite mayor diversidad pero complica la optimización y puede generar redundancias.
- **-image-size / -image-channels**: Determinan el tamaño y tipo de imágenes usadas en el entrenamiento, afectan directamente la complejidad del *autoencoder* y la memoria requerida.

■ Del entrenamiento:

- **-batch-size**: número de imágenes procesadas simultáneamente.
- **-epochs**: Número de iteraciones completas sobre el *dataset*, un mayor número permite convergencia, pero puede inducir sobreajuste si el *dataset* es pequeño.
- **-learning-rate**: Controla el tamaño de las actualizaciones de los pesos, muy alto genera inestabilidad y muy bajo puede dar convergencia lenta.
- **-beta1 / -beta2**: Coeficiente de Adam, regulan la memoria de gradientes pesados en el optimizador Adam.

■ De las perdidas:

- **-disc-start**: Número de pasos antes de activar el discriminador adversarial, si se activa desde el inicio, el generador no logra aprender reconstrucciones básicas.
- **-disc-factor**: Peso relativo de la perdida adversarial dentro de la función objetivo total, valores bajos, reconstrucciones fieles pero mas borrosas, valores altos generan imágenes mas nítidas con riesgo de artefactos y colapso.
- **-rec-loss-factor**: peso de la perdida de reconstrucción (L1/L2), favorece similitud a nivel de pixel, demasiado alto genera imágenes fieles pero poco realistas.
- **-perceptual-loss-factor**: Peso de la perdida perceptual LPIPS que mide similitud en un espacio de características profundas, favorece imágenes perceptuales realistas.
- **-lpips_interval**: Cada cuantos pasos calcular LPIPS.

Los hiperparámetros del *Transformer* autoregresivo controlan la manera en que se da el aprendizaje de la distribución de los índices del *codebook* producidos por el VQGAN, básicamente modela secuencias discretas que representan imágenes comprimidas, lidera la modelación secuencial de *tokens* latentes, su capacidad de generalización y la robustez frente al ruido, busca un equilibrio entre la capacidad de predicción secuencial y regularización.

- De la arquitectura:
 - **-latent-dim / -num-codebook-vectors**: Coinciden con los usados en VQ-GAN, puede determinar el vocabulario de *tokens* que el *Transformer* modela.
 - **-sos-token**: *Token* especial que marca el inicio de la secuencia de índices, es esencial para el entrenamiento autoregresivo.
- Del entrenamiento:
 - **-batch-size**: número de imágenes procesadas simultáneamente, puede ser mayor que en VQ-GAN, porque el *Transformer* no requiere convoluciones pesadas, valores altos aceleran la convergencia.
 - **-epochs**: Número de ciclos de entrenamiento, al ser un modelo secuencial suele requerir mas épocas que el VQ-GAN para aprender dependencias largas.
 - **-learning-rate**: Controla el tamaño de las actualizaciones de los pesos, muy alto genera inestabilidad y muy bajo puede dar convergencia lenta.
 - **-beta1 / -beta2**: Coeficiente de Adam, regulan la memoria de gradientes pesados en el optimizador Adam.
- De regularización y robustez:
 - **-pkeep**: Probabilidad de mantener un token, se usa en *masked training*, aleatoriamente se sustituyen *tokens* por ruido para entrenar el modelo en modo denoising, valores en torno a 0.5 equilibran reconstrucción fiel y robustez al ruido, muy bajo mucho enmascaramiento, el modelo se confunde, muy alto, casi sin enmascarar, se vuelve un modelo memorístico.

En el caso del VQ-GAN se exploraron ocho configuraciones diferentes de hiperparámetros, el propósito no fue únicamente analizar posibles mejoras en el rendimiento del entrenamiento, sino también examinar cómo dichas configuraciones inciden en la calidad del *codebook*, es decir, en la riqueza y diversidad de los *tokens* aprendidos, un *codebook* más robusto permite obtener representaciones discretas más expresivas, lo que se traduce en una base *tokenizada* más sólida para el entrenamiento posterior del modelo *Transformer*.

Tabla 5-4: Evaluación de pros y contras de diferentes configuraciones de hiperparámetros en VQ-GAN

Experimento	Hiperparámetros	Pros	Contras
VG1 – Base / Balanceado	latent-dim=128, codebook=512, beta=0.25, rec-loss-factor=1.0, perceptual-loss=1.0, disc-factor=1.0	Equilibrio entre fidelidad estructural y realismo visual; sirve como referencia comparativa estable.	No explota ninguna ventaja en particular; desempeño medio en todas las métricas.

Experimento	Hiperparámetros	Pros	Contras
VG2 – Alta calidad perceptual	latent-dim=128, codebook=512, beta=0.25, rec-loss-factor=0.5, perceptual-loss=2.0, disc-factor=1.0, lpips_interval=1	Enfocado en similitud perceptual: imágenes más nítidas y visualmente atractivas, mejor FID/LPIPS.	Puede deformar estructuras anatómicas; más lento por el cálculo frecuente de LPIPS.
VG3 – Latente expresivo	latent-dim=256, codebook=1024, beta=0.25, rec-loss-factor=1.0, perceptual-loss=1.0, disc-factor=1.0	Mayor capacidad para representar texturas y detalles; mejora SSIM y PSNR.	Entrenamiento más costoso y pesado; riesgo de infrautilizar el <i>codebook</i> si el <i>dataset</i> es pequeño.
VG4 – Latente compacto	latent-dim=64, codebook=512, beta=0.25, rec-loss-factor=1.0, perceptual-loss=1.0, disc-factor=0.5	Compresión fuerte, genera <i>tokens</i> más pequeños; acelera el entrenamiento del <i>Transformer</i> .	Reconstrucciones más borrosas; pérdida de detalles finos y menor diversidad latente.
VG5 – Codebook pequeño	latent-dim=128, codebook=256, beta=0.25, rec-loss-factor=1.0, perceptual-loss=1.0, disc-factor=0.5	Espacio latente más simple; obliga al modelo a reutilizar códigos y generalizar.	Reducción significativa de detalle; riesgo de patrones repetitivos y artefactos.
VG6 – Codebook grande	latent-dim=128, codebook=1024, beta=0.25, rec-loss-factor=1.0, perceptual-loss=1.0, disc-factor=0.5	Diccionario amplio que permite codificar patrones visuales muy específicos.	Riesgo de infrautilización del diccionario y de inestabilidad en el entrenamiento.
VG7 – Reconstrucción dominante	latent-dim=64, codebook=512, beta=0.25, rec-loss-factor=2.0, perceptual-loss=0.5, disc-factor=0.5	Conserva bien la estructura original; altas métricas SSIM y PSNR.	Imágenes poco realistas y planas; menor percepción visual de naturalidad.
VG8 – Adversarial fuerte	latent-dim=64, codebook=512, beta=0.25, rec-loss-factor=1.0, perceptual-loss=1.0, disc-factor=2.0, disc-start=0	Reconstrucciones muy nítidas y con apariencia fotorrealista desde etapas tempranas.	Alta probabilidad de inestabilidad, oscilaciones de pérdida o colapso de modos.

A partir de esta exploración se seleccionó la configuración de hiperparámetros de VQ-GAN que ofreció un mejor equilibrio entre fidelidad de reconstrucción y calidad perceptual, dicha configuración fue empleada como punto de partida para el entrenamiento del *Transformer*, sin embargo, debido a su muy alta exigencia computacional solo se escogió una configuración para su implementación.

Tabla 5-5: Evaluación de pros y contras de diferentes configuraciones de hiperparámetros en el *Transformer* para la configuración de VQ-GAN

Experimento	Hiperparámetros	Pros	Contras
VT1 – Base / Balanceado	layers=8, heads=8, emb-dim=512, pkeep=1.0, batch=32, lr=3e-5	Configuración estándar y balanceada; ofrece un punto de referencia para comparar el impacto de otros hiperparámetros, buen compromiso entre capacidad de modelado y costo computacional.	No optimiza ninguna dimensión particular (profundidad, regularización o eficiencia); desempeño medio sin destacar en generalización o detalle fino.

Tabla 5-6: Evaluación de pros y contras de diferentes configuraciones de hiperparámetros en el *Transformer* para la configuración de VQ-GAN

Experimento	Hiperparámetros	Pros	Contras
VT1 – Base / Balanceado	layers=8, heads=8, emb-dim=512, pkeep=1.0, batch=32, lr=3e-5	Configuración estándar y balanceada; ofrece un punto de referencia para comparar el impacto de otros hiperparámetros, buen compromiso entre capacidad de modelado y costo computacional.	No optimiza ninguna dimensión particular (profundidad, regularización o eficiencia); desempeño medio sin destacar en generalización o detalle fino.
VT2 – Profundidad aumentada	layers=12, heads=8, emb-dim=512, pkeep=1.0, batch=32, lr=3e-5	Mayor número de capas incrementa la capacidad para capturar dependencias de largo alcance y patrones jerárquicos en la secuencia de tokens, mejora la generación en escenarios complejos.	Entrenamiento más lento y costoso; riesgo de sobreajuste si el <i>dataset</i> no es suficientemente grande; puede incrementar la inestabilidad en etapas iniciales de entrenamiento.
VT3 – Más atención	layers=8, heads=16, emb-dim=512, pkeep=1.0, batch=32, lr=3e-5	Aumentar el número de cabezas permite modelar interacciones más diversas entre tokens en paralelo, favoreciendo detalles locales y globales, útil en imágenes con texturas o estructuras complejas.	Demanda mayor memoria GPU y cómputo; las cabezas extra pueden volverse redundantes y no aportar mejoras significativas, especialmente con <i>dataset</i> limitados.
VT4 – Embedding enriquecido	layers=8, heads=8, emb-dim=768, pkeep=1.0, batch=32, lr=3e-5	Embeddings de mayor dimensión permiten representaciones más expresivas, capturando matices semánticos y estructurales en los tokens, mejora la calidad perceptual de las imágenes reconstruidas.	Mayor costo en memoria y tiempo de cómputo; riesgo de sobreparametrización que no se justifique si la cantidad de datos no es suficiente.
VT5 – Regularización por enmascarado	layers=8, heads=8, emb-dim=512, pkeep=0.5, batch=32, lr=3e-5	El enmascarado parcial (pkeep=0.5) obliga al modelo a inferir tokens ausentes, lo que aumenta la robustez, fomenta la generalización y previene el sobreajuste.	Mayor dificultad de aprendizaje en etapas iniciales; ralentiza la convergencia; puede degradar reconstrucciones si el enmascarado es excesivo.
VT6 – Lote grande	layers=8, heads=8, emb-dim=512, pkeep=1.0, batch=64, lr=2e-5	Gradientes más estables, mejor estimación de la distribución de tokens por actualización; acelera el entrenamiento en hardware potente y mejora la calidad global.	Requiere alta capacidad de memoria GPU; limita la reproducibilidad en entornos de recursos reducidos; tasa de aprendizaje reducida puede hacer más lenta la exploración de soluciones iniciales.
VT7 – Lote pequeño + LR alto	layers=8, heads=8, emb-dim=512, pkeep=1.0, batch=16, lr=5e-5	Reduce requerimientos de memoria, haciéndolo viable en hardware limitado; la tasa de aprendizaje más alta acelera el aprendizaje inicial y permite explorar soluciones variadas.	Gradientes ruidosos, menor estabilidad de entrenamiento; riesgo de divergencia o resultados inconsistentes entre corridas.

Experimento	Hiperparámetros	Pros	Contras
VT8 – Compacto / eficiente	layers=6, heads=4, emb-dim=256, pkeep=1.0, batch=32, lr=3e-5	Modelo ligero y rápido de entrenar; adecuado para validaciones preliminares, prototipado rápido o contextos con recursos computacionales limitados.	Menor capacidad de representación; captura limitada de dependencias de largo alcance; puede generar imágenes menos detalladas y de menor realismo.

5.3 Construcción y adaptación de las arquitecturas

El entrenamiento de todos los modelos se llevó a cabo utilizando Google Colab, aprovechando las GPU asignadas por la plataforma (principalmente Tesla T4 y P100), lo cual permitió una aceleración considerable en los cálculos de retropropagación y optimización, dado que Colab impone restricciones de tiempo y disponibilidad, se utilizó una cuenta independiente para cada configuración experimental, de manera que los diferentes modelos pudieran entrenarse en paralelo o de forma secuencial sin pérdida de progreso.

5.3.1 Implementación StyleGAN2-ADA

Para la implementación de StyleGAN2-ADA, se partió del repositorio oficial adaptado por el repositorio de dvshultz en PyTorch, el cual fue clonado y configurado para su uso en Google Colab, este entorno permitió aprovechar los recursos de GPU gratuitos (Tesla T4, que usa entre 12GB a 16GB de VRAM), útiles para el entrenamiento prolongado de modelos generativos, una rápida ejecución y superar limitaciones propias de equipos con la capacidad de correr estos modelos, la instalación incluyó versiones específicas de dependencias que se pueden observar en el pseudocódigo del Algoritmo 4 como PyTorch 1.9.0 con CUDA 11.1, torchvision 0.10.0 y librerías auxiliares como timm, ftfy, ninja y opensimplex, asegurando compatibilidad con el código y evitando conflictos con paquetes de Colab.

Algorithm 4 Preparación del entorno y dependencias

```

1: function PREPARARENTORNO(repo_dir, data_dir)
2:   if repo_dir no existe then
3:     Clonar repositorio stylegan2-ada-pytorch en repo_dir
4:     Crear directorios: datasets, downloads, pretrained
5:   end if
6:   Cambiar directorio a repo_dir
7:   Desinstalar librerías conflictivas: jax, jaxlib, torch, torchvision,
   torchaudio
8:   Instalar versiones específicas compatibles con CUDA 11:
9:     jax[cuda11_cudnn805]==0.3.10
10:    torch==1.9.0+cu111, torchvision==0.10.0+cu111
11:    timm==0.4.12, ftfy==6.1.1, ninja==1.10.2, opensimplex
12:   Crear estructura de directorios organizada:
13:    results ← almacenar modelos y resultados entrenados
14:    progreso ← almacenar métricas y registros del entrenamiento
15:    datasets ← almacenar datasets preparados
16:    pretrained ← almacenar modelos base preentrenados
17: end function

```

El *dataset* de entrada cargado desde Google Drive fue enlazado al directorio de trabajo del repositorio, para que el entrenamiento pudiera acceder a estos de forma consistente en todas las ejecuciones, se diseñó una estrategia para reanudar entrenamientos a partir de *snapshots* previos usando **-resume**, lo que resultó fundamental dado que Colab interrumpe sesiones al cabo de ciertas horas por día (ya sea por desgaste de la cantidad de GPU permitida por día, tiempo de inactividad, entre otras), con este mecanismo el entrenamiento se retomaba desde el último punto guardado, evitando pérdidas de progreso y asegurando continuidad hacia el objetivo de 2000 king (miles de imágenes procesadas) para todas las configuraciones de hiperparámetros en StyleGAN2-ADA.

En promedio, cada ejecución logró un avance aproximado de 128 a 144 king por día — donde king corresponde a la unidad empleada en StyleGAN2-ADA para medir el progreso del entrenamiento, equivalente a mil imágenes procesadas por la red —, lo que permitió completar el entrenamiento de cada modelo en un periodo de alrededor de 25 días continuos, este ritmo estuvo condicionado por la disponibilidad de GPU, el tamaño del *batch* y las variaciones en la velocidad de entrenamiento propias de cada configuración de hiperparámetros.

Cada cuenta de Colab tenía una configuración según el experimento tal y como se observa en la Tabla 5-1, aparte de estos hiperparámetros, se tenían unos como base que se presentan con claridad en el pseudocódigo del Algoritmo 5, tales como, la configuración de red seleccionada (*11gb-gpu*), diseñada para entornos con memoria limitada, como Colab, no se empleó *data*

augmentation basada en espejado horizontal (`-mirror=0`), debido a que los datos médicos requieren preservar la orientación anatómica.

Algorithm 5 Parámetros en celda de preparación para el entrenamiento

```

1: function CONFIGURARPARAMETROS
2:   outdir  $\leftarrow$  /content/drive/MyDrive/Proyecto_Grado/colab-sg2-ada-pytorch/
     resultsE#
3:   data  $\leftarrow$  /content/drive/MyDrive/Proyecto_Grado/data/frames.zip
4:   gpus  $\leftarrow$  1
5:   batch  $\leftarrow$  segunE#
6:   cfg  $\leftarrow$  11gb - gpu
7:   gamma  $\leftarrow$  segunE#
8:   mirror  $\leftarrow$  0
9:   aug  $\leftarrow$  segunE#
10:  target  $\leftarrow$  segunE#
11:  lr  $\leftarrow$  segunE#
12:  sn  $\leftarrow$  4
13:  resume  $\leftarrow$  ultimo.pkl
14: end function

```

El resto de hiperparámetros (*batch size*, *gamma*, *learning rate*, esquema de aumento ADA y *target*), fueron ajustados según el experimento como se detalla en la Tabla 5-1 para explorar su impacto en la estabilidad, velocidad de convergencia y calidad de las imágenes generadas, durante el entrenamiento, se registraron automáticamente las pérdidas del generador y discriminador en formato *jsonl* —*jsonl* es un formato caracterizado por almacenar cada registro en una línea independiente en estructura JSON, lo que facilita la lectura secuencial, el manejo de grandes volúmenes de datos y el procesamiento incremental - , posteriormente, estos archivos fueron procesados en Python para consolidar la información en archivos CSV, incorporando métricas complementarias como FID, SSIM, PSNR, LPIPS, KID y JSD, se estableció un sistema de acumulación de *king* mediante *offsets*, lo que permitió unir múltiples ejecuciones parciales en una misma línea de tiempo de entrenamiento sin perder continuidad en la interpretación de resultados.

Se usó el *dataset* descomprimido en drive y referenciado mediante el argumento `-data` con la ruta que se ve en el Algoritmo 5, lo que simplificó su uso en Colab, se implementó un mecanismo automático para reanudar el entrenamiento desde el último *snapshot* disponible (*network-snapshot-XXXX.pkl*), reduciendo el riesgo de pérdida de progreso por desconexiones o reinicios de Colab, el parámetro `-sn=4` permitió guardar *snapshots* cada cierto intervalo de progreso, equilibrando la necesidad de seguimiento, el guardado de *checkpoints* controlado y el consumo de almacenamiento.

Algorithm 6 Búsqueda y configuración del último *snapshot*

```

1: function CONFIGURARSNAPSHOT(results_dir)
2:   snapshots := sorted( glob( results_dir/0*/network-snapshot-*.pkl ) )
3:   if snapshots no está vacío then
4:     resume_path := último elemento de snapshots
5:   else
6:     resume_path := None
7:   end if
8:   usar_snapshot := True
9:   if usar_snapshot y resume_path  $\neq$  None then
10:    Imprimir “Reanudando entrenamiento desde:” concatenado con resume_path
11:    resume_flag := la cadena “--resume” concatenada con resume_path
12:   else
13:    Imprimir “Iniciando entrenamiento desde cero”
14:    resume_flag := cadena vacía
15:   end if
16:   return resume_flag
17: end function

```

Esta metodología no implicó modificaciones directas al código fuente de StyleGAN2-ADA, pero sí una adaptación cuidadosa a las condiciones del entorno de entrenamiento, a las características particulares del *dataset* médico, al uso de las métricas implementadas, y al manejo de guardado del progreso de perdidas, métricas y *snapshots*, en términos prácticos, la construcción del modelo se centró en la correcta preparación del entorno, la integración del *dataset* y la parametrización del entrenamiento para maximizar estabilidad y calidad de las imágenes generadas.

5.3.2 Implementación MedGAN

Para la implementación de MedGAN se partió del repositorio oficial publicado por Guo et al. [22], el cual fue adaptado a un entorno de ejecución en Google Colab para facilitar su entrenamiento en GPU Tesla T4, aprovechando los recursos gratuitos de la plataforma, a diferencia de StyleGAN2-ADA, que utiliza un mapeo latente y un sintetizador de alto nivel, MedGAN se basa en una arquitectura tipo DCGAN, donde $\mathcal{G}$ aprende a transformar un vector de ruido latente $z \in \mathbb{R}^{nz}$ en imágenes sintéticas, mientras que $\mathcal{D}$ busca distinguir entre imágenes reales y generadas.

El código se organiza en módulos que encapsulan arquitecturas y funciones auxiliares como se ve en el pseudocódigo del Algoritmo 8, la celda con mayor relevancia es la de *dcgan*,

constituye el núcleo arquitectónico de MedGAN, ya que define explícitamente la estructura del generador (netG) y el discriminador (netD) tal y como se ve en el pseudocódigo del Algoritmo 7, siguiendo la filosofía de DCGAN, se basa en ciertos principios de diseño que han demostrado ser eficaces en la generación de imágenes: (i) $\mathcal{G}$ utiliza convoluciones transpuestas (también conocidas como desconvoluciones) para transformar un vector latente $z \sim \mathcal{N}(0, 1)$ en una imagen sintética de dimensión (C, H, W) , donde C es el número de canales, y (ii) $\mathcal{D}$ aplica convoluciones estratificadas que reducen progresivamente la resolución de entrada y devuelven una probabilidad escalar que indica si la muestra proviene del *dataset* real o fue sintetizada.

Algunas características del Generador (netG):

- Entrada: vector latente z de dimensión nz .
- Flujo: el vector z primero es proyectado mediante una capa lineal y *reshaped* a un bloque inicial de baja resolución (por ejemplo, 4×4 píxeles).
- A continuación, el bloque pasa por múltiples capas de ConvTranspose2D con *stride*=2, que duplican la resolución en cada etapa, aplicando además *BatchNorm* y ReLU para estabilizar el entrenamiento y asegurar que las activaciones se distribuyan adecuadamente.
- Finalmente, una capa ConvTranspose2D + Tanh proyecta la salida al rango $[-1, 1]$, que representa la imagen sintética generada.

Para el Discriminador (netD):

- Entrada: una imagen (real o generada).
- Flujo: la imagen se procesa con una serie de Conv2D con *stride*=2, reduciendo su resolución progresivamente mientras se incrementa el número de canales.
- Cada convolución está seguida por LeakyReLU(0.2), lo cual evita el problema de unidades muertas y permite gradientes más informativos.
- A diferencia de $\mathcal{G}$, aquí no se utiliza *BatchNorm* en la capa de entrada, ya que podría suavizar demasiado los patrones de bajo nivel.
- La última capa es una convolución que colapsa a una sola dimensión, seguida por *Sigmoid*, que devuelve la probabilidad $P(real|x)$.
- El diseño permite que el discriminador actúe como un clasificador binario, aprendiendo a identificar las sutilezas que diferencian imágenes sintéticas de reales.

Algorithm 7 Definición de netG y netD en dcgan

```

1: procedure GENERATORDCGAN( $z$ )
2:    $h_0 \leftarrow \text{Linear}(z \rightarrow (ngf \times 8) \cdot 4 \cdot 4)$ 
3:    $h_0 \leftarrow \text{Reshape}(h_0, [ngf \times 8, 4, 4])$ 
4:   for cada bloque en  $\{ngf \times 4, ngf \times 2, ngf\}$  do
5:      $h \leftarrow \text{ConvTranspose2D}(h, \text{out\_channels}=\text{valor}, \text{stride}=2, \text{padding}=1)$ 
6:      $h \leftarrow \text{BatchNorm2D}(h)$ 
7:      $h \leftarrow \text{ReLU}(h)$ 
8:   end for
9:    $out \leftarrow \text{ConvTranspose2D}(h, \text{out\_channels}=nc, \text{stride}=2, \text{padding}=1)$ 
10:   $img\_fake \leftarrow \text{Tanh}(out)$ 
11:  return  $img\_fake$ 
12: end procedure
13: procedure DISCRIMINATORDCGAN( $x$ )
14:   $h \leftarrow \text{Conv2D}(x, \text{out\_channels}=ndf, \text{stride}=2, \text{padding}=1)$ 
15:   $h \leftarrow \text{LeakyReLU}(h, 0.2)$ 
16:  for cada bloque en  $\{ndf \times 2, ndf \times 4, ndf \times 8\}$  do
17:     $h \leftarrow \text{Conv2D}(h, \text{out\_channels}=\text{valor}, \text{stride}=2, \text{padding}=1)$ 
18:     $h \leftarrow \text{BatchNorm2D}(h)$ 
19:     $h \leftarrow \text{LeakyReLU}(h, 0.2)$ 
20:  end for
21:   $out \leftarrow \text{Conv2D}(h, \text{out\_channels}=1, \text{kernel}=4, \text{stride}=1, \text{padding}=0)$ 
22:   $prob \leftarrow \text{Sigmoid}(out)$ 
23:  return  $prob$ 
24: end procedure

```

La celda que contiene el *script* de *toolsdf.py* contiene rutinas de soporte, como se observa en el pseudocódigo del Algoritmo 8, principalmente funciones de inicialización de pesos (*weights_init*), normalización de tensores y utilidades para entrenar y visualizar resultados, siendo fundamental para mantener la estabilidad del entrenamiento, la celda con *mlp* implementa una variante de $\mathcal{G}$ y $\mathcal{D}$ basados en perceptrones multicapa, pensados para datos tabulares o representaciones vectoriales más simples; en la versión con imágenes médicas 2D, este módulo no se utiliza directamente en la celda principal, pero proporciona flexibilidad para extender MedGAN a dominios no visuales.

Algorithm 8 Estructura modular de MedGAN

```

toolsdf
1: function WEIGHTS_INIT( $m$ )
2:   if  $m$  es conv then
3:     Inicializar pesos  $\sim \mathcal{N}(0, 0,02)$ 
4:   else if  $m$  es batchnorm then
5:     Inicializar pesos  $\sim \mathcal{N}(1, 0,02)$ , bias = 0
6:   end if
7: end function
drgan
8: procedure GENERATORDCGAN.FORWARD( $z$ )
9:   ...
10:  return  $img\_fake$ 
11: end procedure
12: procedure DISCRIMINATORDCGAN.FORWARD( $x$ )
13:   ...
14:  return  $prob\_real\_fake$ 
15: end procedure
mlp
16: procedure MLPGENERATOR.FORWARD( $z$ )
17:   $h \leftarrow \text{Linear}(z \rightarrow hidden)$ 
18:   $h \leftarrow \text{ReLU}(h)$ 
19:  return  $\text{Linear}(h \rightarrow output)$ 
20: end procedure
21: procedure MLPDISCRIMINATOR.FORWARD( $x$ )
22:   $h \leftarrow \text{Linear}(x \rightarrow hidden)$ 
23:   $h \leftarrow \text{LeakyReLU}(h)$ 
24:  return  $\text{Sigmoid}(\text{Linear}(h))$ 
25: end procedure
resnet
26: procedure RESBLOCK.FORWARD( $x$ )
27:   $y \leftarrow \text{ConvBNReLU}(x)$ 
28:   $y \leftarrow \text{ConvBN}(y)$ 
29:  return  $\text{ReLU}(x + y)$  ▷ skip connection
30: end procedure

```

Finalmente, *resnet* ofrece implementaciones de bloques residuales que pueden sustituir o complementar a $\mathcal{G}/\mathcal{D}$ de DCGAN, mejorando la capacidad de representación y el gradiente en arquitecturas más profundas, así, *drgan* es el componente central para el entrenamiento con imágenes, su rol es tan fundamental que cualquier cambio en sus capas (profundidad, tamaño de *kernel*, normalización) impacta directamente en la calidad de las imágenes generadas y en la estabilidad del entrenamiento, mientras que *mlp* y *resnet* son alternativas arquitectónicas.

La principal adaptación de MedGAN radica en la incorporación de un esquema de *entrenamiento adaptativo* controlado por dos umbrales ($T1$, $T2$), los cuales regulan dinámicamente el número de iteraciones dedicadas a $\mathcal{G}$ y a $\mathcal{D}$ en cada ciclo de entrenamiento, esta dinámica puede observarse en ejecución en el Algoritmo 9, este mecanismo permite balancear la competencia entre ambas redes y mitigar problemas comunes como el colapso de modo o el sobreajuste de $\mathcal{D}$, favoreciendo así una mayor estabilidad en la generación de imágenes médicas.

El código fue modificado para integrar distintas configuraciones experimentales (M1–M8 en la Tabla 5-2), donde se evaluaron aspectos como la presencia o ausencia de Batch Normalization, el tamaño del vector latente (nz), la capacidad de representación (ngf , ndf), la proporción de pasos de entrenamiento entre $\mathcal{G}$ y $\mathcal{D}$ ($Diters$), y el optimizador empleado (Adam o RMSprop con diferentes LR), estas variantes se diseñaron con el objetivo de explorar el impacto de la arquitectura y de los hiperparámetros sobre la estabilidad y la calidad de las imágenes generadas.

En términos prácticos, el entrenamiento se gestionó desde Google Drive, donde se almacenó el *dataset* descomprimido, los *snapshots* periódicos y los registros de progreso, se implementó un sistema de guardado y reanudación de entrenamiento a partir de puntos de control, lo que resultó valioso debido a las interrupciones propias de Colab, adicionalmente, se programó un registro automático de pérdidas de $\mathcal{G}$ y $\mathcal{D}$ en formato CSV, junto con métricas de evaluación externas como FID, KID, JSD, SSIM, PSNR y LPIPS, lo que permitió un análisis comparativo sistemático entre las configuraciones.

A diferencia de StyleGAN2-ADA, donde la construcción del modelo se centró en parametrizar un *framework* ya altamente optimizado, la adaptación de MedGAN requirió una exploración más amplia de combinaciones de hiperparámetros y arquitecturas, lo que permitió identificar configuraciones con una posible mayor estabilidad para datos médicos, este proceso fue crucial para determinar qué variantes ofrecían un mejor compromiso entre calidad visual, diversidad de muestras y estabilidad del entrenamiento.

La estrategia de entrenamiento implementada buscó maximizar la estabilidad y reproducibilidad de los experimentos en un entorno restringido como Colab, la combinación de *checkpoints* frecuentes, reanudación automática, y registro detallado de métricas garantizó que el análisis posterior pudiera enfocarse en comparar el efecto de los hiperparámetros sin verse sesgado por limitaciones técnicas.

Algorithm 9 Entrenamiento adaptativo de MedGAN en Colab

```

1: function ENTRENARMEDGAN(data_dir, out_dir, params)
2:   Inicializar dataset desde data_dir usando torch.utils.data.DataLoader
3:   Definir netG  $\leftarrow$  Generator(nz, ngf, nc, noBN)
4:   Definir netD  $\leftarrow$  Discriminator(ndf, nc, noBN)
5:   Inicializar pesos de netG y netD con weights_init
6:   Seleccionar optimizadores (Adam o RMSprop) con tasas de aprendizaje lrG, lrD
7:   Inicializar variables de pérdidas, CSV writer y carpeta de resultados
8:   step  $\leftarrow$  0
9:   for cada epoch en num_epochs do
10:    for cada batch en dataloader do
11:      step  $\leftarrow$  step + 1
12:      Obtener real_images del batch
13:      b_size  $\leftarrow$  tamaño del batch
14:      for i  $\leftarrow$  1 hasta Diters do
15:        noise  $\sim \mathcal{N}(0, I)$  con dimensión nz
16:        fake  $\leftarrow$  netG(noise)
17:        Calcular loss_D  $\leftarrow$  criterion(netD(real_images), 1) +
criterion(netD(fake.detach()), 0)
18:        Retropropagar y actualizar parámetros de netD
19:      end for
20:      noise  $\sim \mathcal{N}(0, I)$ 
21:      fake  $\leftarrow$  netG(noise)
22:      Calcular loss_G  $\leftarrow$  criterion(netD(fake), 1)
23:      Retropropagar y actualizar parámetros de netG
24:      if loss_D < T1 then
25:        Reforzar entrenamiento de G (iteración extra sobre netG)
26:      else if loss_G < T2 then
27:        Reforzar entrenamiento de D (iteración extra sobre netD)
28:      end if
29:      Guardar loss_G, loss_D en archivo CSV
30:      if step mód snap == 0 then
31:        Guardar snapshot de netG, netD en out_dir
32:      end if
33:    end for
34:  end for
35: end function

```

5.3.3 Implementación WGAN

Al igual que en los anteriores modelos se mantiene una estructura en módulos funcionales que abarcan la carga de configuraciones, mecanismos de inicialización y restauración de *check-points*, configuración de los optimizadores, cálculo de métricas de calidad y la construcción del ciclo principal de entrenamiento, es así como estas organizaciones modulares facilitan la obtención de los resultados del modelo, la reanudación automática del proceso cuando existen sesiones previas y el análisis continuo del desempeño mediante métricas cuantitativas, la estructura general está diseñada para ejecutarse en Google Colab, integrándose con el sistema de archivos de Google Drive.

La WGAN implementada utiliza *critic* — en lugar de un discriminador convencional — que aproxima la distancia Wasserstein entre las distribuciones real y generada, esto se refleja en que los $\mathcal{D}$ implementados no utilizan funciones de activación de tipo sigmoide en su salida ni intentan predecir una probabilidad; por el contrario, producen un valor escalar no acotado, la estructura base es muy similar a la implementada en MedGAN con el uso de *drgan*, *mlp* y *resnet* que contienen las arquitecturas de $\mathcal{G}$ y $\mathcal{D}$.

Aunque su nombre sugiere el uso de DCGAN, su diseño fue modificado para ser compatible con la formulación de Wasserstein, $\mathcal{G}$ finaliza con una función de activación tanh que permite producir imágenes normalizadas, mientras que $\mathcal{D}$ antecede cualquier activación final para entregar un valor escalar utilizado por la pérdida Wasserstein, internamente, el modelo emplea bloques de convolución y funciones de activación como ReLU o LeakyReLU, sin embargo, lo que realmente lo convierte en una WGAN es precisamente la forma en que $\mathcal{D}$ produce su salida utilizando una activación softmax y cómo esta se integra con la función de pérdida durante el entrenamiento.

Para este caso, *toolsf* contiene los procedimientos para guardar modelos, generar imágenes sintéticas por unidad y por lote, y registrar métricas del entrenamiento, estas funciones permiten monitorear la calidad de las imágenes producidas y la evolución del *critic* a lo largo del proceso, también se incluyen utilidades como un sistema de *logging* — es decir, el registro continuo de valores clave como pérdidas, gradientes y métricas durante el entrenamiento — y herramientas para medición de promedios móviles, que facilitan la evaluación del comportamiento del modelo durante la optimización.

Se definen los hiperparámetros del experimento mediante una función de *parser* — es decir, un módulo encargado de recibir y organizar los argumentos del experimento usando *argparse*, como la lista de parámetros que se pasan al *script* antes de ejecutar el entrenamiento — este uso de *parser* se puede observar en el pseudocódigo del Algoritmo 10, que permite configurar todos los hiperparámetros presentados en la Tabla 5-3, el código permite seleccionar arquitecturas tipo DCGAN o MLP tanto para $\mathcal{G}$ como para el crítico.

Algorithm 10 Definición y carga de hiperparámetros $W\#$ mediante `argsget()`

```

1: procedure ARGSGET
2:   Crear parser  $\leftarrow$  ArgumentParser()
                                      $\triangleright$  --- Parámetros del dataset ---
3:   AgregarArgumento(dataset, valor_defecto=other)
4:   AgregarArgumento(dataroot, valor_defecto=path_dataset)
5:   AgregarArgumento(workers, tipo=int, defecto=4)
                                      $\triangleright$  --- Atributos del lote e imágenes ---
6:   AgregarArgumento(batchSize, tipo=int, defecto= $W\#$ )
7:   AgregarArgumento(img_size, tipo=int, defecto= $W\#$ )
8:   AgregarArgumento(nc, tipo=int, defecto=1)
                                      $\triangleright$  --- Dimensionalidades del modelo ---
9:   AgregarArgumento(nz, tipo=int, defecto= $W\#$ )
10:  AgregarArgumento(ngf, tipo=int, defecto= $W\#$ )
11:  AgregarArgumento(ndf, tipo=int, defecto= $W\#$ )
                                      $\triangleright$  --- Parámetros de entrenamiento ---
12:  AgregarArgumento(niter, tipo=int, defecto=1000)
13:  AgregarArgumento(lrD, tipo=float, defecto= $5 \times 10^{-5}$ )
14:  AgregarArgumento(lrG, tipo=float, defecto= $5 \times 10^{-5}$ )
15:  AgregarArgumento(beta1, tipo=float, defecto=0.5)
16:  AgregarArgumento(ngpu, tipo=int, defecto=1)
                                      $\triangleright$  --- Checkpoints ---
17:  AgregarArgumento(netG, defecto= $W\#$ )
18:  AgregarArgumento(netD, defecto= $W\#$ )
                                      $\triangleright$  --- Parámetros propios de WGAN ---
19:  AgregarArgumento(clamp_lower, tipo=float, defecto=-0.01)
20:  AgregarArgumento(clamp_upper, tipo=float, defecto=0.01)
21:  AgregarArgumento(Diters, tipo=int, defecto= $W\#$ )
                                      $\triangleright$  --- Arquitecturas alternativas ---
22:  AgregarArgumento(noBN, flag=True)  $\triangleright$  Sin BatchNorm en G
23:  AgregarArgumento(mlp_G, flag=True)  $\triangleright$  Usar MLP para G
24:  AgregarArgumento(mlp_D, flag=True)  $\triangleright$  Usar MLP para D
25:  AgregarArgumento(n_extra_layers, tipo=int, defecto=0)
                                      $\triangleright$  --- Carpeta de experimentos y metadatos ---
26:  AgregarArgumento(experiment, defecto=path_salida)
27:  AgregarArgumento(adam, flag=True)  $\triangleright$  Usar Adam en lugar de RMSprop
28:  AgregarArgumento(class_name, tipo=str, defecto=."ocordio")
                                      $\triangleright$  --- Parsear hiperparámetros sin argumentos externos (Colab) ---
29:  opt  $\leftarrow$  ParsearArgumentos(args=[])
                                      $\triangleright$  --- Ajustes manuales por defecto ---
30:  opt.adam  $\leftarrow$  False
31:  opt.noBN  $\leftarrow$  False
32:  opt.mlp_G  $\leftarrow$  False
33:  opt.mlp_D  $\leftarrow$  False
34:  return opt
35: end procedure

```

El núcleo de WGAN aparece en la implementación de las funciones *discriminator_train* y *generator_train* como se logra apreciar en el pseudocódigo del Algoritmo 11, donde se utiliza explícitamente el esquema propuesto por Arjovsky et al. [116], el entrenamiento del *critic* aplica *weight clipping*, fijado por los parámetros *clamp_lower* y *clamp_upper*, para cumplir la condición 1-Lipschitz necesaria para estimar la distancia de Wasserstein, el crítico es entrenado múltiples veces (*Diters*) por cada actualización de $\mathcal{G}$, una necesidad propia de WGAN para mantener el crítico cercano al óptimo. Las funciones de pérdida también reflejan la formulación WGAN: la pérdida del *critic* se calcula como $D(\text{real}) - D(\text{fake})$ y $\mathcal{G}$ busca maximizar $D(\text{fake})$, sin funciones de entropía cruzada ni sigmoides.

Para la parte de entrenamiento del WGAN es necesario detallarla, ya que a diferencia de los anteriores modelos que implementan pérdida de entropía cruzada, la función objetivo del crítico se basa en una pérdida Wasserstein, reforzando al *critic* para que otorgue valores más altos a muestras reales y valores bajos a muestras generadas, de forma paralela la pérdida de $\mathcal{G}$ busca maximizar la puntuación del *critic* sobre las imágenes sintéticas, esto se aprecia mejor en el pseudocódigo del Algoritmo 12, lo que lo empuja a producir datos más alineados con la distribución real.

Algorithm 11 Construcción del WGAN: Inicialización del Generador y Crítico

```

1: function INICIALIZARWGAN(params)                                ▷ --- Arquitectura del Generador ---
2:    $G \leftarrow \text{Generator}(nz, ngf, nc)$ 
3:   Aplicar weights_init a  $G$                                        ▷ --- Arquitectura del Crítico (sin sigmoide final) ---
4:    $D \leftarrow \text{Critic}(ndf, nc)$ 
5:   Aplicar weights_init a  $D$                                        ▷ --- Configuración de optimizadores ---
6:   if usar Adam then
7:      $optD \leftarrow \text{Adam}(D, lr_D, \beta_1)$ 
8:      $optG \leftarrow \text{Adam}(G, lr_G, \beta_1)$ 
9:   else
10:     $optD \leftarrow \text{RMSprop}(D, lr_D)$ 
11:     $optG \leftarrow \text{RMSprop}(G, lr_G)$ 
12:  end if
13:  Preparar vector de ruido  $z \sim \mathcal{N}(0, I)$ 
14:  Inicializar contador  $gen\_iters \leftarrow 0$ 
15:  return  $G, D, optG, optD$ 
16: end function

```

Algorithm 12 Entrenamiento del WGAN con Pérdida Wasserstein

```

1: function ENTRENARWGAN( $G, D, optG, optD, dataloader, params$ )
2:   for cada  $epoch$  hasta  $niter$  do
3:     for cada batch  $x_{real}$  en  $dataloader$  do ▷ --- Número de iteraciones del crítico ---
4:       if  $gen\_iters < 25$  or  $gen\_iters \bmod 500 = 0$  then
5:          $Diters \leftarrow 100$ 
6:       else
7:          $Diters \leftarrow$  parámetro  $Diters$ 
8:       end if ▷ --- Entrenamiento del Crítico D ---
9:       for  $i = 1$  to  $Diters$  do
10:        Activar gradientes de  $D$ 
11:         $D.zero\_grad()$ 
12:         $loss_{real} \leftarrow D(x_{real})$ 
13:        Retropropagar con signo positivo
14:        Muestrear  $z \sim \mathcal{N}(0, I)$ 
15:         $x_{fake} \leftarrow G(z)$  (detached)
16:         $loss_{fake} \leftarrow D(x_{fake})$ 
17:        Retropropagar con signo negativo
18:         $loss_D = loss_{real} - loss_{fake}$ 
19:         $optD.step()$ 
20:      end for ▷ --- Entrenamiento del Generador G ---
21:      Congelar gradientes de  $D$ 
22:       $G.zero\_grad()$ 
23:      Muestrear  $z \sim \mathcal{N}(0, I)$ 
24:       $x_{fake} \leftarrow G(z)$ 
25:       $loss_G \leftarrow -D(x_{fake})$ 
26:      Retropropagar con signo positivo
27:       $optG.step()$ 
28:       $gen\_iters \leftarrow gen\_iters + 1$ 
29:    end for
30:  end for
31: end function

```

El WGAN requiere que el crítico sea 1-Lipschitz, y esta restricción se implementa en este código mediante un mecanismo inspirado en el *weight clipping* clásico, aunque se incorporan elementos adaptados (como inspección y ajuste periódico de parámetros), todo el flujo del código se organiza para permitir que la distancia Wasserstein actúe como métrica entrenable, desde la inicialización de pesos consistente, pasando por la generación de ruido latente, hasta rutinas de guardado y evaluación de muestras sintéticas, el paso del clipping se puede ver en el pseudocódigo del Algoritmo 13.

Algorithm 13 Aplicación de la Restricción 1-Lipschitz mediante *Weight Clipping*

```

1: function APLICARCLIPPING( $D, clamp\_lower, clamp\_upper$ )
2:   for cada parámetro  $p$  en  $D$  do
3:      $p \leftarrow \text{clip}(p, clamp\_lower, clamp\_upper)$ 
4:   end for
5: end function
6: function CICLOWGAN( $G, D, optG, optD, dataloader, params$ )
7:   for cada  $epoch$  do
8:     for cada batch  $x_{real}$  do ▷ --- Múltiples actualizaciones del Crítico ---
9:       for  $i = 1$  to  $Diters$  do
10:         $D.zero\_grad()$ 
11:        AplicarClipping( $D$ )
12:        Calcular pérdida Wasserstein del Crítico
13:        Actualizar parámetros con  $optD$ 
14:      end for ▷ --- Actualización del Generador ---
15:      Calcular pérdida Wasserstein del Generador
16:      Actualizar parámetros con  $optG$ 
17:    end for ▷ --- Guardado opcional de imágenes y pesos ---
18:    if  $epoch \bmod save\_interval = 0$  then
19:      Generar muestras y guardarlas
20:      Guardar modelos  $G$  y  $D$ 
21:    end if
22:  end for
23: end function

```

5.3.4 Implementación VQ-GAN + *Transformer*

El desarrollo de VQ-GAN se articuló como una cadena de módulos colaborativos tales como *helper*, *codebook*, *encoder/decoder*, *discriminator*, *mingpt*, *vqgan* y *transformer*, cada uno desempeñando una función precisa dentro del flujo de entrenamiento, se diseñó un sistema robusto para la extracción y agregación de estadísticas del conjunto de datos, para evaluar posteriormente la fidelidad del modelo mediante métricas basadas en espacios de características.

El bloque de código *helper* funciona como un conjunto de bloques modulares que permiten construir arquitecturas neuronales complejas de manera ordenada, en vez de definir capas y operaciones repetitivas dentro del modelo principal, se concentran componentes funcionales que estructuran la red, su objetivo es ofrecer implementaciones estandarizadas de normalización por grupos. Esto es útil cuando el *batch* es reducido; activación suave, no lineal que mejora la propagación del gradiente; bloques residuales que evita que el gradiente se degrade

en redes profundas, esta organización se puede apreciar en el pseudocódigo del Algoritmo 14.

Algorithm 14 Construcción de los Bloques Auxiliares del Modelo (Helper)

```

1: function CONSTRUIRGROUPNORM(channels)
2:   Crear normalización por grupos con 32 grupos y channels
3:   return módulo GroupNorm
4: end function
5: function CONSTRUISWISH
6:   Definir operación  $f(x) = x \cdot \sigma(x)$ 
7:   return módulo Swish
8: end function
9: function CONSTRUIRRESIDUALBLOCK(in_c, out_c)
10:  bloque  $\leftarrow$  secuencia:
11:    GroupNorm(in_c)
12:    Swish()
13:    Conv2D(in_c  $\rightarrow$  out_c)
14:    GroupNorm(out_c)
15:    Swish()
16:    Conv2D(out_c  $\rightarrow$  out_c)
17:  if in_c  $\neq$  out_c then
18:    Definir convolución  $1 \times 1$  para ajustar canales
19:  end if
20:  return módulo con salto residual
21: end function
22: function CONSTRUIRUPSAMPLEBLOCK(channels)
23:   Definir interpolación que aumenta resolución por factor 2
24:   Aplicar convolución  $3 \times 3$  para refinar
25:   return módulo UpSample
26: end function
27: function CONSTRUIRDOWNSAMPLEBLOCK(channels)
28:   Aplicar padding para tamaño par
29:   Usar convolución con stride 2 para reducir resolución
30:   return módulo DownSample
31: end function
32: function CONSTRUIRNONLOCALBLOCK(channels)
33:   Normalizar entrada con GroupNorm(channels)
34:   Definir proyecciones  $Q, K, V$  con convoluciones  $1 \times 1$ 
35:   Definir proyección de salida
36:   Atenuar similitud por factor  $1/\sqrt{\text{channels}}$ 
37:   Aplicar softmax para mapa de atención
38:   Reconstruir respuesta no local y sumarla a la entrada
39:   return módulo NonLocal
40: end function

```

El módulo *codebook* implementa la estructura fundamental del *codebook* utilizada en modelos de tipo VQ-VAE, este componente actúa como un espacio latente discreto donde cada vector latente continuo producido por el *encoder* es reemplazado por la entrada más cercana de un conjunto finito de *embeddings* aprendibles, su función principal es transformar la representación continua en una secuencia discreta de índices, permitiendo que el modelo opere en un espacio comprimido y estructurado, se mantienen un conjunto de vectores prototipo—los *codebook vectors*— almacenados en una capa *Embedding* como se ve en el pseudocódigo del Algoritmo 15, estos funcionan como “anclas” en el espacio latente.

Durante el entrenamiento, el módulo introduce un término de pérdida que empuja la representación del *encoder* hacia los vectores del *codebook* y, simultáneamente, actualiza los *embeddings* para alinearse con los datos, esto permite que la operación de cuantización, inherentemente no diferenciable, pueda integrarse dentro del flujo de gradientes de la red, creando un latente discreto que mantiene información semántica relevante.

Algorithm 15 Construcción y Operación del Módulo Codebook

```

1: function INICIALIZARCODEBOOK(args)
2:    $N \leftarrow args.num\_codebook\_vectors$ 
3:    $D \leftarrow args.latent\_dim$ 
4:    $\beta \leftarrow args.beta$ 
5:   Crear matriz de embeddings de tamaño  $(N, D)$ 
6:   Inicializar pesos en rango uniforme  $[-\frac{1}{N}, \frac{1}{N}]$ 
7:   return módulo Codebook
8: end function
9: function CUANTIZAR(z)
10:  Reordenar z a forma (batch, height, width, channels)
11:  Aplanar a forma (batch · hw, D)  $\triangleright$  --- Cálculo de distancias a cada vector del
    codebook ---
12:   $dist \leftarrow \|z\_flat\|^2 + \|emb\|^2 - 2(z\_flat \cdot emb^T)$ 
13:   $indices \leftarrow \arg \min(dist)$  para cada fila
14:   $z_q \leftarrow emb[indices]$  y devolver a forma espacial  $\triangleright$  --- Cálculo de la pérdida de
    cuantización ---
15:   $loss \leftarrow \mathbb{E}[(z_q - z)^2] + \beta \cdot \mathbb{E}[(z_q - z)_{stopgrad}^2]$   $\triangleright$  --- Straight-Through Estimator ---
16:   $z_q \leftarrow z + (z_q - z)_{stopgrad}$ 
17:  Volver a forma (batch, channels, height, width)
18:  return  $z_q, indices, loss$ 
19: end function

```

El *encoder* transforma la imagen de entrada en un conjunto de características comprimidas que capturan su estructura esencial, aplica una secuencia de convoluciones, bloques residuales y operaciones de reducción de resolución, a medida que la imagen avanza por estas capas, su tamaño espacial disminuye y aumenta la profundidad de sus canales, capturando

progresivamente la información perceptual en un mapa latente compacto, esto no solo reduce dimensionalidad: organiza el contenido visual en una forma que facilita la cuantización posterior y los bloques residuales permiten una extracción de características estable y profunda, mientras que las operaciones *DownSample* controlan la escala espacial, tal como se ve en el pseudocódigo del Algoritmo 16.

Algorithm 16 Construcción del *Encoder*

```

1: function CONSTRUIRENCODER(args)
2:   Definir lista de canales  $C = [64, 128, 256]$ 
3:    $num\_res \leftarrow 2$ 
4:    $res \leftarrow 256$  ▷ --- Capa inicial ---
5:    $layers \leftarrow [\text{Conv2D}(\text{args.image\_channels}, C[0])]$  ▷ --- Bloques de reducción y
   extracción de características ---
6:   for  $i = 0$  to  $|C| - 2$  do
7:      $in \leftarrow C[i]$ 
8:      $out \leftarrow C[i + 1]$ 
9:     for  $j = 1$  to  $num\_res$  do
10:      Añadir ResidualBlock( $in, out$ ) a  $layers$ 
11:       $in \leftarrow out$ 
12:     end for
13:     if  $i$  no es el último nivel then
14:       Añadir DownSampleBlock( $out$ ) a  $layers$ 
15:     end if
16:      $res \leftarrow res/2$ 
17:   end for ▷ --- Bloques finales del encoder ---
18:   Añadir dos ResidualBlock( $C[-1], C[-1]$ )
19:   Añadir GroupNorm( $C[-1]$ ) y Swish
20:   Añadir Conv2D( $C[-1], \text{args.latent\_dim}$ )
21:   return Secuencia  $layers$ 
22: end function

```

El *decoder* realiza la operación inversa del *encoder*, toma el latente cuantizado del *codebook* y lo transforma nuevamente en una imagen sintetizada, utilizando bloques residuales, módulos de atención opcional y operaciones de *UpSampling* como se ve en el pseudocódigo del Algoritmo 17, que van incrementando la resolución hasta el tamaño original, en este proceso, el *decoder* “expande” y refina la información latente, reintroduciendo gradualmente textura, forma y color.

Algorithm 17 Construcción del Decoder

```

1: function CONSTRUIRDECODER(args)
2:   Definir canales decodificador  $C = [256, 128, 64]$ 
3:    $num\_res \leftarrow 2$ 
4:    $res \leftarrow args.image\_size/4$  ▷ --- Capa inicial del decoder ---
5:    $layers \leftarrow [Conv2D(args.latent\_dim, C[0])]$ 
6:   Añadir dos ResidualBlock( $C[0], C[0]$ ) ▷ --- Bloques de expansión y refinamiento ---
7:    $in \leftarrow C[0]$ 
8:   for  $i = 0$  to  $|C| - 1$  do
9:      $out \leftarrow C[i]$ 
10:    for  $j = 1$  to  $num\_res$  do
11:      Añadir ResidualBlock( $in, out$ )
12:       $in \leftarrow out$ 
13:    end for
14:    if  $i \neq 0$  and  $res < args.image\_size$  then
15:      Añadir UpSampleBlock( $in$ )
16:       $res \leftarrow res \times 2$ 
17:    end if
18:  end for ▷ --- Capa final del decoder ---
19:  Añadir GroupNorm( $in$ ) y Swish
20:  Añadir Conv2D( $in, args.image\_channels$ )
21:  return Secuencia  $layers$ 
22: end function

```

Luego esta el modulo *minGPT* actúa como el motor generador de secuencias: recibe símbolos codificados (*tokens*), los convierte en vectores aprendidos y los hace circular por una cadena uniforme de bloques *Transformer*; cada bloque combina normalización, atención causal y una red *feed-forward* como se ve en los pseudocódigos de los Algoritmos 18 y 19, depositando el resultado de manera que se mantenga la estabilidad del flujo de información, permitiendo que el modelo aprenda relaciones internas en la secuencia y produzca predicciones *token* por *token*. El rol central de minGPT es servir como un lenguaje base capaz de modelar distribuciones sobre secuencias, tiene una salida con —una proyección lineal transformada en probabilidades mediante softmax— permite amoldarlo a múltiples tareas: generación de texto, predicción condicional, modelado de acciones discretas o cualquier escenario donde se necesite anticipar el siguiente elemento de una secuencia.

Por ultimo, el VQ-GAN tiene el propósito de transformar una imagen de alta dimensión en un espacio latente discreto compuesto por códigos provenientes de un diccionario aprendido, y luego reconstruirla manteniendo fidelidad semántica y detalles visuales de alta calidad, aquí se implementa el *encoder*, sin embargo, antes de almacenarlo, este mapa atraviesa una capa de cuantización: la *codebook*, un espacio discreto donde cada fragmento del latente se reemplaza por el vector más cercano dentro de un conjunto fijo de prototipos, luego el

decoder reconvierte ese mapa cuantizado en una imagen del dominio original.

Algorithm 18 Construcción del Modelo minGPT

```

1: function CONSTRUIRMINGPT(vocab_size, block_size, args)
2:    $n\_layer \leftarrow args.n\_layer$ 
3:    $n\_head \leftarrow args.n\_head$ 
4:    $n\_embd \leftarrow args.n\_embd$  ▷ --- Embeddings iniciales ---
5:   Crear tok_emb  $\leftarrow$  Embedding(vocab_size,  $n\_embd$ )
6:   Crear pos_emb  $\leftarrow$  Parameter(Zeros(block_size,  $n\_embd$ )) ▷ --- Lista de bloques Transformer ---
7:    $blocks \leftarrow []$ 
8:   for  $i = 1$  to  $n\_layer$  do
9:     Crear bloque  $B$  con:
10:      LayerNorm( $n\_embd$ )
11:      CausalSelfAttention( $n\_head$ ,  $n\_embd$ )
12:      MLP( $4n\_embd \rightarrow n\_embd$ )
13:      Añadir  $B$  a  $blocks$ 
14:   end for ▷ --- Capa final y proyección a vocabulario ---
15:    $ln\_f \leftarrow$  LayerNorm( $n\_embd$ )
16:   head  $\leftarrow$  Linear( $n\_embd$ , vocab_size)
17:   return Modelo  $\mathcal{M} = \{tok\_emb, pos\_emb, blocks, ln\_f, head\}$ 
18: end function

```

Algorithm 19 Paso Forward del Modelo minGPT

```

1: function FORWARDMINGPT( $\mathcal{M}$ , idx) ▷ --- Obtener embeddings ---
2:    $E_t \leftarrow \mathcal{M}.tok\_emb(idx)$ 
3:    $t \leftarrow$  Longitud(idx)
4:    $E_p \leftarrow \mathcal{M}.pos\_emb[1:t]$ 
5:    $x \leftarrow E_t + E_p$  ▷ --- Aplicar bloques Transformer secuencialmente ---
6:   for cada bloque  $B$  en  $\mathcal{M}.blocks$  do
7:      $x \leftarrow B(x)$ 
8:   end for ▷ --- Normalización final y logits ---
9:    $x \leftarrow \mathcal{M}.ln\_f(x)$ 
10:   $logits \leftarrow \mathcal{M}.head(x)$ 
11:  return  $logits$ 
12: end function

```

El VQ-GAN como se ve en el pseudocódigo del Algoritmo 20 introduce además un mecanismo de equilibrio dinámico entre la pérdida perceptual y la pérdida adversarial, este equilibrio permite que el modelo aprenda representaciones compactas sin sacrificar la naturalidad visual, evitando que la compresión excesiva vuelva las imágenes borrosas o artificialmente limpias, haciendo que funcione como una máquina de traducción entre imágenes y *tokens*.

Algorithm 20 Paso Forward del Modelo VQGAN

```

1: function FORWARDVQGAN( $\mathcal{V}, imgs$ )                                ▷ --- Codificación inicial ---
2:    $z_e \leftarrow \mathcal{V}.\text{encoder}(imgs)$ 
3:    $z_e \leftarrow \mathcal{V}.\text{quant\_conv}(z_e)$                                 ▷ --- Cuantización vectorial ---
4:    $(z_q, indices, q\_loss) \leftarrow \mathcal{V}.\text{codebook}(z_e)$           ▷ --- Ajuste posterior a la cuantización ---
5:    $z'_q \leftarrow \mathcal{V}.\text{post\_quant\_conv}(z_q)$                     ▷ --- Decodificación y reconstrucción ---
6:    $x_{rec} \leftarrow \mathcal{V}.\text{decoder}(z'_q)$ 
7:   return ( $x_{rec}, indices, q\_loss$ )
8: end function

```

Con el VQ-GAN entrenado, el *Transformer* que se ve en el pseudocódigo del Algoritmo 21, cumple la función de un modelo generativo autoregresivo encargado de aprender la estructura estadística de las imágenes, pero no en el espacio de píxeles, sino en el espacio discreto del código latente producido por el VQGAN, este proporciona unos índices que actúan como una especie de “alfabeto visual”, de modo que el *Transformer* trabaja no con imágenes continuas, sino con secuencias discretas de símbolos, su rol se centra de aprender las relaciones entre los *tokens* de esta secuencia latente, capturando patrones espaciales y texturales que se distribuyen a lo largo de la imagen. Durante el entrenamiento, el modelo recibe secuencias parcialmente corrompidas mediante una máscara estocástica, lo que incentiva al *Transformer* a predecir los *tokens* faltantes o modificados.

Algorithm 21 Construcción y Paso Forward del VQGAN-Transformer

```

1: function CONSTRUIRVQGANTRANSFORMER( $args$ )                        ▷ --- Inicializar ---
2:    $sos \leftarrow args.sos\_token$ 
3:    $vqgan \leftarrow \text{CARGARVQGAN}(args)$                                 ▷ --- Configuración del Transformer ---
4:    $\mathcal{C} \leftarrow \{vocab\_size = args.num\_codebook\_vectors, block\_size = 512,$ 
5:      $n\_layer = 24, n\_head = 16, n\_embd = 1024\}$ 
6:    $T \leftarrow \text{GPT}(\mathcal{C})$ 
7:    $p_{keep} \leftarrow args.pkeep$ 
8:   return ( $sos, vqgan, T, p_{keep}$ )
9: end function
10: function FORWARDVQGANTRANSFORMER( $\mathcal{M}, x$ )                        ▷ --- Codificación latente ---
11:    $(\_, idx) \leftarrow \mathcal{M}.vqgan.\text{encode}(x)$ 
12:    $idx \leftarrow \text{Flatten}(idx)$                                 ▷ --- Preparar token inicial ---
13:    $sos \leftarrow \text{Tensor}(x.\text{batch\_size}, 1)$  con valor  $\mathcal{M}.sos$ 
14:    $mask \leftarrow \text{Bernoulli}(p_{keep})$  del tamaño de  $idx$ 
15:    $rnd \leftarrow \text{RandomInts}(idx.\text{shape}, vocab\_size)$ 
16:    $idx' \leftarrow mask \cdot idx + (1 - mask) \cdot rnd$ 
17:    $idx' \leftarrow \text{Concatenate}(sos, idx')$ 
18:   return ( $logits, idx$ )
19: end function

```

5.4 Construcción de evaluación de métricas

Basado en los ejemplos de la literatura médica con los rangos en que dichos experimentos eran considerados buenos, se proponen en esta investigación unos rangos orientativos ideales para determinar que tan buenas o malas son las métricas de las imágenes generadas con los diferentes modelos, es decir, estos valores son puramente heurísticos y no un valor absoluto global.

Para FID basado en los valores alcanzados por la arquitectura StyleGAN2-ADA en la generación altamente realista y diversa en CT/MRI/ultrasonido, se propone los valores presentados en la Tabla 5-7.

Tabla 5-7: Umbrales orientativos del (FID) para la evaluación de modelos GAN en imágenes médicas, los rangos son aproximados y deben interpretarse en función del dominio, el extractor de características y el número de muestras utilizadas en el cálculo.

Eficiencia	Rango aproximado de FID	Interpretación y ejemplos reportados
Excelente	0 – 25	Calidad muy alta y difícil de distinguir de los datos reales; Woodland et al. [30] reportaron $FID = 5.22 (\pm 0.17)$ para un <i>dataset</i> de CT y FID record de ~ 10.78 , ~ 3.52 y ~ 21.17 en ACDC, SLIVER07, ChestX-ray14.
Bueno	26 – 50	Imágenes de alta fidelidad, con diferencias menores respecto al conjunto real.
Aceptable	51 – 150	Aún utilizables para <i>data augmentation</i> , pero pueden presentar diferencias perceptibles o estructurales; Gazda et al. [39] en TTE reportó $FID \approx 122.08$ y FID crecientes (≈ 144 – 150) para cortes de <i>outpainting</i> mayores, los autores interpretan que pese a FID altos comparados con dominios “limpios”, las imágenes outpainted mantienen coherencia anatómica comprobada por medidas clínicas.
Malo	151 – 200	Divergencias claras entre distribuciones, con riesgo de artefactos visibles, requiere validación clínica exhaustiva antes de cualquier aplicación.
Muy Malo	> 200	Alta discrepancia, imágenes poco realistas o con pérdida significativa de información anatómica. No se recomienda su uso más allá de pruebas exploratorias.

La métrica de referencia base implementada en StyleGAN2-ADA fue el FID, ya incorporada en el código oficial, parte de su implementación general se puede observar en el pseudocódigo del Algoritmo 22, donde el FID mide la distancia entre las distribuciones de características extraídas por la red Inception-v3, comparando un conjunto de imágenes reales con otro de imágenes generadas, se calculan la media y la covarianza de ambas distribuciones en un espacio de representación semántica profundo, y luego se aplica la distancia de Fréchet, que penaliza tanto desviaciones en la media como discrepancias en la correlación de las características, el FID se considera una métrica robusta para evaluar la realismo visual y la coherencia global de las imágenes sintéticas de este modelo.

Algorithm 22 Cálculo de FID en StyleGAN2-ADA

```

1: function CALCULARFID(modelo, dataset_real, N)
2:   features_real  $\leftarrow$  EXTRAERFEATURESINCEPTION(dataset_real)
3:    $\mu_r, \Sigma_r \leftarrow$  CALCULARMEDIACOV(features_real)
4:   imagenes_fake  $\leftarrow$  GENERARIMAGENES(modelo, N)
5:   features_fake  $\leftarrow$  EXTRAERFEATURESINCEPTION(imagenes_fake)
6:    $\mu_f, \Sigma_f \leftarrow$  CALCULARMEDIACOV(features_fake)
7:   diff  $\leftarrow \|\mu_r - \mu_f\|^2$ 
8:   covmean  $\leftarrow$  RAIZMATRIZ( $\Sigma_r \cdot \Sigma_f$ )
9:   FID  $\leftarrow diff + Trace(\Sigma_r + \Sigma_f - 2 \cdot covmean)$ 
10:  return FID
11: end function

```

Otra de las métricas de distribución implementadas fue KID, se calculó sobre parches extraídos en grillas regulares de las imágenes ecocardiográficas, lo cual permitió evaluar la calidad de las estructuras locales y texturas anatómicas relevantes, el KID normalmente produce valores bastante pequeños (cerca de 0) si las distribuciones son muy similares; rangos de “0.0 a 0.01” se inspiran en cómo se usa MMD / *kernel* divergencias en la literatura de aprendizaje de distribución (aunque no necesariamente reportados en medical GANs), por ende se manejara la evaluación como se ve en la Tabla 5-8.

Tabla 5-8: Umbrales orientativos adoptados para KID en evaluación de modelos GAN (heurísticos, no universales, estos valores son heurísticos y deben usarse solo para comparar modelos dentro del mismo experimento con las mismas condiciones, mismo extractor, mismo número de muestras, mismo *kernel*/ espacio).

Eficiencia	Rango estimado	Interpretación / notas
Excelente	0 a 0.03	Representación casi indistinguible entre muestras reales y generadas en el espacio del kernel; Pan et al. [42] en el uso de estimador no sesgado reportado KID del orden de 0.01–0.05 para modelos muy afinados.
Aceptable	0.031 – 0.07	Buena similitud de distribución, diferencias menores en las características de kernel; Luo et al. [43] en trabajos con 2S-BUSGAN reportando KID entre 0.76-0.33 como indicador de buen ajuste.
Medio	0.07 – 0.15	Diferencias perceptibles en la estructura estadística de las representaciones; podría requerir revisión cualitativa.
Malo	0.16 – 0.3	Discrepancias tangibles; artefactos o falta de diversidad pueden dominar.
Muy malo	>0.3	Alta discrepancia entre distribuciones, desalineación marcada; difícil justificar uso como datos clínicos.

De manera complementaria, se implementó JSD como una métrica estadística para comparar distribuciones de características, la JSD se utilizó para evaluar el grado de solapamiento entre las distribuciones de descriptores extraídos de imágenes reales y sintéticas, proporcionando una

medida acotada entre 0 y 1 que refleja la similitud probabilística entre ambas, su principal aporte radica en identificar casos donde el modelo generativo puede estar produciendo imágenes plausibles pero con distribuciones de características desplazadas respecto al conjunto real, un fenómeno común en situaciones de *mode collapse*. Para JSD, la métrica teórica está acotada entre 0 y $\log 2$ ($\approx 0,693$ en logaritmo base e , ≈ 1 si se usa base 2), entonces, definir como “bueno” los valores menores a 0,3 bits es coherente como se observa en la Tabla 5-9.

Tabla 5-9: Umbrales orientativos adoptados para JSD en evaluación de modelos GAN (heurísticos, no universales).

Eficiencia	Rango estimado (bits)	Interpretación / notas
Excelente	0.0—0.05	Distribuciones casi idénticas, divergencia mínima; en ultrasonido/IVUS Tom et al. [45], los autores interpretan valores bajos (≈ 0.0458 en lumen) como indicativos de muy buena coincidencia estadística local del <i>speckle</i> .
Bueno	0.05—0.1	Diferencias moderadas pero aceptables; Tom et al. [45], obtuvo en la etapa inicial (<i>Stage 0</i> : 0.0957, 0.5245, 0.6685)
Aceptable	0.1—0.2	Divergencia perceptible; se recomienda validación clínica adicional.
Malo	0.2—0.5	Distribuciones claramente distintas; limitaciones en la utilidad del modelo.
Muy malo	> 0,6	Alta divergencia; el modelo no representa bien la distribución real.

En el caso de las métricas por similitud, como se ha mencionado antes, no son adecuadas para ser usadas en GANs no pareados, lo que es igual a GANs generativos, debido a que no existe una imagen de referencia o de correspondencia directa perdiendo sentido el uso de este tipo de métricas, pero basados en la literatura y aplicaciones en GANs pareados tales como pix2pix o CycleGAN, se hizo uso de estas métricas para ver su comportamiento, comenzando con PSNR, luego SSIM y por ultimo LPIPS, por ende, se hizo una propuesta para esta investigación puramente heurística para poder discernir si los resultados son o no apropiados, de manera que se establezca el rango para estas métricas teniendo en cuenta los valores reportados por diferentes autores en la literatura, pero para GANs de tipo generativo, disminuir un poco la exigencia y determinar su comportamiento, estos valores se pueden apreciar en las Tablas 5-10 para PSNR, 5-11 para SSIM y 5-12 para LPIPS.

Todos estos cálculos fueron integrados en un *pipeline* automatizado en Google Colab, que recolectaba periódicamente las imágenes generadas por los *snapshots* de entrenamiento y los contrastaba con un conjunto de referencia de imágenes reales, el código se estructuró de forma modular para extraer características mediante Inception-v3, calcular métricas de manera incremental y fusionar los resultados en un archivo CSV centralizado, este registro consolidado incluyó la evolución de pérdidas de $\mathcal{G}$ y $\mathcal{D}$, los valores de FID, KID, JSD, SSIM, PSNR y LPIPS calculados en una función como se ve en el pseudocódigo del Algoritmo 23 a lo largo de las épocas, permitiendo un análisis a través del tiempo del proceso de entrenamiento.

Tabla 5-10: Umbrales orientativos adoptados para PSNR en evaluación de modelos GAN, adaptación y valor que fue ajustado para GANs sin correspondencia de imagen directa.

Eficiencia	Rango ideal PSNR (dB)	PSNR ajustado GANs no pareados (dB)	Interpretación
Excelente	$> 35dB$	$> 30dB$	Muy alta fidelidad, diferencias mínimas perceptuales; Cammarasana et al. [46] usando una CNN PSNR de 36.13 al usar un <i>dataset</i> mas amplio señalándolo como muy buenos valores.
Bueno	25--35 dB	20--30 dB	Calidad aceptable, diferencias leves; Cammarasana et al. [46] reporto PSNR de 26.58 con menor cantidad de datos, indicándolo como un valor bueno.
Aceptable	20--25 dB	15--20 dB	Visibles discrepancias, pero posiblemente útiles con validación; Raposo et al. [31] reporto PSNR ~ 21.08 dB señalándolo como una mejora estructural apreciable.
Malo	$< 20dB$	$< 10dB$	Imagen muy degradada, difícil uso clínico.

Tabla 5-11: Umbrales orientativos adoptados para SSIM en evaluación de modelos GAN, adaptación y valor que fue ajustado para GANs sin correspondencia de imagen directa.

Eficiencia	Rango ideal SSIM	SSIM ajustado GANs no pareados	Interpretación
Excelente	0.95--1.0	0.90--1.0	Muy alta similitud estructural, difícil de alcanzar en GANs no pareados; Cammarasana et al. [46] usando una CNN obtuvo un SSIM de 0.964 al usar un <i>dataset</i> mas amplio señalándolo como muy buenos valores.
Bueno	0.85--0.95	0.75--0.90	Similitud robusta; Cammarasana et al. [46] reporto SSIM de 0.881, indicándolo como un valor bueno.
Aceptable	0.75--0.85	0.60--0.75	Diferencias estructurales perceptibles pero tolerables; Raposo et al. [31] reporto SSIM ≈ 0.789 señalándolo como una mejora estructural apreciable.
Malo	0.65--0.75	0.40--0.60	Pérdida estructural evidente, artefactos presentes; dudosa utilidad clínica.
Muy malo	$< 0,65$	$< 0,40$	Imágenes poco confiables, fuerte distorsión anatómica.

Tabla 5-12: Umbrales orientativos adoptados para LPIPS en evaluación de modelos GAN, adaptación y valor que fue ajustado para GANs sin correspondencia de imagen directa.

Eficiencia	Rango heurístico LPIPS	Interpretación
Excelente	$< 0,10$	Excelente perceptual (ej.: mejores métodos SR pareados en la literatura médica). Ejemplo: Real-ESRGAN variantes en retinal SR alcanzaron 0.081 en el paper citado.
Bueno	0.10--0.25	Bueno / aceptable perceptualmente en tareas médicas (reconstrucción / SR / traducción cuando se busca realismo). Ejemplo: MRI SR reportó 0.276
Medio	0.25--0.40	Medio; perceptible diferencia pero potencialmente útil para aumentos / entrenamiento de modelos.
Malo	0.40--0.60	Malo / perceptual gap significativo (ej.: algunos resultados de CycleGAN en ultrasound mostraron 0.48). En ultrasonido y ecocardiografía, valores en este rango indican que todavía hay artefactos o stylization perceptible.
Muy malo	$> 0,60$	Muy malo: grandes discrepancias perceptuales.

En StyleGAN2-ADA, las métricas se encuentran altamente estandarizadas y optimizadas en el código oficial (particularmente FID y KID sobre 50k imágenes) ya que se utilizó la implementación de *Inception-v3* de torchvision, en lugar de la variante *TorchScript* entrenada y calibrada por NVIDIA, esto implica que las distribuciones de características no son directamente comparables con los valores reportados en la literatura para StyleGAN, el número de muestras fue menor y variable, determinado por el *batchSize* del entrenamiento y por el número máximo de imágenes generadas en cada época (usualmente $\leq 10k$), en contraste con las 50k imágenes fijas de StyleGAN2-ADA, esto hace que el FID en las demás arquitecturas presente mayor varianza y menor robustez estadística ya que se adoptó una estrategia de evaluación más flexible y heterogénea, ajustada a las limitaciones computacionales y a las características específicas del dominio médico.

Algorithm 23 Pipeline de evaluación de métricas adicionales

```

1: function EVALUARMETRICAS(imagenes_reales, imagenes_sinteticas)
2:   resultados  $\leftarrow$  diccionario vacío
3:   for all (real, fake) en pares de imágenes do
4:     ssim  $\leftarrow$  CALCULARSSIM(real, fake)
5:     psnr  $\leftarrow$  CALCULARPSNR(real, fake)
6:     lpips  $\leftarrow$  CALCULARLPIPS(real, fake)
7:     AGREGAR(resultados, ssim, psnr, lpips)
8:   end for
9:   features_real  $\leftarrow$  EXTRAERFEATURESINCEPTION(imagenes_reales)
10:  features_fake  $\leftarrow$  EXTRAERFEATURESINCEPTION(imagenes_sinteticas)
11:  KID  $\leftarrow$  CALCULARMMD(features_real, features_fake, kernel=polinomial)
12:  JSD  $\leftarrow$  CALCULARJSD(features_real, features_fake)
13:  return PROMEDIARMETRICAS(resultados)  $\cup$  {KID, JSD}
14: end function

```

Para StyleGAN2-ADA se comparo una imagen real fija (reals.jpg) contra las imágenes de salida fakesXXXX.png (snapshots), en las demas arquitecturas normalmente se usan *batches* de imágenes reales del *dataset* para estimar las distribuciones (μ, Σ) de FID/KID, haciendo que en StyleGAN2-ADA, FID/KID sea, más *patch-wise* y dependiente de esa *reals.jpg*, mientras que en los demás modelos es más global al *dataset*; en StyleGAN2-ADA, se dividió las imágenes en parches 8×8 (*extract_grid_patches*) para aumentar el número de muestras y capturar variabilidad local, en los otros modelos se extrajo features de la imagen completa o de *batches* completos, esto introdujo un sesgo: en StyleGAN2-ADA FID/KID reflejan también consistencia local (detalles), no solo global; para JSD en StyleGAN2-ADA se calcula sobre histogramas de los *features* de Inception (1D *flatten* + *histogram*), en los demás modelos JSD se aplico a histogramas de intensidad de píxeles (no de *features*), los resultados de JSD no son directamente comparables con los modelos, porque se llevaron a un espacio semántico más profundo (*features* de Inception).

El aporte de esta metodología garantiza la reproducibilidad experimental, al almacenar de manera sistemática todas las métricas asociadas a cada experimento y ofrece una visión multidimensional de la calidad del modelo, combinando perspectivas estadísticas, estructurales y perceptuales, en el caso específico de la generación de imágenes ecocardiográficas, esta combinación de métricas es especialmente valiosa, pues no solo interesa la calidad visual global de las imágenes, sino también la fidelidad anatómica local.

5.4.1 Evaluación perceptual por especialistas médicos

Una vez identificado el modelo con mejor desempeño cuantitativo, junto con su configuración óptima y el punto de entrenamiento asociado a las mejores métricas, se procedió a la generación masiva de imágenes sintéticas con el fin de evaluar su realismo en un contexto clínico, en este punto, se considero apropiada la generación de un conjunto de 10.000 imágenes sintéticas, este conjunto permitió disponer de una muestra suficientemente amplia para realizar evaluaciones perceptuales sin riesgo de sobreajuste o selección sesgada de ejemplos.

Posteriormente, se diseñó un procedimiento automatizado en Python para emparejar imágenes sintéticas con las imágenes reales que fueron usadas para el entrenamiento, este proceso se realizó mediante un algoritmo de selección aleatoria que asigna las imágenes a posiciones izquierda o derecha de forma igualmente aleatoria, garantizando así la ausencia de sesgo por parte del investigador en la selección y disposición de las muestras.

Con el propósito de facilitar la participación de especialistas en cardiología y radiología, el instrumento de evaluación se implementó mediante un formulario digital estructurado en diferentes tipos de preguntas:

- En primer lugar, se incluyeron diez preguntas compuestas por pares de imágenes, donde cada par contiene una imagen real y una sintética, solicitando al especialista identificar cuál considera real, lo que representa una probabilidad de acierto del 50 %.
- En segundo lugar, se incluyeron cinco preguntas con cuatro imágenes cada una, compuestas por dos imágenes reales y dos sintéticas, donde el evaluador debía identificar las imágenes reales.
- Adicionalmente, se incluyeron tres preguntas con una imagen real y tres sintéticas con el fin de aumentar la dificultad de discriminación.
- Finalmente, se dispuso un espacio abierto para observaciones con el objetivo de recoger apreciaciones clínicas sobre artefactos, coherencia anatómica o posibles inconsistencias visuales.

Este diseño experimental busca simular un escenario de evaluación tipo prueba de Turing perceptual, en el cual se analiza la capacidad de las imágenes generadas para pasar desapercibidas ante observadores expertos, así como la habilidad de los especialistas para discriminar entre imágenes reales y sintéticas en distintos niveles de dificultad.

Una vez recolectadas las respuestas, se realizó un análisis estadístico basado en:

- *Matriz de confusión*: Con el objetivo de caracterizar detalladamente el comportamiento de los evaluadores en la tarea de discriminación, esta herramienta constituye una representación tabular que permite analizar la distribución conjunta entre las etiquetas verdaderas y las clasificaciones asignadas por los especialistas. En el contexto de una clasificación binaria (imagen real vs. imagen sintética), se define como:

	$\hat{y} = 1$	$\hat{y} = 0$
$y = 1$	TP	FN
$y = 0$	FP	TN

donde:

- TP (True Positives): número de imágenes reales clasificadas correctamente como reales.
 - TN (True Negatives): número de imágenes sintéticas clasificadas correctamente como sintéticas.
 - FP (False Positives): número de imágenes sintéticas clasificadas incorrectamente como reales.
 - FN (False Negatives): número de imágenes reales clasificadas incorrectamente como sintéticas.
- *Exactitud (Accuracy)*: Es una métrica de desempeño en problemas de clasificación binaria que mide la proporción de predicciones correctas respecto del total de observaciones evaluadas, utilizando los términos de la matriz de confusión, la exactitud puede expresarse como:

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}, \quad (5-1)$$

- *Fooling Rate*: Se define como la proporción de imágenes sintéticas que fueron clasificadas como reales por los especialistas, cuantifica directamente la capacidad del modelo para producir imágenes perceptualmente indistinguibles de las reales.

En conjunto, este enfoque metodológico permite evaluar de manera rigurosa la calidad perceptual de las imágenes sintéticas desde una perspectiva clínica, proporcionando evidencia complementaria a las métricas cuantitativas tradicionales, permitiendo determinar el grado en que el modelo es capaz de generar imágenes capaces de engañar a especialistas en cardiología y áreas afines.

6 Resultados y Análisis

En esta sección se presentan y analizan los resultados obtenidos a partir del entrenamiento de las arquitecturas StyleGAN2-ADA, MedGAN, WGAN y VQ-GAN cada una evaluada bajo ocho configuraciones experimentales distintas, el objetivo principal es identificar, dentro de cada arquitectura, la configuración que logra un mejor equilibrio entre calidad de generación, diversidad de muestras, capacidad de generalización, conservación de morfología y ruido speckle elemental en ecocardiografías, una vez seleccionada la mejor configuración de cada modelo, se procede a un entrenamiento extendido con el fin de validar la estabilidad y robustez de los resultados a mayor escala.

La comparación entre configuraciones no se basa en un único indicador, sino en un conjunto de métricas complementarias que permiten evaluar diferentes dimensiones de desempeño, en primer lugar, se emplean métricas de distribución como el FID y KID, que cuantifican la similitud estadística entre las distribuciones de características extraídas de imágenes reales y generadas, estas métricas permiten evaluar qué tan cercanas son las muestras sintéticas a la distribución real del dominio, adicionalmente, se considera la JSD sobre histogramas de características, lo cual brinda una medida de discrepancia en la cobertura de la distribución, para capturar la fidelidad perceptual, se incluyen métricas de similitud estructural y calidad de reconstrucción como el SSIM y PSNR; aunque se reconoce que, al tratarse de modelos generativos no condicionados, su interpretación debe realizarse con cautela, complementariamente, se utiliza el LPIPS, una métrica basada en características profundas que se correlaciona mejor con la percepción visual humana.

El rendimiento para estas métricas es plasmada en, gráficas de líneas para mostrar el rendimiento de cada métrica respecto a una época o *king* respectivo, y un diagrama de caja junto a un violín plot por que muestra la forma completa de la distribución, junto con los valores mínimo, máximo, mediana, cuartiles y promedio, ofreciendo una gran cantidad de información de una forma claramente visual.

Finalmente, se analizan las pérdidas de $\mathcal{G}$ y $\mathcal{D}$ registradas durante el entrenamiento, con el fin de identificar posibles problemas de colapso de modo, sobreajuste o inestabilidad adversarial,

el análisis combinado de estas métricas proporciona un marco integral que permite seleccionar, de manera fundamentada, la mejor configuración de cada arquitectura y valorar su capacidad para producir imágenes sintéticas fidedignas y diversas.

6.1 Resultados preliminares de las 8 configuraciones

6.1.1 Resultados en configuraciones de StyleGAN2-ADA

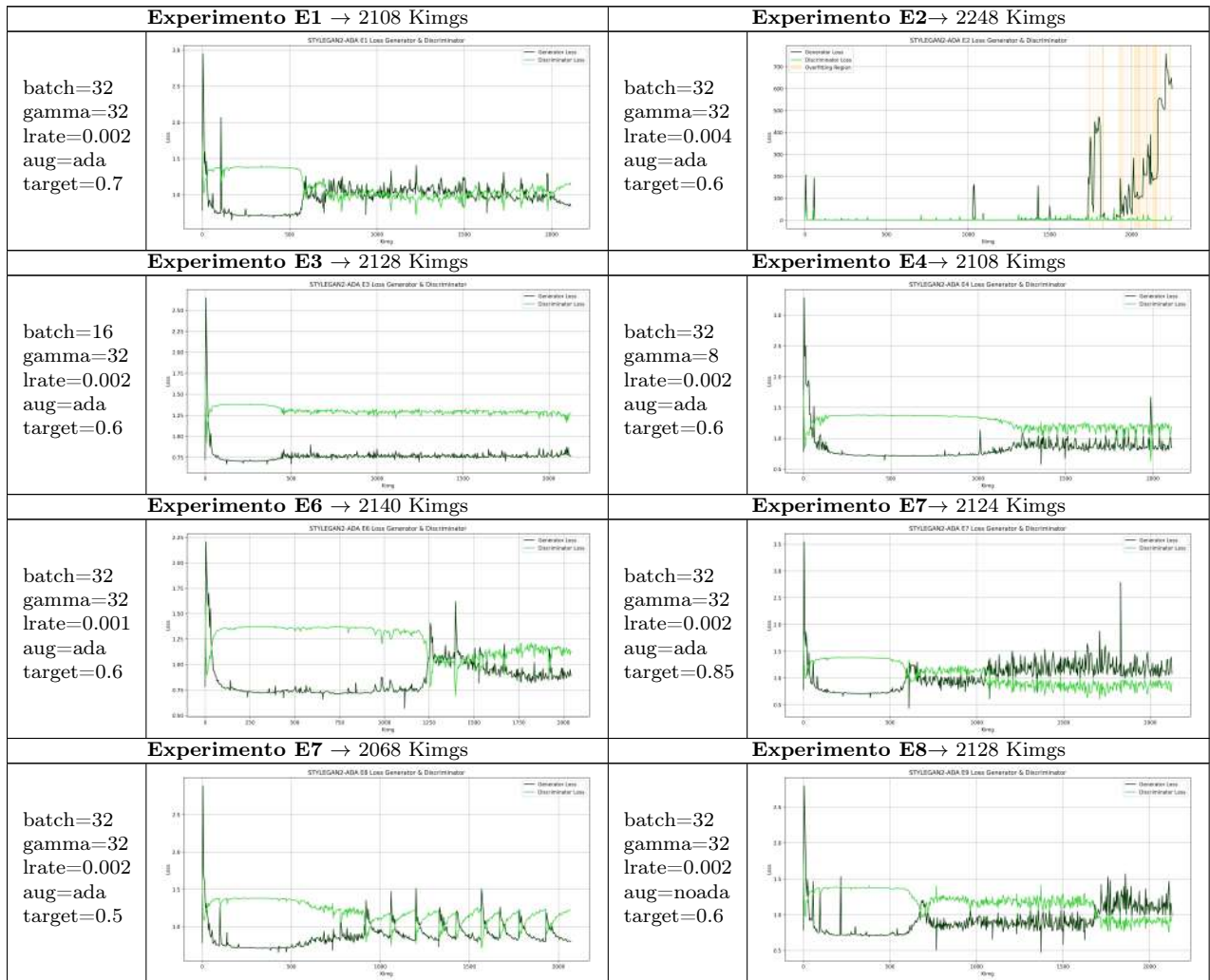


Tabla 6-2: Comparación de las pérdidas del Generador y Discriminador para las diferentes configuraciones de StyleGAN2-ADA

El análisis de las pérdidas del generador ($\mathcal{G}_{\text{loss}}$, en verde oscuro) y del discriminador ($\mathcal{D}_{\text{loss}}$ en

verde claro) se pueden observar de forma gráfica en las figuras de la Tabla **6-2**, estas permiten interpretar la estabilidad y el comportamiento global del entrenamiento en cada configuración evaluada, en general, valores de $\mathcal{G}_{\text{loss}}$ cercanos a 1 y $\mathcal{D}_{\text{loss}}$ alrededor de 1 a 1.3 suelen indicar un equilibrio competitivo entre ambas redes, mientras que desviaciones marcadas hacia valores muy altos o muy bajos reflejan desbalance en la dinámica adversaria, bajo esta premisa, las configuraciones E1, E3, E4 y E6 presentaron curvas estables, con promedios en torno a $\mathcal{G}_{\text{loss}} \approx 0,7\text{--}0,9$ y $\mathcal{D}_{\text{loss}} \approx 1,1\text{--}1,3$, esto sugiere que $\mathcal{G}$ logró producir ejemplos suficientemente convincentes para retar a $\mathcal{D}$, sin evidenciar sobreajuste ni debilitamiento excesivo de alguna de las dos partes.

El conjunto de registros de la configuración E1 cubre desde ($\text{king} \approx 0,032$) hasta ($\text{king} \approx 2108,61$)(546 registros) mostró una dinámica adversarial estable, las pérdidas de $\mathcal{G}$ oscilaron entre 0.647 y 2.95 (promedio $\approx 0,957$), y las de $\mathcal{D}$ entre 0.724 y 1.799 (promedio $\approx 1,104$), la correlación entre ambas pérdidas fue fuertemente negativa ($\approx -0,70$), indicando la típica competición adversarial, tras una fase inicial de inestabilidad (picos en $\mathcal{G}_{\text{loss}}$ alrededor de 4--12 king) típico de $\mathcal{G}$ en su inicio que a partir de ruido experimenta ajustes bruscos, el sistema alcanzó una meseta de comportamiento con oscilaciones moderadas hasta el final del registro (~ 2108 king), no hay evidencia numérica concluyente de colapso de $\mathcal{G}$ ni de divergencia numérica.

Sin embargo, $\mathcal{D}$ presentó un descenso notable hacia valores bajos en el tramo medio (mínimo $\approx 0,724$ en ~ 1224 king), esta caída de $\mathcal{D}_{\text{loss}}$ a valores más bajos indica que, en cierto tramo medio, $\mathcal{D}$ estaba clasificando con mayor confianza (más “fuerte” frente a las muestras del $\mathcal{G}$), no se observan pérdidas que exploten hacia infinito ni que se vuelvan NaN (resultados inválidos); el entrenamiento no muestra la clásica “divergencia” en la que $\mathcal{G}_{\text{loss}}$ o $\mathcal{D}_{\text{loss}}$ aumentan sin control, por tanto no hay indicios de colapso numérico/divergencia, desde ~ 200 king en adelante la pérdida de $\mathcal{G}$ permanece relativamente estable alrededor de $\sim 0,8\text{--}1,0$ y $\mathcal{D}$ también oscila en una banda moderada, esto sugiere que el entrenamiento entró en un equilibrio dinámico (no en una estabilización perfecta, sino en un punto donde ambos continúan compitiendo y ninguna de las dos pérdidas cae a extremos).

En contraste, la configuración E2 mostró un comportamiento anómalo y evidencia clara de inestabilidad, su pérdida final de $\mathcal{G}$ superó los 600, con un promedio muy elevado y gran variabilidad, mientras que $\mathcal{D}$ mantuvo valores persistentemente altos (promedio $\mathcal{D}_{\text{loss}} \approx 3,0$), esta divergencia es un indicio típico de colapso en el entrenamiento: $\mathcal{G}$ no logra aproximar la distribución de datos reales y sus salidas son fácilmente rechazadas, lo que se traduce en un aumento desmesurado de la pérdida, indicando que la configuración de hiperparámetros no era la adecuada y un posible desbalance en la regularización, lo que impidió la convergencia adversarial.

En términos de interpretación, como se ha mencionado a lo largo del presente trabajo, si FID tiene valores más bajos implican mayor similitud (mejor calidad/fidelidad), un FID

de valores en el rango bajo (e.g., $\approx 0 - 20$) suelen indicar imágenes de alta calidad para muchos dominios. En todas las configuraciones tal como se observa en la gráfica de líneas con series temporales **6-1** FID arrancan con valores altos, lo cual es esperable al inicio del entrenamiento, y luego descienden hasta un mínimo antes de rebotar al alza, ese “rebote” después del mejor punto es consistente con sobreajuste o degradación de la dinámica de entrenamiento: en todos los experimentos el FID empeora tras alcanzar su mínimo (señal clara para aplicar early stopping por FID).

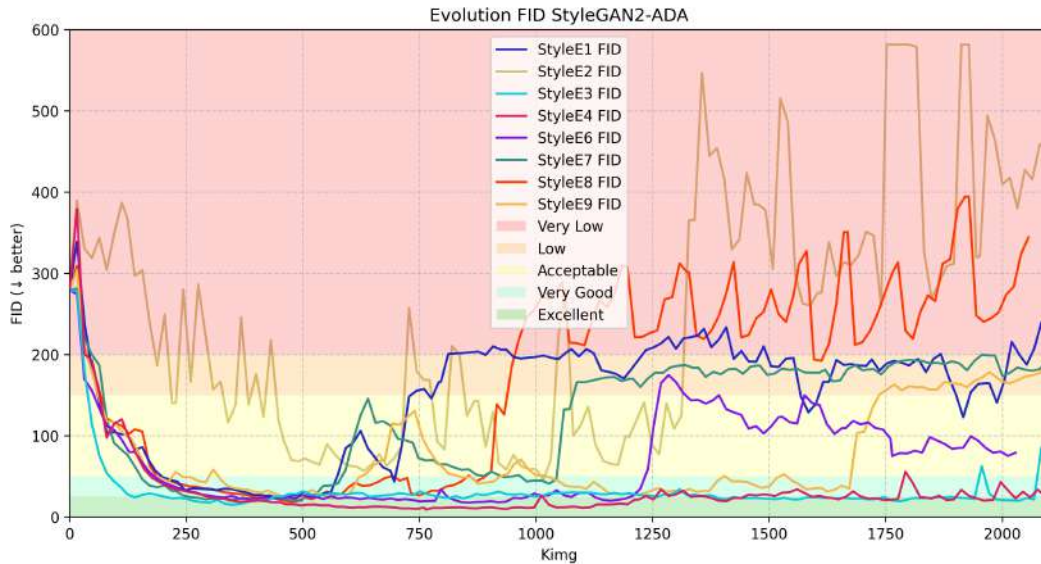


Figura 6-1: Evolución temporal del FID (en función de kimg) para todas las configuraciones de hiperparámetros en StyleGAN2-ADA.

En la Figura **6-2** se evidencia que para la mayoría de configuraciones, FID tiene una distribución amplia y altamente dispersa, indicando que los entrenamientos presentan una alta variabilidad producto de una baja estabilización en la generación de imágenes coherentes, sin embargo, E3 y E4 mantiene una distribución compacta sugiriendo un comportamiento consistente a lo largo del entrenamiento, esto se complementa con la Tabla **6-3** donde se observa que E3 y E4 alcanzan una calidad muy alta en el rango de 9 - 15, siendo valores excelentes para sintetizar imágenes ecocardiográficas con buena fidelidad.

Por el contrario E2 queda significativamente por detrás, teniendo la dispersión mas amplia junto con un promedio elevado de 254.4 que marca un rendimiento claramente inferior; a partir de la evolución temporal en FID se presentan dos patrones principales, un patrón de descenso rápido y temprano (e.g., E3 alcanza su mejor FID en (~ 348 kimg) y otro de descenso más gradual que logra su mejor punto más tarde (E4 en ~ 764 kimg y E6 en ~ 856 kimg), la fracción de pasos con mejora (caída de FID entre pasos consecutivos) oscila entre $\sim 0,43$ y $\sim 0,59$, lo que confirma que el proceso no es monótono, alterna bajadas y subidas pequeñas hasta alcanzar un valle pronunciado; tras ese valle, el FID vuelve a

subir, en términos prácticos, el checkpoint óptimo para generar nuevos datos debería fijarse exactamente en el king del FID mínimo de cada experimento, no al final del entrenamiento.

Si se compara inicio vs. final, E4 muestra una mejora neta muy grande (de 279.60 a 26.72; $\sim 90,44\%$ de reducción), pero aun así el final es peor que su propio mínimo (9.17), lo que subraya la conveniencia del early stopping, casos como E8 y E2 son ilustrativos de decaimiento severo hacia el final: E8 termina en 344.25 (peor que su inicio), y E2 finaliza en 494.16 (también peor que el inicio), lo que indica colapso o degradación tardía, en el resto (E1, E3, E6, E7, E9) el final mejora frente al inicio, pero está lejos del mejor punto intermedio.

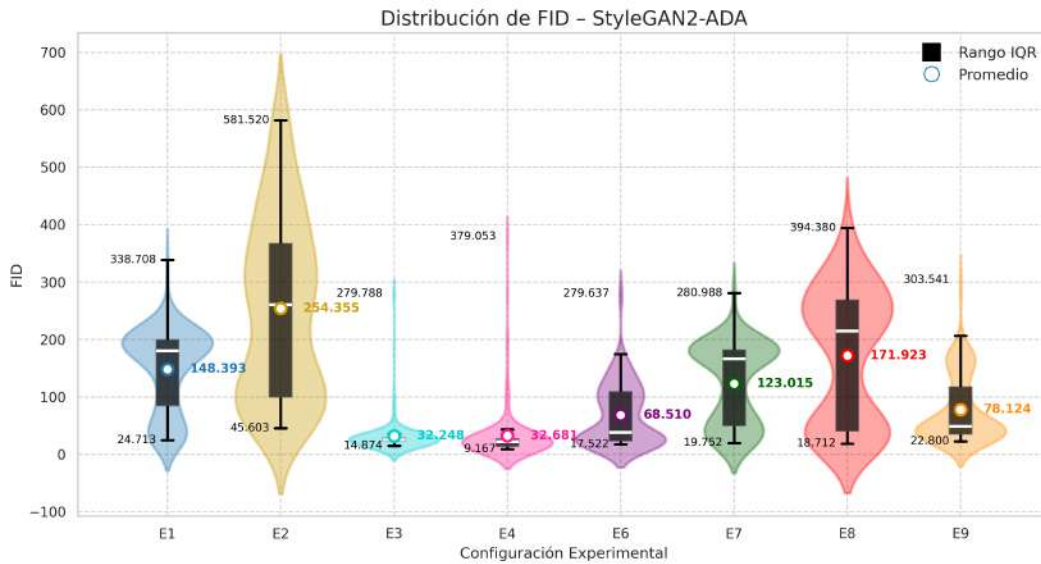


Figura 6-2: Diagrama de violín con boxplot superpuesto representando la distribución del FID para cada configuración experimental de StyleGAN2-ADA.

Tabla 6-3: Mínimos, máximos y promedios de FID registrados para los diferentes experimentos de StyleGAN2-ADA, de mejor a peor.

Experimento	FID_min ↑	king	FID_max	king	FID_promedio
E4	9.17	764	379.05	16	32.68
E3	14.87	348	279.79	0	32.25
E6	17.52	856	279.64	0	68.51
E8	18.71	472	394.38	1920	171.92
E7	19.75	400	280.99	16	123.01
E9	22.80	556	303.54	16	78.12
E1	24.71	516	338.70	16	148.39
E2	45.60	1016	581.51	1752	254.36

En consecuencia, sí se llega a valores “ideales” para la síntesis en varios experimentos: E4

(≈ 9), E3 (≈ 15) y E6 (≈ 18) se sitúan en rangos típicamente considerados de muy buena calidad para generación de imágenes, siempre condicionado por el conjunto de referencia y el protocolo de cálculo del FID, la mejor corrida para generar nuevos datos ecocardiográficos es E4 en el checkpoint de 764 king; E3 ofrece una alternativa excelente siendo más rápida y temprana (~ 384 king), y E6 es también sólida, aunque llega un poco más tarde (~ 856 king), la peor en términos de FID mínimo y estabilidad es E2, seguida por E1 y E9; en E8, pese a un buen mínimo, el comportamiento posterior sugiere inestabilidad marcada.

A diferencia del FID, el KID se estima vía MMD con un kernel polinómico y ofrece un estimador insesgado; en la práctica, valores muy cercanos a 0 indican que las imágenes sintéticas se asemejan fuertemente a las reales, como se observa en la Figura 6-3 todos los entrenamientos comienzan con $KID \approx 0,80$ en king iniciales — lo esperado al inicio — y luego descienden hasta un mínimo bien definido antes de volver a empeorar (subir) si el entrenamiento continúa, el promedio de mejora relativa (inicio→final) fue $\approx 60,9\%$, pero en los 8 experimentos el KID empeora después de su mínimo (señal clara de sobreentrenamiento y de que conviene early stopping en el punto óptimo).

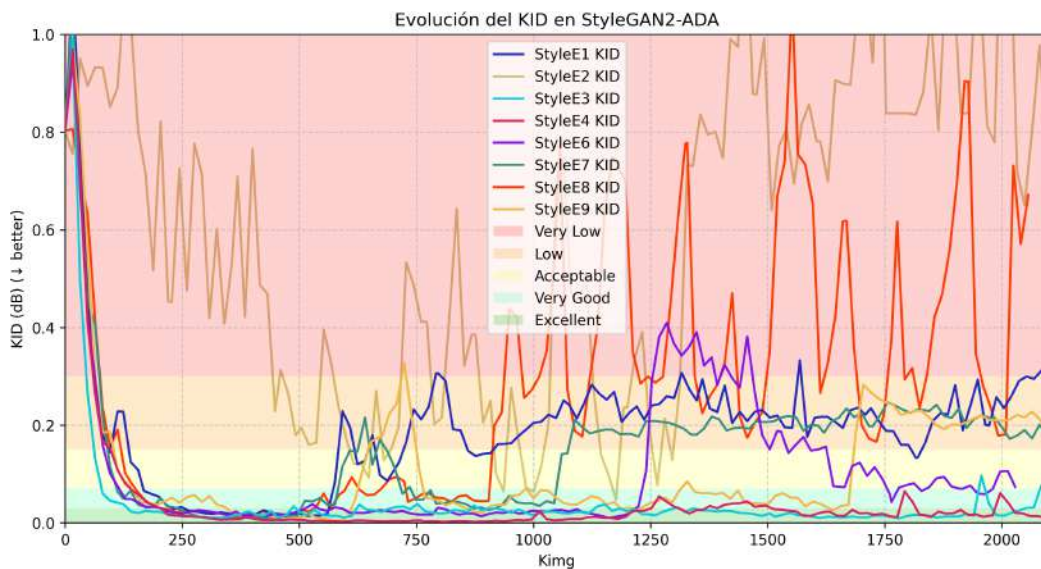


Figura 6-3: Evolución temporal del FID (en función de king) para todas las configuraciones de hiperparámetros en StyleGAN2-ADA.

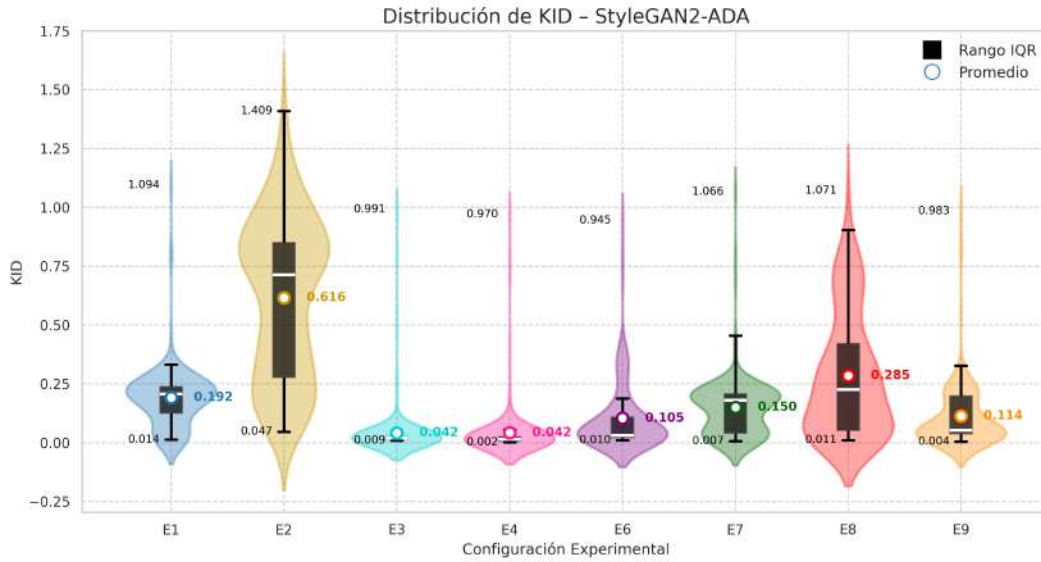


Figura 6-4: Diagrama de violín con boxplot superpuesto representando la distribución del KID para cada configuración experimental de StyleGAN2-ADA.

Como se observa en la Figura 6-4 y en la Tabla 6-4 E4 se posiciona como el modelo de mayor fidelidad, con un valor extremadamente bajo (del orden de 10^{-3}), seguido por un grupo (E9, E7, E3, E6, E8, E1) también en rangos muy buenos (0.004–0.014), E2 queda rezagado por un margen amplio, si se observa dinámica y estabilidad. De manera general aparecen dos patrones, un patrón de convergencia temprana (e.g., E3 y E7 alcanzan su mejor KID en $\sim 300 - 340$ king), útil cuando se busca reducir tiempo de cómputo; y otro tardío (e.g., E4 ~ 808 king y E6 ~ 1160 king), que logra mínimos más bajos pero exige entrenamientos más prolongados, la fracción de pasos con mejora (porcentaje de intervalos king donde el KID baja) se sitúa $\sim 0,49 - 0,58$, evidenciando que el descenso no es monótono; se alternan pequeñas subidas y bajadas hasta el valle global, tras el cual KID tiende a degradar, operativamente, el checkpoint óptimo para síntesis debe fijarse exactamente en el king del KID mínimo de cada corrida.

Tabla 6-4: Mínimos, máximos y promedios de KID registrados para los diferentes experimentos de StyleGAN2-ADA, de mejor a peor.

Experimento	KID_min $\uparrow$	king	KID_max	king	KID_promedio
E4	0.002	808	0.969	16	0.042
E9	0.004	572	0.983	16	0.114
E7	0.007	340	1.066	16	0.150
E3	0.009	300	0.991	16	0.042
E6	0.010	1160	0.945	16	0.105
E8	0.011	384	1.071	1552	0.285
E1	0.013	456	1.093	16	0.192
E2	0.047	1176	1.409	1740	0.616

En términos de competencia para generar nuevos datos ecocardiográficos, los valores mínimos alcanzados por E4 ($\sim 0,0023$) y el grupo E9–E7–E3–E6–E8–E1 ($\sim 0,004 - 0,014$) son consistentes con alta calidad y cercanía a la distribución real en el espacio de características, de este modo, E4 es la mejor elección para la generación (checkpoint ~ 808 king), seguido de E9 (572 king), E7 (340 king) y E3 (300 king) si se prioriza convergencia más temprana, E2 debe evitarse, dado su peor KID mínimo y su inestabilidad tardía.

En JSD los valores más bajos indican mayor similitud entre ambas distribuciones, en términos prácticos, una caída del JSD a lo largo del entrenamiento sugiere que $\mathcal{G}$ está alineando mejor su distribución con la de referencia; un aumento posterior suele indicar sobreentrenamiento o degradación de la dinámica de entrenamiento, para los experimentos (E1–E4, E6–E9) se observa en la Figura 6-5 un patrón consistente: el entrenamiento inicia con JSD $\approx 0,13$ a king muy bajos ($\approx 0,016 - 0,0320$) y luego desciende hasta un mínimo bien definido; si el entrenamiento continúa, el JSD vuelve a subir, esto último ocurre en los 8 experimentos, lo que respalda la aplicación de early stopping en el punto de mínimo JSD, en promedio, la fracción de pasos con mejora (proporción de intervalos king donde JSD disminuye) ronda 0.46–0.52, evidenciando una convergencia no monótona: pequeñas oscilaciones sobre una tendencia global descendente hasta el valle.

Por calidad según un JSD mínimo sitúa a E4 como la opción de mayor fidelidad, con un valor notablemente bajo tal y como se observa en la Tabla 6-5, teniendo en cuenta las distribuciones y promedios los mejores modelos son E3, E4 y E6 como se ve en la Figura 6-6, el grupo E3/E1/E6/E8/E9/E7 presenta mínimos cercanos ($\approx 0,017 - 0,019$), también compatibles con buena aproximación a la distribución real; E2 queda rezagado de forma clara.

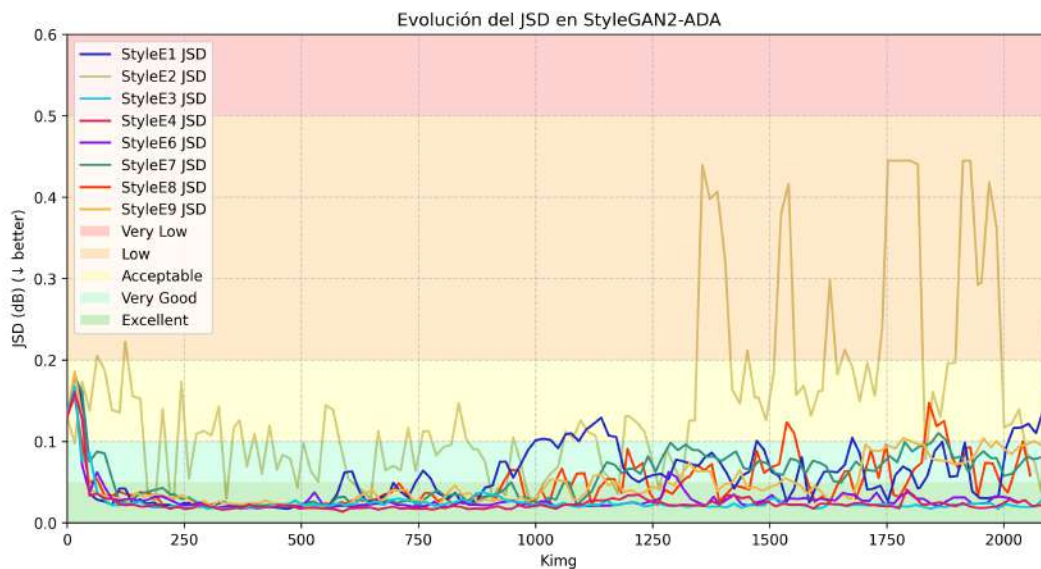


Figura 6-5: Evolución temporal del JSD (en función de king) para todas las configuraciones de hiperparámetros en StyleGAN2-ADA.

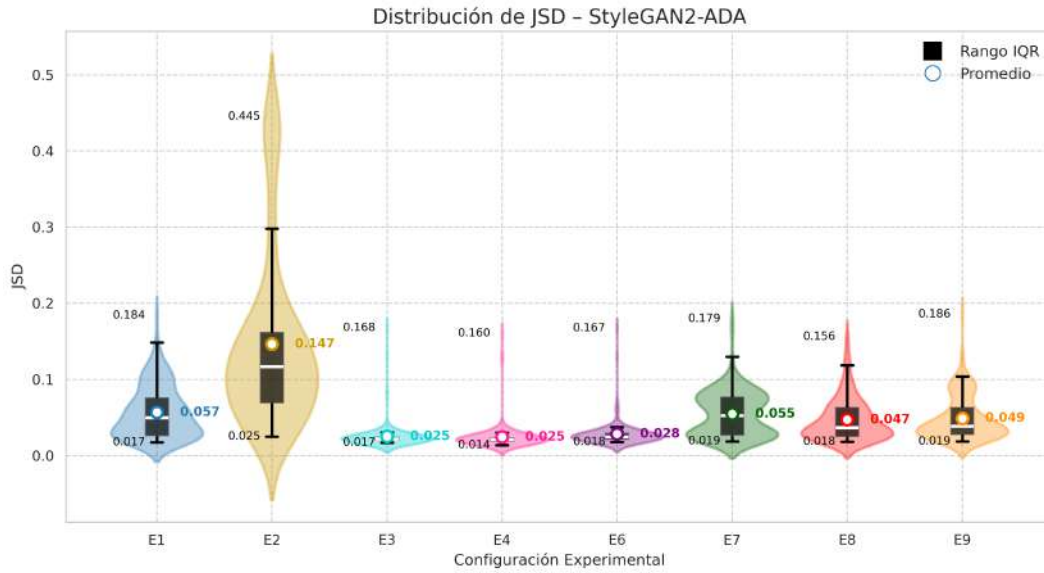


Figura 6-6: Diagrama de violín con boxplot superpuesto representando la distribución del KID para cada configuración experimental de StyleGAN2-ADA.

Tabla 6-5: Mínimos, máximos y promedios de JSD registrados para los diferentes experimentos de StyleGAN2-ADA, de mejor a peor.

Experimento	JSD_min ↑	king	JSD_max	king	JSD_promedio
E4	0.0137	588	0.160	16	0.025
E3	0.0170	1988	0.168	16	0.025
E1	0.0173	472	0.183	16	0.057
E6	0.0178	408	0.168	16	0.028
E8	0.0181	520	0.156	16	0.047
E9	0.0187	536	0.186	16	0.049
E7	0.0187	324	0.179	16	0.055
E2	0.0251	1000	0.445	1752	0.147

Analizando la evolución temporal, se distinguen dos perfiles útiles operativamente, un perfil de convergencia temprana (e.g., E7 alcanza su mejor JSD hacia ~ 324 king y E6 hacia ~ 408 king), interesante si se prioriza costo computacional reducido; y un perfil de convergencia tardía (e.g., E3 logra su mínimo recién en ~ 1998 king, y E4 en ~ 588 king), que puede entregar mínimos más bajos pero exige entrenamientos más largos y control cuidadoso para evitar degradación posterior, las configuraciones mejoran en porcentaje (inicio→final) de la siguiente manera: por ejemplo, E4 mejora 84.46 % ($0.12994 \rightarrow 0.02019$), E6 80.13 %, E3 74.11 % y E8 55.41 %, en contraste, E2 termina peor que al inicio ($0.13048 \rightarrow 0.41841$; -220.67 %), un caso clásico de colapso tardío, pese a tener un mínimo intermedio razonable (0.02508), continuar entrenando lo aleja de la solución óptima.

Para evaluar SSIM, como se mencionaba antes, al ser una métrica de similitud es necesario

tener en cuenta que para GANs de tipo generativo no se espera alcanzar valores cercanos a 1, sino observar tendencias relativas, estabilidad y presencia (o ausencia) de sobreentrenamiento; en todos los experimentos, tal como se ve en la Figura 6-7 el SSIM inicial comienza consistentemente en torno a 0.109, lo que refleja el comportamiento típico de $\mathcal{G}$ al inicio: produce imágenes prácticamente aleatorias, con apenas una coincidencia estructural mínima respecto al dominio ecocardiográfico real, conforme avanza el entrenamiento, el SSIM aumenta progresivamente, indicando que el modelo comienza a aprender patrones estructurales recurrentes, como bordes cardiacos, cavidades o texturas específicas del ultrasonido, el pico de SSIM varía entre 0.382 y 0.397, siendo E7 el que alcanza el mayor valor (0.3967), seguido muy de cerca por E8 y E2 ($\approx 0,394 - 0,392$), estos serían, por tanto, los modelos con mejor asimilación estructural global del dominio real, al menos en términos de consistencia visual estadística.

Sin embargo, un aspecto importante emerge: en la mayoría de los experimentos (E1, E3, E4, E6, E7, E8, E9) como se observa en la distribución de la Figura 6-10 tiene distribuciones sumamente similares y como se ve en la clasificación de la Tabla 6-6, el valor final de SSIM no mejora con respecto al máximo, se podría pensar que el entrenamiento no se detuvo en el punto óptimo, o que algunos registros finalizaron antes de estabilizarse; sin embargo, las distribuciones similares en diferentes configuraciones que se sabe generaron datos diferentes, permite ver que existe un problema en la capacidad de la métrica en este tipo de GAN, y no un problema en la configuración como tal.

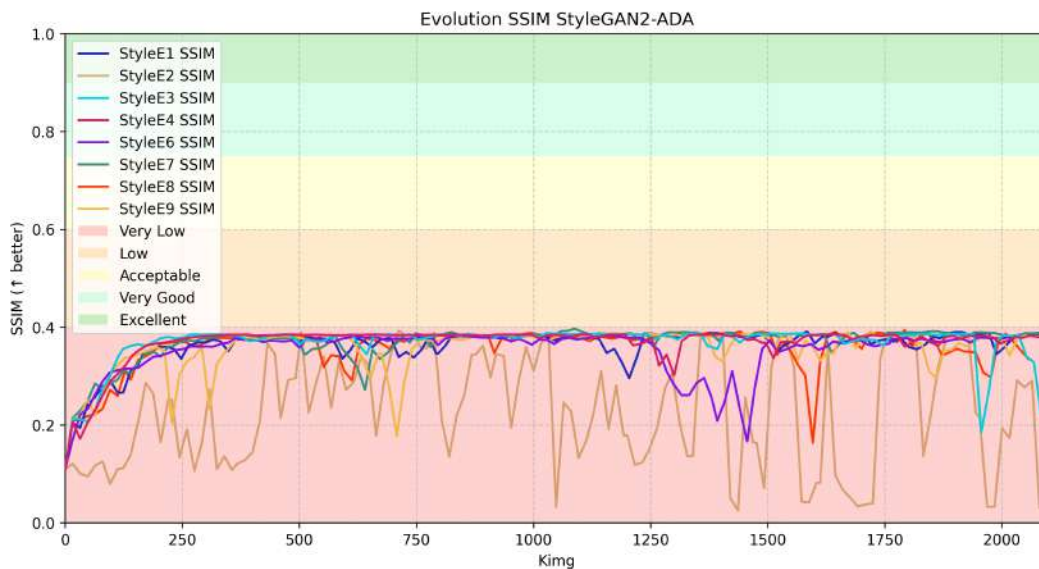


Figura 6-7: Evolución temporal del SSIM (en función de kimg) para todas las configuraciones de hiperparámetros en StyleGAN2-ADA.

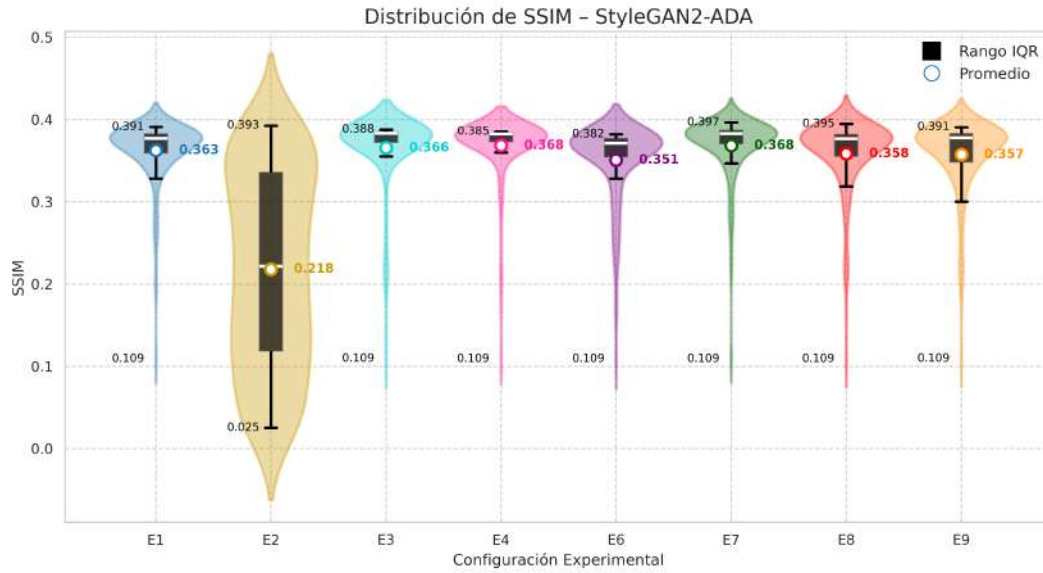


Figura 6-8: Diagrama de violín con boxplot superpuesto representando la distribución del SSIM para cada configuración experimental de StyleGAN2-ADA.

En contraste, analizando lo proporcionado por SSIM se observa que E4 y E9 se mantiene elevado (0.3787 y 0.3858 respectivamente), aunque ligeramente por debajo del pico máximo, indicando que después de cierto punto el SSIM deja de mejorar y comienza a estabilizar o decrecer levemente, lo cual es una señal clásica de sobreentrenamiento estructural, en otras palabras, el modelo aprende demasiado algunos patrones y pierde diversidad estructural, afectando la métrica global.

Tabla 6-6: Mínimos, máximos y promedios de SSIM registrados para los diferentes experimentos de StyleGAN2-ADA, de mejor a peor.

Experimento	SSIM_max ↑	kimg	SSIM_min	kimg	SSIM_promedio
E7	0.397	1088	0.109	0	0.368
E8	0.394	1792	0.109	0	0.358
E2	0.393	712	0.025	1436	0.218
E1	0.391	1584	0.109	0	0.363
E9	0.390	1720	0.109	0	0.357
E3	0.388	1028	0.109	0	0.366
E4	0.385	1648	0.109	0	0.368
E6	0.382	1160	0.109	0	0.351

En conclusión, el SSIM en este contexto confirma que para StyleGAN2-ADA al ser un GAN no pareado no será estructuralmente parecida a imágenes individuales del *dataset*, esta métrica para estos GANs están sesgados negativamente, entonces para este tipo de GAN esos valores son aceptables, indicando que no solo genera imágenes realistas visualmente, sino

que captura progresivamente estructuras inherentes al dominio médico incluso en ausencia de supervisión directa, el hecho de que los valores converjan en torno a 0.38–0.39 sugiere la existencia de un punto de saturación estructural, a partir del cual entrenamientos más prolongados no aportan mejoras sustanciales e incluso comienzan a degradar la distribución aprendida.

En este estudio, el PSNR — al igual que en SSIM — se calcula sobre GANs de tipo generativo, donde no existe una imagen pareada ni un emparejamiento 1:1; en este contexto, valores mayores de PSNR sugieren menor distorsión promedio y, por tanto, una señal sintética más estable y coherente, debido a que el cálculo se realiza sobre snapshots de $\mathcal{G}$ mediante un pipeline estandarizado, los valores absolutos no deben compararse con tareas clásicas de denoising o super-resolución; lo informativo es la tendencia temporal, la ubicación del máximo y la estabilidad posterior.

Como se ve en la Figura 6-9, todos los experimentos exhiben el mismo patrón global: el entrenamiento parte con PSNR bajo ($\approx 5,88dB$ en king iniciales), luego mejora con rapidez hasta un máximo bien definido y, si se continúa entrenando, decae, confirmando sobreentrenamiento radiométrico después del punto óptimo, la fracción de pasos con incremento (0.48–0.51) muestra una convergencia no monótona: el PSNR no sube de forma estrictamente creciente, pero su tendencia es claramente ascendente hasta el máximo, operativamente, esto respalda aplicar early stopping exactamente en el king del PSNR máximo para consolidar el mejor checkpoint.

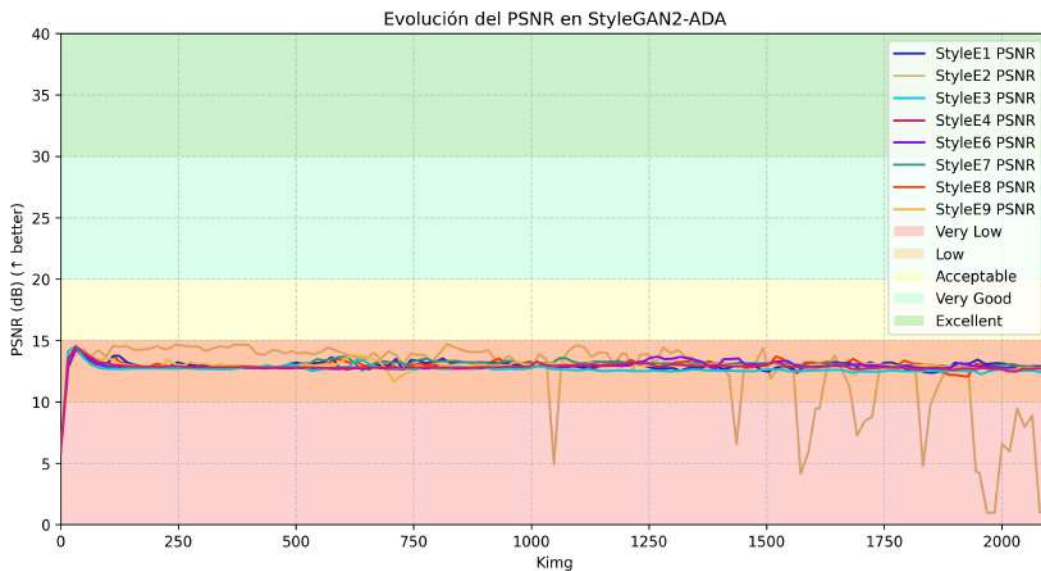


Figura 6-9: Evolución temporal del PSNR (en función de king) para todas las configuraciones de hiperparámetros en StyleGAN2-ADA.

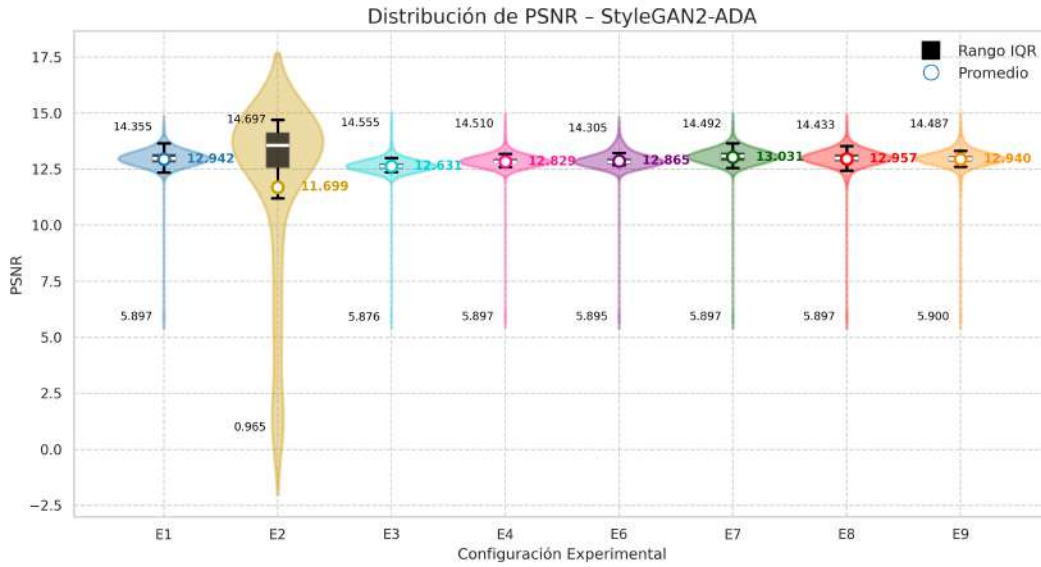


Figura 6-10: Diagrama de violín con boxplot superpuesto representando la distribución del PSNR para cada configuración experimental de StyleGAN2-ADA.

Tal como ocurre en SSIM los experimentos (E1, E3, E4, E6, E7, E8, E9) tienen una distribución casi idéntica como se observa en la Figura 6-10 y se detalla en la Tabla 6-7, E2 logra el pico más alto, pero lo hace tardíamente (≈ 820 king) y presenta inestabilidad severa: al final cae a 0.965 dB, convirtiéndose también en el peor final por amplio margen (indicativo de degradación/colapso si se entrena más allá del óptimo), segundo, en el resto de experimentos (E1, E3, E4, E6, E7, E8, E9) el máximo ocurre muy temprano (≈ 32 king), lo que se pensaría es ventajoso en costo de cómputo.

Tabla 6-7: Mínimos, máximos y promedios de PSNR registrados para los diferentes experimentos de StyleGAN2-ADA, de mejor a peor.

Experimento	PSNR_max ↑	king	PSNR_min	king	PSNR_promedio
E2	14.69	820	0.97	1968	11.69
E3	14.56	32	5.88	0	12.63
E4	14.51	32	5.89	0	12.82
E7	14.49	32	5.89	0	13.03
E9	14.49	32	5.89	0	12.94
E8	14.43	32	5.89	0	12.96
E1	14.35	32	5.89	0	12.94
E6	14.30	32	5.89	0	12.86

Los resultados refuerzan que el PSNR medido sin pares directos sirve como proxy estadístico de la estabilidad de $\mathcal{G}$, en todos los casos, la métrica progresa desde valores iniciales muy bajos hasta un óptimo identificable, a partir del cual no mejora e incluso degrada.

La métrica LPIPS al igual que en las demás métricas de similitud empleadas, no existe correspondencia 1:1 entre imágenes reales y sintéticas, en este contexto, valores más bajos de LPIPS indican que $\mathcal{G}$ está produciendo imágenes cuya estructura visual y texturas locales se alinean mejor con las características perceptuales del dominio real ecocardiográfico, por tanto, la métrica se interpreta como un indicador de fidelidad visual global, y no como una métrica de reconstrucción o diversidad.

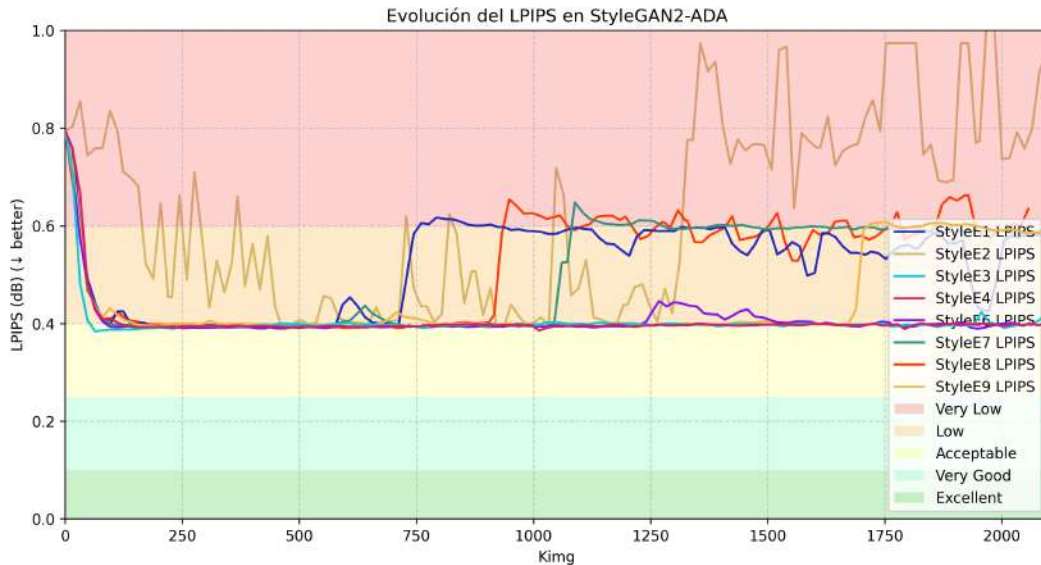


Figura 6-11: Evolución temporal del LPIPS (en función de king) para todas las configuraciones de hiperparámetros en StyleGAN2-ADA.

Como se observa en la Figura 6-11, en todos los experimentos el entrenamiento inició con valores de LPIPS elevados, en torno a 0.79, reflejando una marcada disimilitud perceptual inicial, conforme avanza el entrenamiento, todas las métricas convergen hacia valores sustancialmente menores, oscilando entre 0.38 y 0.40, lo que evidencia una mejora clara en la capacidad de $\mathcal{G}$ para reproducir patrones visuales ecocardiográficos realistas, sin embargo, el comportamiento no es estrictamente monótono: tras alcanzar un mínimo, algunas corridas muestran un ligero rebote o deterioro, evidenciando que $\mathcal{G}$ puede sobreajustarse o desviarse perceptualmente si se continúa entrenando más allá del punto óptimo.

A diferencia de las otras métricas de similitud se puede apreciar un comportamiento mas cercano a las métricas de distribución, tanto SSIM como PSNR presentaron valores incongruentes, sin aporte significativo que diera a entender el rendimiento de las configuraciones, sin embargo, esta métrica mantiene distribuciones diferentes como se observa en la Figura 6-12 y aporta información significativa como se detalla en la Tabla 6-8,

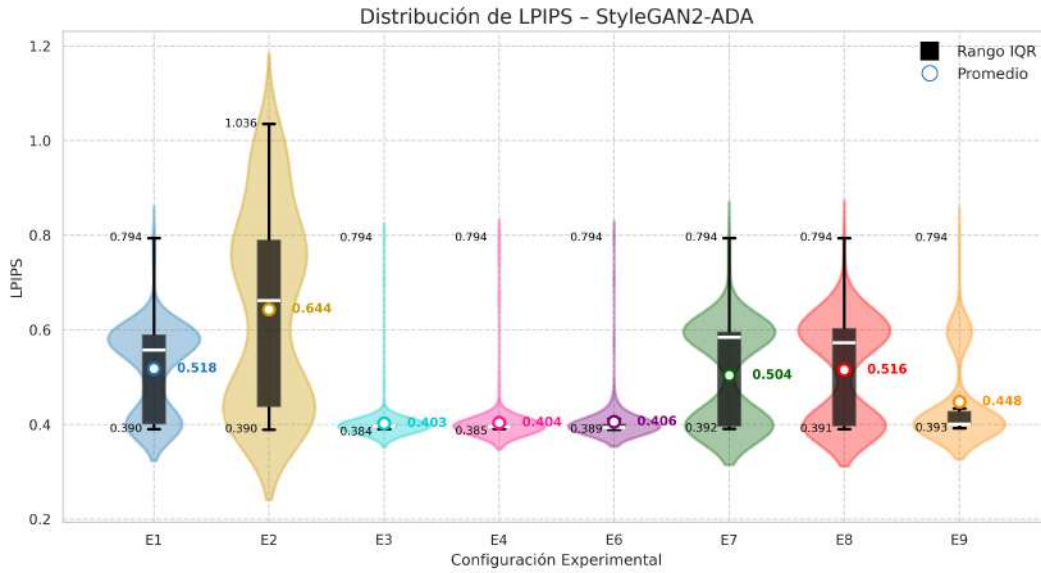


Figura 6-12: Diagrama de violín con boxplot superpuesto representando la distribución del LPIPS para cada configuración experimental de StyleGAN2-ADA.

Por ende el experimento E3 alcanzó el mejor valor de LPIPS (0.3838), seguido muy de cerca por E4 (0.3852) y E6 (0.3892), estos valores constituyen los estadios de mayor similitud perceptual alcanzados entre las imágenes reales y las sintéticas, situando a dichos experimentos como los más exitosos en términos de realismo visual aprendido, en contraste, E9 alcanzó el peor valor mínimo de la serie (0.3925) y además terminó con un LPIPS final elevado ($\approx 0,5853$), reflejando un deterioro significativo tras el óptimo. E2, aunque alcanzó un mínimo intermedio (0.3896), mostró una alta inestabilidad, llegando a registrar valores máximos por encima de 1.0, lo cual indica un comportamiento inconsistente de $\mathcal{G}$.

Tabla 6-8: Mínimos, máximos y promedios de LPIPS registrados para los diferentes experimentos de StyleGAN2-ADA, de mejor a peor.

Experimento	LPIPS_min ↑	king	LPIPS_max	king	LPIPS_promedio
E3	0.384	64	0.794	0	0.403
E4	0.385	1012	0.794	0	0.404
E6	0.389	1948	0.794	0	0.406
E2	0.389	476	1.036	1968	0.644
E1	0.390	232	0.794	0	0.518
E8	0.391	708	0.794	0	0.516
E7	0.392	764	0.794	0	0.504
E9	0.393	968	0.794	0	0.448

En definitiva para las diferentes configuraciones de StyleGAN2-ADA, las métricas calculadas sobre snapshots de $\mathcal{G}$ en lugar de comparaciones 1:1, produce métricas estadísticas de distri-

bución (no de reconstrucción puntual), por lo que la interpretación se centro en tendencias y valores relativos entre experimentos, al comparar los mínimos/máximos relevantes por experimento, E4 es el único experimento que gana en tres métricas clave de distribución, tal como se observa en las Tablas: **6-3**, **6-4**, **6-5**, con un mejor FID de $\sim 9,17$, mejor KID de $\sim 0,00227$ y un mejor JSD de $\sim 0,01367$; es decir, E4 produce la mejor aproximación estadística al dominio real, con sus mínimos concentrados en un rango aproximadamente 600 – 820 king, esto define una ventana operativa donde E4 muestra la mejor fidelidad de distribución.

Respecto a las métricas perceptuales existen matices, donde, el máximo SSIM lo alcanza E7 ($\sim 0,3927$), lo que sugeriría que modela mejor la estructura local de las imágenes (e.g., disposición de bordes y texturas ecocardiográficas), su máximo se observa relativamente tarde en entrenamiento (1088 king); el máximo PSNR lo alcanza E2 ($\sim 14,70$ dB, 820 king), sin embargo, E2 presenta múltiples problemas graves como ya se ha mencionado y el mínimo LPIPS lo alcanza E3 ($\sim 0,3838$, 64 king), indicando que E3 logra la mejor similitud perceptual en su punto óptimo (además E3 está muy cerca de E4 en la puntuación global).

6.1.2 Resultados en configuraciones de MedGAN

Experimento M1 → 1008 Épocas	Experimento M2 → 1098 Épocas
batchSize=20, nz=100, ngf=64, ndf=64, lrD=5e-5, lrG=5e-5, Ditters=5, opt=RMS, T1=0.3, T2=0.5	batchSize=64, nz=100, ngf=64, ndf=64, lrD=2e-5, lrG=2e-5, Ditters=1, opt=Adam, T1=0.3, T2=0.5
Experimento M3 → 925 Épocas	Experimento M4 → 1020 Épocas
batchSize=16, nz=200, ngf=128, ndf=128, lrD=5e-5, lrG=5e-5, Ditters=5, opt=RMS, T1=0.4, T2=0.6	batchSize=32, nz=150, ngf=128, ndf=128, lrD=5e-5, lrG=1e-5, Ditters=3, opt=Adam, T1=0.2, T2=0.5
Experimento M5 → 1039 Épocas	Experimento M6 → 1284 Épocas
batchSize=32, nz=100, ngf=64, ndf=128, lrD=1e-5, lrG=5e-5, Ditters=7, opt=RMS, T1=0.3, T2=0.6	batchSize=32, nz=100, ngf=64, ndf=64, lrD=5e-5, lrG=5e-5, Ditters=5, noBN=True, opt=RMS, T1=0.3, T2=0.5
Experimento M7 → 1493 Épocas	Experimento M8 → 1065 Épocas
batchSize=40, nz=120, ngf=96, ndf=96, lrD=1e-5, lrG=5e-5, Ditters=2, opt=RMS, T1=0.25, T2=0.55	batchSize=64, nz=100, ngf=128, ndf=128, lrD=2e-5, lrG=2e-5, Ditters=1, opt=Adam, T1=0.1, T2=0.3

Tabla 6-10: Comparación de las pérdidas del Generador y Discriminador para las diferentes configuraciones de MedGAN

A partir de este punto se hará énfasis únicamente en aspectos relevantes, sin profundizar en comportamientos específicos de las funciones de pérdida, ni contribución detallada de cada métrica o que no se tomara relevancia las métricas de SSIM y PSNR, dado que los comportamientos de pérdidas y métricas mantienen similitud a lo presentado en StyleGAN2-ADA.

En el caso de MedGAN el tiempo de entrenamiento presento una mayor variación en comparación con las diferentes configuraciones de StyleGAN2-ADA, para la arquitectura MedGAN el objetivo fue completar un mínimo de 1000 épocas de entrenamiento, bajo este objetivo, la configuración M7 fue la mas rápida alcanzando dicha cantidad de entrenamiento en 5 días, le siguieron M1 y M6 que lo lograron en 6 días, M2 en 8 días, M4 en 14 días, M5 en 15 días, M8 en 23 días, siendo M3 la configuración mas lenta ocupando un lapso 27 días donde aun no logra alcanzar el objetivo de 1000 épocas, debido a limitaciones de tiempo y considerando que en la ultima etapa a M3 le estaba tomando mas tiempo por época, se decidió finalizar su entrenamiento hasta 925 épocas.

Cabe resaltar que para MedGAN $\mathcal{D}_{\text{loss}}$ las perdidas toman valores negativos con alta frecuencia en todas las configuraciones debido a que está basado en WGAN y DCGAN, en WGAN el “critic” no produce una probabilidad y la pérdida puede ser cualquier número real, para este modelo se puede observar un comportamiento mucho más competitivo, siendo frecuente la oscilación producto de una adecuada competencia adversarial entre $\mathcal{G}$ y $\mathcal{D}$.

Partiendo el análisis desde M1, se observa en las figuras de la Tabla **6-10** que su $\mathcal{G}_{\text{loss}}$ (en morado oscuro) se mantiene en un rango medio (promedio $\approx 0,74$) y con variabilidad moderada ($\text{std} \approx 0,25$), $\mathcal{D}_{\text{loss}}$ (en morado claro) presenta valores consistentemente negativos (promedio $\approx -0,68$) y una variabilidad similar a la de $\mathcal{G}$, las curvas muestran oscilaciones continuas pero sin una tendencia marcada de divergencia en las últimas épocas; el entrenamiento parece alcanzar un equilibrio dinámico donde $\mathcal{G}$ y $\mathcal{D}$ que siguen compitiendo --- no hay colapso de modo evidente ni una convergencia limpia a un mínimo --- pero el ruido y la variabilidad sugieren que ambos modelos no han reducido completamente la incertidumbre de su juego adversarial.

En cuanto a M2, es la configuración que presenta mayor estabilidad estadística: las desviaciones estándar de las pérdidas de $\mathcal{G}$ y $\mathcal{D}$ son las más bajas (0.188 y 0.158, respectivamente), el promedio de $\mathcal{G}_{\text{loss}}$ es moderada ($\approx 0,58$) y la de $\mathcal{D}_{\text{loss}}$ es la más cercana a cero ($\approx -0,54$), las pendientes en el tramo final son prácticamente nulas, las pérdidas oscilan menos y se mantienen en bandas relativamente estrechas; por otra parte, M3 muestra la mayor inestabilidad: medias de $\mathcal{G}_{\text{loss}}$ elevadas ($\approx 1,61$) y $\mathcal{D}_{\text{loss}}$ con promedio $\approx -1,13$ y gran dispersión ($\text{std}_D \approx 0,94$, $\text{std}_G \approx 0,585$), además, aparecen picos muy negativos en $\mathcal{D}_{\text{loss}}$ (mínimos hasta $\approx -3,48$), estos números y las curvas (saltos pronunciados) son indicativos de una dinámica inestable entre $\mathcal{G}$ y $\mathcal{D}$.

Las demás configuraciones también presentan oscilaciones continuas sin percibirse un colapso de modo, de manera breve se menciona que, M4 presenta pérdidas promedios bajas de $\mathcal{G}_{\text{loss}}$ ($\approx 0,56$) y de $\mathcal{D}_{\text{loss}}$ relativamente cercanas a cero ($\approx -0,49$), con desviaciones estándar moderadas y controladas ($\text{std}_G \approx 0,213$, $\text{std}_D \approx 0,156$); M5 exhibe promedios de la pérdida de $\mathcal{G}$ y $\mathcal{D}$ más altas que M2/M4 (promedio $\mathcal{G}_{\text{loss}} \approx 1,09$, promedio $\mathcal{D}_{\text{loss}} \approx -0,93$) y dispersión notable ($\text{std}_G \approx 0,418$, $\text{std}_D \approx 0,572$), la mayor variación en M5 sugiere condiciones de

entrenamiento menos controladas y una dinámica adversaria que, en ocasiones, deja a $\mathcal{G}$ y $\mathcal{D}$ en estados extremos; M6 muestra una dinámica más controlada, $\mathcal{G}_{\text{loss}}$ mantiene una pérdida media de $\approx 0,67$ (rango entre 0.04 y 1.29), mientras que $\mathcal{D}_{\text{loss}}$ oscila entre 1.9 y 0.17, con promedio de -1.09 ; en M7, la dinámica adversarial se vuelve más activa, $\mathcal{G}_{\text{loss}}$ oscila en un rango más amplio (0.003 a 2.51, promedio $\approx 1,13$), mientras que $\mathcal{D}_{\text{loss}}$ varía de -4.65 a 0.84, con promedio -0.46 , la correlación ligeramente negativa entre ambas pérdidas (-0.10) indica un comportamiento adversarial clásico; finalmente, la configuración M8 muestra las fluctuaciones más amplias en $\mathcal{G}_{\text{loss}}$ (-0.01 a 2.85, promedio $\approx 0,65$), acompañadas de $\mathcal{D}_{\text{loss}}$ con rango -5.65 a 0.82 y promedio -0.15 , en esta configuración, $\mathcal{D}$ aún muestra predominio (84% de valores negativos), pero con menor intensidad que en las demás, lo que sugiere un equilibrio más cercano entre ambos adversarios.

En conclusión, se observaron comportamientos distintos según la configuración de hiperparámetros, las configuraciones M2, M4, M6 y M8 mostraron el comportamiento más estable: bandas de pérdida estrechas y pendientes próximas a cero en el tramo final, sugiriendo un equilibrio dinámico entre $\mathcal{G}$ y $\mathcal{D}$, M5 muestra el desequilibrio más pronunciado (dominancia de $\mathcal{D}$), M7 exhibe mayor variabilidad, aunque sin divergencia, por el contrario, M3 presentó la mayor inestabilidad (picos extremos en la pérdida de $\mathcal{D}$ y alta varianza), lo que apunta a un ajuste deficiente de hiperparámetros, teniendo sentido y correlación en la gran cantidad de tiempo requerido para su entrenamiento.

El FID para las diferentes configuraciones destaca a M2 que presentó el desempeño más favorable, alcanzando el menor valor promedio de FID ($\approx 80,35$) tal como se aprecia en la Figura **6-14** y se remarca en la Tabla **6-11**, registrando además el valor mínimo absoluto del conjunto (63.47) cerca del final del entrenamiento, este comportamiento es indicativo de un proceso de aprendizaje progresivo y sostenido, por el contrario, la configuración M3 fue la de peor rendimiento, reportando el FID promedio más elevado ($\approx 149,96$) y exhibiendo el pico máximo más pronunciado (458.69) presente en etapas finales del entrenamiento indicando un claro deterioro.

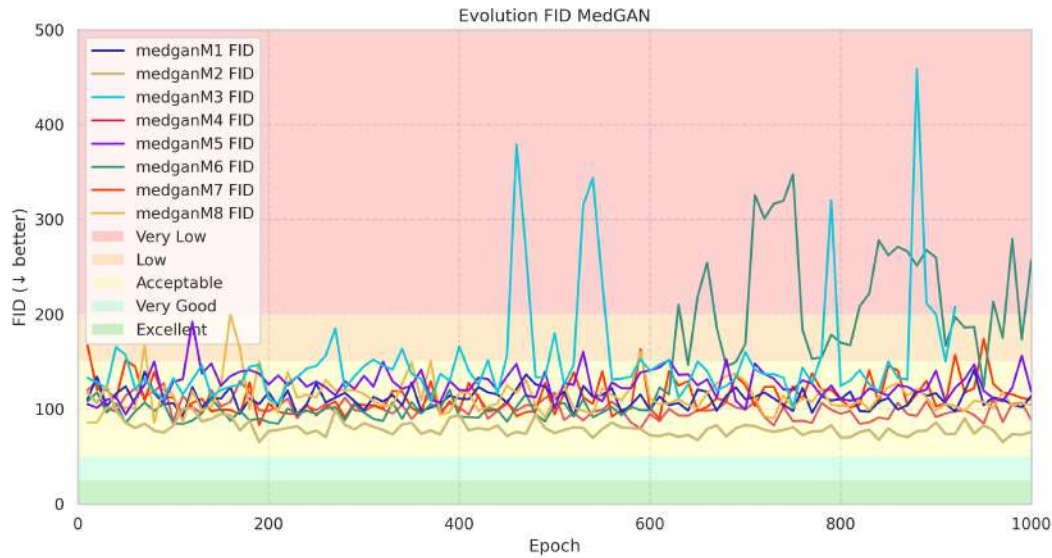


Figura 6-13: Evolución temporal del FID (en función de época) para todas las configuraciones de hiperparámetros en MedGAN.

M4 evidenció un patrón relativamente estable, apreciable en su distribución compacta como se ve en la Figura 6-14, manteniendo valores cercanos a su mínimo sin variaciones abruptas, M1, M5, M7 y M8 se sitúan en un punto intermedio, mostrando aprendizaje pero con fluctuaciones que denotan inestabilidad del entrenamiento adversarial, aunque lograron valores bajos de FID en las primeras fases, posteriormente se observaron repuntes negativos, lo que sugiere posibles procesos de sobreentrenamiento o pérdida de capacidad generativa en etapas posteriores; en particular, M8 destacó porque alcanzó su mejor FID de forma prematura, seguido de un deterioro posterior, lo cual es característico de un modelo que requiere ajustes en las tasas de aprendizaje o mayor regularización y M6 evidencia inestabilidad y falta de convergencia, reflejada en un FID promedio elevado y oscilaciones amplias, indicando menor capacidad de generalización y posible colapso de modos parcial, situándola en la segunda peor configuración para MedGAN.

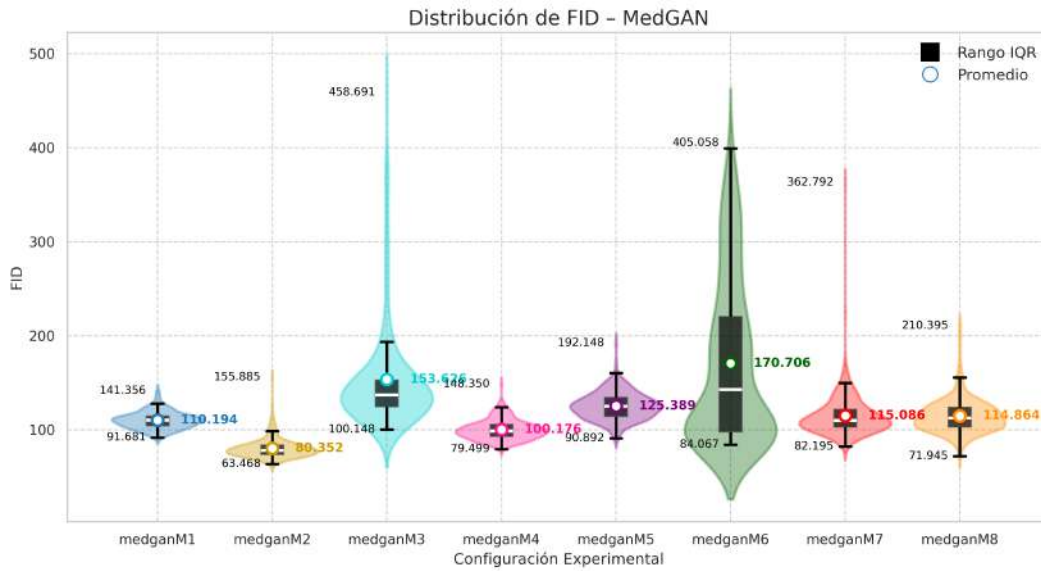


Figura 6-14: Diagrama de violín con boxplot superpuesto representando la distribución del FID para cada configuración experimental de MedGAN.

Tabla 6-11: Mínimos, máximos y promedios de FID registrados para los diferentes experimentos de MedGAN, de mejor a peor.

Experimento	FID_min ↑	época	FID_max	época	FID_promedio
M2	63.46	1070	155.88	5	80.352
M8	71.95	75	210.39	245	114.86
M4	79.49	590	148.35	5	100.18
M7	82.19	255	362.79	5	115.09
M6	84.07	215	405.05	1200	170.71
M5	90.89	45	192.15	120	125.39
M1	91.68	255	141.36	940	110.19
M3	100.15	105	458.69	905	153.62

Según KID, la configuración M2 presentó el menor valor medio (0.027), siendo muy buena y con fuerte mejora, promedio bajo de (0.0237) aunque inicia alto (first KID = 0.1397) termina muy bajo (last = 0.00952) teniendo una mejora del 93 %, tiene pocos picos, buena convergencia; para M1 promedio muy bajo (0.0135) y baja desviación relativa, indicando buena similitud entre generadas y reales, mantiene una tendencia de ligera mejora del 22.7 %, 4 picos detectados (cambios bruscos pequeños) observables en la Figura 6-15, configuración con mejor comportamiento de KID como se observa en la Tabla 6-12 teniendo una convergencia suave y estable según estos números.

De manera aceptable, M5 tiene un promedio de 0.054, desviación moderada, last < first (-57 %), mejora a lo largo del entrenamiento, tiene pocos picos relevantes, converge razonablemente

bien pero es mas ruidoso que los anteriores; M7 tiene una subida inicial importante, con un promedio de 0.058, se observa que el primer registro (epoch 5) es 0.48245 (muy alto) y luego baja mucho (last = 0.06113), sugiriendo que el entrenamiento arrancó inestable pero se estabilizó luego, comportamiento típico por usar una tasa alta de aprendizaje o cambios drásticos de pesos al inicio;

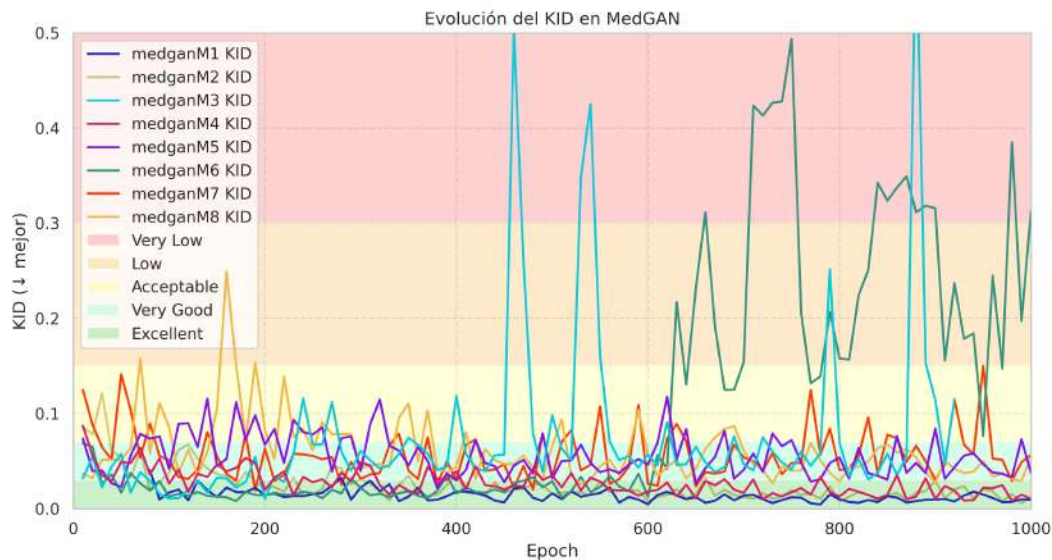


Figura 6-15: Evolución temporal del KID (en función de época) para todas las configuraciones de hiperparámetros en MedGAN.

El M6 tiene un promedio de 0.1596, indicando alta variabilidad, tendiendo a empeorar notablemente desde el inicio al final, mayor número de picos detectados y valores máximos muy altos (0.634), indicador fuerte de inestabilidad, tiene una peor configuración del conjunto; en contraste, el modelo M3 evidenció un descenso inicial en KID hasta alcanzar un mínimo alrededor de la mitad del entrenamiento, seguido de un aumento leve en las últimas épocas, este patrón en forma de “U” indica un posible inicio de sobreentrenamiento o desequilibrio entre $\mathcal{G}$ y $\mathcal{D}$, donde el modelo deja de mejorar la distribución global de las imágenes y empieza a memorizar características particulares del conjunto de entrenamiento.

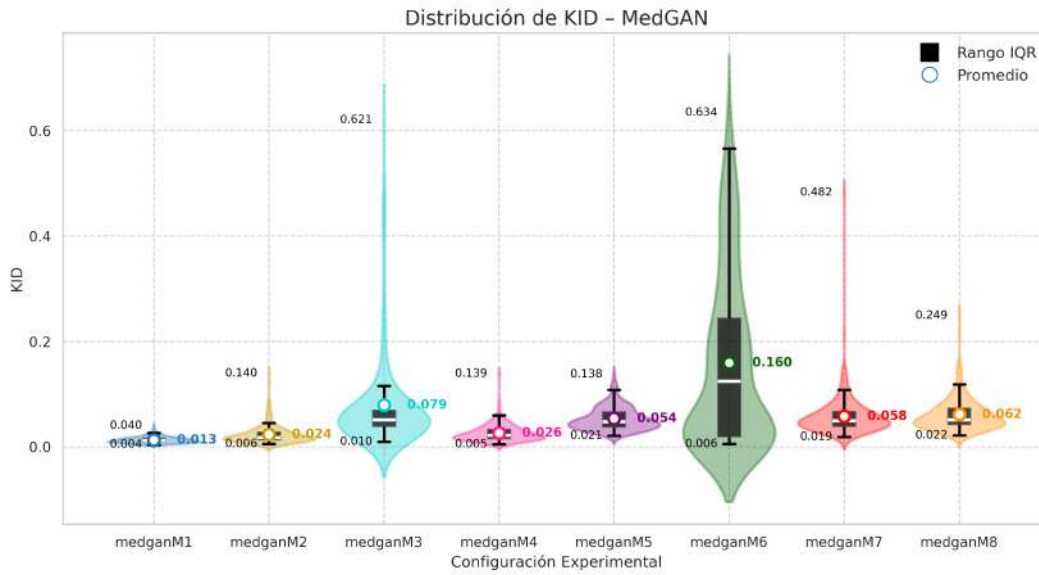


Figura 6-16: Diagrama de violín con boxplot superpuesto representando la distribución del KID para cada configuración experimental de MedGAN.

Tabla 6-12: Mínimos, máximos y promedios de KID registrados para los diferentes experimentos de MedGAN, de mejor a peor.

Experimento	KID_min ↑	época	KID_max	época	KID_promedio
M1	0.0044	780	0.0405	80	0.0135
M4	0.0051	925	0.1388	5	0.0236
M6	0.0056	145	0.6342	1120	0.1596
M2	0.0058	990	0.1397	5	0.0237
M3	0.0097	125	0.6205	880	0.0793
M7	0.0186	190	0.4824	5	0.0582
M5	0.0214	380	0.1378	195	0.4967
M8	0.0218	765	0.2492	160	0.0616

Según los resultados de JSD, M2 demostró ser la configuración mas efectiva, todas las configuraciones en este modelo oscilan en un rango cercano a 0.5 — lo que dificulta distinguir visualmente diferencias significativas — impidiendo que la Figura 6-17 sea distinguible, la superioridad de M2 se puede apreciar con mayor claridad en la Figura 6-18 donde se destaca como la más estable y efectiva, su rendimiento sobresale no sólo por presentar uno de los promedios más bajos (0.495), sino también por exhibir la distribución más compacta, esta compactación indica que, más allá de valores puntuales, el entrenamiento mantuvo una dinámica consistente, evitando fluctuaciones amplias que suelen relacionarse con inestabilidad en la generación, la estrechez de su distribución sugiere que el modelo logró un equilibrio sólido favoreciendo un proceso de aprendizaje controlado.

Por el contrario, las configuraciones M3 y M6 muestran comportamientos menos favorables,

aunque M3 presenta el JSD mínimo más bajo de todos los experimentos (0.452), este valor corresponde a un instante aislado que no refleja la tendencia general de su entrenamiento, su distribución más dispersa y un JSD máximo relativamente alto (0.696) demuestran períodos prolongados de inestabilidad, lo que dificulta mantener una dinámica de aprendizaje coherente, en cuanto a M6 comparte un JSD mínimo comparable al de M2 (0.457), pero su promedio más alto (0.517), junto con picos elevados de JSD, evidencia un proceso de entrenamiento irregular y poco saludable para la generación de imágenes de calidad.

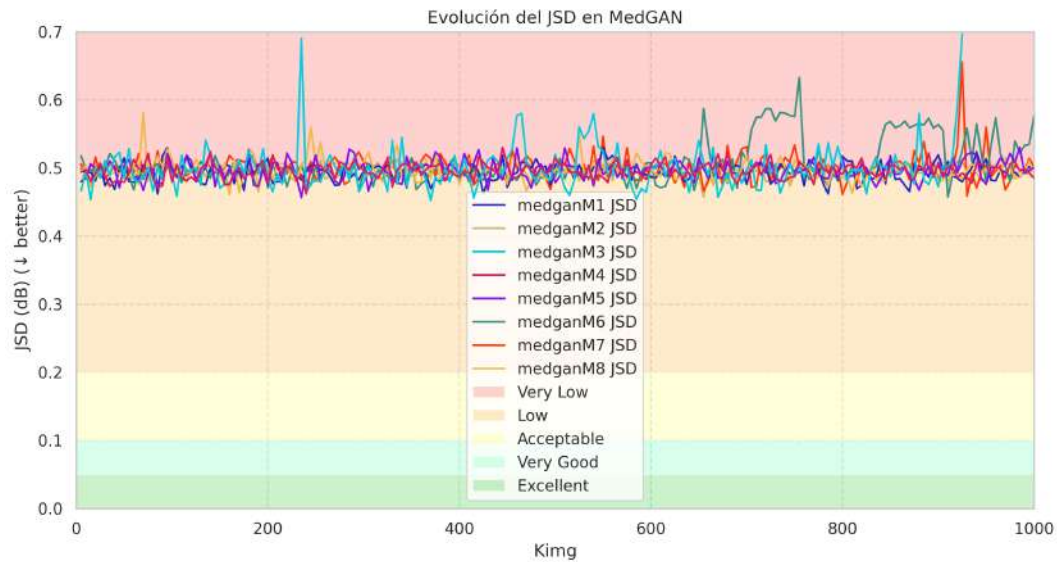


Figura 6-17: Evolución temporal del JSD (en función de época) para todas las configuraciones de hiperparámetros en MedGAN

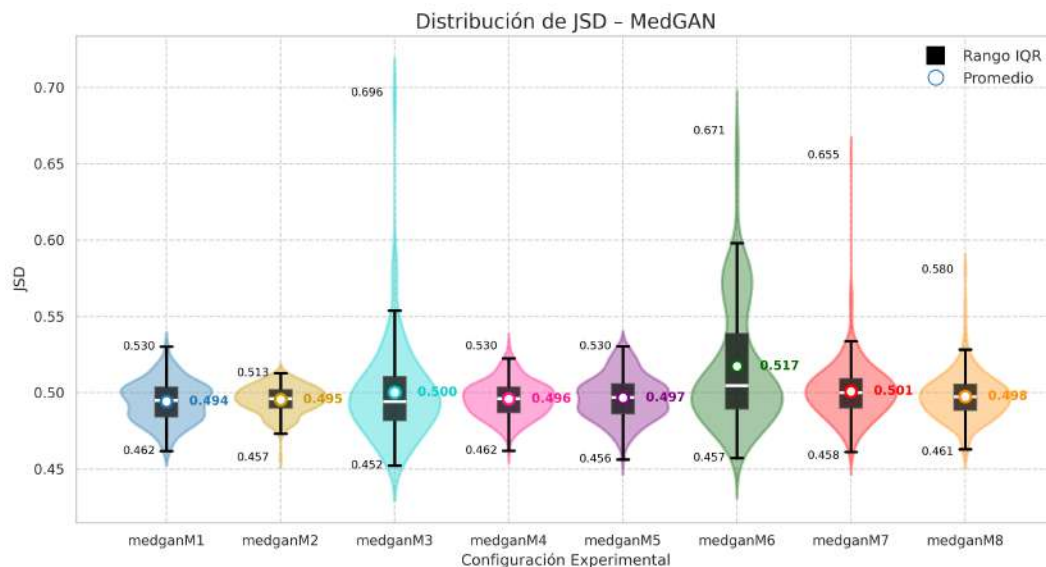


Figura 6-18: Diagrama de violín con boxplot superpuesto representando la distribución del JSD para cada configuración experimental de MedGAN.

El resto de las configuraciones para JSD mantienen comportamientos moderadamente consistentes, con distribuciones compactas que indican estabilidad razonable, aunque no superan a M2 en cuanto a la relación entre compactación y promedio, tampoco presentan los niveles de variabilidad extrema observados en M3 y M6, sugiriendo que si bien estas configuraciones son funcionales, no logran optimizar el equilibrio del entrenamiento tan eficientemente como lo hace M2.

Tabla 6-13: Mínimos, máximos y promedios de JSD registrados para los diferentes experimentos de MedGAN, de mejor a peor.

Experimento	JSD_min ↑	época	JSD_max	época	JSD_promedio
M3	0.452	370	0.696	925	0.500
M5	0.456	235	0.530	985	0.497
M2	0.457	655	0.513	1080	0.495
M6	0.457	910	0.671	1190	0.517
M7	0.458	930	0.655	925	0.0500
M8	0.461	160	0.580	190	0.497
M1	0.462	425	0.530	550	0.494
M4	0.462	240	0.530	445	0.496

Para la métrica SSIM se observa un comportamiento particular, las métricas con el SSIM mas alto fueron M1, M3, M7 y M8, sin embargo, las configuraciones M1, M2, M4 y M5 mostraron el desempeño más sólido y consistente en términos generales, esto puede apreciarse en las distribuciones y promedios representados en la Figura **6-20**, mantienen rangos acotados entre sus valores mínimos y máximos, lo que indicaría una evolución estable durante el entrenamiento, la ausencia de valores extremadamente bajos en estas configuraciones sugiere que el modelo no sufrió colapsos severos y fue capaz de mantener una representación estructural relativamente coherente en la mayor parte del proceso.

Tabla 6-14: Mínimos, máximos y promedios de SSIM registrados para los diferentes experimentos de MedGAN, de mejor a peor.

Experimento	SSIM_max ↑	época	SSIM_min	época	SSIM_promedio
M3	0.421	150	0.015	465	0.355
M8	0.408	190	0.298	70	0.372
M7	0.403	1238	0.089	5	0.371
M1	0.402	970	0.349	35	0.375
M5	0.400	155	0.354	460	0.375
M4	0.396	595	0.353	445	0.374
M2	0.392	840	0.355	110	0.374
M6	0.391	60	0.014	740	0.269

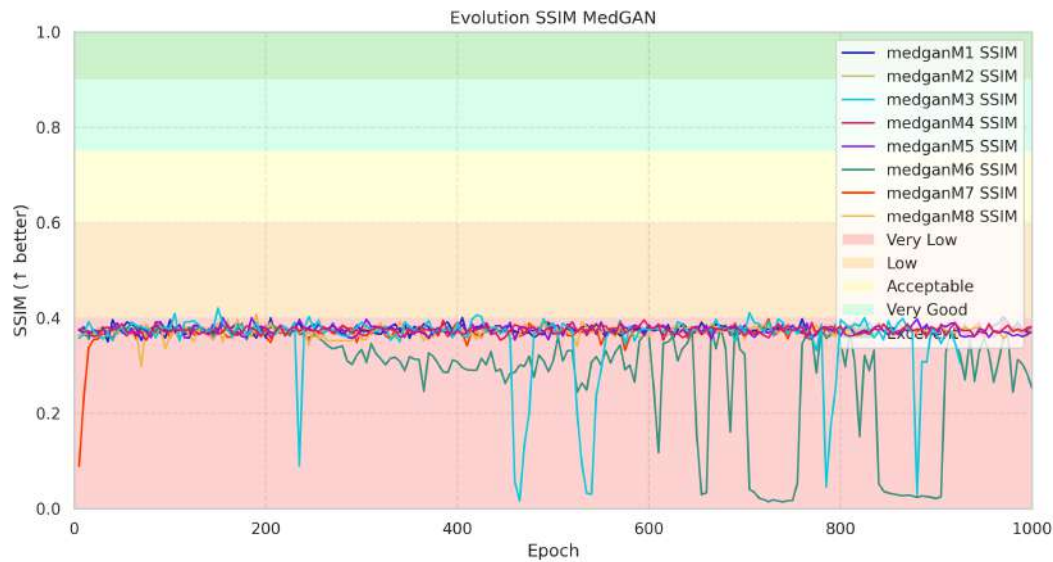


Figura 6-19: Evolución temporal del SSIM (en función de época) para todas las configuraciones de hiperparámetros en MedGAN.

Aun que M3 logro el SSIM mas alto de SSIM, al ver la Figura 6-19 se aprecia que presenta el segundo comportamiento mas errático e inestable, y esta inestabilidad también se evidencia en la Figura 6-20 donde evidencia la segunda distribución mas dispersa, indicando que tuvo un momento con la mayor similitud de imágenes pero no es una configuración apropiada para el entrenamiento, mientras que M6 mostró ser la arquitectura mas inestable, con el promedio mas bajo (0.269) y una distribución altamente dispersa.

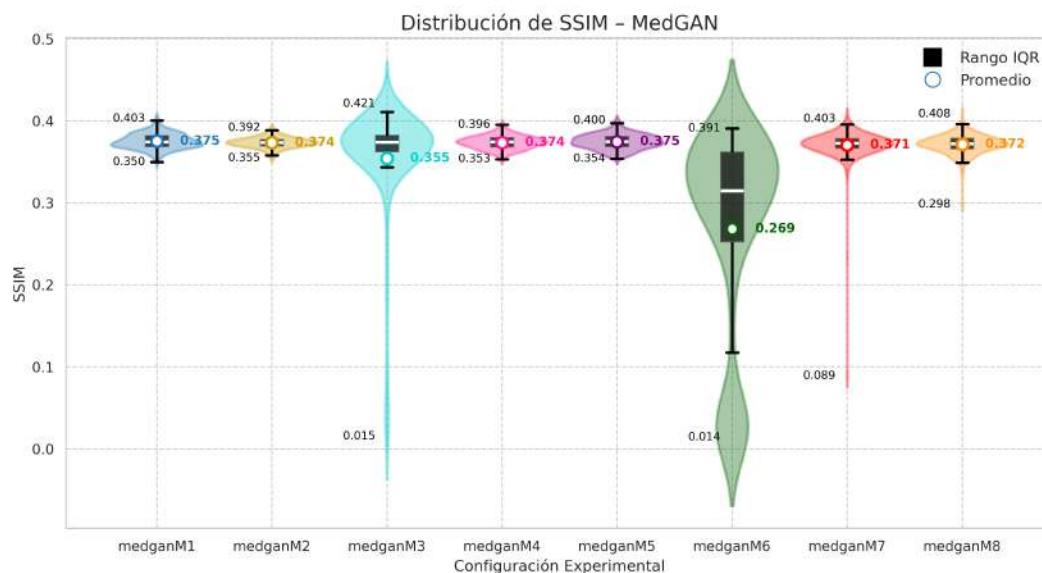


Figura 6-20: Diagrama de violín con boxplot superpuesto representando la distribución del SSIM para cada configuración experimental de MedGAN.

En el caso de la métrica PSNR, se evidencia que no aporta información relevante, para destacar configuraciones por encima de otras, como se observa en las Figuras 6-21 y 6-22, las distribuciones de todas las variantes son muy similares, con máximos y mínimos cercanos entre si, al igual que en SSIM las configuraciones M3 y M6 presentan el mayor grado de dispersión y los promedios mas altos, sugiriendo la poca estabilidad de estos modelos, sin embargo, esta metrica resulta poco coherente para el caso de M3, con su comportamiento inestable obtiene el mayor PSNR (14,55).

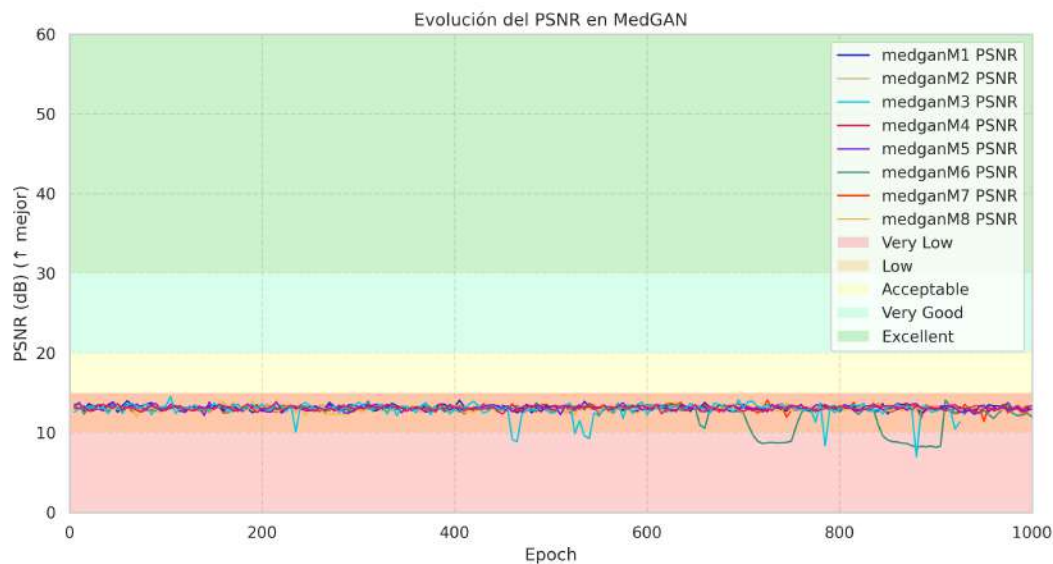


Figura 6-21: Evolución temporal del PSNR (en función de época) para todas las configuraciones de hiperparámetros en MedGAN.

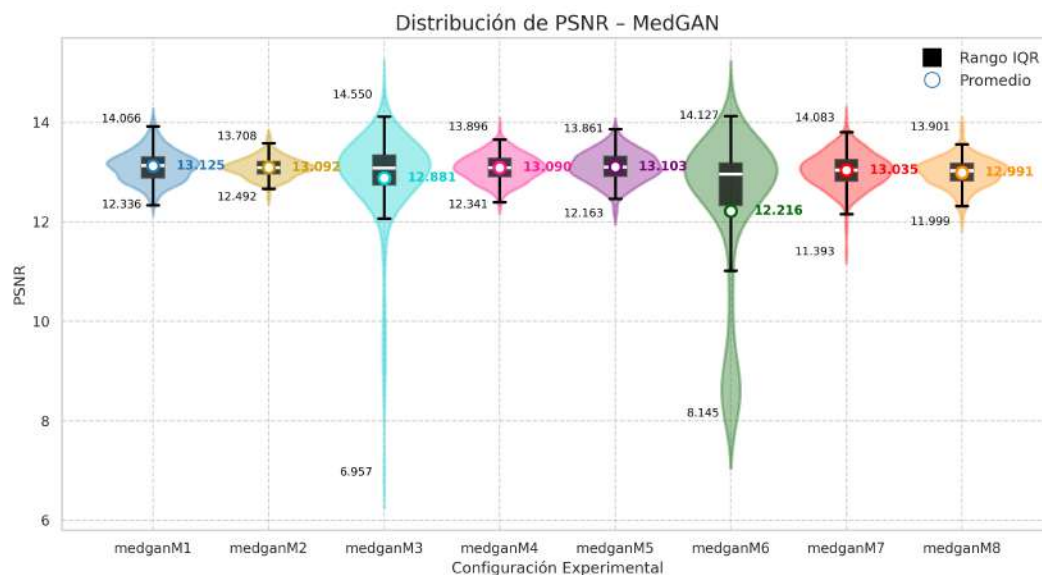
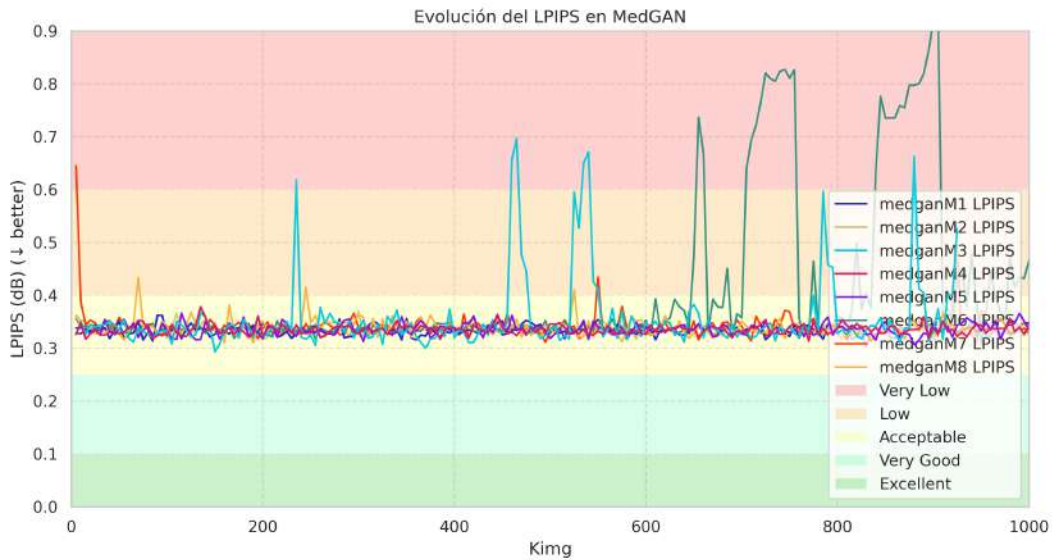


Figura 6-22: Diagrama de violín con boxplot superpuesto representando la distribución del PSNR para cada configuración experimental de MedGAN.

Tabla 6-15: Mínimos, máximos y promedios de PSNR registrados para los diferentes experimentos de MedGAN, de mejor a peor.

Experimento	PSNR_max ↑	época	PSNR_min	época	PSNR_promedio
M3	14.550	105	6.957	880	12.881
M6	14.127	1220	8.145	900	12.216
M7	14.083	725	11.392	950	13.034
M1	14.066	405	12.366	850	13.124
M8	13.901	160	11.998	70	12.991
M4	13.896	215	12.341	450	13.089
M5	13.861	45	12.162	140	13.102
M2	13.708	840	12.491	145	13.092

Finalmente para la métrica LPIPS se evidencia la misma particularidad de PSNR, teniendo la mayoría de las variantes de MedGAN presentan valores mínimos cercanos a 0.308, máximos al rededor de 0.382 y promedios aproximados de 0.336, por lo que esta métrica y con este modelo no permite identificar diferencias significativas entre los modelos que permita determinar los que mas destacan, no obstante, es posible distinguir que las configuraciones M3 y M6 son las mas deficientes, teniendo distribuciones dispersas como se ve en la Figura 6-24, así como una marcada inestabilidad evidenciado en la Figura 6-23.

**Figura 6-23:** Evolución temporal del LPIPS (en función de época) para todas las configuraciones de hiperparámetros en MedGAN.

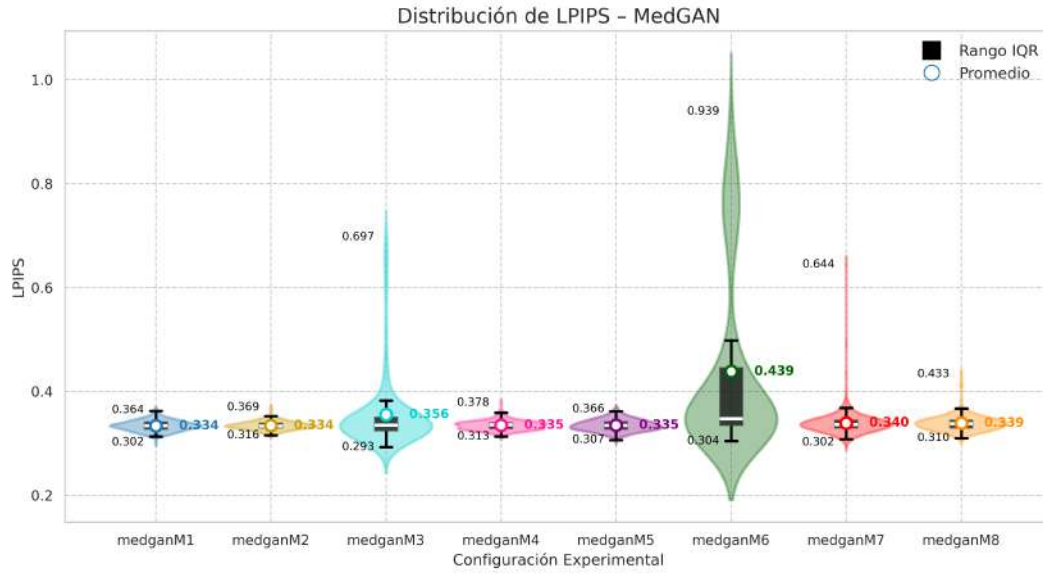


Figura 6-24: Diagrama de violín con boxplot superpuesto representando la distribución del LPIPS para cada configuración experimental de MedGAN.

Tabla 6-16: Mínimos, máximos y promedios de LPIPS registrados para los diferentes experimentos de MedGAN, de mejor a peor.

Experimento	LPIPS_min ↑	época	LPIPS_max	época	LPIPS_promedio
M3	0.293	150	0.697	465	0.356
M1	0.302	830	0.364	445	0.334
M6	0.304	950	0.939	905	0.439
M5	0.307	880	0.366	115	0.335
M4	0.313	770	0.378	135	0.335
M2	0.316	200	0.369	300	0.334
M7	0.458	930	0.655	925	0.0500
M8	0.461	160	0.580	190	0.497

6.1.3 Resultados en configuraciones de WGAN

Experimento W1 → 1008 Épocas batchSize=64, nz=100, ngf=64, ndf=64, lrD=5e-5, lrG=5e-5, Diters=5, opt=RMSprop, BN=True, mlpG=False		Experimento W2 → 1098 Épocas batchSize=128, nz=128, ngf=128, ndf=128, lrD=5e-5, lrG=5e-5, Diters=5, opt=RMSprop, BN=True, mlpG=False	
Experimento W3 → 925 Épocas batchSize=64, nz=100, ngf=64, ndf=64, lrD=5e-5, lrG=5e-5, Diters=5, opt=RMSprop, BN=False, mlpG=False		Experimento W4 → 1020 Épocas batchSize=64, nz=256, ngf=64, ndf=64, lrD=1e-4, lrG=1e-4, Diters=5, opt=Adam, BN=False, mlpG=True	
Experimento W5 → 1039 Épocas batchSize=64, nz=100, ngf=64, ndf=128, lrD=1e-4, lrG=5e-5, Diters=10, opt=RMSprop, BN=True, mlpG=False		Experimento W6 → 1284 Épocas batchSize=64, nz=128, ngf=128, ndf=64, lrD=5e-5, lrG=1e-4, Diters=3, noBN=True, opt=Adam, BN=True, mlpG=False	
Experimento W7 → 1493 Épocas batchSize=64, nz=100, ngf=64, ndf=64, lrD=1e-4, lrG=1e-4, Diters=5, opt=Adam, BN=True, mlpG=False		Experimento W8 → 1065 Épocas batchSize=16, nz=100, ngf=32, ndf=32, lrD=5e-5, lrG=5e-5, Diters=5, opt=RMSprop, BN=True, mlpG=False	

Ta6la 6-18: Comparación de las pérdidas del Generador y Discriminador para las diferentes configuraciones de WGAN

El tiempo de entrenamiento de las diferentes configuraciones propuestas para WGAN presento variaciones significativas, aunque, en general, este modelo resulto mas rápido que MedGAN, la calidad de las muestras generadas fue drásticamente menor, al igual que en MedGAN el objetivo era completar 1000 épocas de entrenamiento por configuración,; las configuraciones mas rápidas fueron W7 y W8 alcanzando el limite previsto en el transcurso de 6 días, les siguieron W1, W2 y W3 que completaron su entrenamiento en 7 días, posteriormente W6 con 13 días, finalmente, las configuraciones W2 y W5 excedieron los 15 días de entrenamiento y

aun así no lo lograron alcanzar el objetivo; W5 solo llego a 665 épocas, mientras que W2 llego a las 560 épocas.

En ambos casos, el entrenamiento fue detenido debido a un colapso evidente del modelo, acompañado de una generación de muestras altamente distorsionadas y carentes de coherencia visual, dado el deterioro progresivo y la ausencia de señales de recuperación, continuar entrenando estas configuraciones representaba un uso ineficiente del tiempo computacional.

Para WGAN sucede una particularidad en sus pérdidas que no se había visto hasta el momento en las otras arquitecturas, siendo el completo colapso de varias configuraciones y como estas se pueden apreciar en las pérdidas de $\mathcal{G}$ y $\mathcal{D}$. En las configuraciones W1, W3, W6 y W8, se observa un entrenamiento equilibrado entre $\mathcal{G}$ y $\mathcal{D}$, distinguiéndose un entrenamiento adversarial típico; para W1 y W3 las pérdidas de $\mathcal{G}$ se mantienen en promedios relativamente bajos (≈ 1.12) y $\mathcal{D}$ presenta valores negativos moderados (≈ -2.5), no aparecen señales de colapso de modo ni explosiones de gradientes, lo que indica que la dinámica de aprendizaje está fluyendo correctamente y que ambas redes están ajustándose mutuamente sin perder capacidad de representación.

Las configuraciones W6 y W8 también presentan comportamientos estables y balanceados. Para W8, $\mathcal{G}$ oscila entre -0.63 y 0.68, con un promedio de 0.26 y desviación estándar de 0.21, lo que refleja oscilaciones suaves alrededor de valores cercanos a cero. $\mathcal{D}$ muestra pérdidas entre -1.46 y 0.10, con un promedio de -0.55 y desviación estándar de 0.28, lo que indica que ninguna de las dos redes domina completamente a la otra. Esta cercanía de magnitudes es un fuerte indicador de equilibrio adversarial.

En contraste, el modelo W2 muestra una tendencia en la que $\mathcal{D}$ se vuelve más fuerte, reflejada en un descenso más pronunciado de su pérdida (≈ -5.63). Esto provoca que $\mathcal{G}$ deba incrementar su esfuerzo, lo cual se manifiesta en un $\mathcal{G}_{\text{loss}}$ promedio mayor (≈ 2.43), existe un desbalance evidente, $\mathcal{D}$ está aprendiendo más rápido y con mayor precisión que $\mathcal{G}$, lo cual puede afectar la diversidad de las muestras generadas e incluso conducir, en etapas prolongadas, hacia un posible estancamiento del aprendizaje de $\mathcal{G}$, teniendo coherencia en su gran cantidad de tiempo consumida.

Esta tendencia se amplifica en el modelo W4, donde el desbalance se vuelve más marcado, $\mathcal{D}_{\text{loss}}$ alcanza valores muy negativos (≈ -11.38), indicando que $\mathcal{D}$ distingue con demasiada facilidad entre muestras reales y generadas, como consecuencia, $\mathcal{G}$ incrementa notablemente su pérdida promedio (≈ 4.97), lo que sugiere un entrenamiento forzado, con dificultades para aproximarse a la distribución real, existiendo un mayor riesgo de pérdida de estabilidad, reducción en la calidad de los ejemplos generados.

En W5 se observa un incremento drástico en la magnitud de las pérdidas, $\mathcal{G}$ alcanza un

máximo de 80.18, con un promedio elevado de 62.86 y una desviación estándar de 3.02, lo que indica una mayor inestabilidad, incluso se registra un valor mínimo negativo (-7.10), lo que sugiere fluctuaciones importantes en la señal de entrenamiento de $\mathcal{G}$, $\mathcal{D}$ presenta pérdidas extremadamente grandes en magnitud, con un mínimo de -159.68 y un promedio de -143.28, lo que evidencia un $\mathcal{D}$ excesivamente dominante, esta gran separación entre las magnitudes de ambos modelos indica un desequilibrio fuerte en la dinámica adversarial, donde $\mathcal{G}$ no logra alcanzar un nivel competitivo.

Para finalizar W7 muestra un $\mathcal{G}$ con pérdidas entre 122.55 y 161.71, con un promedio de 130.25 y desviación estándar de solo 1.60, lo que refleja un entrenamiento extremadamente inestable con un régimen de alta pérdida, esto indica que $\mathcal{G}$ no logra mejorar sustancialmente su capacidad de engaño, manteniéndose estancado en un rango elevado, $\mathcal{D}$ varía entre -318.88 y -270.63, con un promedio de -290.12 confirmando un dominio absoluto del discriminador, esta relación de fuerzas es un claro indicio de un entrenamiento adversarial desbalanceado, donde $\mathcal{G}$ recibe gradientes pobres.

Teniendo en cuenta este análisis detallado necesario, es claro determinar por las pérdidas que las configuraciones W2, W4, W5 y W7 serán completamente incapaces de generar datos útiles, son configuraciones completamente estancadas, mientras que para W1, W3, W6 y W8, presentan modelos estables donde existe un proceso de competencia, de manera que se puede afirmar que fueron capaces de crear imágenes coherentes y cercanas a un ecocardiograma, para evaluar que tan útiles fueron las imágenes generadas es necesario recurrir como antes a las métricas de evaluación.

Al hacer una comparación general para los valores de FID de las configuraciones de WGAN, se identifica W3 como la mejor configuración ya que combina el FID más bajo entre todas las configuraciones, un promedio significativamente mejor y una estabilidad aceptable como se observa en las Figuras 6-25 y 6-26, aunque presenta momentos de inestabilidad, seguido de W1 con un FID mínimo alcanzado de 75.76, lo cual indica que esta configuración consiguió acercarse razonablemente a la distribución real, sin embargo, el valor máximo (726.05) es extremadamente elevado aunque al inicio y, junto con una desviación estándar muy alta (105.81), revela una fuerte inestabilidad. El promedio global (135.63) sugiere que, aunque en ciertos momentos el modelo generó imágenes de alta calidad, el comportamiento general fue oscilante.

La configuración W6 presentó un desempeño global, obteniendo un valor mínimo de FID (68.30) entre todos los experimentos, así como la media más baja (88.65), acompañada de una desviación estándar moderada (19.77), esto sugiere no solo una calidad superior de las imágenes generadas, sino también un entrenamiento más estable y consistente. De manera similar; en contraste, las configuraciones W4, W5 y W7 exhibieron valores de FID considerablemente más altos con, W5 que registró la media más elevada (345.37), W7 también mostró una media alta (258.12) con una desviación estándar significativa (60.54), indicando

fluctuaciones pronunciadas, estos resultados sugieren que dichas configuraciones generaron imágenes con menor fidelidad perceptual, probablemente debido a parámetros subóptimos en el espacio de aprendizaje, pesos del discriminador, o estabilidad del critic loss.

En W2 la desviación estándar de FID (80.17) refleja que esta configuración se mantuvo en un rango de calidad de imágenes significativamente peor que las otras configuraciones, aunque menos inestable que W1, W2 nunca logró valores bajos de FID, indicando un entrenamiento deficiente, se observa su mínimo FID obtenido de 173.25, este valor se encuentra muy por encima de las configuraciones con mejor desempeño — como W6 y W3, cuyos mínimos están alrededor de 68 – 70 —, lo que confirma que W2 nunca logró acercarse a una distribución visualmente plausible de los datos reales; su máximo FID de 454.92 también es uno de los más altos, lo que sugiere que el modelo no solo generaba imágenes de baja calidad, sino que además atravesaba fases donde el proceso de entrenamiento se alejaba aún más de la distribución objetivo.

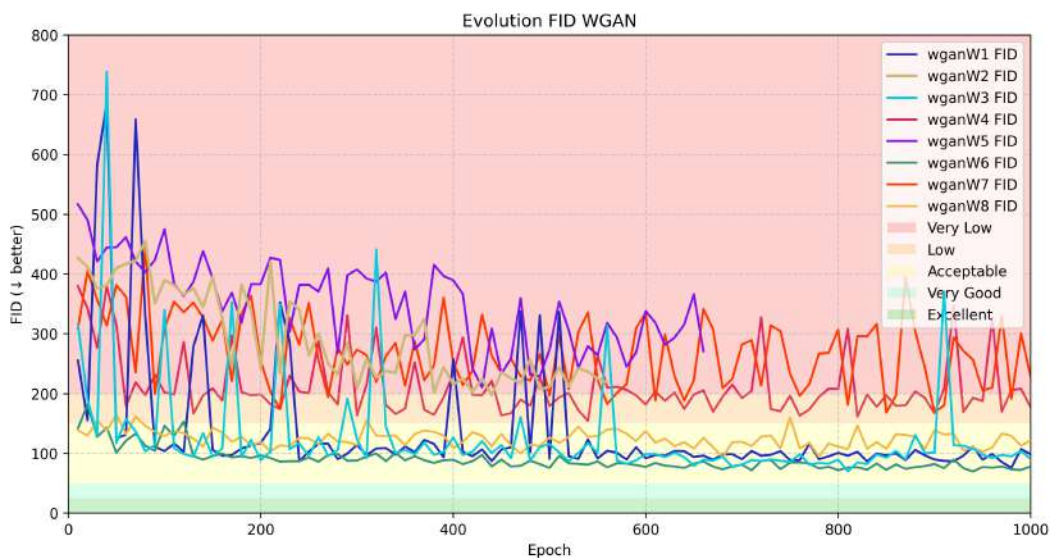


Figura 6-25: Evolución temporal del FID (en función de época) para todas las configuraciones de hiperparámetros en WGAN.

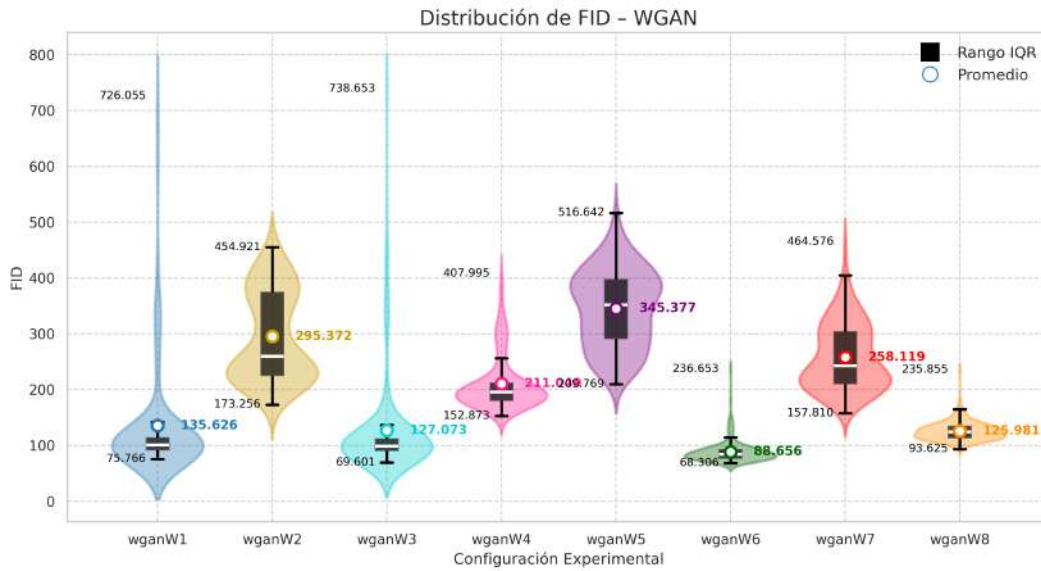


Figura 6-26: Diagrama de violín con boxplot superpuesto representando la distribución del FID para cada configuración experimental de WGAN.

Tabla 6-19: Mínimos, máximos y promedios de FID registrados para los diferentes experimentos de WGAN, de mejor a peor.

Experimento	FID_min ↑	época	FID_max	época	FID_promedio
W6	68.31	1005	236.65	5	88.66
W3	69.60	810	738.65	40	127.07
W1	75.77	980	727.05	45	135.63
W8	93.62	680	235.85	5	125.98
W4	152.87	305	407.99	5	211.04
W7	157.80	575	464.58	5	258.12
W2	173.25	395	454.92	80	295.37
W5	209.77	430	516.64	10	345.38

Para el KID en WGAN, W3 presenta el valor mínimo de KID de 0.02236, lo que demuestra su capacidad para producir imágenes buenas, aunque presenta valores máximos altos (1.4054), el promedio (0.0880) es el más bajo de todos los modelos evaluados sumado a su distribución como se ve en la Figura 6-28, en conjunto, W1 se posiciona como la segunda mejor con un FID mínimo de 0.025, que indica momentos en los que el modelo logró generar imágenes con alta similitud a la distribución real, sin embargo, el valor máximo alcanza 1.2816, lo cual es muy elevado y sugiere episodios de severa divergencia en el entrenamiento, el promedio (0.1058) es relativamente bajo en comparación, lo que demuestra que, a pesar de su inestabilidad, esta configuración logró mantener periodos prolongados de buen desempeño.

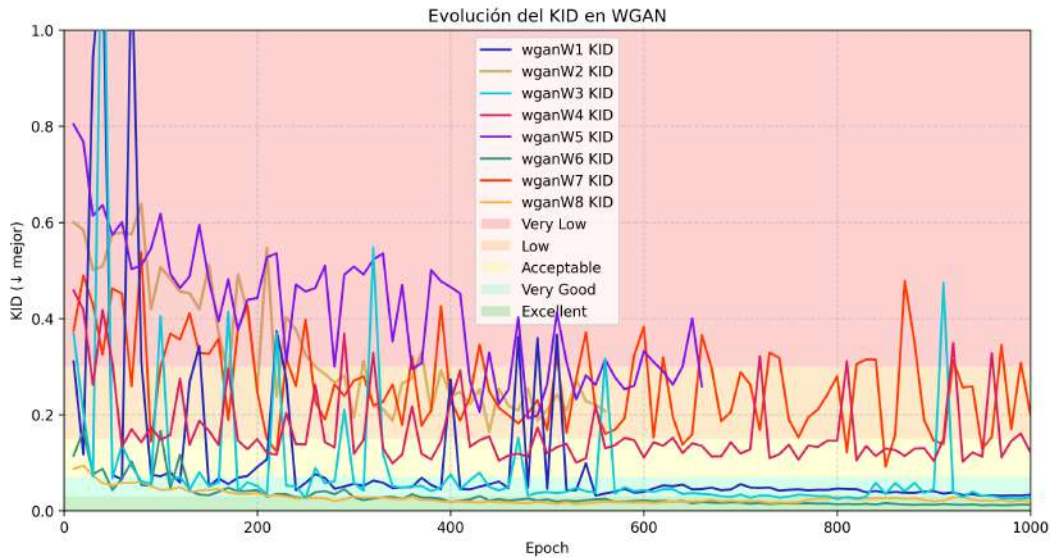


Figura 6-27: Evolución temporal del KID (en función de época) para todas las configuraciones de hiperparámetros en WGAN.

Las configuraciones W6 y W8 destacan por su baja variabilidad y valores promedio reducidos como se ve en la Figura 6-28, también oscilan en valores bajos de KID lo que refleja un flujo de aprendizaje más estable y un equilibrio adecuado entre $\mathcal{G}$ y $\mathcal{D}$, las configuraciones con valores altos y variaciones pronunciadas, aunque W8 posee una media ligeramente inferior, la capacidad de W6 para alcanzar valores mínimos más pequeños y mantener un rendimiento estable sugiere una mayor fidelidad en la reconstrucción de la distribución real y una mayor efectividad global en el proceso de entrenamiento adversarial.

En cuanto a las demás configuraciones, reflejan un rendimiento menos favorable, W5 fue la configuración con peor desempeño, con un valor mínimo de 0.1925, un máximo de 0.8040, y una media significativamente elevada de 0.4050, esta combinación indica un entrenamiento inestable, en el que $\mathcal{G}$ produjo imágenes con baja similitud perceptual respecto a los datos reales y con un comportamiento altamente fluctuante entre iteraciones, W7 presentó una media de 0.2470 y una desviación estándar de 0.0947, lo cual también evidencia un entrenamiento errático y una pobre capacidad del modelo para aproximar la distribución real, y para W4 sus valores se mantienen entre 0.0987 y 0.563, demostrando que, aunque nunca alcanza una calidad sobresaliente, sí mantiene un rendimiento estable y predecible.

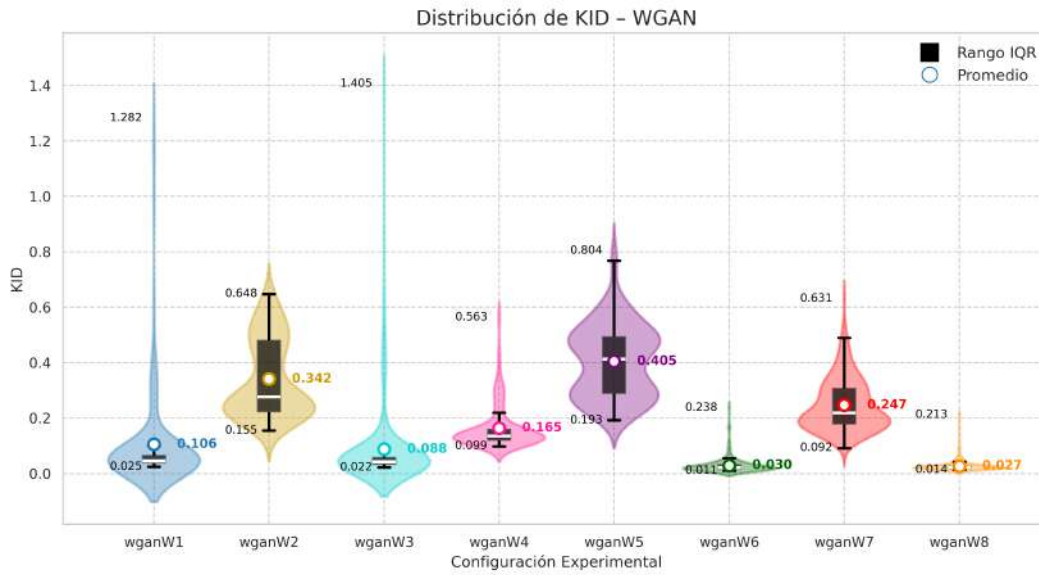


Figura 6-28: Diagrama de violín con boxplot superpuesto representando la distribución del KID para cada configuración experimental de WGAN.

Tabla 6-20: Mínimos, máximos y promedios de KID registrados para los diferentes experimentos de WGAN, de mejor a peor.

Experimento	KID_min ↑	época	KID_max	época	KID_promedio
W6	0.0109	1030	0.2375	5	0.0296
W8	0.0180	530	0.2150	5	0.0275
W3	0.0223	915	1.4055	40	0.0880
W1	0.0246	1070	1.2816	45	0.1058
W7	0.0919	850	0.6311	5	0.2470
W4	0.0987	340	0.5633	15	0.1646
W2	0.1551	395	0.6478	5	0.3415
W5	0.1925	480	0.8040	10	0.4050

La métrica JSD, detalla algo particular en las configuraciones W1, W3 y W5, estas presentan los promedios más bajos, con W5 ligeramente por delante, sin embargo, esta aparente ventaja de W5 debe interpretarse con cautela debido a su baja variabilidad, el modelo no está mejorando, sino que se mantiene estático en una región de menor calidad y con poca capacidad para aproximarse mejor a la distribución real tal y como se aprecia en la Figura 6-30, W3 se establece como la mejor opción en términos generales, coherentemente con su desempeño, tiene el segundo JSD mínimo, un promedio competitivo, mayor dinámica de aprendizaje, aunque no mantiene estabilidad como se observa en la Figura 6-29.

La mejor configuración según JSD es W8, debido a que exhibe los valores más bajos y estables, lo cual implica una aproximación más consistente y precisa a la distribución real de

los datos, este comportamiento es coherente con los valores también competitivos observados en las otras métricas, y confirma que W8 logra un equilibrio adecuado entre diversidad y fidelidad en las muestras generadas, aunque W6 presenta un rendimiento sólido, W8 supera ligeramente tanto en estabilidad como en similitud estadística global.

En contraste, las configuraciones W2, W4, W5 y W7 mostraron valores significativamente mayores de JSD, evidenciando una menor similitud entre sus distribuciones generadas y reales, W4 con un promedio de 0.6084, mientras que W7 alcanzó valores más altos, con un promedio de 0.5948 y un rango que se extiende hasta 0.6941, reflejando mayor divergencia entre distribuciones entre todas las configuraciones, W5, aunque menos extrema, también mantuvo valores altos con una media de 0.5433, lo que indica una menor capacidad para aproximar la distribución real de los datos.

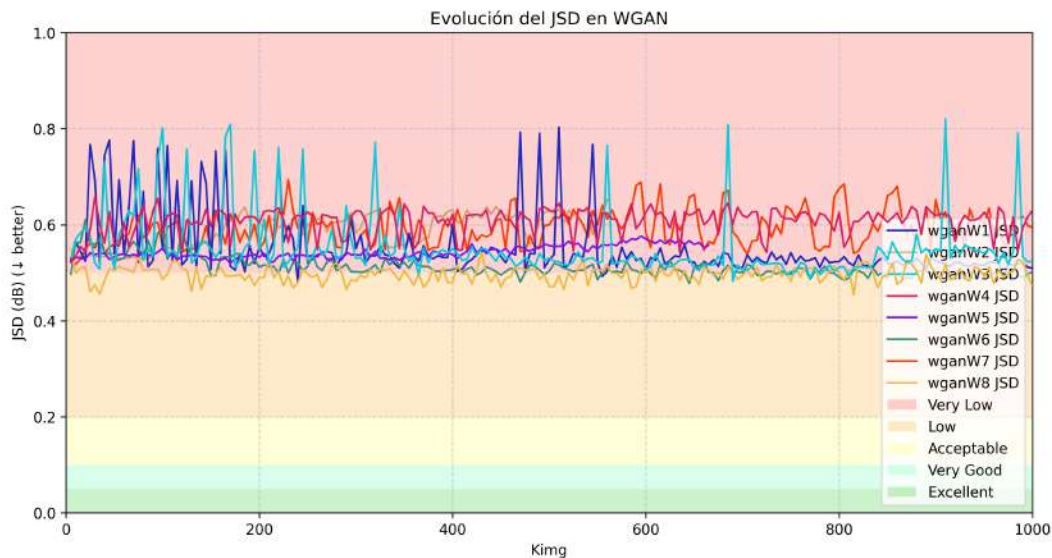


Figura 6-29: Evolución temporal del JSD (en función de época) para todas las configuraciones de hiperparámetros en WGAN

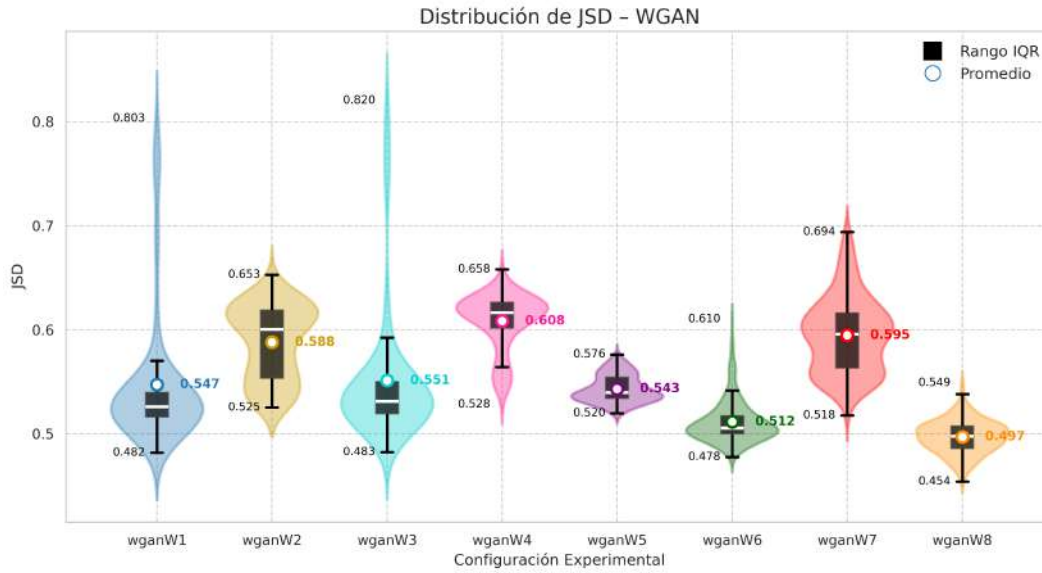


Figura 6-30: Diagrama de violín con boxplot superpuesto representando la distribución del JSD para cada configuración experimental de WGAN.

Tabla 6-21: Mínimos, máximos y promedios de JSD registrados para los diferentes experimentos de WGAN, de mejor a peor.

Experimento	JSD_min ↑	época	JSD_max	época	JSD_promedio
W8	0.4541	815	0.5490	5	0.4971
W6	0.4777	645	0.6102	20	0.5117
W1	0.4818	240	0.8032	510	0.5473
W3	0.4825	690	0.8204	910	0.5514
W7	0.5176	10	0.6941	230	0.5949
W5	0.5198	170	0.5763	595	0.5433
W2	0.5252	5	0.6529	560	0.5881
W4	0.5283	5	0.6582	30	0.6084

De manera particularmente interesante, en este caso SSIM para la arquitectura WGAN muestra una mayor variabilidad de información de utilidad, la mejor configuración que mantiene mas una similitud estructural es W8 debido a que alcanza el promedio más alto, la mayor similitud estructural máxima y la menor variabilidad entre muestras, esto se refuerza en las Figuras 6-31 y 6-32 donde se ve que esta configuración tiene el promedio más alto entre todas las configuraciones (0.371), con una distribución completa denotando estabilidad, le sigue W6 como una segunda opción sólida, pero ligeramente inferior en estabilidad y similitud perceptual, que mostró un promedio de 0.3453, un máximo de 0.3813, y una desviación estándar moderada (0.034).

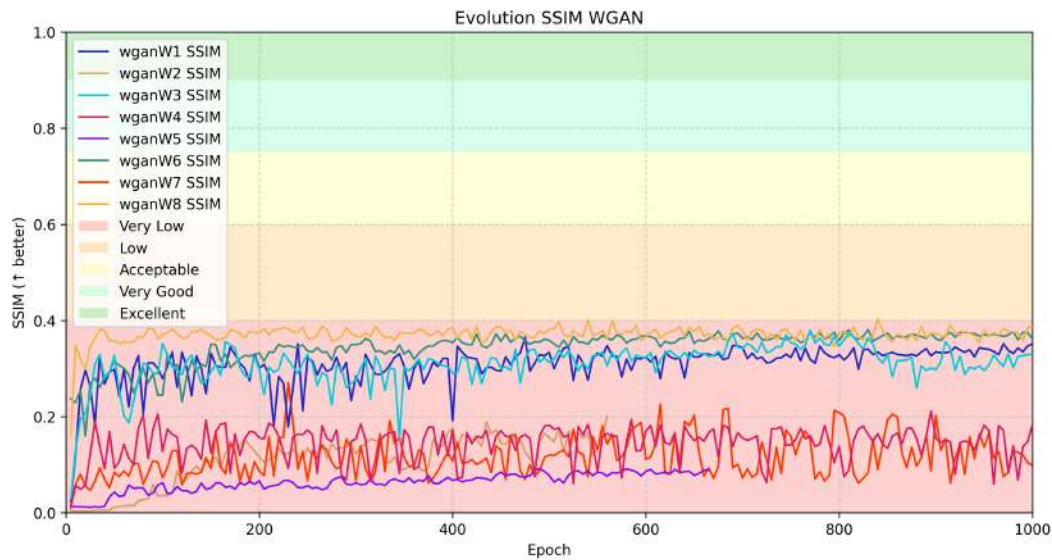


Figura 6-31: Evolución temporal del SSIM (en función de época) para todas las configuraciones de hiperparámetros en WGAN.

Aquellas configuraciones que tuvieron el estancamiento en sus pérdidas (W2, W4, W5 y W7) son las mas castigadas por SSIM, proporcionando un mayor panorama que este tipo de métricas de similitud son fuertes para detectar la mala capacidad de un modelo al momento de generar nuevos datos, siendo W2 la peor configuración segun SSIM, con un mínimo es 0.003 y el máximo apenas llegando a 0.201.

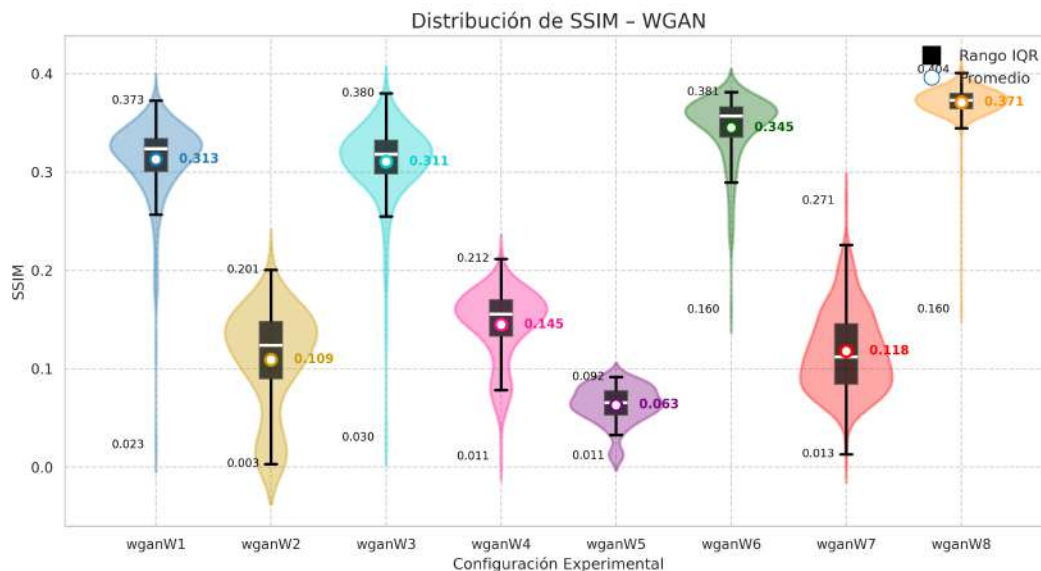
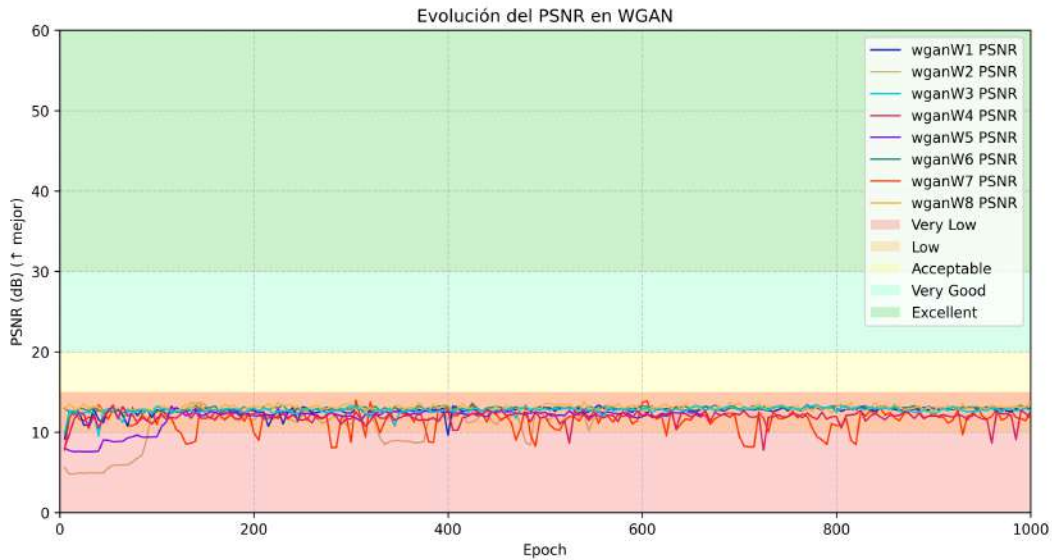


Figura 6-32: Diagrama de violín con boxplot superpuesto representando la distribución del SSIM para cada configuración experimental de WGAN.

Tabla 6-22: Mínimos, máximos y promedios de SSIM registrados para los diferentes experimentos de WGAN, de mejor a peor.

Experimento	SSIM_max ↑	época	SSIM_min	época	SSIM_promedio
W8	0.4037	840	0.1604	5	0.3710
W3	0.3800	770	0.0299	5	0.3107
W6	0.3813	830	0.1601	20	0.3452
W1	0.3727	1075	0.0226	5	0.3131
W7	0.2708	230	0.0132	5	0.1181
W4	0.2118	895	0.1093	5	0.1451
W2	0.2006	560	0.0033	25	0.1094
W5	0.0916	490	0.0113	25	0.0634

Al igual que en las otras arquitecturas GANs, PSNR no es capaz de proporcionar información relevante para determinar una configuración superior, la poca información relevante se centra en W3 como la mejor configuración, siguiendo la Figura 6-34 al tener una distribución tan concentrada, el promedio mas alto, los valores máximos altos y los mínimos elevados indicando que no cae en una mala reconstrucción, lo hacen ver como una configuración fuerte en comparación al resto, sin embargo al detallar la Figura 6-33 se aprecia que W3 tiene varios picos donde decae bruscamente su capacidad de generación de datos confiables.

**Figura 6-33:** Evolución temporal del PSNR (en función de época) para todas las configuraciones de hiperparámetros en WGAN.

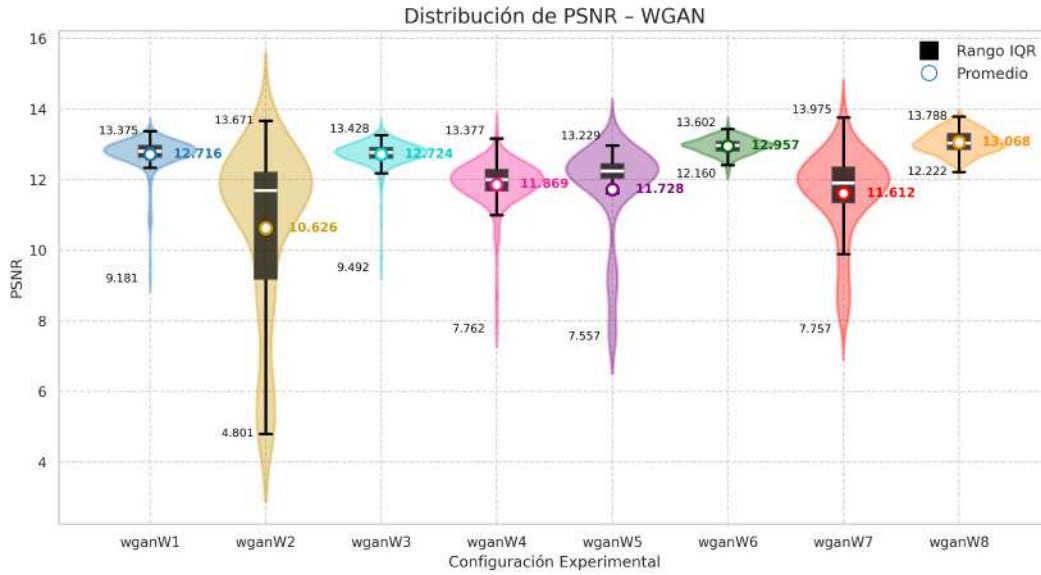


Figura 6-34: Diagrama de violín con boxplot superpuesto representando la distribución del PSNR para cada configuración experimental de WGAN.

Tabla 6-23: Mínimos, máximos y promedios de PSNR registrados para los diferentes experimentos de WGAN, de mejor a peor.

Experimento	PSNR_max ↑	época	PSNR_min	época	PSNR_promedio
W7	13.97	305	7.76	5	11.61
W8	13.78	1015	12.22	465	13.07
W2	13.67	135	4.80	10	10.62
W6	13.60	425	12.16	15	12.96
W3	13.43	780	9.49	40	12.72
W4	13.38	55	7.76	725	11.87
W1	13.37	555	9.18	5	12.72
W5	13.23	115	7.56	15	11.73

En el caso de LPIPS, al igual que con PSNR, no se presenta data o información relevante, pero sí se identifican diferencias suficientes para establecer un orden de desempeño, aunque las curvas temporales de LPIPS no muestran variaciones abruptas Figura 6-35, sí es posible notar que las configuraciones W1 y W3 mantienen trayectorias relativamente estables en los rangos más bajos, en la Figura 6-36, ambas configuraciones presentan distribuciones más compactas y concentradas hacia valores inferiores en comparación con configuraciones como W2, W4 o W5, cuya dispersión es mayor y cuyas medianas se sitúan en regiones menos favorables.

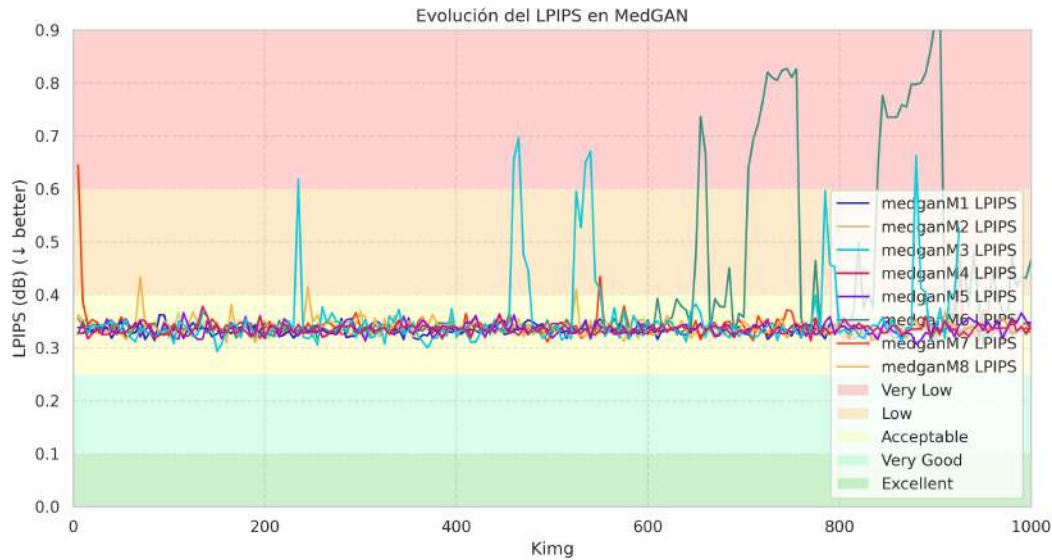


Figura 6-35: Evolución temporal del LPIPS (en función de época) para todas las configuraciones de hiperparámetros en WGAN.

De acuerdo con la Tabla6-24, la elección de W1 como la mejor configuración se fundamenta en su LPIPS promedio (0.4045), que es el más bajo entre todas las configuraciones, donde W3 presenta un desempeño muy cercano (promedio de 0.4097), la ligera ventaja de W1 la posiciona como la configuración con la mejor calidad perceptual global, además, ambas configuraciones alcanzan valores mínimos bastante bajos (0.3267 para W1 y 0.3251 para W3), lo que indica que en ciertos momentos del entrenamiento lograron generar imágenes con una similitud perceptual notablemente alta respecto a los datos reales, estos mínimos son importantes porque evidencian que el modelo, bajo estas configuraciones, fue capaz de alcanzar picos de desempeño perceptual muy competitivos, aunque no los haya sostenido de manera prolongada.

En contraste, configuraciones como W2, W5 y W4 muestran los promedios más altos (0.6571, 0.6304 y 0.5351, respectivamente), lo cual refleja no solo menor calidad perceptual en términos absolutos, sino también una mayor propensión a generar imágenes perceptualmente distantes de los datos reales, esto se ve reforzado por sus valores máximos extremadamente altos (superiores a 1.0 en algunos casos), lo que sugiere que estas configuraciones experimentaron fases donde la calidad perceptual se deterioró de manera importante, además, el rango amplio entre sus mínimos y máximos indica que estas configuraciones fueron menos estables durante el entrenamiento, con valores que oscilan de manera más pronunciada y sin alcanzar los niveles perceptualmente favorables de las configuraciones más competitivas.

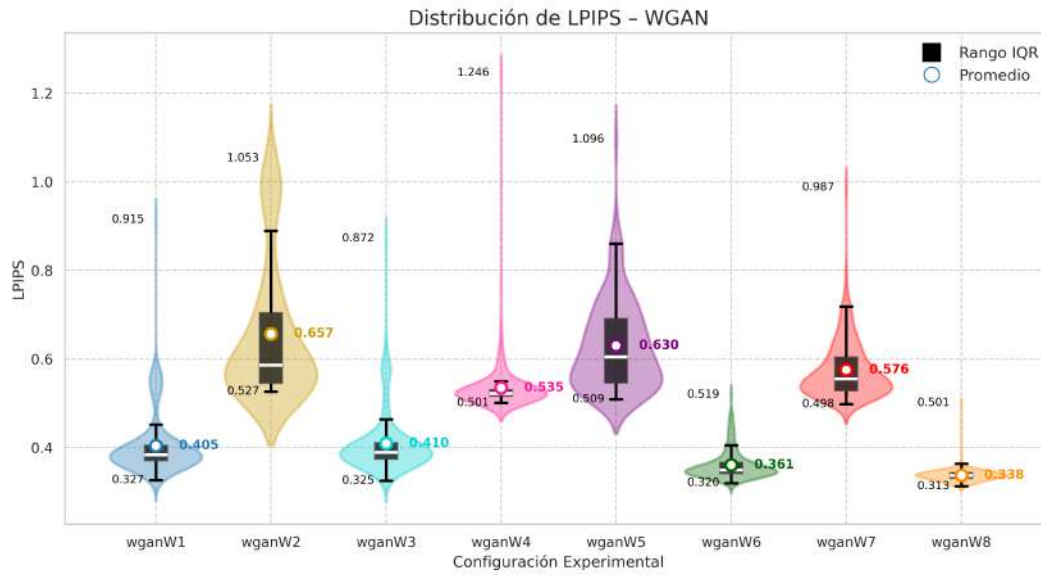


Figura 6-36: Diagrama de violín con boxplot superpuesto representando la distribución del LPIPS para cada configuración experimental de WGAN.

Tabla 6-24: Mínimos, máximos y promedios de LPIPS registrados para los diferentes experimentos de WGAN, de mejor a peor.

Experimento	LPIPS_min ↑	época	LPIPS_max	época	LPIPS_promedio
W7	0.0983	835	0.9871	5	0.5755
W8	0.3129	1095	0.5010	5	0.3376
W6	0.3202	830	0.5193	20	0.3608
W3	0.3251	770	0.8723	5	0.4097
W1	0.3267	1075	0.9148	5	0.4045
W4	0.5009	740	1.2462	5	0.5351
W5	0.5094	490	1.0956	5	0.6304
W2	0.5267	275	1.0529	5	0.6571

6.1.4 Resultados en configuraciones de VQGAN

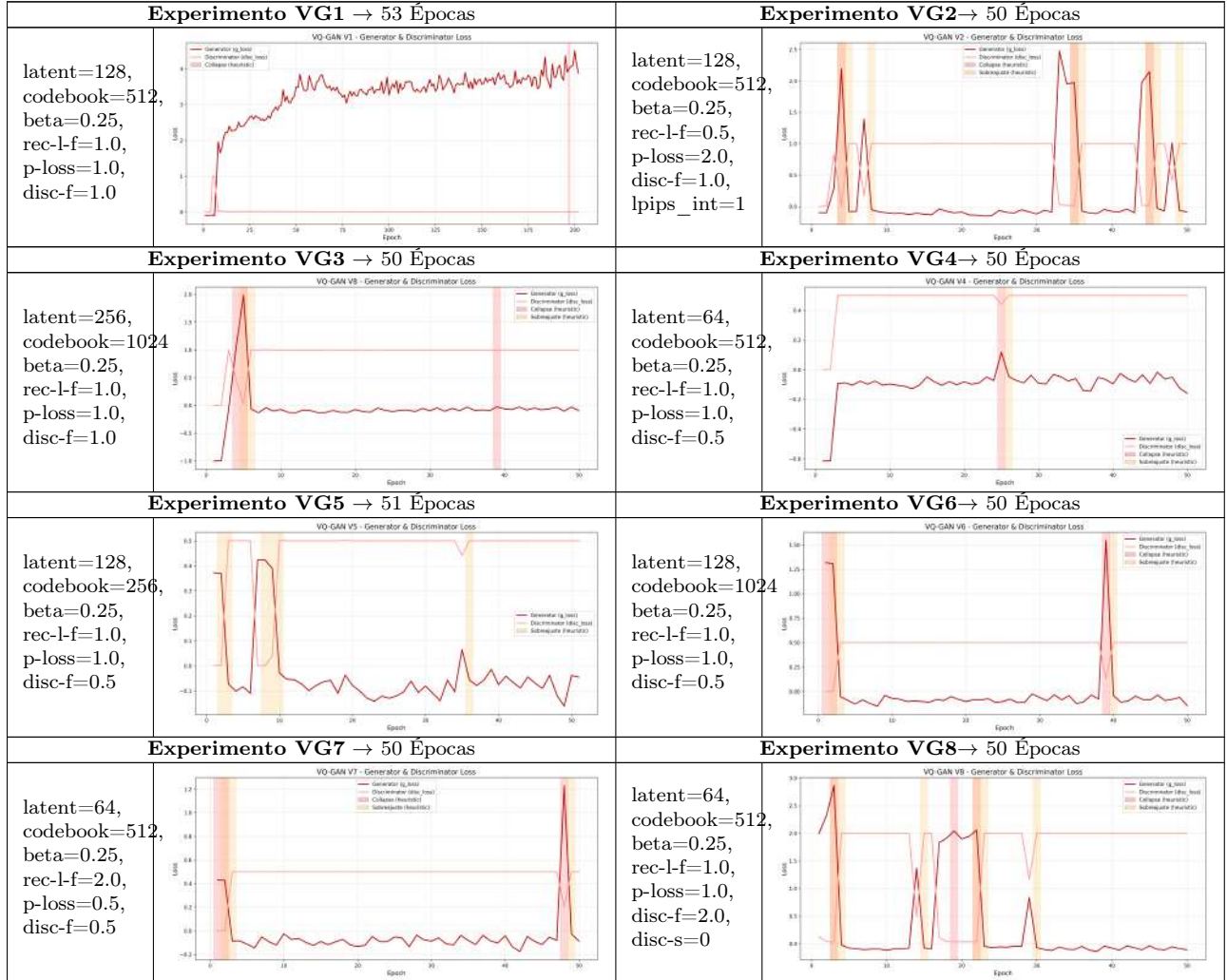


Tabla 6-26: Comparación de las pérdidas del Generador y Discriminador para las diferentes configuraciones de VQGAN

Como análisis superficial, para todas las configuraciones la pérdida del discriminador aumenta con el tiempo, lo cual generalmente indica que el discriminador está perdiendo capacidad de distinguir entre muestras reales y generadas, mostrando una distinción de los anteriores GANs, y es que se puede ver que como tal la arquitectura hace que el generador mejore demasiado rápido y $\mathcal{D}$ se quede atrás, el discriminador se satura con gradientes inestables, cambios en las distribuciones generadas, y momentos que como se ven en la Tabla 6-26 sucede un breve colapso parcial de modos que engaña a $\mathcal{D}$ de forma constante, en VQ5 la pérdida del discriminador no aumenta con la misma tendencia, lo que ya sugiere una configuración más estable.

A nivel general, el análisis de las pérdidas revela que en todas las configuraciones la pérdida del discriminador tiende a aumentar conforme avanza el entrenamiento, este comportamiento suele interpretarse como una pérdida progresiva de la capacidad de $\mathcal{D}$ para diferenciar entre muestras reales y generadas, a diferencia de las arquitecturas GAN anteriores, en VQ-GAN se observa que $\mathcal{G}$ mejora demasiado rápido, dejando al discriminador rezagado y, en ocasiones, saturado debido a gradientes inestables o cambios abruptos en la distribución de salida. Esto, junto con episodios de colapso parcial de modos evidenciados en la Tabla 6-26, provoca que el generador engañe de forma consistente a $\mathcal{D}$, una excepción es la configuración VQ5, en la cual la pérdida de $\mathcal{D}$ no presenta un crecimiento marcado, lo que sugiere un comportamiento más estable en comparación con las demás variantes.

Según FID indica que VQ1 tiene un FID promedio alto y termina también en un valor elevado, aunque en cierto momento bajó, la configuración no logró mantener ese nivel, lo que sugiere episodios de inestabilidad y degradación posterior, no muestra señales claras de colapso completo, pero sí entrenamiento inconsistente; VQ3 es el caso más interesante, aunque el promedio es alto (posiblemente por un mal comienzo), el modelo termina en un FID bajo, probablemente colapsó, porque un FID perfecto casi nunca ocurre en entrenamiento real, VQ4 mantiene los valores más consistentes y estables, apreciables en la Figura 6-38, mientras que VQ5 mantiene un rendimiento mixto, el mínimo nunca es muy bajo, por lo que probablemente nunca logró buena calidad.

Como se observa en la Figura 6-37, en general prácticamente todas las configuraciones oscilan en un rango muy cercano y van mejorando a medida que avanza el entrenamiento, denotando que aun les falta bastante para llegar a un punto de convergencia, en general solo oscilan entre valores aceptables o buenos.

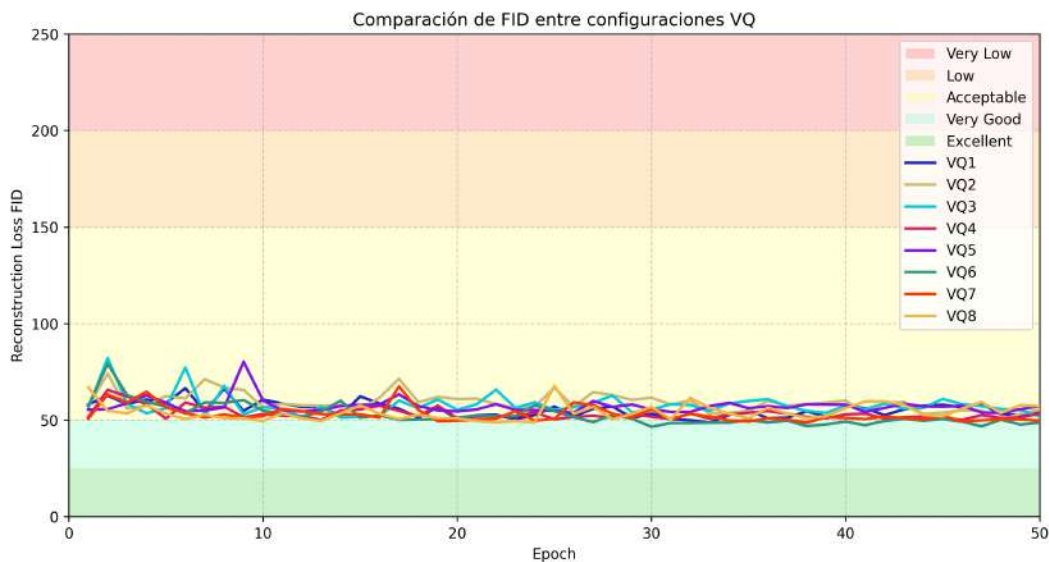


Figura 6-37: Evolución temporal del FID (en función de época) para todas las configuraciones de hiperparámetros en VQ-GAN.

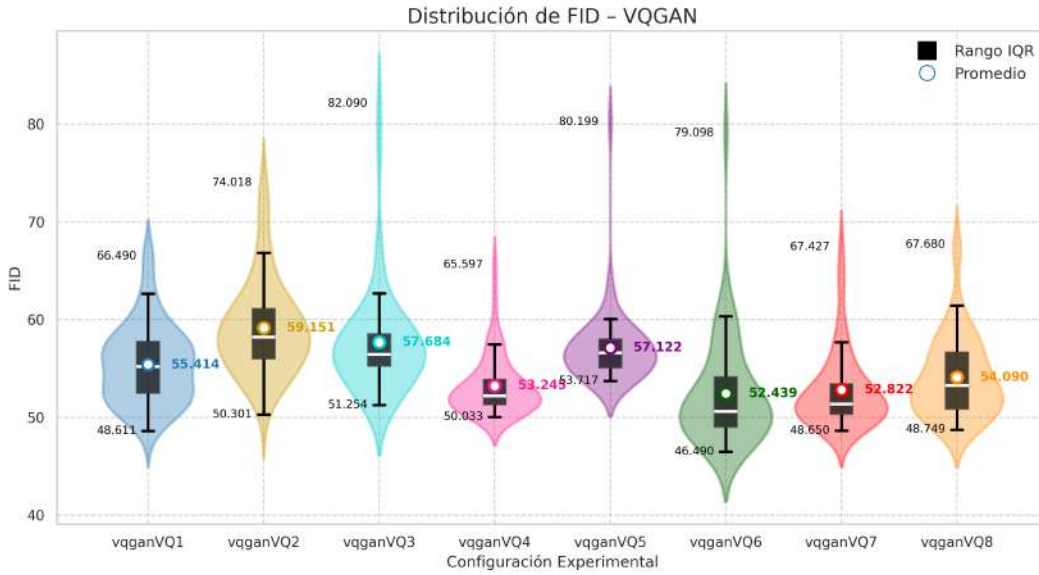


Figura 6-38: Diagrama de violín con boxplot superpuesto representando la distribución del FID para cada configuración experimental de VQ-GAN.

Tabla 6-27: Mínimos, máximos y promedios de FID registrados para los diferentes experimentos de VQ-GAN, de mejor a peor.

Experimento	FID_min ↑	época	FID_max	época	FID_promedio
VQ6	46.49	30	79.09	2	52.44
VQ1	48.61	33	66.49	6	0.0073
VQ7	48.65	38	67.43	17	57.12
VQ8	48.75	22	67.68	25	54.09
VQ4	50.03	13	65.59	2	53.24
VQ2	50.30	48	74.18	2	59.15
VQ3	51.25	46	82.09	2	0.0149
VQ5	63.71	18	80.19	9	0.0114

Con esta arquitectura surge una particularidad importante, VQ-GAN, estrictamente hablando, no es un modelo generativo, sino un modelo de reconstrucción, aunque puede codificar y reconstruir imágenes, su función principal es construir un codebook que posteriormente servirá como insumo para un *Transformer* más puro, el cual será el encargado de generar imágenes completamente nuevas. Por esta razón, VQ-GAN sí cuenta con un equivalente directo respecto a las imágenes reales y, en consecuencia, métricas de similitud como SSIM, PSNR y LPIPS resultan no solo aplicables, sino especialmente útiles para evaluar su desempeño.

En la configuración VQ4, el SSIM muestra un patrón inicial muy bueno, los valores que comienzan cerca de 0.89 – 0.95 desde las primeras épocas, lo que ya sugiere una red que logra capturar una parte significativa de la estructura visual incluso antes de estabilizarse, a lo

largo del entrenamiento, el índice asciende progresivamente; VQ5 tiene un comportamiento inicial similar al de VQ4, aunque en la Figura 6-39 se observan oscilaciones más amplias.

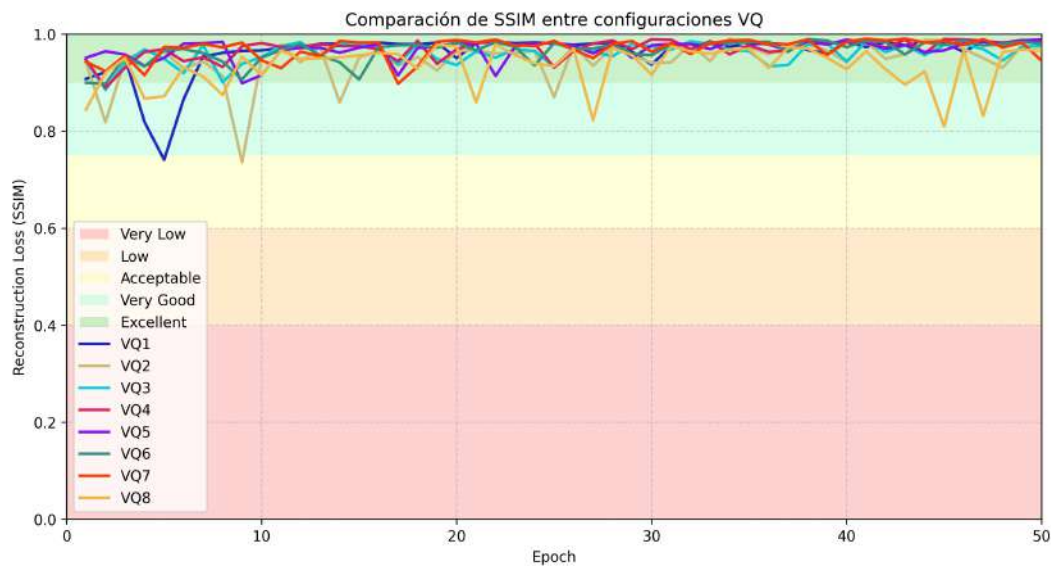


Figura 6-39: Evolución temporal del SSIM (en función de época) para todas las configuraciones de hiperparámetros en VQ-GAN.

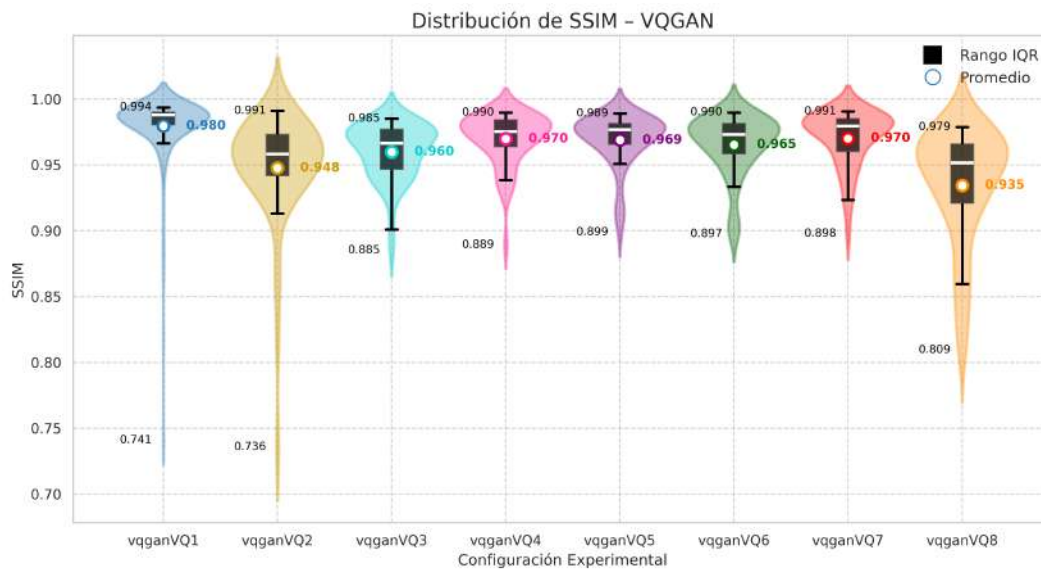


Figura 6-40: Diagrama de violín con boxplot superpuesto representando la distribución del SSIM para cada configuración experimental de VQ-GAN.

En VQ6, el comportamiento es notablemente diferente, aunque el valor inicial sigue siendo bueno (alrededor de 0.89–0.90), el índice no alcanza la misma madurez que en VQ4 o VQ5,

la mejora existe, pero se nota más tímida, más lenta, en VQ8 sigue el mismo patrón con valores iniciales cercanos a 0.81–0.90, sin embargo, presenta una dualidad, por un lado, los valores válidos muestran estabilidad y un ascenso saludable, por otro, la dispersión de los valores contaminados en el archivo es más marcada que en los otros casos como se ve en la Figura6-40 aun así, cuando se depura visualmente la señal real, VQ8 sí mantiene una evolución ascendente sin colapsos, aunque con cierta irregularidad intermedia.

Tabla 6-28: Mínimos, máximos y promedios de SSIM registrados para los diferentes experimentos de VQ-GAN, de mejor a peor.

Experimento	SSIM_max ↑	época	SSIM_min	época	SSIM_promedio
VQ5	0.9891	50	0.8988	9	0.9687
VQ1	0.9938	166	0.7411	5	0.9798
VQ2	0.9912	34	0.7358	9	0.9477
VQ7	0.9907	43	0.8976	17	0.9702
VQ6	0.9899	38	0.8975	2	0.9654
VQ4	0.9898	45	0.8893	2	0.9699
VQ3	0.9851	32	0.8851	2	0.9599
VQ8	0.9787	22	0.8094	45	0.9345

Para PSNR se ve un comportamiento para cada configuración similar a lo observado en SSIM, se ve en la Figura6-41 como todas las configuraciones siguen mejorando mientras avanza el entrenamiento, donde ninguna tiene cambios considerablemente bruscos, VQ4 se mantiene como la configuración más estable, con ascenso claro y convergencia fina, mientras que V6, Vq3 y VQ8 aprende estructura, pero no logra gran precisión pixel-a-pixel en comparacion a las demas configuraciones.

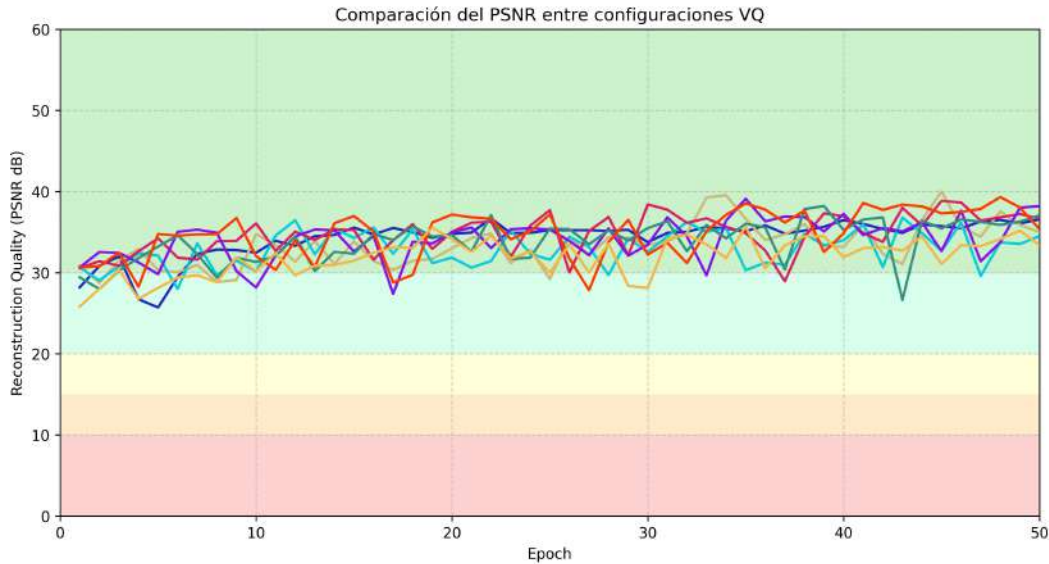


Figura 6-41: Evolución temporal del PSNR (en función de época) para todas las configuraciones de hiperparámetros en VQ-GAN.

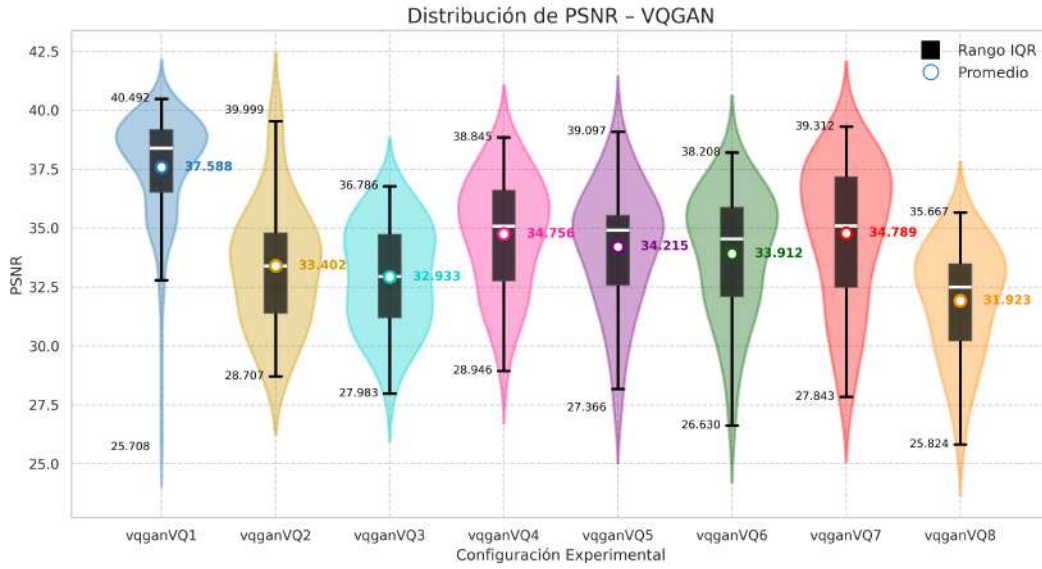


Figura 6-42: Diagrama de violín con boxplot superpuesto representando la distribución del PSNR para cada configuración experimental de VQ-GAN.

Tabla 6-29: Mínimos, máximos y promedios de PSNR registrados para los diferentes experimentos de VQ-GAN, de mejor a peor.

Experimento	PSNR_max ↑	época	PSNR_min	época	PSNR_promedio
VQ1	40.49	139	25.71	5	37.59
VQ2	39.99	45	28.70	2	33.41
VQ7	39.31	48	27.84	27	34.79
VQ5	39.09	35	27.37	17	34.16
VQ6	38.21	39	26.66	43	33.91
VQ4	38.85	45	28.95	37	34.76
VQ8	35.67	35	25.82	1	31.92
VQ3	36.79	43	27.98	6	32.93

Tabla 6-30: Mínimos, máximos y promedios de LPIPS registrados para los diferentes experimentos de VQ-GAN, de mejor a peor.

Experimento	LPIPS_min ↑	época	LPIPS_max	época	LPIPS_promedio
VQ1	0.0029	131	0.0812	4	0.0073
VQ5	0.0039	49	0.0375	10	0.0114
VQ7	0.0040	43	0.0554	4	0.0133
VQ4	0.0041	46	0.0311	3	0.0105
VQ6	0.0048	39	0.0427	2	0.0128
VQ3	0.0065	46	0.0601	2	0.0149
VQ8	0.0074	35	0.0902	1	0.0228
VQ2	0.2006	560	0.0033	25	0.1094

Por ultimo para LPIPS, VQ4 y VQ7 tienen los mismos descensos limpios y estables, una reconstrucción perceptuales excelente y menos sensibilidad o fluctuaciones, VQ8 tiene una mejora clara pero con una irregularidad intermedia, ademas de tener el LPIPS mas alto en comparacion, su desempeño final es bueno, pero la trayectoria deja ver un modelo que depende un poco más del azar del proceso estocástico y de la interacción entre batches, de ultimas se puede ubicar VQ6 con LPIPS alto, una mejora lenta, texturas mas pobres y menos detalles perceptuales en comparación de las demas configuraciones.

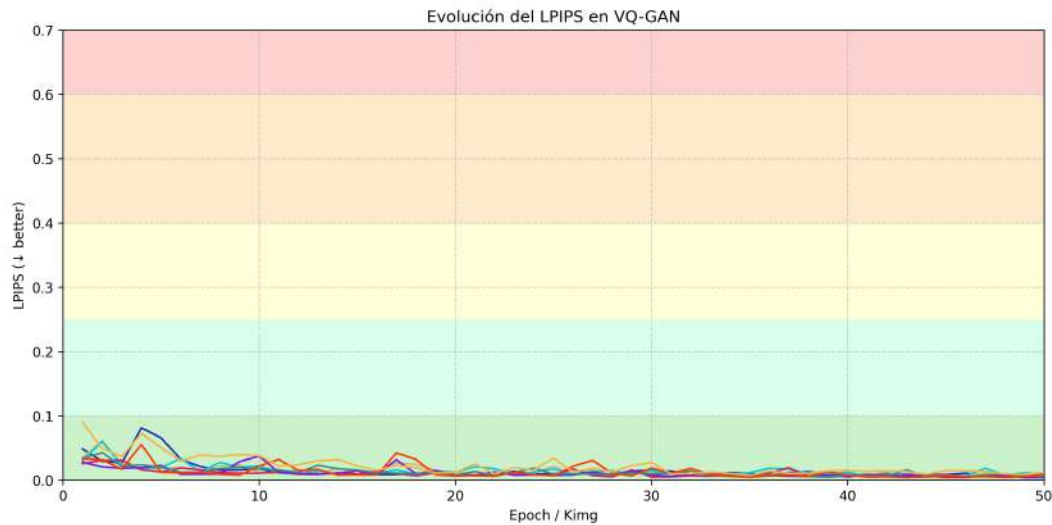


Figura 6-43: Evolución temporal del LPIPS (en función de época) para todas las configuraciones de hiperparámetros en VQ-GAN.

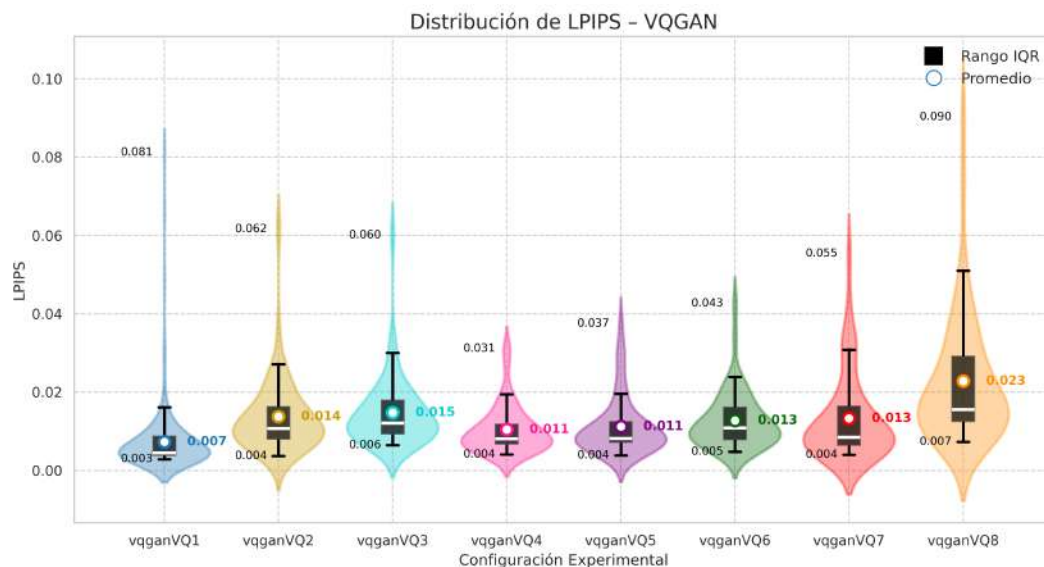


Figura 6-44: Diagrama de violín con boxplot superpuesto representando la distribución del LPIPS para cada configuración experimental de VQ-GAN.

6.2 Mejores configuraciones de cada arquitectura



Tabla 6-31: Diagramas del mejor valor obtenido por cada configuración en cada métrica para las 4 diferentes arquitecturas.

Para determinar la mejor configuración de cada arquitectura GAN, se inicia el análisis presentando los mejores resultados obtenidos por cada configuración en las diferentes métricas implementadas como se observa en las figuras de la Tabla 6-31, se observa que cada métrica incluye, en la esquina superior derecha, una estrella blanca o negra que indica cuál fue la mejor configuración en cada métrica, para determinar cual configuración fue superior teniendo en cuentas todas las metricas en conjunto, se aplicó un procedimiento de ranking multicriterio, asignando a cada métrica un peso proporcional a su nivel de importancia.

Tal como se menciona anteriormente, las métricas de distribución fueron las más relevantes en el contexto de modelos generativos, en particular, FID y KID reflejaron de forma consistente el comportamiento y la calidad de los datos generados: cuando el FID empeoraba, las imágenes tendían a verse ruidosas o incoherentes ante la inspección humana; por el contrario, un FID bajo se asociaba con imágenes capaces de engañar al observador no experto, por el contrario, la métrica PSNR fue la que menos apporto información relevante o coherente, a

excepciona de un GAN de reconstrucción como lo fue VQ-GAN, por esta razón se estableció un conjunto de pesos para darle mayor o menor relevancia a cada métrica en el presente análisis.

Los valores finales fueron: FID con una ponderación de 0.30, KID con 0.25, JSD con 0.20, SSIM con 0.15, LPIPS con 0.07 y PSNR con 0.03, esta última representando el menor peso debido a su baja contribución a la evaluación global. Posteriormente, se aplicó una normalización min-max, requerida para unificar la dirección de evaluación de las métricas — menor valor es mejor en FID, KID, JSD y LPIPS, ocurre lo contrario en SSIM y PSNR donde mayor es mejor —, luego la normalización transforma todas las métricas a una escala común entre 0 y 1, donde valores más cercanos a 1 indican ser mejor. Finalmente, se calculó una puntuación ponderada para cada configuración, lo que permitió obtener un ranking general, tal como se presenta en la Tabla **6-32**, este ranking resume las mejores configuraciones para cada arquitectura junto con su valor de evaluación final.

Tabla 6-32: Top mejores configuraciones para cada modelo de GANs.

Exp StyleGAN	Puntaje	Exp MedGAN	Puntaje	Exp WGAN	Puntaje	Exp VQGAN	Puntaje
E4	0.99	M2	0.63	W8	0.92	VQ6	0.81
E3	0.86	M3	0.62	W6	0.89	VQ1	0.72
E6	0.78	M6	0.50	W3	0.84	VQ7	0.71
E8	0.74	M4	0.45	W1	0.82	VQ4	0.68
E7	0.73	M1	0.43	W7	0.44	VQ2	0.62
E9	0.71	M7	0.38	W4	0.32	VQ5	0.46
E1	0.70	M8	0.34	W2	0.23	VQ3	0.36
E2	0.02	M5	0.29	W5	0.02	VQ8	0.28

Las puntuaciones obtenidas sitúan a E4 con $\sim 0,99$ y a E3 con $\sim 0,83$, lo que indica que ambas presentan un rendimiento significativamente superior al del resto de configuraciones; cabe señalar que, en las métricas SSIM y PSNR, se observa un comportamiento atípico: E7 aparece como la mejor en SSIM y E2 — a pesar de ser la peor en el ranking global — figura como la mejor en PSNR, esta inconsistencia confirma que dichas métricas no ofrecen información fiable para evaluar el desempeño de estos modelos generativos y, por tanto, su contribución en el puntaje global es penalizada.

Para determinar en qué king las configuraciones E3 y E4 alcanzan sus mejores resultados en cada métrica, es necesario definir una ventana operativa, dado que las configuraciones no siempre alcanzan su óptimo en el mismo valor de king, se consultaron los datos presentados en las Tablas **6-3**, **6-4**, **6-5**, **6-6**, **6-7** y **6-8**, a partir de estos valores se construyó la Tabla **6-33**, donde se resumen los kings en los que cada métrica alcanza su mejor rendimiento.

Para definir la ventana operativa se priorizaron los rangos donde aparecen los valores óptimos de FID y KID, dado su peso e importancia en la evaluación, se estableció una ventana entre

300 y 348 king para la configuración E3, y entre 348 y 764 king para E4, estos resultados pueden apreciarse con mayor claridad en las imágenes generadas en la Tabla **6-34**, donde la imagen de E4 dentro de los rangos óptimos se muestran resaltadas en verde para el king 750 y para el mejor rango de E3 que se mantiene como el segundo mejor modelo se muestra la imagen de la época 500 resaltada en amarillo ya que en este valor aun los rangos eran similares.

Tabla 6-33: Kings para configuración E3 y E4 donde tuvieron sus mejores resultados según cada métrica.

Métricas	King E3	King E4
FID	348	764
KID	300	808
JSD	1988	588
SSIM	1028	1648
PSNR	32	32
LPIPS	64	1012

Para MedGAN como se observa en la Tabla **6-32**, las configuraciones con el mejor puntaje son M2 con $\sim 0,63$ y M3 con $\sim 0,62$, mostrando coherencia con lo observado en la Tabla **6-31**, M2 obtiene los mejores valores en FID, SSIM y PSNR, mientras que M3 presenta los mejores resultados en JSD y LPIPS, no obstante, al considerar también la estabilidad de sus distribuciones y el hecho de que M3 alcanza el peor valor mínimo de FID, esta configuración no resulta la más adecuada para un entrenamiento confiable y robusto, aun así, se mantiene en la evaluación para respetar los criterios establecidos, por ende, M2 no solo presenta el mejor FID, sino que muestra una distribución consistente y compacta en todas las métricas, lo que la convierte en la opción más confiable entre las configuraciones analizadas.

Al igual que con StyleGAN2-ADa, para determinar en qué época las configuraciones M2 y M3 alcanzan sus mejores resultados en cada métrica, se definirá la ventana operativa teniendo en cuenta los datos presentados en las Tablas **6-11**, **6-12**, **6-13**, **6-14**, **6-15** y **6-16**, a partir de estos valores se construyó la Tabla **6-35**, que contiene los épocas en los que cada métrica alcanza su mejor rendimiento. M2 alcanza sus mejores valores de evaluación dentro de una ventana comprendida entre las épocas 990 y 1070, es decir al final del entrenamiento, lo que indica que el modelo continúa mejorando conforme avanza el entrenamiento y que aún no ha alcanzado su punto de convergencia; por el contrario, M3 presenta su intervalo óptimo entre las épocas 105 y 125, mostrando un decaimiento progresivo conforme continúa el entrenamiento, estos resultados posicionan a M2 como la mejor configuración de MedGAN, al evidenciar un comportamiento más robusto y un mayor potencial de mejora, de este modo en la Tabla **6-37**, se remarco con un reborde verde la Ecocardiografía generada en la época 1000 para M2 y con un reborde amarillo en la época 100 para M3.

Tabla 6-34: Resultados de imágenes generadas en diferentes kings a medida que avanza el entrenamiento del StyleGAN2-ADA

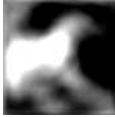
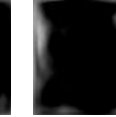
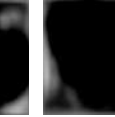
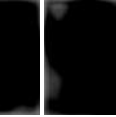
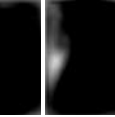
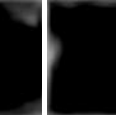
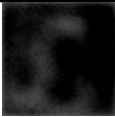
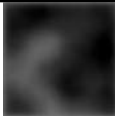
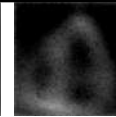
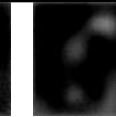
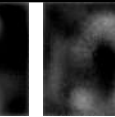
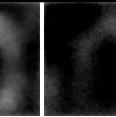
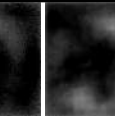
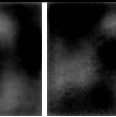
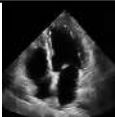
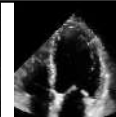
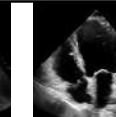
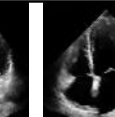
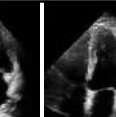
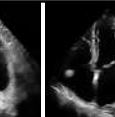
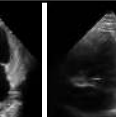
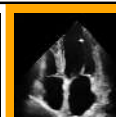
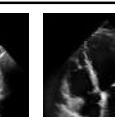
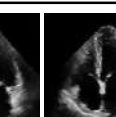
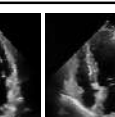
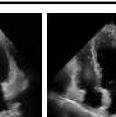
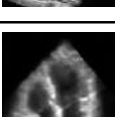
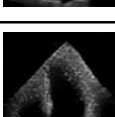
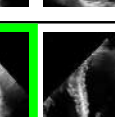
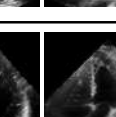
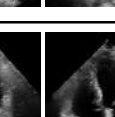
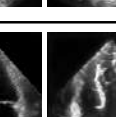
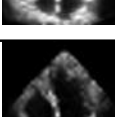
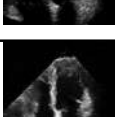
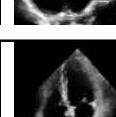
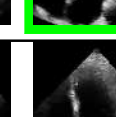
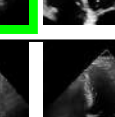
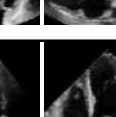
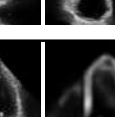
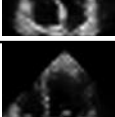
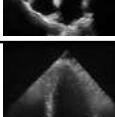
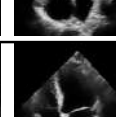
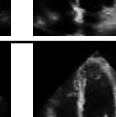
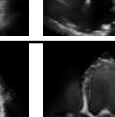
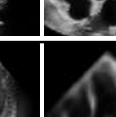
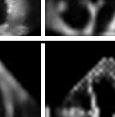
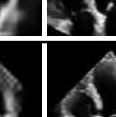
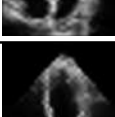
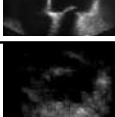
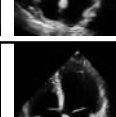
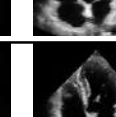
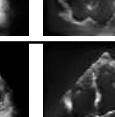
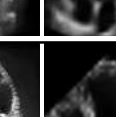
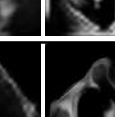
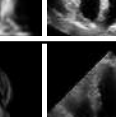
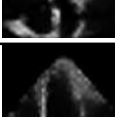
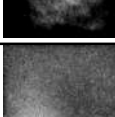
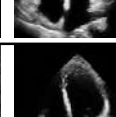
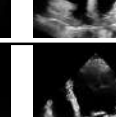
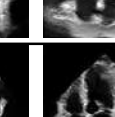
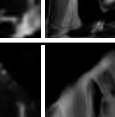
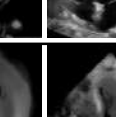
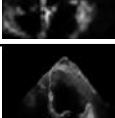
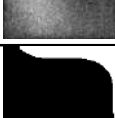
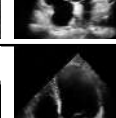
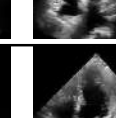
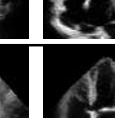
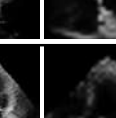
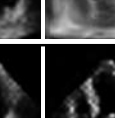
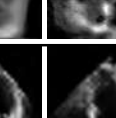
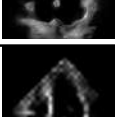
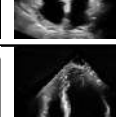
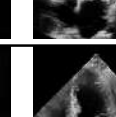
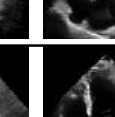
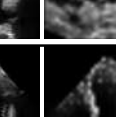
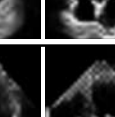
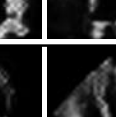
kings	E1	E2	E3	E4	E6	E7	E8	E9
0								
16								
250								
500								
750								
1000								
1250								
1500								
1750								
2000								
2100								

Tabla 6-35: Épocas para configuración M2 y M3 donde tuvieron sus mejores resultados según cada métrica.

Métricas	Epocá M2	Epocá M3
FID	1070	105
KID	990	125
JSD	655	370
SSIM	840	150
PSNR	840	105
LPIPS	200	150

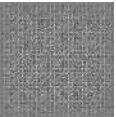
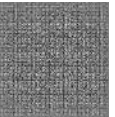
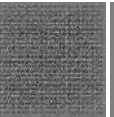
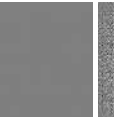
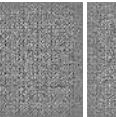
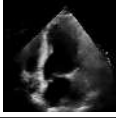
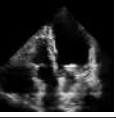
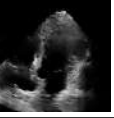
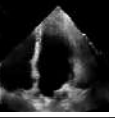
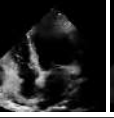
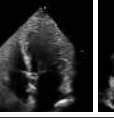
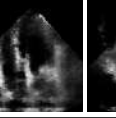
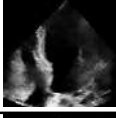
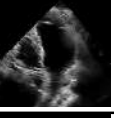
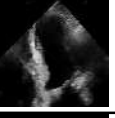
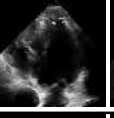
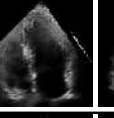
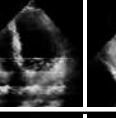
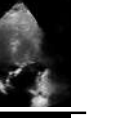
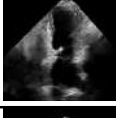
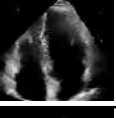
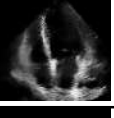
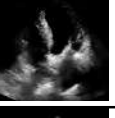
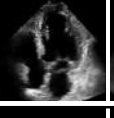
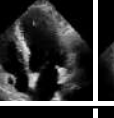
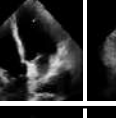
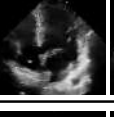
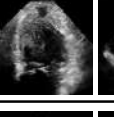
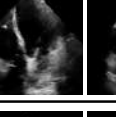
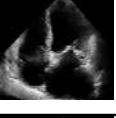
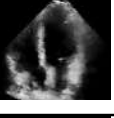
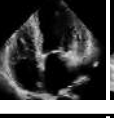
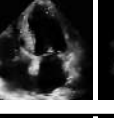
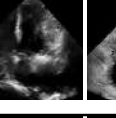
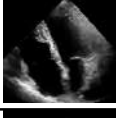
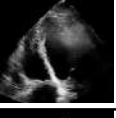
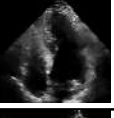
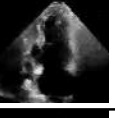
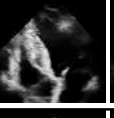
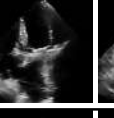
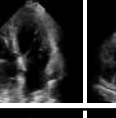
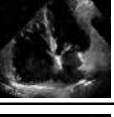
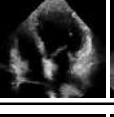
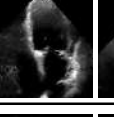
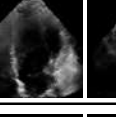
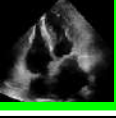
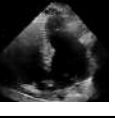
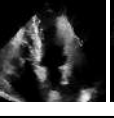
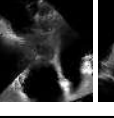
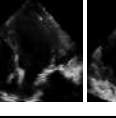
Para WGAN como se observa en la Tabla **6-32**, las configuraciones cuyos $\mathcal{G}$ y $\mathcal{D}$ mostraron estancamiento en sus funciones de pérdida, ocuparon los últimos 4 lugares para todas las métricas y por ende también el puntaje final (W7, W4, W2 y W5), las configuraciones con el mejor puntaje fueron W8 con $\sim 0,92$ y M6 con $\sim 0,89$, mostrando a su vez coherencia con lo observado en la Tabla **6-31**, W8 obtiene los mejores valores en JSD, SSIM y LPIPS, mientras que W6 presenta los mejores resultados en FID y KID.

Al igual que en las anteriores GANs, para determinar en qué época las configuraciones W8 y W6 alcanzan sus mejores resultados en cada métrica, se definirá la ventana operativa teniendo en cuenta los datos presentados en las Tablas **6-19**, **6-20**, **6-21**, **6-22**, **6-23** y **6-24**, a partir de estos valores se construyó la Tabla **6-36**, que contiene los épocas en los que cada métrica alcanza su mejor rendimiento. W8 alcanza sus mejores valores de evaluación dentro de una ventana comprendida entre las épocas 530 y 680; por el contrario, W6 presenta su intervalo óptimo entre las épocas 1005 y 1030, demostrando ser una configuración con la capacidad de seguir mejorando, de este modo en la Tabla **6-37**, se remarco con un reborde verde la Ecocardiografía generada en la época 600 para W8 y con un reborde amarillo en la época 1000 para W6.

Tabla 6-36: Épocas para configuración W8 y W6 donde tuvieron sus mejores resultados según cada métrica.

Métricas	Epocá W8	Epocá W6
FID	680	1005
KID	530	1030
JSD	815	645
SSIM	840	830
PSNR	1015	425
LPIPS	1095	830

Tabla 6-37: Resultados de imágenes generadas en diferentes épocas a medida que avanza el entrenamiento de MedGAN

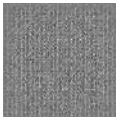
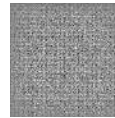
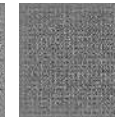
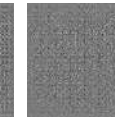
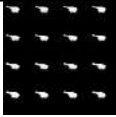
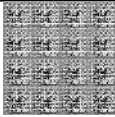
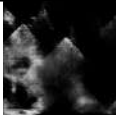
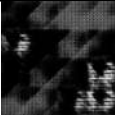
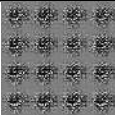
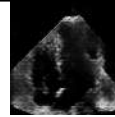
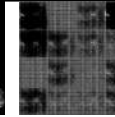
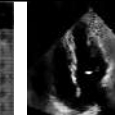
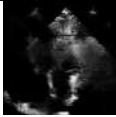
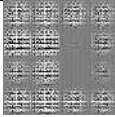
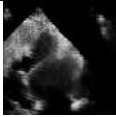
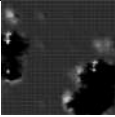
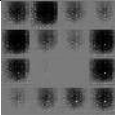
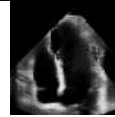
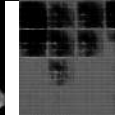
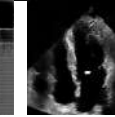
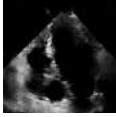
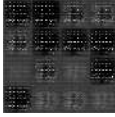
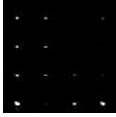
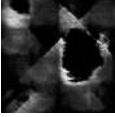
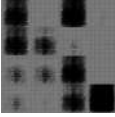
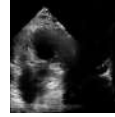
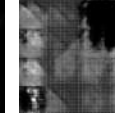
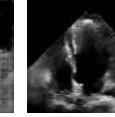
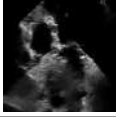
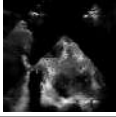
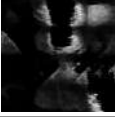
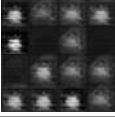
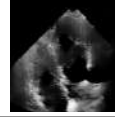
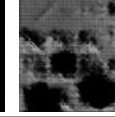
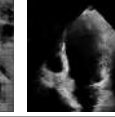
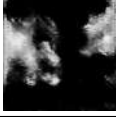
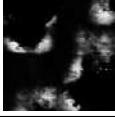
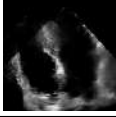
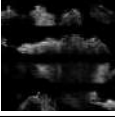
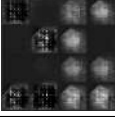
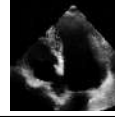
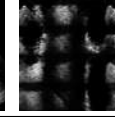
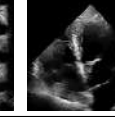
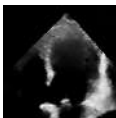
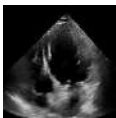
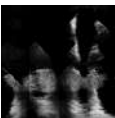
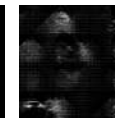
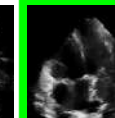
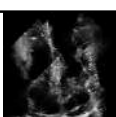
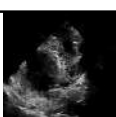
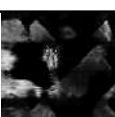
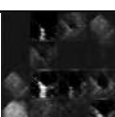
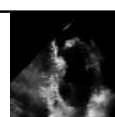
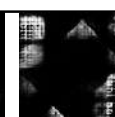
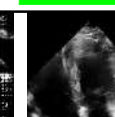
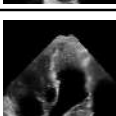
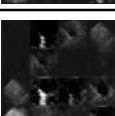
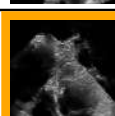
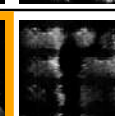
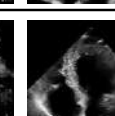
epoch	M1	M2	M3	M4	M5	M6	M7	M8
0								
25								
50								
100								
200								
400								
600								
800								
1000								

Para VQ-GAN se realizó una ligera modificación en la ponderación utilizada para el cálculo del ranking de las mejores configuraciones, en este caso, se asignó un peso de 0.4 al FID, mientras que SSIM, PSNR y LPIPS recibieron un peso de 0.2 cada una, esta decisión se fundamenta en que las métricas de similitud presentaron comportamientos muy similares entre sí y aportaron esencialmente la misma información relevante, por ello, se optó por mantener pesos equivalentes entre dichas métricas.

Como se observa en la Tabla 6-32, las configuraciones con mejor puntaje global fueron VQ6

con aproximadamente 0.81 y VQ1 con alrededor de 0.72, lo cual resulta coherente con lo mostrado en la Tabla 6-31. VQ6 obtuvo el mejor valor en FID, mientras que VQ1 registró los valores más altos en SSIM y PSNR, evidenciando su capacidad para reconstruir imágenes altamente similares a su respectivo par real, con diferencias leves principalmente asociadas al ruido *speckle*.

Tabla 6-38: Resultados de imágenes generadas en diferentes épocas a medida que avanza el entrenamiento de WGAN

epoch	W1	W2	W3	W4	W5	W6	W7	W8
0								
25								
50								
100								
200								
400								
600								
800								
1000								

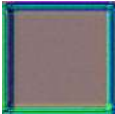
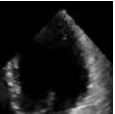
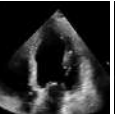
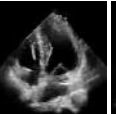
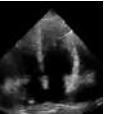
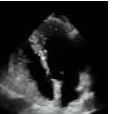
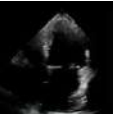
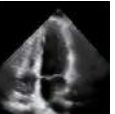
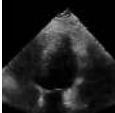
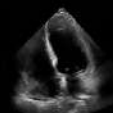
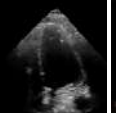
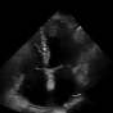
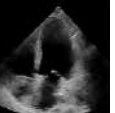
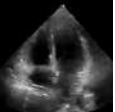
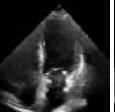
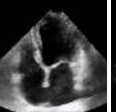
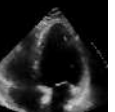
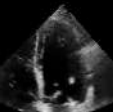
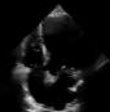
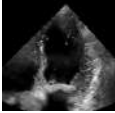
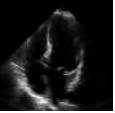
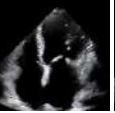
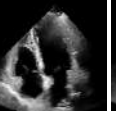
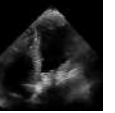
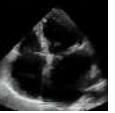
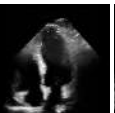
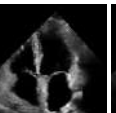
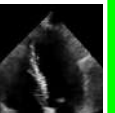
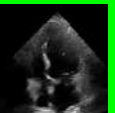
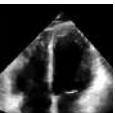
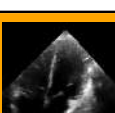
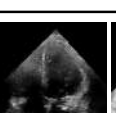
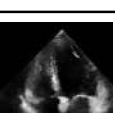
Al igual que con las anteriores metricas, para determinar en qué época las configuraciones VQ6 y VQ1 alcanzan sus mejores resultados en cada métrica, se definirá la ventana operativa

teniendo en cuenta los datos presentados en las Tablas **6-27**, **6-28**, **6-29** y **6-30**, a partir de estos valores se construyó la Tabla **6-39**, que contiene los épocas en los que cada métrica alcanza su mejor rendimiento. VQ6 alcanza sus mejores valores de evaluación dentro de una ventana comprendida entre las épocas 30-39, mientras que, VQ1 presenta su intervalo óptimo entre las épocas 131-166, mostrando como el resto de configuraciones requieren mas entrenamiento y a que aun tienen un gran margen de mejor, de este modo en la Tabla **6-40**, se remarco con un reborde verde la Ecocardiografía generada en la época 40 para VQ6 y con un reborde amarillo en la época 50 para VQ1.

Tabla 6-39: Épocas para configuración VQ6 y VQ1 donde tuvieron sus mejores resultados según cada métrica.

Métricas	Epocá VQ6	Epocá VQ1
FID	30	33
SSIM	38	166
PSNR	39	139
LPIPS	39	131

Tabla 6-40: Resultados de imágenes generadas en diferentes épocas a medida que avanza el entrenamiento de WGAN

epoch	W1	W2	W3	W4	W5	W6	W7	W8
0								
10								
20								
30								
40								
50								

6.3 Muestras sintéticas generadas por el mejor modelo

Teniendo en cuenta los resultados previamente analizados, se identificó con claridad el modelo con mejor rendimiento, su configuración más óptima y el punto de entrenamiento más favorable, siendo StyleGAN2-ADA bajo la configuración E4 en la época 764 con un valor de 9.17 según la métrica FID, este punto se tomó como referencia para utilizar el checkpoint correspondiente, con el propósito de proporcionar evidencia visual del rendimiento del modelo con mejor resultado cuantitativo, estas muestras permiten observar la diversidad visual y la coherencia estructural y morfológica alcanzada por el modelo en la síntesis de imágenes.

Figura 6-45: Muestras sintéticas generadas por StyleGAN2-ADA (Configuración E4)





Teniendo en cuenta las limitaciones de espacio del documento, se presentan muestras aleatorias del conjunto generado, el *dataset* completo compuesto por 10.000 imágenes generadas se encuentra disponible públicamente en el repositorio del proyecto para fines de reproducibilidad y análisis adicional: GitHub¹.

¹https://github.com/SalomonHernandezVelandia/EchocardiogramGenerator/tree/main/experiments/stylegan2_ada/E4/best_generated

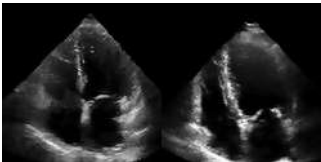
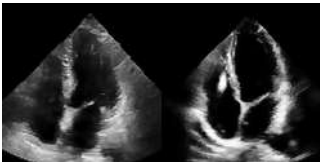
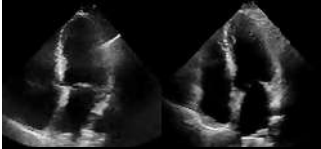
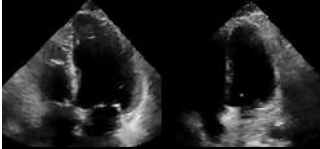
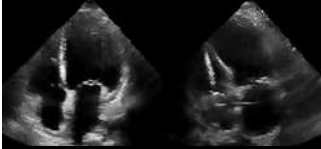
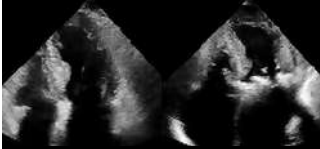
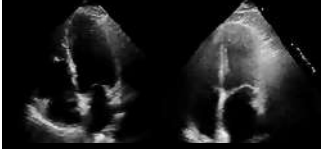
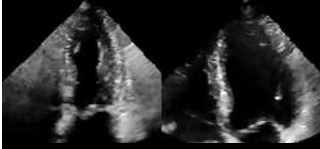
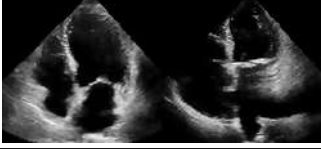
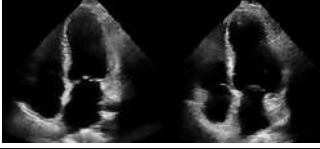
6.4 Resultados de la evaluación perceptual por especialistas

La evaluación fue realizada por cinco profesionales del área de la salud, conformados por tres especialista en cardiología y dos médicos generales, cada participante indico que imágenes consideraba reales una sola vez, evitando así efectos de aprendizaje o familiarización.

El instrumento incluyó distintos niveles de dificultad con el objetivo de analizar el comportamiento de los especialistas en escenarios de elección progresivamente más complejos, permitiendo evaluar tanto la capacidad de detección como la susceptibilidad al engaño por parte de las imágenes generadas:

- Por comparación binaria, en este primer nivel se presentaron pares de imágenes, cada uno compuesto por una imagen real y una imagen generada, el médico debía seleccionar cuál consideraba real.

Tabla 6-42: Nivel 1 – Evaluación binaria, pares de imágenes compuestos por una imagen real y una sintética, la columna “Real” indica la posición correcta.

Imagen de Evaluación	Real	Imagen de Evaluación	Real
	Derecha		Derecha
	Izquierda		Derecha
	Derecha		Derecha
	Derecha		Izquierda
	Derecha		Derecha

- Para el segundo y tercer nivel se presentaron grupos de cuatro imágenes, en el segundo nivel compuestos por dos reales, dos sintéticas y en el tercer nivel únicamente una era real y las tres restantes eran sintéticas distribuidas aleatoriamente, aquí se reduce la probabilidad de acierto por selección aleatoria y exige un análisis comparativo más fino.

Tabla 6-43: Nivel 2 – Conjuntos de cuatro imágenes con dos reales y dos sintéticas, se indica la distribución correcta (R = real, F = sintética).

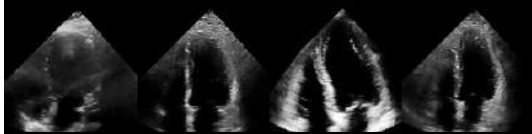
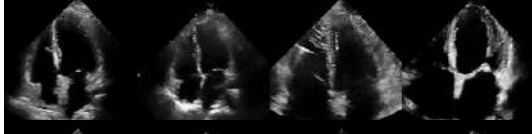
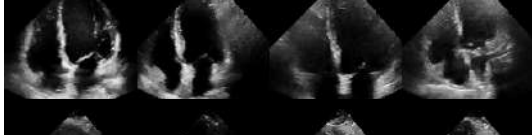
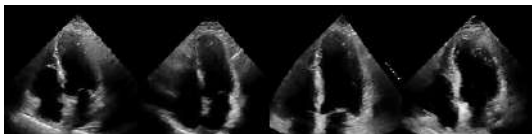
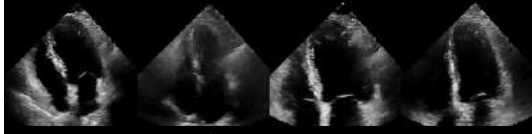
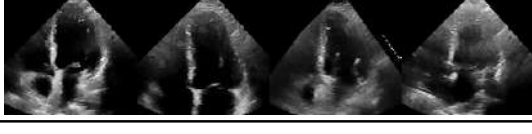
Imagen de Evaluación	Etiqueta real
	R F R F
	F R R F
	F F R R
	R F F R
	F R R F

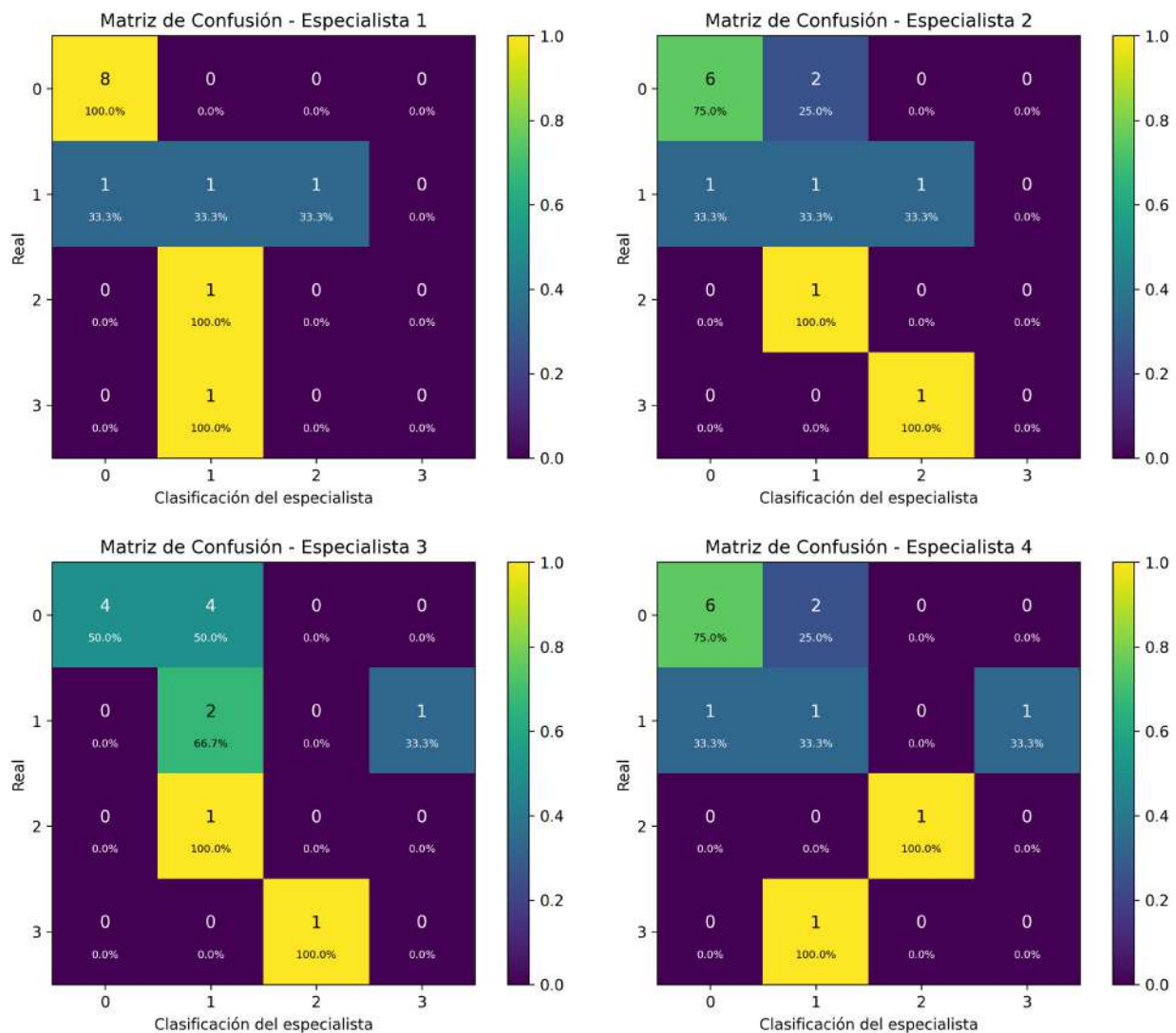
Tabla 6-44: Nivel 3 – Conjuntos de cuatro imágenes con una real y tres sintéticas, se indica la posición correcta (R = real, F = sintética).

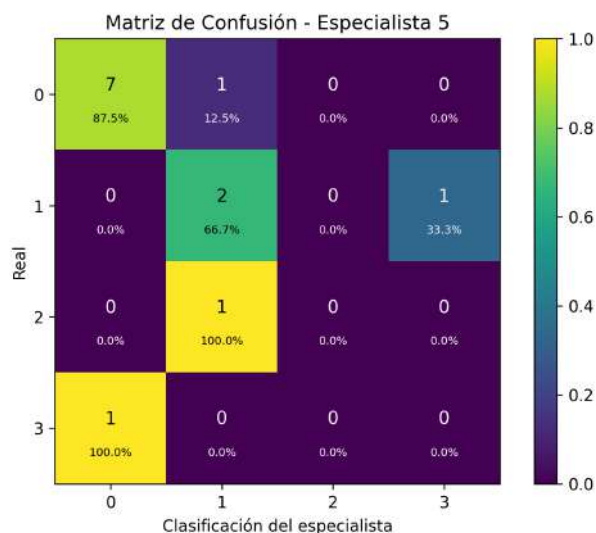
Imagen de Evaluación	Etiqueta real
	F F F R
	F R F F
	F F R F

6.4.1 Análisis de matrices de confusión por especialista

Con el fin de identificar el desempeño individual de los participantes en la tarea de discriminación entre imágenes reales y sintéticas, se construyeron matrices de confusión para cada evaluador, permitiendo observar de forma mas visual la distribución de aciertos y errores, así como identificar patrones de clasificación.

Figura 6-46: Matrices de confusión de cada evaluador





El cardiólogo que puede ver en la matriz como Especialista 1, muestra un comportamiento relativamente consistente en la identificación de imágenes reales, con una proporción elevada de clasificaciones correctas en la categoría principal, sin embargo, se observa la presencia de errores en los cuales algunas imágenes fueron clasificadas en categorías incorrectas, lo que indica que ciertas muestras sintéticas lograron simular características visuales plausibles desde una perspectiva clínica. Estos resultados indican que, aunque el cardiólogo mantiene una buena capacidad discriminativa, el modelo presenta una capacidad moderada de engaño.

La matriz correspondiente al primer médico general (Especialista 2) evidencia un mayor grado de dispersión en las clasificaciones, con errores tanto en la identificación de imágenes reales como sintéticas, se observa una menor concentración en la diagonal principal en comparación con el especialista, lo que sugiere una menor precisión global, para el segundo médico general (Especialista 3) muestra un patrón interesante, con aciertos claros en algunas categorías y errores significativos en otras, la presencia de clasificaciones incorrectas en imágenes sintéticas refuerza la evidencia de que el modelo es capaz de generar estructuras visuales que pueden confundirse con patrones anatómicos reales, especialmente en observadores con menor especialización.

El segundo cardiólogo (Especialista 4) muestra, buen desempeño en categoría 0, mayor dispersión en categoría 1, clasificación perfecta en categoría 2, error total en categoría 3), indicando dificultad cuando el número de imágenes sintéticas aumenta. Es interesante notar que incluso un especialista presenta vulnerabilidad cuando la proporción de imágenes sintéticas es mayor, lo que sugiere que el contexto influye en la toma de decisión clínica.

Y el tercer cardiólogo (Especialista 5) muestra 87.5 % de acierto en categoría 0, buen desem-

peño en categoría 1, error completo en categoría 3, indicando alta capacidad discriminativa en configuraciones simples, incremento notable del engaño cuando la proporción de imágenes sintéticas es mayor, influencia clara del diseño del experimento en la toma de decisiones. A nivel global, aunque mantiene el mejor desempeño entre los participantes, no logra superar consistentemente la ambigüedad inducida por el modelo en escenarios complejos.

6.4.2 Análisis de la exactitud por especialista

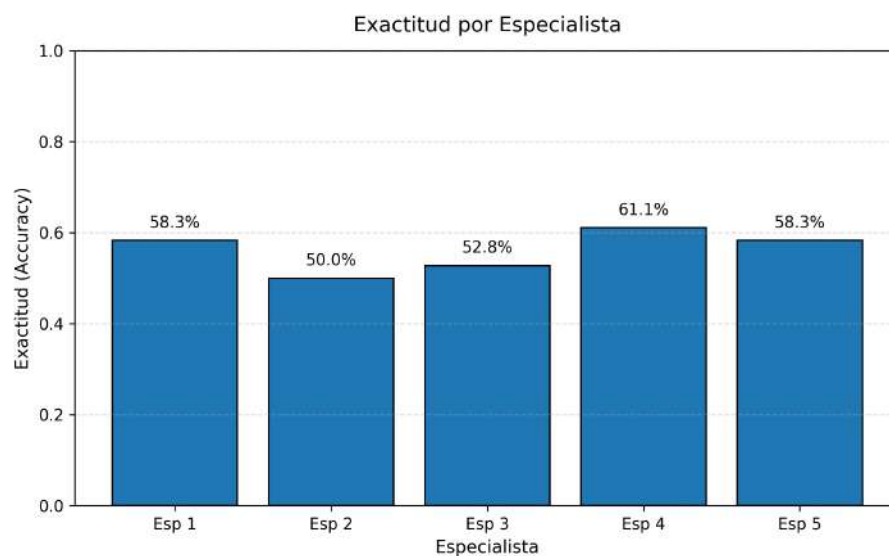


Figura 6-47: Exactitud (Accuracy) para cada especialista.

La Figura 6-47 presenta la exactitud obtenida por cada evaluador, esta métrica representa la proporción de clasificaciones correctas respecto al total de decisiones realizadas, permitiendo evaluar la capacidad global de los especialistas para identificar correctamente la naturaleza de las imágenes.

El especialista 1 presenta una exactitud del 58.3 %, lo cual indica un desempeño ligeramente superior al nivel de azar (50 %), sugiriendo cierta capacidad para distinguir patrones característicos entre imágenes reales y generadas, no obstante, la diferencia respecto al azar no es amplia, lo que evidencia que las imágenes sintéticas alcanzan un nivel de realismo suficiente para generar confusión incluso en un evaluador con experiencia previa.

El especialista 2 obtuvo una exactitud del 50 %, lo que indica un desempeño equivalente al esperado por selección aleatoria, este resultado tiene cierto grado de relevancia, ya que sugiere

que el evaluador no pudo distinguir de manera consistente entre imágenes reales y sintéticas, un *accuracy* cercano al 50 % indica que el modelo generativo logró producir imágenes perceptualmente indistinguibles para este evaluador, evidenciando una alta capacidad de camuflaje visual, esto puede interpretarse como un indicador fuerte del realismo de las imágenes generadas.

El especialista 3 obtuvo una exactitud del 52.8 %, lo cual representa un desempeño marginalmente superior al azar, este resultado sugiere que el evaluador pudo identificar correctamente algunas características diferenciales, aunque con un nivel de incertidumbre considerable, la proximidad al 50 % indica que las imágenes sintéticas presentan características visuales suficientemente realistas como para dificultar la discriminación confiable.

Con la inclusión de dos nuevos cardiólogos, se observa una variación interesante en los resultados, el especialista 4 obtuvo la mayor exactitud global (61.1 %), superando ligeramente el desempeño del resto de los participantes, sugiriendo que la formación clínica especializada podría aportar una ventaja moderada en la identificación de inconsistencias estructurales o patrones fisiológicos sutiles presentes en las imágenes sintéticas. Sin embargo, incluso en este caso, la exactitud no supera ampliamente el 60 %, lo que indica que el modelo generativo logra mantener un nivel de realismo suficientemente alto como para evitar una discriminación contundente.

Por su parte, el especialista 5 obtuvo una exactitud del 58.3 %, valor comparable al del especialista 1, este desempeño refuerza la idea de que, aunque la experiencia clínica puede contribuir a mejorar la capacidad discriminativa, el margen de mejora respecto al azar sigue siendo limitado.

De manera global, los resultados muestran que todos los evaluadores presentan valores de exactitud en un rango estrecho entre 50 % y 61.1 %, la proximidad de estos valores al nivel de azar sugiere que las imágenes sintéticas generadas poseen un alto grado de realismo perceptual, dificultando su identificación incluso por especialistas con formación en cardiología, la baja dispersión entre evaluadores también indica que la dificultad de la tarea es consistente entre distintos perfiles profesionales.

6.4.3 Análisis del *Fooling Rate*

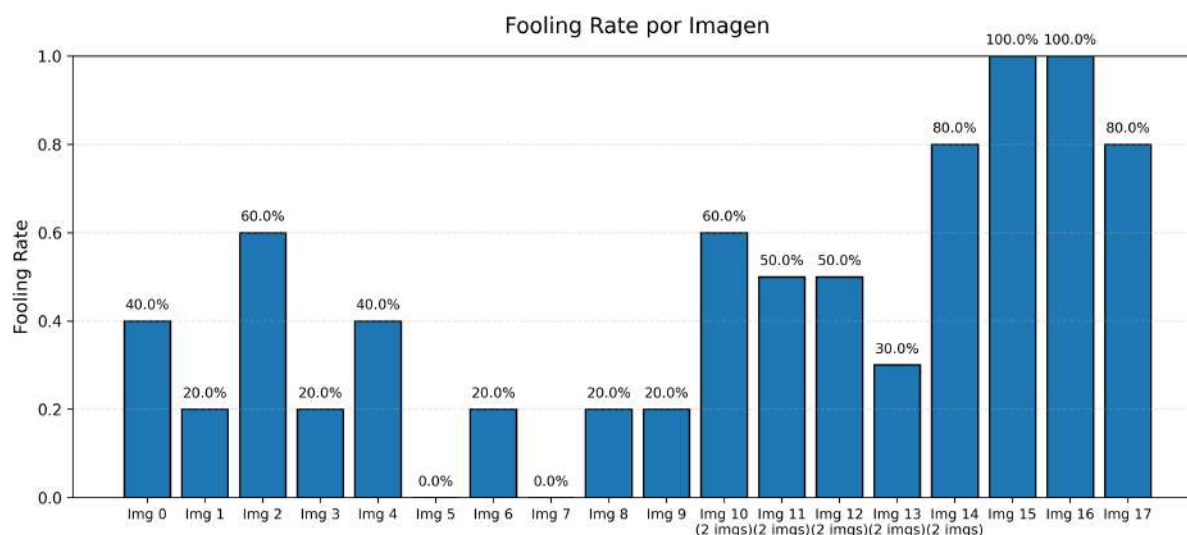


Figura 6-48: *Fooling Rate* indicando las imágenes que tuvieron mas capacidad de pasar como reales.

La Figura 6-48 presenta el comportamiento del *Fooling Rate* por imagen, entendido como la proporción de especialistas que clasificaron incorrectamente una imagen sintética como real, permitiendo evaluar de manera directa la capacidad del modelo generativo para producir imágenes plausibles desde el punto de vista clínico, constituyendo un indicador clave del grado de realismo alcanzado.

En términos globales, los resultados evidencian de forma considerable la capacidad de engaño entre las diferentes imágenes, lo cual sugiere que el realismo perceptual no es uniforme en el conjunto generado, se observan valores que oscilan entre 0 % y 100 %, lo que indica la coexistencia de muestras claramente distinguibles como sintéticas y otras prácticamente indistinguibles de las reales para los especialistas.

Las imágenes 5 y 7 presentan un *Fooling Rate* del 0 %, lo que implica que fueron correctamente identificadas como sintéticas por todos los especialistas. Estos casos sugieren la presencia de artefactos perceptibles o inconsistencias estructurales que facilitaron su detección, en coherencia con lo señalado por uno de los cardiólogos participantes: “Había imágenes que eran muy obvias”.

Las imágenes con 20 % (Imgs 1, 3, 6, 8 y 9) indican que solo uno de los cinco evaluadores fue engañado, aunque el modelo logra cierto grado de plausibilidad, estas muestras aún contienen señales detectables que permiten su correcta clasificación en la mayoría de los casos.

Las imágenes 0 y 4 presentan un 40 %, lo que implica que dos de los cinco especialistas fueron engañados, este rango representa un punto de transición donde el realismo comienza a ser clínicamente convincente para una proporción relevante de evaluadores.

Las imágenes 2 y 10 alcanzan un 60 %, indicando que la mayoría de los especialistas las clasificaron como reales, en estos casos, el modelo parece haber capturado con mayor fidelidad características fundamentales de la distribución de datos reales, como coherencia anatómica global, textura consistente y patrones de contraste adecuados.

Las imágenes 14 y 17 presentan un 80 % de *Fooling Rate*, es decir, cuatro de cinco especialistas fueron engañados, estas muestras reflejan un alto nivel de realismo perceptual, donde solo un evaluador logró detectar su naturaleza sintética.

Particularmente destacables son el grupo de imágenes 15 y 16, que alcanzan un 100 % de *Fooling Rate*, implicando que ningún especialista logró identificarlas como sintéticas, estos casos constituyen evidencia sólida del alto grado de realismo alcanzado por el modelo generativo, sugiriendo que ha logrado capturar no solo patrones globales, sino también detalles finos de textura, ruido y representación anatómica coherente con adquisiciones ecocardiográficas reales.

6.4.4 Análisis integrador de resultados y consideraciones clínicas de los evaluadores

El análisis conjunto de las métricas cuantitativas y perceptuales, complementado con las observaciones cualitativas de los especialistas, permite realizar una interpretación integral del desempeño del modelo generativo en un contexto clínico simulado, los resultados evidencian que el modelo es capaz de generar imágenes con un nivel de realismo suficiente para inducir confusión y pasar como reales incluso en profesionales del área médica, lo cual constituye un indicador sólido de la plausibilidad visual alcanzada.

Los valores de exactitud cercanos al azar, junto con la presencia consistente de falsos positivos en todos los evaluadores, sugieren que las imágenes sintéticas comparten características visuales relevantes con las imágenes reales, dificultando su identificación. No obstante, la variabilidad observada en el *Fooling Rate* entre imágenes individuales indica que el realismo perceptual no es completamente homogéneo dentro del conjunto generado, este fenómeno sugiere la existencia de regiones en el espacio latente con distintos niveles de calidad visual, lo cual es consistente con la literatura sobre modelos generativos basados en GAN, donde la estabilidad y coherencia pueden variar dependiendo de la complejidad anatómica.

Desde la perspectiva clínica, las observaciones cualitativas aportadas por los especialistas resultan especialmente relevantes para contextualizar los resultados.

En primer lugar, se destacó que la evaluación basada en fotogramas individuales limita la capacidad diagnóstica, dado que la interpretación ecocardiográfica se apoya en la dinámica temporal del movimiento cardíaco, la ausencia de información temporal reduce el contexto clínico disponible y puede influir tanto en la dificultad de elección como en la percepción global de realismo.

Asimismo, señalaron una notable cercanía morfológica entre imágenes reales y sintéticas, indicando que varias muestras lograron simular adecuadamente características visuales propias de los datos reales, incluyendo patrones que reflejan la presión o manipulación del transductor por parte del ecografista, respaldando la capacidad del modelo para capturar propiedades visuales complejas del proceso de adquisición.

Por otra parte, uno de los especialistas identificó limitaciones relacionadas con el tamaño y la resolución de las imágenes utilizadas en la evaluación, así como con ajustes de ganancia y brillo considerados subóptimos. Estas condiciones pueden haber afectado la percepción de detalles finos y la evaluación de la calidad visual, constituyendo un factor a considerar en la interpretación de los resultados y futuros estudios.

7 Conclusiones

El objetivo central de esta tesis consistió en evaluar de manera rigurosa el desempeño de tres arquitecturas generativas —StyleGAN2-ADA, MedGAN, WGAN y VQ-GAN— para la generación de imágenes ecocardiográficas completamente nuevas, de manera que fueran capaces de preservar morfología cardíaca, de textura propias de ecocardiogramas, ruido nativo (speckle) y coherencia estadística de los datos reales, con el fin de determinar el potencial de cada arquitectura, se implementó un análisis cuantitativo por medio de un esquema comparativo basado en diferentes métricas.

Los resultados obtenidos confirman que StyleGAN2-ADA fue el único capaz de lograr los criterios de calidad sugeridos en la literatura médica y las rúbricas heurística establecidas en la presente tesis como mínimas para considerar un dato generado como excelente, para StyleGAN2-ADA su mejor configuración fue E4, alcanzando un FID de 9.17, un valor que no solo cumple el estándar, sino que evidencia un nivel notablemente superior de fidelidad estadística; en contraste, la mejor configuración de MedGAN (M2) logró un FID de 63.469, mientras que la mejor configuración de WGAN (W8) obtuvo 93.63, ambos muy lejos del umbral clínico, estos resultados establecen a StyleGAN2-ADA como la arquitectura más adecuada para la generación sintética de ecocardiogramas dentro de los criterios establecidos en el presente trabajo, además, cabe resaltar que se evidenció que StyleGAN2-ADA no solo fue la arquitectura más precisa, sino también la más rápida y estable durante el entrenamiento, todas sus configuraciones lograron en determinado momento del entrenamiento un FID mínimo que superó en desempeño a las mejores variantes de MedGAN y WGAN.

A nivel de hallazgos, los resultados demuestran que la modificación de hiperparámetros desempeña un papel fundamental en el comportamiento de un modelo GAN, estos son sumamente sensibles a la elección de hiperparámetros, aunque la arquitectura sea buena una mala elección puede afectar fuertemente al modelo; en el caso de StyleGAN2-ADA, una regularización más baja, combinada con un *batch size* mayor y un parámetro *gamma* reducido, contribuyó a una mejora significativa en la estabilidad y la calidad de las muestras generadas, para MedGAN, la versión basada en un DCGAN clásico potenciada con un optimizador Adam de mayor tamaño, un incremento en el *batch size* y una reducción de la

tasa de aprendizaje mostró un desempeño superior frente al uso de RMSProp y para WGAN, la eliminación del *batch normalization* resultó ser el ajuste más determinante, aunque aun así la arquitectura no logró comportarse de manera completamente estable.

Se evidencio que métricas como PSNR y LPIPS, mostraron poca sensibilidad ante las variaciones en la calidad generada, de hecho, se observó que estas métricas tendían a estancarse o converger en valores similares, incluso cuando la calidad real de las imágenes variaba de manera notable, demostrando que son sumamente ineficientes y hasta innecesarias en el uso de GANs generativos que no tiene una imagen como correspondiente directo, sin embargo, se notó que la métrica SSIM — aunque tampoco es eficiente en este tipo de GANs —, fue ser útil en modelos que no generan imágenes congruentes, como WGAN, cuando las imágenes sintéticas presentaban distorsiones severas, SSIM penalizo adecuadamente esas inconsistencias, actuando como un indicador de deformación morfológica, esto permitió concluir que SSIM aporta una dimensión evaluativa valiosa en arquitecturas poco estables, mientras que PSNR y LPIPS no resultan adecuadas para determinar el mejor modelo en GANs centradas en ecocardiografía.

Asimismo, el uso de VQ-GAN — un modelo orientado a la reconstrucción de información y no a su generación desde cero — permitió comprobar que las métricas de similitud como SSIM, PSNR y LPIPS resultan altamente útiles y coherentes con los datos producidos, ya que facilitan un seguimiento preciso del estado del entrenamiento, a partir de eso, se concluye que para modelos generativos cuyo objetivo es producir imágenes completamente nuevas, estas métricas no son apropiadas, su aporte es limitado o incluso contraproducente, pues pueden inducir interpretaciones incorrectas y dificultar la evaluación global del modelo, como fue el caso de M3, que al tener métricas SSIM y PSNR más altas que las demás, le permitió estar en el top 2 de las configuraciones de MedGAN, siendo M3 una de las configuraciones más conflictivas e inestables de esa arquitectura. Por el contrario, para modelos de reconstrucción o aquellos que disponen de un par de referencia por cada imagen generada, dichas métricas se vuelven herramientas valiosas y potentes para evaluar la calidad del proceso.

En cuanto a la estabilidad del entrenamiento, MedGAN demostró un comportamiento significativamente más robusto que WGAN, su arquitectura con mecanismos adaptativo con el objetivo de impedir que el generador o el discriminador se vuelvan más fuertes respecto al otro permitió evitar colapsos de modo y estancamientos prolongados, problemas que sí estuvieron presentes en la mitad de configuraciones de WGAN. Gracias a WGAN se permitió observar y determinar que los entrenamientos que se prolongaba más allá del tiempo esperado eran un signo temprano de colapso, lo cual no solo afectó la calidad final, sino que también incrementó los costos computacionales, reforzando la conclusión de que la eficiencia de las GANs no depende solo de la arquitectura base, sino de la interacción cuidadosa entre hiperparámetros, optimizadores, regularización y dinámicas adversariales.

Entre los hallazgos visuales más importantes, StyleGAN2-ADA fue la única arquitectura

7. Conclusiones

capaz de generar ecocardiogramas altamente realistas, preservando morfología cardíaca, ruido speckle y evitando artefactos no fisiológicos, este nivel de realismo representa un avance significativo para el aumento de datos en este campo médico; los otros GANs con dificultades lograban crear las 4 cámaras del corazón.

En términos de aporte, esta tesis proporciona una aproximación cuantitativa amplia y detallada sobre el comportamiento de diferentes arquitecturas GAN aplicadas a ecocardiografía, así como el impacto directo que los hiperparámetros y estrategias de entrenamiento tienen sobre su desempeño, esta no solo establece qué arquitectura funciona mejor, sino que expone con claridad qué configuraciones y combinaciones de parámetros tienden a favorecer o deteriorar el rendimiento adversarial.

No obstante, se presentaron limitaciones importantes, especialmente desde el punto de vista computacional, para garantizar entrenamientos adecuados fue necesario disponer de máquinas con GPUs de al menos 12 GB de VRAM, y aun así varios modelos mostraron restricciones severas en estabilidad o en tiempos de convergencia, el tiempo total de entrenamiento acumulado para todas las configuraciones superó los 450 días computacionales, lo cual impuso límites prácticos sobre la exploración más exhaustiva de combinaciones de parámetros o arquitecturas adicionales.

Al realizar el análisis con dos médicos generales y tres especialistas en cardiología, se observó que el modelo generativo, en su mejor configuración, demuestra una capacidad significativa para producir imágenes que pueden pasar desapercibidas incluso ante la evaluación de profesionales de la salud. Los resultados evidencian que aproximadamente la mitad de las imágenes generadas no pudieron ser distinguidas de manera consistente de las reales, lo cual constituye un indicador relevante del nivel de realismo alcanzado por el modelo.

Aunque todas las muestras fueron generadas utilizando la misma arquitectura y configuración, el hecho de que una proporción de imágenes resulte perceptualmente indistinguible mientras que otra sea identificada como sintética pone de manifiesto la influencia del espacio latente en la variabilidad de la calidad visual, sugiriendo que el modelo ha aprendido representaciones distinguibles de la distribución de datos reales, pero que la calidad perceptual no es completamente homogénea en todo el conjunto generado, lo cual es consistente con el comportamiento reportado en modelos generativos basados en GAN.

Para trabajos futuros, se plantea la incorporación de una red más adecuada para el cálculo de métricas perceptuales en imágenes médicas, reemplazando Inception por modelos más relacionados al dominio clínico, asimismo, resulta pertinente profundizar en arquitecturas que no generan datos a partir de ruido, sino a partir de representaciones discretas — como VQ-GAN —, las cuales ofrecen un mayor potencial para mejorar la compresión latente y la consistencia estructural de las imágenes reconstruidas.

Teniendo en consideración las observaciones realizadas por los profesionales medicos durante, se plantea como línea de trabajo futuro la exploración de arquitecturas generativas que permitan mejorar la calidad visual y resolución. En particular, se propone la implementación de modelos basados en arquitecturas tipo U-Net dentro de esquemas de GAN, los cuales incorporan estructuras de encoder-decoder que, mediante el uso de redes convolucionales profundas y conexiones de salto (skip connections), permiten incrementar la resolución espacial de las imágenes de manera consistente, preservando detalles finos, texturas y características de ruido relevantes para mantener la fidelidad y coherencia visual. Adicionalmente, considerando la naturaleza dinámica y la importancia del contexto temporal en la interpretación ecocardiográfica, se propone como línea futura el desarrollo de modelos generativos capaces de sintetizar secuencias temporales completas en formato de video, en lugar de limitarse a la generación de fotogramas individuales, para ello, se contemplará el uso de los mejores modelos e hiperparámetros identificados en este trabajo como punto de partida, extendiendo la arquitectura hacia enfoques que integren modelado espacio-temporal. Estas mejoras permitirían no solo incrementar el realismo de las muestras generadas, sino también ampliar su potencial aplicabilidad en escenarios clínicos, educativos y de investigación, donde la coherencia temporal y la alta resolución resultan factores críticos para la interpretación médica.

Finalmente, un trabajo futuro de gran relevancia consiste en implementar el *Transformer* generativo encargado de producir nuevas imágenes ecocardiográficas utilizando los codebooks obtenidos de las distintas configuraciones de VQ-GAN; no obstante, este proceso requiere un equipo con una GPU que disponga de al menos 20 GB de VRAM, dado el alto costo computacional asociado a este tipo de modelos.

Referencias Bibliográficas

- [1] Z. Nie, M. Xu, Z. Wang, X. Lu, and W. Song, "A review of application of deep learning in endoscopic image processing," *Journal of Imaging*, vol. 10, no. 11, p. 275, Nov. 2024. [Online]. Available: <https://www.mdpi.com/2313-433X/10/11/275>
- [2] M. Aledhari, S. Joji, M. Hefeida, and F. Saeed, "Optimized cnn-based diagnosis system to detect the pneumonia from chest radiographs," in *Proceedings of the 2019 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*. San Diego, CA, USA: IEEE, Nov. 2019, pp. 2405--2412.
- [3] M. Szymkowski, E. Saeed, M. Omieljanowicz, A. Omieljanowicz, K. Saeed, and Z. Mariak, "A novelty approach to retina diagnosing using biometric techniques with svm and clustering algorithms," *IEEE Access*, vol. 8, pp. 125 849--125 862, 2020.
- [4] E. Gündüzvar, A. Kayık, and M. A. Altuncu, "Alzheimer's disease diagnosis in mri images using transfer learning methods: Evaluation of different model performances," *Balkan Journal of Electrical and Computer Engineering*, vol. 13, no. 2, p. 119--127, 2025.
- [5] M. Masud, N. Sikder, A.-A. Nahid, A. K. Bairagi, and M. A. AlZain, "A machine learning approach to diagnosing lung and colon cancer using a deep learning-based classification framework," *Sensors*, vol. 21, no. 3, p. 748, 2021. [Online]. Available: <https://doi.org/10.3390/s21030748>
- [6] X. Cai, X. Li, N. Razmjoo, and N. Ghadimi, "Breast cancer diagnosis by convolutional neural network and advanced thermal exchange optimization algorithm," *Computational and Mathematical Methods in Medicine*, vol. 2021, p. 5595180, 2021. [Online]. Available: <https://doi.org/10.1155/2021/5595180>
- [7] M. S. Kumar, K. V. Rao, and G. A. Kumar, "Mri image based classification model for lung tumor detection using convolutional neural networks," *Traitement du Signal*, vol. 38, no. 6, pp. 1837--1842, 2021. [Online]. Available: <https://doi.org/10.18280/ts.380628>
- [8] Y. Wang, X. Ge, H. Ma, S. Qi, G. Zhang, and Y. Yao, "Deep learning in medical ultrasound image analysis: A review," *IEEE Access*, vol. 9, pp. 54 310--54 324, 2021.
- [9] K. Drukker, P. Yan, A. Sibley, and G. Wang, "Chapter 4 - biomedical imaging and analysis through deep learning," in *Artificial Intelligence in Medicine*, L. Xing, M. L. Giger, and J. K. Min, Eds. Academic Press, 2021, pp. 49--74. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/B9780128212592000041>
- [10] K. Balaji and K. Lavanya, "Chapter 5 - medical image analysis with deep neural networks," in *Deep Learning and Parallel Computing Environment for Bioengineering Systems*, A. K. Sangaiah, Ed. Academic Press, 2019, pp. 75--97. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/B9780128167182000129>
- [11] F. Garcea, A. Serra, F. Lamberti, and L. Morra, "Data augmentation for medical imaging: A systematic literature review," *Computers in Biology and Medicine*, vol. 152, p. 106391, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S001048252201099X>
- [12] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Proceedings of the 27th International Conference on Neural Information Processing Systems (NeurIPS)*, vol. 2. MIT Press, 2014, pp. 2672--2680. [Online]. Available: <https://dl.acm.org/doi/10.5555/2969033.2969125>
- [13] M. Frid-Adar, I. Diamant, E. Klang, M. Amitai, J. Goldberger, and H. Greenspan, "Gan-based synthetic medical image augmentation for increased cnn performance in liver lesion classification," *Neurocomputing*, vol. 321, pp. 321--331, 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925231218310749>

- [14] C. Tiago, S. R. Snare, J. Šprem, and K. McLeod, "A domain translation framework with an adversarial denoising diffusion model to generate synthetic datasets of echocardiography images," *IEEE Access*, vol. 11, pp. 17 594--17 602, 2023.
- [15] H. Reynaud, Q. Meng, M. Dombrowski, A. Ghosh, T. Day, A. Gomez, P. Leeson, and B. Kainz, "Echonet-synthetic: Privacy-preserving video generation for safe medical data sharing," in *Medical Image Computing and Computer Assisted Intervention -- MICCAI 2024*, M. G. Linguraru, Q. Dou, A. Feragen, S. Giannarou, B. Glocker, K. Lekadir, and J. A. Schnabel, Eds. Cham: Springer Nature Switzerland, 2024, pp. 285--295. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-72104-5_28
- [16] J. C. Carvalho, P. Farand, H. D. Do, M. C. Brochu, F. Bonenfant, and S. Lepage, "Effect of age and sex on echocardiographic left ventricular diastolic function parameters in patients with preserved ejection fraction and normal valvular function," *Cardiology Journal*, vol. 20, no. 5, pp. 513--518, 2013. [Online]. Available: <https://doi.org/10.5603/CJ.2013.0137>
- [17] I. Kardys, J. W. Deckers, B. H. Stricker, W. B. Vletter, A. Hofman, and J. Witteman, "Distribution of echocardiographic parameters and their associations with cardiovascular risk factors in the rotterdam study," *European Journal of Epidemiology*, vol. 25, no. 7, pp. 481--490, 2010. [Online]. Available: <https://doi.org/10.1007/s10654-010-9453-5>
- [18] M. Daimon, H. Watanabe, Y. Abe, K. Hirata, T. Hozumi, K. Ishii, H. Ito, K. Iwakura, C. Izumi, M. Matsuzaki, S. Minagoe, H. Abe, K. Murata, S. Nakatani, K. Negishi, K. Yoshida, K. Tanabe, N. Tanaka, K. Tokai, J. Yoshikawa, and J. N. V. for Echocardiographic Measurements Project (JAMP) Study Investigators, "Gender differences in age-related changes in left and right ventricular geometries and functions. echocardiography of a healthy subject group," *Circulation Journal: Official Journal of the Japanese Circulation Society*, vol. 75, no. 12, pp. 2840--2846, 2011. [Online]. Available: <https://doi.org/10.1253/circj.cj-11-0364>
- [19] T. Karras, M. Aittala, J. Hellsten, S. Laine, J. Lehtinen, and T. Aila, "Training generative adversarial networks with limited data," in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NIPS '20. Red Hook, NY, USA: Curran Associates Inc., 2020. [Online]. Available: <https://arxiv.org/abs/2006.06676>
- [20] P. Esser, R. Rombach, and B. Ommer, "Taming transformers for high-resolution image synthesis," in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 12 868--12 878.
- [21] E. Choi, S. Biswal, B. Malin, J. Duke, W. F. Stewart, and J. Sun, "Generating multi-label discrete patient records using generative adversarial networks," in *Proceedings of the 2nd Machine Learning for Healthcare Conference*, ser. Proceedings of Machine Learning Research, F. Doshi-Velez, J. Fackler, D. Kale, R. Ranganath, B. Wallace, and J. Wiens, Eds., vol. 68. PMLR, 18--19 Aug 2017, pp. 286--305. [Online]. Available: <https://proceedings.mlr.press/v68/choi17a.html>
- [22] K. Guo, J. Chen, T. Qiu, S. Guo, T. Luo, T. Chen, and S. Ren, "Medgan: An adaptive gan approach for medical image generation," *Computers in Biology and Medicine*, vol. 163, p. 107119, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S001048252300584X>
- [23] Y. Skandarani, A. Lalande, J. Afilalo, and P.-M. Jodoin, "Generative adversarial networks in cardiology," *Canadian Journal of Cardiology*, vol. 38, no. 2, pp. 196--203, 2022, focus Issue: New Digital Technologies in Cardiology. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0828282X21008606>
- [24] K. V. Bunting, R. P. Steeds, K. Slater, J. K. Rogers, G. V. Gkoutos, and D. Kotecha, "A practical guide to assess the reproducibility of echocardiographic measurements," *Journal of the American Society of Echocardiography*, vol. 32, no. 12, pp. 1505--1515, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0894731719309460>
- [25] E. Gayat, H. Ahmad, L. Weinert, R. M. Lang, and V. Mor-Avi, "Reproducibility and inter-vendor variability of left ventricular deformation measurements by three-dimensional speckle-tracking echocardiography," *Journal of the American Society of Echocardiography*, vol. 24, no. 8, pp. 878--885, 2011. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0894731711003257>
- [26] M. Moinuddin, S. Khan, A. U. Alsaggaf, M. J. Abdulaal, U. M. Al-Saggaf, and J. C. Ye, "Medical ultrasound image speckle reduction and resolution enhancement using texture compensated multi-resolution convolution neural network," *Frontiers in Physiology*, vol. Volume 13 - 2022, 2022. [Online]. Available: <https://www.frontiersin.org/journals/physiology/articles/10.3389/fphys.2022.961571>
- [27] G. Van De Vyver, L. Fidon, E. Puyol-Antón, G. Piella, K. Lekadir, B. Glocker, and G. Tarroni, "Generative augmentations for improved cardiac ultrasound segmentation using diffusion models," *arXiv preprint arXiv:2502.20100*, 2025, blog post: <https://gillesvandyver.com/#/projects/generative-ai>. [Online]. Available: <https://arxiv.org/abs/2502.20100>
- [28] C. Tiago, A. Gilbert, A. S. Beela, S. A. Aase, S. R. Snare, J. Šprem, and K. McLeod, "A data augmentation pipeline to generate synthetic labeled datasets of 3d echocardiography images using a gan," *IEEE Access*, vol. 10, pp. 98 803--98 815, 2022.

Generación de imágenes de ecocardiogramas mediante estrategias de aprendizaje generativo

- [29] P. A. Amir H. Abdi, Teresa Tsang, “Gan-enhanced conditional echocardiogram generation,” *ArXiv*, 2019. [Online]. Available: <https://arxiv.org/abs/1911.02121>
- [30] M. Woodland, J. Wood, B. M. Anderson, S. Kundu, E. Lin, E. Koay, B. Odisio, C. Chung, H. C. Kang, A. M. Venkatesan, S. Yedururi, B. De, Y.-M. Lin, A. B. Patel, and K. K. Brock, “Evaluating the performance of stylegan2-ada on medical images,” in *Simulation and Synthesis in Medical Imaging*, C. Zhao, D. Svoboda, J. M. Wolterink, and M. Escobar, Eds. Cham: Springer International Publishing, 2022, pp. 142–153. [Online]. Available: https://link-springer-com.ezproxy.umng.edu.co/chapter/10.1007/978-3-031-16980-9_14#citeas
- [31] A. Raposo, A. Azeitona, M. Afonso, and J. Sanches, “Ultrasound denoising using the pix2pix gan,” in *27th Portuguese Conference on Pattern Recognition*. Évora, Portugal: RECPAD, 11 2021, pp. 91–92. [Online]. Available: https://afonsoraposo.com/pdf/2021-RECPAD-US_GAN_pix2pix.pdf
- [32] X. Yi, E. Walia, and P. Babyn, “Generative adversarial network in medical imaging: A review,” *Medical Image Analysis*, vol. 58, p. 101552, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1361841518308430>
- [33] K. Armanious, C. Jiang, M. Fischer, T. Küstner, T. Hepp, K. Nikolaou, S. Gatidis, and B. Yang, “Medgan: Medical image translation using gans,” *Computerized Medical Imaging and Graphics*, vol. 79, p. 101684, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0895611119300990>
- [34] B. Macedo, I. Ribeiro Vaz, and T. Taveira Gomes, “Medgan: optimized generative adversarial network with graph convolutional networks for novel molecule design,” *Scientific Reports*, vol. 14, p. 1212, 2024. [Online]. Available: <https://doi.org/10.1038/s41598-023-50834-6>
- [35] K. Armanious, C. Jiang, S. Abdulatif, T. Küstner, S. Gatidis, and B. Yang, “Unsupervised medical image translation using cycle-medgan,” *2019 27th European Signal Processing Conference (EUSIPCO)*, pp. 1–5, 2019. [Online]. Available: <https://api.semanticscholar.org/CorpusID:72940886>
- [36] F. Khader, G. Müller-Franzes, S. Tayebi Arasteh, T. Köhler, S. Engelhardt, A. F. Frangi, and A. Maier, “Denoising diffusion probabilistic models for 3d medical image generation,” *Scientific Reports*, vol. 13, p. 7303, 2023. [Online]. Available: <https://doi.org/10.1038/s41598-023-34341-2>
- [37] A. Zhao, M. Xu, A. H. Shahin, W. Wuyts, M. G. Jones, J. Jacob, and D. C. Alexander, “4d vq-gan: Synthesising medical scans at any time point for personalised disease progression modelling of idiopathic pulmonary fibrosis,” *arXiv preprint arXiv:2502.05713*, 2025, electrical Engineering and Systems Science >Image and Video Processing. GitHub repository: <https://github.com/anzhao920/4DVQGAN>. [Online]. Available: <https://arxiv.org/abs/2502.05713>
- [38] H. Reynaud, Q. Meng, M. Dombrowski, A. Ghosh, T. G. Day, A. Gomez, P. Leeson, and B. Kainz, “Echonetsynthetic: Privacy-preserving video generation for safe medical data sharing,” *ArXiv*, vol. abs/2406.00808, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:270220857>
- [39] M. Gazda, J. Gazda, S. Kadoury, R. Kanasz, and P. Drotar, “echogan: Extending the field of view in transthoracic echocardiography through conditional gan-based outpainting,” *Computer Methods and Programs in Biomedicine*, vol. 269, p. 108869, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S016926072500286X>
- [40] H. Anaya-Sánchez, L. Altamirano-Robles, R. Díaz-Hernández, and S. Zapotecas-Martínez, “Wgan-gp for synthetic retinal image generation: Enhancing sensor-based medical imaging for classification models,” *Sensors*, vol. 25, no. 1, p. 167, 2025. [Online]. Available: <https://doi.org/10.3390/s25010167>
- [41] M. Alruily, W. Said, A. M. Mostafa, M. Ezz, and M. Elmezain, “Breast ultrasound images augmentation and segmentation using GAN with identity block and modified U-Net 3,” *Sensors (Basel)*, vol. 23, no. 20, p. 8599, October 2023.
- [42] H. Pan, J. Mo, H. Lin, C. Zhang, Z. Wu, Y. Wang, and Q. Zheng, “Clinical prior-guided tumor generation for breast ultrasound with cross domain adaptation,” in *Medical Image Computing and Computer Assisted Intervention – MICCAI 2025*, J. C. Gee, D. C. Alexander, J. Hong, J. E. Iglesias, C. H. Sudre, A. Venkataraman, P. Golland, J. H. Kim, and J. Park, Eds. Cham: Springer Nature Switzerland, 2026, pp. 57–67.
- [43] J. Luo, H. Zhang, Y. Zhuang, L. Han, K. Chen, Z. Hua, C. Li, and J. Lin, “2s-busgan: Una nueva red generativa antagónica para imágenes de ultrasonido mamario realistas con el contorno tumoral correspondiente, basada en pequeños conjuntos de datos,” *Sensors*, vol. 23, no. 20, p. 8614, 2023.
- [44] M. Sinn and A. Rawat, “Non-parametric estimation of jensen-shannon divergence in generative adversarial network training,” in *International Conference on Artificial Intelligence and Statistics*, 2017. [Online]. Available: <https://api.semanticscholar.org/CorpusID:4858180>
- [45] F. Tom and D. Sheet, “Simulating patho-realistic ultrasound images using deep generative networks with adversarial learning,” *2018 IEEE 15th International Symposium on Biomedical Imaging (ISBI 2018)*, pp. 1174–1177, 2017. [Online]. Available: <https://api.semanticscholar.org/CorpusID:20179615>

- [46] S. Cammarasana, P. Nicolardi, and G. Patanè, “Real-time denoising of ultrasound images based on deep learning,” *Medical & Biological Engineering & Computing*, vol. 60, pp. 2229–2244, 2022.
- [47] Y. Skandarani, P. M. Jodoin, and A. Lalande, “Gans for medical image synthesis: An empirical study,” *Journal of Imaging*, vol. 9, no. 3, p. 69, 2023.
- [48] N. Jain, A. Olmo, S. Sengupta, L. Manikonda, and S. Kambhampati, “Imperfect imagination: Implications of gans exacerbating biases on facial data augmentation and snapchat face lenses,” *Artificial Intelligence*, vol. 304, p. 103652, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0004370221002034>
- [49] P. Friedrich, Y. Frisch, and P. C. Cattin, “Deep generative models for 3d medical image synthesis,” in *Generative Machine Learning Models in Medical Image Computing*, L. Zhang, C. Chen, Z. Li, and G. Slabaugh, Eds. Cham: Springer, 2025.
- [50] R. Kurapati, J. Heaton, and D. R. Lowery, *Atrial Kick*. Treasure Island, FL: StatPearls Publishing, 2025, retrieved from StatPearls, National Center for Biotechnology Information (NCBI) Bookshelf. [Online]. Available: <https://www.ncbi.nlm.nih.gov/books/NBK482421/>
- [51] C. Mitchell, P. S. Rahko, L. A. Blauwet, B. Canaday, J. A. Finstuen, M. C. Foster, K. Horton, K. O. Ogunyankin, R. A. Palma, E. J. Velazquez *et al.*, “Guidelines for performing a comprehensive transthoracic echocardiographic examination in adults: Recommendations from the american society of echocardiography,” *Journal of the American Society of Echocardiography*, vol. 32, no. 1, pp. 1–64, 2019. [Online]. Available: [https://www.onlinejase.com/article/S0894-7317\(18\)30318-3/fulltext](https://www.onlinejase.com/article/S0894-7317(18)30318-3/fulltext)
- [52] D. Ouyang, B. He, A. Ghorbani, M. P. Lungren, E. A. Ashley, D. H. Liang, and J. Y. Zou, “Echonet-dynamic: a large new cardiac motion video data resource for medical machine learning,” 2019. [Online]. Available: <https://api.semanticscholar.org/CorpusID:225101287>
- [53] R. M. Lang, L. P. Badano, V. Mor-Avi, J. Afilalo, A. Armstrong, L. Ernande, F. A. Flachskampf, E. Foster, S. A. Goldstein, T. Kuznetsova, P. Lancellotti, D. Muraru, M. H. Picard, E. R. Rietzschel, L. Rudski, K. T. Spencer, W. Tsang, and J.-U. Voigt, “Recommendations for cardiac chamber quantification by echocardiography in adults: An update from the american society of echocardiography and the european association of cardiovascular imaging,” *European Heart Journal - Cardiovascular Imaging*, vol. 16, no. 3, pp. 233–271, 02 2015. [Online]. Available: <https://doi.org/10.1093/ehjci/jev014>
- [54] Z. Y. Sun, Q. Li, J. Li, M. W. Zhang, L. Zhu, and J. Geng, “Echocardiographic evaluation of the right atrial size and function: Relevance for clinical practice,” *American Heart Journal Plus: Cardiology Research and Practice*, vol. 27, p. 100274, 2023. [Online]. Available: <https://doi.org/10.1016/j.ahjo.2023.100274>
- [55] L. Thomas, D. Muraru, B. A. Popescu, M. Sitges, M. Rosca, G. Pedrizzetti, M. Y. Henein, E. Donal, and L. P. Badano, “Evaluation of left atrial size and function: Relevance for clinical practice,” *Journal of the American Society of Echocardiography: Official Publication of the American Society of Echocardiography*, vol. 33, no. 8, pp. 934–952, 2020. [Online]. Available: <https://doi.org/10.1016/j.echo.2020.03.021>
- [56] M. Mukherjee, L. G. Rudski, K. Addetia, J. Afilalo, M. D’Alto, B. H. Freed, L. B. Friend, L. Gargani, J. Grapsa, P. M. Hassoun, L. Hua, J. Kim, V. Mercurio, R. Saggat, and A. Vonk-Noordegraaf, “Guidelines for the echocardiographic assessment of the right heart in adults and special considerations in pulmonary hypertension: Recommendations from the american society of echocardiography,” *Journal of the American Society of Echocardiography*, vol. 38, no. 3, pp. 141–186, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0894731725000379>
- [57] D. Ouyang, B. He, A. Ghorbani *et al.*, “Video-based ai for beat-to-beat assessment of cardiac function,” *Nature*, vol. 580, pp. 252–256, 2020. [Online]. Available: <https://doi.org/10.1038/s41586-020-2145-8>
- [58] S. Upadhyay, A. S. Beevi, and S. Kalady, “Left ventricle segmentation of 2d echocardiography using deep learning,” in *Computer Vision and Image Processing*, D. Gupta, K. Bhurchandi, S. Murala, B. Raman, and S. Kumar, Eds. Cham: Springer Nature Switzerland, 2023, pp. 87–98.
- [59] R. B. Hinton and K. E. Yutzey, “Heart valve structure and function in development and disease,” *Annual Review of Physiology*, vol. 73, pp. 29–46, 2011. [Online]. Available: <https://doi.org/10.1146/annurev-physiol-012110-142145>
- [60] A. Dahou, D. Levin, M. Reisman, and R. T. Hahn, “Anatomy and physiology of the tricuspid valve,” *JACC: Cardiovascular Imaging*, vol. 12, no. 3, pp. 458–468, 2019, FOCUS ISSUE: IMAGING THE TRICUSPID VALVE--PART I. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1936878X1930097X>
- [61] Y. Topilsky, “Mitral regurgitation: Anatomy, physiology, and pathophysiology—lessons learned from surgery and cardiac imaging,” *Frontiers in Cardiovascular Medicine*, vol. 7, p. 84, 2020. [Online]. Available: <https://www.frontiersin.org/articles/10.3389/fcvm.2020.00084>

Generación de imágenes de ecocardiogramas mediante estrategias de aprendizaje generativo

- [62] M. Lansakara and S. Unai, “An overview of aortic valve anatomy: the current understanding,” *Indian Journal of Thoracic and Cardiovascular Surgery*, vol. 39, no. Suppl 2, pp. 246–252, 2023. [Online]. Available: <https://doi.org/10.1007/s12055-023-01645-x>
- [63] G. Duffy, P. P. Cheng, N. Yuan *et al.*, “High-throughput precision phenotyping of left ventricular hypertrophy with cardiovascular deep learning,” *JAMA Cardiology*, vol. 7, no. 4, pp. 386–395, 2022.
- [64] O. A. Smiseth, H. Torp, A. Opdahl, K. H. Haugaa, and S. Urheim, “Myocardial strain imaging: how useful is it in clinical decision making?” *European Heart Journal*, vol. 37, no. 15, pp. 1196–1207, 2016.
- [65] S. X. Morita, K. Kusunose, A. Haga, M. Sata, K. Hasegawa, Y. Raita, M. P. Reilly, M. A. Fifer, M. S. Maurer, and Y. J. Shimada, “Deep learning analysis of echocardiographic images to predict positive genotype in patients with hypertrophic cardiomyopathy,” *Frontiers in Cardiovascular Medicine*, vol. 8, p. 669860, 2021. [Online]. Available: <https://doi.org/10.3389/fcvm.2021.669860>
- [66] B. Yi, W. Zeng, L. Lv, and P. Hua, “Changing epidemiology of calcific aortic valve disease: 30-year trends of incidence, prevalence, and deaths across 204 countries and territories,” *Aging*, vol. 13, no. 9, pp. 12 710–12 732. [Online]. Available: <https://doi.org/10.18632/aging.202942>
- [67] J. Park, J. Kim, J. Jeon, Y. E. Yoon, Y. Jang, H. Jeong, Y. Hong, S.-A. Lee, H.-M. Choi, I.-C. Hwang, G.-Y. Cho, and H.-J. Chang, “Artificial intelligence-enhanced comprehensive assessment of the aortic valve stenosis continuum in echocardiography,” *eBioMedicine*, vol. 112, p. 105560, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2352396425000040>
- [68] A. Long, C. M. Haggerty, J. Finer, D. Hartzel, L. Jing, A. Keivani, C. Kelsey, D. Rocha, J. Ruhl, D. vanMaanen, G. Metser, E. Duffy, T. Mawson, M. Maurer, A. J. Einstein, A. Beecy, D. Kumaraiah, S. Homma, Q. Liu, V. Agarwal, M. Lebehn, M. Leon, R. Hahn, P. Elias, and T. J. Poterucha, “Deep learning for echo analysis, tracking, and evaluation of mitral regurgitation (delineate-mr),” *Circulation*, vol. 150, no. 12, pp. 911–922, 2024. [Online]. Available: <https://www.ahajournals.org/doi/abs/10.1161/CIRCULATIONAHA.124.068996>
- [69] B. Liu, H. Chang, D. Yang *et al.*, “A deep learning framework assisted echocardiography with diagnosis, lesion localization, phenogrouping heterogeneous disease, and anomaly detection,” *Scientific Reports*, vol. 13, no. 3, 2023. [Online]. Available: <https://doi.org/10.1038/s41598-022-27211-w>
- [70] J. A. Naser, E. Lee, S. V. Pislaru, G. Tsaban, J. G. Malins, J. I. Jackson, D. M. Anisuzzaman, B. Rostami, F. Lopez-Jimenez, P. A. Friedman, G. C. Kane, P. A. Pellikka, and Z. I. Attia, “Artificial intelligence-based classification of echocardiographic views,” *European Heart Journal. Digital Health*, vol. 5, no. 3, pp. 260–269, 2024. [Online]. Available: <https://doi.org/10.1093/ehjdh/ztae015>
- [71] T. M. Mitchell, *Machine Learning*. New York, NY, USA: McGraw-Hill, 1997.
- [72] S. Shalev-Shwartz and S. Ben-David, *Understanding Machine Learning: From Theory to Algorithms*. Cambridge, UK: Cambridge University Press, 2014.
- [73] F. Bach, *Learning Theory from First Principles*, ser. Adaptive Computation and Machine Learning series. MIT Press, 2024. [Online]. Available: https://books.google.com.co/books?id=R_T8EAAAQBAJ
- [74] M. Mohri, A. Rostamizadeh, and A. Talwalkar, *Foundations of Machine Learning*, 2nd ed. Cambridge, MA: The MIT Press, 2018.
- [75] S. Geman, E. Bienenstock, and R. Doursat, “Neural networks and the bias/variance dilemma,” *Neural Computation*, vol. 4, no. 1, pp. 1–58, 1992.
- [76] R. O. Duda, P. E. Hart, and D. G. Stork, *Pattern Classification*, 2nd ed. New York: Wiley-Interscience, 2000.
- [77] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022. [Online]. Available: <https://probml.github.io/pml-book/book1.html>
- [78] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Pearson, 2009.
- [79] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, 1st ed. Cambridge, MA: MIT Press, 2016, hardcover, 66 color illustrations, 100 b&w illustrations. [Online]. Available: <https://www.deeplearningbook.org/>
- [80] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *CoRR*, vol. abs/1412.6980, 2014. [Online]. Available: <https://api.semanticscholar.org/CorpusID:6628106>

- [81] S. Ioffe and C. Szegedy, "Batch normalization: accelerating deep network training by reducing internal covariate shift," in *Proceedings of the 32nd International Conference on Machine Learning - Volume 37*, ser. ICML'15. JMLR.org, 2015, p. 448–456.
- [82] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proceedings of the 30th International Conference on Machine Learning - Volume 28*, ser. ICML'13. JMLR.org, 2013, p. III–1310–III–1318.
- [83] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [84] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in *Computer Vision -- ECCV 2014*, D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars, Eds. Cham: Springer International Publishing, 2014, pp. 818–833.
- [85] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, 2006. [Online]. Available: <https://www.science.org/doi/10.1126/science.1127647>
- [86] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," in *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2014. [Online]. Available: <http://arxiv.org/abs/1312.6114>
- [87] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*, 1st ed. Cambridge, MA: MIT Press, 2012, hardcover, 300 color illustrations, 165 b&w illustrations.
- [88] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," *CoRR*, vol. abs/1511.06434, 2015. [Online]. Available: <https://api.semanticscholar.org/CorpusID:11758569>
- [89] A. Odena, V. Dumoulin, and C. Olah, "Deconvolution and checkerboard artifacts," *Distill*, 2016. [Online]. Available: <http://distill.pub/2016/deconv-checkerboard>
- [90] A. L. Maas, "Rectifier nonlinearities improve neural network acoustic models," 2013. [Online]. Available: <https://api.semanticscholar.org/CorpusID:16489696>
- [91] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, p. 1929–1958, Jan. 2014.
- [92] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, "Improved techniques for training gans," in *Proceedings of the 30th International Conference on Neural Information Processing Systems*, ser. NIPS'16. Red Hook, NY, USA: Curran Associates Inc., 2016, p. 2234–2242.
- [93] T. Che, Y. Li, A. Jacob, Y. Bengio, and W. Li, "Mode regularized generative adversarial networks," in *International Conference on Learning Representations (ICLR)*, 2017. [Online]. Available: <https://openreview.net/forum?id=HJKkY35le>
- [94] A. Borji, "Pros and cons of gan evaluation measures," *Computer Vision and Image Understanding*, vol. 179, pp. 41–65, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1077314218304272>
- [95] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2818–2826.
- [96] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "Gans trained by a two time-scale update rule converge to a local nash equilibrium," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 6629–6640.
- [97] M. Bińkowski, D. J. Sutherland, M. Arbel, and A. Gretton, "Demystifying MMD GANs," in *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: <https://openreview.net/forum?id=r1lUOzWCW>
- [98] J. Lin, "Divergence measures based on the shannon entropy," *IEEE Transactions on Information Theory*, vol. 37, no. 1, pp. 145–151, 1991.
- [99] S. Nowozin, B. Cseke, and R. Tomioka, "f-gan: training generative neural samplers using variational divergence minimization," in *Proceedings of the 30th International Conference on Neural Information Processing Systems*, ser. NIPS'16. Red Hook, NY, USA: Curran Associates Inc., 2016, p. 271–279.
- [100] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 5967–5976.
- [101] Z. Wang and A. C. Bovik, "Mean squared error: Love it or leave it? a new look at signal fidelity measures," *IEEE Signal Processing Magazine*, vol. 26, no. 1, pp. 98–117, 2009.

Generación de imágenes de ecocardiogramas mediante estrategias de aprendizaje generativo

- [102] A. Horé and D. Ziou, “Image quality metrics: Psnr vs. ssim,” in *2010 20th International Conference on Pattern Recognition*, 2010, pp. 2366–2369.
- [103] C. Ledig, L. Theis, F. Huszar, J. Caballero, A. Cunningham, A. Acosta, A. Aitken, A. Tejani, J. Totz, Z. Wang, and W. Shi, “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Los Alamitos, CA, USA: IEEE Computer Society, Jul. 2017, pp. 105–114. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/CVPR.2017.19>
- [104] Z. Wang, A. Bovik, H. Sheikh, and E. Simoncelli, “Image quality assessment: from error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [105] R.-c. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, “The unreasonable effectiveness of deep features as a perceptual metric,” in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 586–595.
- [106] A. Dosovitskiy and T. Brox, “Generating images with perceptual similarity metrics based on deep networks,” in *Proceedings of the 30th International Conference on Neural Information Processing Systems*, ser. NIPS’16. Red Hook, NY, USA: Curran Associates Inc., 2016, p. 658–666.
- [107] T. Karras, S. Laine, and T. Aila, “A style-based generator architecture for generative adversarial networks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 12, pp. 4217–4228, 2021.
- [108] A. Creswell, T. White, V. Dumoulin, K. Arulkumaran, B. Sengupta, and A. A. Bharath, “Generative adversarial networks: An overview,” *IEEE Signal Processing Magazine*, vol. 35, no. 1, pp. 53–65, 2018.
- [109] S. Laine, “Feature-based metrics for exploring the latent space of generative models,” in *Proceedings of the International Conference on Learning Representations (ICLR), Workshop Track*, 2018, poster. [Online]. Available: <https://openreview.net/forum?id=By-7dz-AZ>
- [110] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen, and T. Aila, “Analyzing and improving the image quality of stylegan,” in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 8107–8116.
- [111] X. Huang and S. Belongie, “Arbitrary style transfer in real-time with adaptive instance normalization,” in *2017 IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 1510–1519.
- [112] T. Karras, T. Aila, S. Laine, and J. Lehtinen, “Progressive growing of gans for improved quality, stability, and variation,” in *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: <https://openreview.net/forum?id=Hk99zCeAb>
- [113] A. Sauer, K. Schwarz, and A. Geiger, “Stylegan-xl: Scaling stylegan to large diverse datasets,” in *ACM SIGGRAPH 2022 Conference Proceedings*, ser. SIGGRAPH ’22. New York, NY, USA: Association for Computing Machinery, 2022. [Online]. Available: <https://doi.org/10.1145/3528233.3530738>
- [114] Y. Rubner, C. Tomasi, and L. J. Guibas, “The earth mover’s distance as a metric for image retrieval,” *International Journal of Computer Vision*, vol. 40, no. 2, pp. 99–121, Nov. 2000.
- [115] N. Tupitsa, P. Dvurechensky, D. Dvinskikh, and A. Gasnikov, *Computational Optimal Transport*. Cham: Springer International Publishing, 2020, pp. 1–17. [Online]. Available: https://doi.org/10.1007/978-3-030-54621-2_861-1
- [116] M. Arjovsky, S. Chintala, and L. Bottou, “Wasserstein generative adversarial networks,” in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 06–11 Aug 2017, pp. 214–223. [Online]. Available: <https://proceedings.mlr.press/v70/arjovsky17a.html>
- [117] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, “Neural discrete representation learning,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS’17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 6309–6318.
- [118] T. Karras, M. Aittala, S. Laine, E. Härkönen, J. Hellsten, J. Lehtinen, and T. Aila, “Alias-free generative adversarial networks,” *arXiv preprint arXiv:2106.12423*, 2021. [Online]. Available: <https://arxiv.org/pdf/2106.12423.pdf>
- [119] A. Brock, J. Donahue, and K. Simonyan, “Large scale GAN training for high fidelity natural image synthesis,” in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: <https://openreview.net/forum?id=B1xsqj09Fm>
- [120] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, “Unpaired Image-to-Image Translation Using Cycle-Consistent Adversarial Networks,” in *2017 IEEE International Conference on Computer Vision (ICCV)*. Los Alamitos, CA, USA: IEEE Computer Society, Oct. 2017, pp. 2242–2251. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/ICCV.2017.244>

- [121] A. B. L. Larsen, S. K. Sønderby, H. Larochelle, and O. Winther, “Autoencoding beyond pixels using a learned similarity metric,” in *Proceedings of The 33rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. F. Balcan and K. Q. Weinberger, Eds., vol. 48. New York, New York, USA: PMLR, 20--22 Jun 2016, pp. 1558--1566. [Online]. Available: <https://proceedings.mlr.press/v48/larsen16.html>
- [122] R. Osuala, G. Skorupko, N. Lazrak, L. Garrucho, E. García, S. Joshi, S. Jouide, M. Rutherford, F. Prior, K. Kushibar, O. Díaz, and K. Lekadir, “medigan: a python library of pretrained generative models for medical image synthesis,” *Journal of Medical Imaging*, vol. 10, no. 6, p. 061403, Nov. 2023, epub 2023 Feb 20. [Online]. Available: <https://doi.org/10.1117/1.JMI.10.6.061403>
- [123] H. Huang, Z. Ge, H. Wang, J. Wu, C. Hu, N. Li, X. Wu, and C. Pan, “Segmentation of echocardiography based on deep learning model,” *Electronics*, vol. 11, no. 11, p. 1714, 2022. [Online]. Available: <https://doi.org/10.3390/electronics11111714>
- [124] D. Wolpert and W. Macready, “No free lunch theorems for optimization,” *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 67--82, 1997.
- [125] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” *CoRR*, vol. abs/1312.6114, 2013. [Online]. Available: <https://api.semanticscholar.org/CorpusID:216078090>
- [126] J. Hussain, M. Bâth, and J. Ivarsson, “Generative adversarial networks in medical image reconstruction: A systematic literature review,” *Computers in Biology and Medicine*, vol. 191, p. 110094, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0010482525004457>
- [127] M. Arjovsky and L. Bottou, “Towards Principled Methods for Training Generative Adversarial Networks,” 1 2017.
- [128] M. Lucic, K. Kurach, M. Michalski, O. Bousquet, and S. Gelly, “Are gans created equal? a large-scale study,” in *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, ser. NIPS’18. Red Hook, NY, USA: Curran Associates Inc., 2018, p. 698--707.
- [129] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, “Learning transferable visual models from natural language supervision,” in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021, pp. 8748--8763. [Online]. Available: <http://proceedings.mlr.press/v139/radford21a/radford21a.pdf>
- [130] R. De Castro Korgi, *El Universo LaTeX*, 2nd ed. Bogota DC: Universidad Nacional de Colombia, 2010.